COLOR-CODED STUDY BOOK | BASICS TO ADVANCED

AIML BOOK

Complete notes for HTML, CSS, JavaScript, Python OOP, Python, and Machine Learning

01 HTML	**02 CSS**
03 JavaScript	**04 Python OOP**
05 Machine Learning	**06 Python: Beginner to Advanced**
07 Python: 34 Topics with Code	**08 ML Interview Cheat Sheet**

Built from the supplied notes: examples, code, dry runs, formulas, mind maps, interview guidance, and revision tables are retained.

Contents

PART 1 | HTML MASTERY — COMPLETE STUDY GUIDE 3

PART 2 | CSS MASTERY — COMPLETE STUDY GUIDE 63

PART 3 | JavaScript — Complete Mastery Guide 132

PART 4 | Chapter 11: Object-Oriented Programming 199

PART 5 | Complete Machine Learning for Interview 244

PART 6 | Complete Python: Beginner to Advanced 347

PART 7 | Complete Python: 34 Topics with Detailed Code 409

PART 8 | Complete Machine Learning Interview Cheat Sheet 488

PART 1 | HTML MASTERY — COMPLETE STUDY GUIDE

CONCEPT MAP

```
Beginner → Advanced | Concepts + Code + Diagrams + Projects + Interview Questions
```

HTML is the foundation of every web page. If you want to become a Full-Stack, AI/ML, or web developer, HTML should be extremely strong.

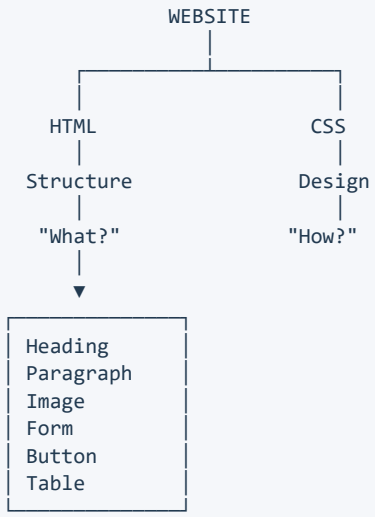
> Master formula:

HTML FOUNDATIONS TO ADVANCED

```
HTML = Structure + Meaning + Content
```

Think of a website like a human body:

CONCEPT MAP



1. What is HTML?

HTML FOUNDATIONS TO ADVANCED

```
HTML = HyperText Markup Language
```

Break the difficult words:

Word	Easy meaning
HyperText	Text connected to other pages
Markup	Special tags that describe content
Language	Rules used to write something

> Simple definition

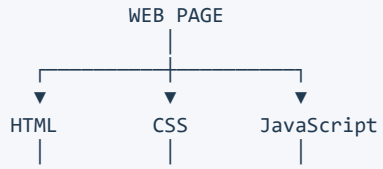
HTML is a markup language used to create the structure and content of web pages.

HTML is not a programming language because it does not normally contain programming logic like loops, conditions, or functions.

2. HTML vs CSS vs JavaScript

This is one of the most important concepts.

CONCEPT MAP



Structure Design Behavior

CONCEPT MAP

What? How? Action?

> Example:

HTML FOUNDATIONS TO ADVANCED

```
<button>Click Me</button>
```

HTML creates the button.

button {

HTML FOUNDATIONS TO ADVANCED

```
    color: white;
    background: blue;
```

}

CSS designs it.

HTML FOUNDATIONS TO ADVANCED

```
button.addEventListener("click", function () {
    alert("Hello!");
```

});

JavaScript gives it behavior.

> Remember

HTML FOUNDATIONS TO ADVANCED

```
HTML = Skeleton
CSS = Clothes/Design
JavaScript = Brain/Behavior
```

3. COMPLETE HTML MIND MAP

HTML

CONCEPT MAP

```
│
├── 1. Basics
│   ├── HTML document
│   ├── Tags
│   ├── Elements
│   ├── Attributes
│   └── Comments
│
├── 2. Text
│   ├── h1-h6
│   ├── p
│   ├── br
│   ├── hr
│   ├── strong
│   ├── em
│   ├── mark
│   ├── small
│   ├── del
│   ├── ins
│   ├── sub
│   └── sup
│
├── 3. Structure
│   ├── header
│   ├── nav
│   ├── main
│   ├── section
│   ├── article
│   ├── aside
│   └── footer
│
├── 4. Links
│   ├── a
│   ├── href
```

CONCEPT MAP

```
│   ├── target
│   ├── mailto
│   ├── tel
│   └── page anchors
│
├── 5. Images
│   ├── img
│   ├── src
│   ├── alt
│   ├── width
│   └── height
│
├── 6. Lists
│   ├── ul
│   ├── ol
│   ├── li
│   └── dl
│
├── 7. Tables
│   ├── table
│   ├── tr
│   ├── th
│   ├── td
│   ├── thead
│   ├── tbody
│   └── tfoot
│
├── 8. Forms
│   ├── form
│   ├── input
│   ├── label
│   ├── textarea
│   ├── select
│   ├── option
```

CONCEPT MAP

```
│   ├── button
│   └── validation
│
├── 9. Multimedia
│   ├── audio
│   ├── video
│   ├── iframe
│   └── track
│
├── 10. Semantic HTML
│   ├── header
│   ├── nav
│   ├── main
│   ├── section
│   ├── article
│   ├── aside
│   └── footer
│
├── 11. Advanced
│   ├── metadata
│   ├── accessibility
│   ├── SEO
│   ├── responsive HTML
│   ├── data attributes
│   ├── template
│   ├── details
│   └── dialog
│
└── 12. Real Projects
    ├── Portfolio
    ├── Login
    ├── Registration
    ├── Dashboard
    ├── Product page
```

CONCEPT MAP

```
    └── AI/ML application
```

4. Basic HTML Document

Every beginner should memorize this structure.

HTML FOUNDATIONS TO ADVANCED

```
<!DOCTYPE html>

<html lang="en">

<head>
    <meta charset="UTF-8">

    <meta
        name="viewport"
        content="width=device-width, initial-scale=1.0"
    >

    <title>My Website</title>
</head>

<body>

    <h1>Hello World</h1>

    <p>Welcome to my website.</p>

</body>

</html>
```

5. Understand Every Line

HTML FOUNDATIONS TO ADVANCED

```
<!DOCTYPE html>
```

Tells the browser:

"This document uses modern HTML."

HTML FOUNDATIONS TO ADVANCED

```
<html>
```

Root element.

CONCEPT MAP

```
<html>
   │
   ├── head
   │
   └── body
</html>
<head>
```

Contains information about the page.

Examples:

HTML FOUNDATIONS TO ADVANCED

```
<head>

    <title>My Page</title>

    <meta charset="UTF-8">

    <meta
        name="description"
        content="My personal portfolio"
    >

</head>
<body>
```

Contains what the user sees.

HTML FOUNDATIONS TO ADVANCED

```
<body>

    <h1>Hello</h1>

    <p>This is visible.</p>

</body>
```

6. Tag vs Element vs Attribute

Very important.

HTML FOUNDATIONS TO ADVANCED

```
<p class="intro">Hello World</p>
```

Breakdown:

CONCEPT MAP

```
<p class="intro">Hello World</p>
| |       |        |        |
| |       |        |        └── Closing tag
| |       |        └─────────── Content
| |       └──────────────────── Attribute value
| └──────────────────────────── Attribute
└────────────────────────────── Tag
```

Element

The complete structure:

HTML FOUNDATIONS TO ADVANCED

```
<p class="intro">Hello World</p>
```

Tag

HTML FOUNDATIONS TO ADVANCED

```
<p>
</p>
```

Attribute

HTML FOUNDATIONS TO ADVANCED

```
class="intro"
```

> Remember

HTML FOUNDATIONS TO ADVANCED

```
Tag = instruction
Attribute = extra information
Element = complete HTML unit
```

7. Opening and Closing Tags

Normal element:

HTML FOUNDATIONS TO ADVANCED

```
<h1>Hello</h1>
```

Opening:

HTML FOUNDATIONS TO ADVANCED

```
<h1>
```

Closing:

HTML FOUNDATIONS TO ADVANCED

```
</h1>
```

Content:

Hello

8. Void Elements

Some HTML elements don't have closing tags.

Examples:

HTML FOUNDATIONS TO ADVANCED

```
<img>
<br>
<hr>
<input>
<meta>
<link>
```

> Example:

HTML FOUNDATIONS TO ADVANCED

```
<img src="photo.jpg" alt="Profile photo">
```

9. Comments

Comments are ignored by the browser.

HTML FOUNDATIONS TO ADVANCED

```
<!-- This is a comment -->
```

> Example:

HTML FOUNDATIONS TO ADVANCED

```
<!-- User profile section -->

<h2>Profile</h2>
```

10. Headings

HTML provides six heading levels.

HTML FOUNDATIONS TO ADVANCED

```
<h1>Main Heading</h1>
<h2>Section</h2>
<h3>Subsection</h3>
<h4>Small Heading</h4>
<h5>Smaller Heading</h5>
<h6>Smallest Heading</h6>
```

Hierarchy:

h1

CONCEPT MAP

```
│
├── h2
│   ├── h3
│   └── h3
│
└── h2
    └── h3
```

> Important

Don't choose heading levels simply because of their visual size.

Use them for document structure.

CSS controls visual size.

11. Paragraph

HTML FOUNDATIONS TO ADVANCED

```
<p>
    HTML is used to structure web pages.
</p>
```

Multiple paragraphs:

HTML FOUNDATIONS TO ADVANCED

```
<p>First paragraph.</p>

<p>Second paragraph.</p>
```

 12. Line Break

Hello

World

Result:

Hello

World

Don't use
 for page layout.

Use CSS for layout.

13. Horizontal Rule

HTML FOUNDATIONS TO ADVANCED

```
<hr>
```

Used to represent a thematic separation.

14. Text Formatting

Bold importance

HTML FOUNDATIONS TO ADVANCED

```
<strong>Important</strong>
```

Emphasis

HTML FOUNDATIONS TO ADVANCED

```
<em>Important idea</em>
```

Highlight

HTML FOUNDATIONS TO ADVANCED

```
<mark>Important</mark>
```

Small text

HTML FOUNDATIONS TO ADVANCED

```
<small>Terms and conditions</small>
```

Deleted text

HTML FOUNDATIONS TO ADVANCED

```
<del>₹999</del>
```

Inserted text

HTML FOUNDATIONS TO ADVANCED

```
<ins>₹799</ins>
```

Subscript

H₂O

Superscript

x²

15. <b> vs

HTML FOUNDATIONS TO ADVANCED

```
<b>Bold</b>
```

mainly represents styling/attention.

HTML FOUNDATIONS TO ADVANCED

```
<strong>Important</strong>
```

represents strong importance.

> Interview answer

HTML FOUNDATIONS TO ADVANCED

```
<strong> has semantic meaning, while <b> is primarily a stylistic attention element.
```

16. <i> vs

HTML FOUNDATIONS TO ADVANCED

```
<i>Technical term</i>
```

versus:

HTML FOUNDATIONS TO ADVANCED

```
<em>Important emphasis</em>
<em> has semantic emphasis.
```

17. Links

The anchor element:

HTML FOUNDATIONS TO ADVANCED

```
<a href="https://example.com">
    Visit Website
</a>
```

Structure

CONCEPT MAP

```
<a>
 |
 ├── href → destination
 |
 └── text → clickable content
```

18. Open Link in New Tab

HTML FOUNDATIONS TO ADVANCED

```
<a
    href="https://example.com"
    target="_blank"
    rel="noopener noreferrer"
```

>

HTML FOUNDATIONS TO ADVANCED

```
    Open Website
</a>
```

Security note

When using _blank, rel="noopener noreferrer" is a good defensive practice.

19. Email Link

HTML FOUNDATIONS TO ADVANCED

```
<a href="mailto:aamir@example.com">
    Email Me
</a>
```

20. Telephone Link

HTML FOUNDATIONS TO ADVANCED

```
<a href="tel:+911234567890">
    Call Me
</a>
```

21. Internal Page Navigation

HTML FOUNDATIONS TO ADVANCED

```
<a href="#projects">
    Projects
</a>

<section id="projects">

    <h2>Projects</h2>

</section>
```

Flow:

Click Projects

CONCEPT MAP

```
      ↓
href="#projects"
      ↓
```

Find id="projects"

CONCEPT MAP

```
      ↓
```

Scroll to section

22. Images

Basic:

HTML FOUNDATIONS TO ADVANCED

```
<img
    src="photo.jpg"
    alt="Profile photo"
```

>

Important attributes

Attribute	Meaning
src	Image location
alt	Alternative text
width	Width
height	Height
loading	Loading behavior

> Example:

HTML FOUNDATIONS TO ADVANCED

```
<img
    src="profile.jpg"
    alt="Aamir profile photo"
    width="300"
    height="300"
    loading="lazy"
```

>

23. Why alt is Important

If image cannot load:

Image

CONCEPT MAP

```
  ↓
```

Cannot load

CONCEPT MAP

```
  ↓
```

alt text

Screen readers can also use alternative text.

Bad:

HTML FOUNDATIONS TO ADVANCED

```
<img src="cat.jpg">
```

[ ] Better:

HTML FOUNDATIONS TO ADVANCED

```
<img
    src="cat.jpg"
    alt="White cat sitting on a chair"
```

>

24. Unordered List

HTML FOUNDATIONS TO ADVANCED

```
<ul>

    <li>Python</li>
    <li>SQL</li>
    <li>Machine Learning</li>

</ul>
```

Result:

- Python
- SQL
- Machine Learning

25. Ordered List

HTML FOUNDATIONS TO ADVANCED

```
<ol>

    <li>Learn HTML</li>
    <li>Learn CSS</li>
    <li>Learn JavaScript</li>

</ol>
```

Result:

1. Learn HTML
2. Learn CSS

3. Learn JavaScript

26. Description List

Useful for terms and definitions.

HTML FOUNDATIONS TO ADVANCED

```
<dl>

    <dt>HTML</dt>

    <dd>
        Structure of a web page.
    </dd>

    <dt>CSS</dt>

    <dd>
        Styling of a web page.
    </dd>

</dl>
```

Think:

Term

CONCEPT MAP

```
 ↓
```

Definition

27. HTML Tables

Basic:

HTML FOUNDATIONS TO ADVANCED

```
<table>

    <tr>
        <th>Name</th>
        <th>Age</th>
        <th>Role</th>
    </tr>

    <tr>
        <td>Aamir</td>
        <td>24</td>
        <td>AI/ML</td>
    </tr>

</table>
```

28. Table Structure

CONCEPT MAP

```
<table>
   │
   ├── <thead>
   │       └── <tr>
   │            ├── <th>
   │            └── <th>
   │
   ├── <tbody>
   │       └── <tr>
   │            ├── <td>
   │            └── <td>
   │
   └── <tfoot>
```

Professional table:

HTML FOUNDATIONS TO ADVANCED

```
<table>

    <thead>

        <tr>
            <th>Name</th>
            <th>Skill</th>
        </tr>

    </thead>

    <tbody>

        <tr>
            <td>Aamir</td>
            <td>Python</td>
        </tr>

        <tr>
            <td>Rahul</td>
            <td>Java</td>
        </tr>

    </tbody>

</table>
```

29. colspan

Combine columns.

HTML FOUNDATIONS TO ADVANCED

```
<table>

    <tr>
        <th colspan="2">
            Student Information
        </th>
    </tr>

    <tr>
        <td>Aamir</td>
        <td>AI/ML</td>
    </tr>

</table>
```

Concept:

Before:

CONCEPT MAP

```
┌──────────┬──────────┐
│ Name     │ Skill    │
└──────────┴──────────┘
```

After colspan:

CONCEPT MAP

```
┌────────────────────┐
│ Student Info       │
└────────────────────┘
```

↕ 30. rowspan

Combine rows.

HTML FOUNDATIONS TO ADVANCED

```
<td rowspan="2">
    AI/ML
</td>
```

31.

div is a generic block container.

HTML FOUNDATIONS TO ADVANCED

```
<div>

    <h2>Profile</h2>

    <p>Hello!</p>

</div>
```

Think:

CONCEPT MAP

```
<div>
 ├── heading
 ├── paragraph
 └── image
</div>
```

32.

span is a generic inline container.

HTML FOUNDATIONS TO ADVANCED

```
<p>
    I love
    <span>Python</span>.
</p>
```

Difference

div

CONCEPT MAP

```
 ↓
```

Block-level grouping

span

CONCEPT MAP

```
 ↓
```

Inline grouping

33. Semantic HTML

This is very important for interviews.

Semantic elements tell the browser and developer what the content means.

Instead of:

HTML FOUNDATIONS TO ADVANCED

```
<div class="header">
```

use:

HTML FOUNDATIONS TO ADVANCED

```
<header>
```

Instead of:

HTML FOUNDATIONS TO ADVANCED

```
<div class="navigation">
```

use:

HTML FOUNDATIONS TO ADVANCED

```
<nav>
```

34. Semantic HTML Diagram

CONCEPT MAP

```
<body>
│
├── <header>
│      └── Logo
│
├── <nav>
│      └── Navigation
│
├── <main>
│      │
│      ├── <section>
│      │
│      ├── <article>
│      │
│      └── <aside>
│
└── <footer>
```

35. Important Semantic Elements

HTML FOUNDATIONS TO ADVANCED

```
<header>
```

Introductory content.

HTML FOUNDATIONS TO ADVANCED

```
<header>
    <h1>My Portfolio</h1>
</header>
<nav>
```

Navigation links.

HTML FOUNDATIONS TO ADVANCED

```
<nav>

    <a href="/">Home</a>
    <a href="#about">About</a>
    <a href="#contact">Contact</a>

</nav>
<main>
```

Main page content.

HTML FOUNDATIONS TO ADVANCED

```
<main>

    <h1>AI/ML Engineer</h1>

</main>
```

A document should normally have one primary <main>.

36. <section>

A thematic section.

HTML FOUNDATIONS TO ADVANCED

```
<section id="skills">

    <h2>Skills</h2>

    <p>Python, SQL, Machine Learning</p>

</section>
```

37. <article>

Self-contained content.

Examples:

Blog post

News article

Product card

Forum post

HTML FOUNDATIONS TO ADVANCED

```
<article>

    <h2>My ML Project</h2>

    <p>
        This project predicts crop yield.
    </p>

</article>
```

38. <aside>

Related but secondary content.

HTML FOUNDATIONS TO ADVANCED

```
<aside>

    <h3>Related Projects</h3>

    <p>AI Chatbot</p>

</aside>
```

39. <footer>

Bottom/closing information.

HTML FOUNDATIONS TO ADVANCED

```
<footer>

    <p>
        © 2026 My Portfolio
    </p>

</footer>
```

40. Forms — Extremely Important

Forms collect user input.

USER

CONCEPT MAP

```
  ↓
```

FORM

CONCEPT MAP

```
  ↓
```

INPUT

CONCEPT MAP

```
  ↓
```

VALIDATION

CONCEPT MAP

```
  ↓
```

SERVER

CONCEPT MAP

```
  ↓
```

DATABASE / MODEL

Basic:

HTML FOUNDATIONS TO ADVANCED

```
<form>

    <label for="name">
        Name
    </label>

    <input
        type="text"
        id="name"
        name="name"
    >

    <button type="submit">
        Submit
    </button>

</form>
```

41. Why <label> is Important

HTML FOUNDATIONS TO ADVANCED

```
<label for="email">
    Email
</label>

<input
    id="email"
    type="email"
```

>

The for value should match the input's id.

label for="email"

CONCEPT MAP

```
        ↓
```

input id="email"

Benefits:

Accessibility

Better usability

Clicking label can focus input

42. Input Types

HTML provides many input types.

HTML FOUNDATIONS TO ADVANCED

```
<input type="text">

<input type="password">

<input type="email">

<input type="number">

<input type="date">

<input type="time">

<input type="file">

<input type="checkbox">

<input type="radio">

<input type="range">

<input type="color">

<input type="search">

<input type="tel">

<input type="url">

<input type="hidden">

<input type="submit">
```

43. Password Input

HTML FOUNDATIONS TO ADVANCED

```
<label for="password">
    Password
</label>

<input
    type="password"
    id="password"
    name="password"
```

>

Important:

HTML FOUNDATIONS TO ADVANCED

```
type="password" hides characters visually; it does not encrypt the password.
```

44. Email Input

HTML FOUNDATIONS TO ADVANCED

```
<input
    type="email"
    name="email"
    required
```

>

Browser can perform basic validation.

45. Number Input

HTML FOUNDATIONS TO ADVANCED

```
<input
    type="number"
    min="18"
    max="100"
```

>

46. Date

HTML FOUNDATIONS TO ADVANCED

```
<input
    type="date"
    name="dob"
```

>

47. 47. Checkbox

HTML FOUNDATIONS TO ADVANCED

```
<label>

    <input
        type="checkbox"
        name="skills"
        value="python"
    >

    Python

</label>
```

Multiple:

HTML FOUNDATIONS TO ADVANCED

```
<label>
    <input type="checkbox" name="skills" value="python">
    Python
</label>

<label>
    <input type="checkbox" name="skills" value="sql">
    SQL
</label>

<label>
    <input type="checkbox" name="skills" value="ml">
    Machine Learning
</label>
```

48. Radio Buttons

Radio buttons normally allow one selection within the same group.

HTML FOUNDATIONS TO ADVANCED

```
<label>
    <input
        type="radio"
        name="gender"
        value="male"
    >
    Male
</label>

<label>
    <input
        type="radio"
        name="gender"
        value="female"
    >
    Female
</label>
```

> Important

Same name:

CONCEPT MAP

```
name="gender"
      ↓
```

same group

49. Select Dropdown

HTML FOUNDATIONS TO ADVANCED

```
<label for="city">
    City
</label>

<select id="city" name="city">

    <option value="delhi">
        Delhi
    </option>

    <option value="jaipur">
        Jaipur
    </option>

    <option value="patna">
        Patna
    </option>

</select>
```

50. Textarea

For long text:

HTML FOUNDATIONS TO ADVANCED

```
<label for="message">
    Message
</label>

<textarea
    id="message"
    name="message"
    rows="5"
    cols="40"
```

></textarea>

51. Button

HTML FOUNDATIONS TO ADVANCED

```
<button type="button">
    Click
</button>
```

Submit:

HTML FOUNDATIONS TO ADVANCED

```
<button type="submit">
    Submit
</button>
```

Reset:

HTML FOUNDATIONS TO ADVANCED

```
<button type="reset">
    Reset
</button>
```

52. required

HTML FOUNDATIONS TO ADVANCED

```
<input
    type="text"
    required
```

>

User must provide a value before submitting.

53. min, max

HTML FOUNDATIONS TO ADVANCED

```
<input
    type="number"
    min="1"
    max="100"
```

>

54. minlength, maxlength

HTML FOUNDATIONS TO ADVANCED

```
<input
    type="text"
    minlength="3"
    maxlength="50"
```

>

55. pattern

You can use a regular expression for browser-side validation.

> Example:

HTML FOUNDATIONS TO ADVANCED

```
<input
    type="text"
    pattern="[A-Za-z]+"
```

>

This accepts letters according to that pattern.

56. Complete Registration Form

HTML FOUNDATIONS TO ADVANCED

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta
        name="viewport"
        content="width=device-width, initial-scale=1.0"
    >

    <title>Registration</title>

</head>

<body>

    <main>

        <h1>Create Account</h1>

        <form>

            <div>

                <label for="name">
                    Full Name
                </label>

                <input
                    type="text"
                    id="name"
```

HTML FOUNDATIONS TO ADVANCED

```
            name="name"
            required
            minlength="3"
        >

    </div>

    <br>

    <div>

        <label for="email">
            Email
        </label>

        <input
            type="email"
            id="email"
            name="email"
            required
        >

    </div>

    <br>

    <div>

        <label for="password">
            Password
        </label>

        <input
            type="password"
```

```
            id="password"
            name="password"
            required
            minlength="8"
        >

    </div>

    <br>

    <div>

        <label for="age">
            Age
        </label>

        <input
            type="number"
            id="age"
            name="age"
            min="18"
            max="100"
            required
        >

    </div>

    <br>

    <div>

        <label for="city">
            City
        </label>
```



```
        <select
            id="city"
            name="city"
            required
        >

            <option value="">
                Select city
            </option>

            <option value="delhi">
                Delhi
            </option>

            <option value="jaipur">
                Jaipur
            </option>

            <option value="patna">
                Patna
            </option>

        </select>

    </div>

    <br>

    <div>

        <p>Gender</p>

        <label>
```

```
                    <input
                        type="radio"
                        name="gender"
                        value="male"
                    >
                    Male
                </label>

                <label>
                    <input
                        type="radio"
                        name="gender"
                        value="female"
                    >
                    Female
                </label>

            </div>

            <br>

            <label>

                <input
                    type="checkbox"
                    required
                >

                I agree to the terms.

            </label>

            <br><br>
```



```
            <button type="submit">
                Register
            </button>

            <button type="reset">
                Reset
            </button>

        </form>

    </main>

</body>

</html>
```

57. Form Dry Run

Suppose user enters:

Field	Value
Name	Aamir
Email	aamir@example.com
Password	****
Age	24
City	Jaipur

Field	Value
Gender	Male
Terms	Checked

Flow:

User enters data

CONCEPT MAP

↓

Browser validation

CONCEPT MAP

↓

required?

CONCEPT MAP

↓

email valid?

CONCEPT MAP

↓

age valid?

CONCEPT MAP

↓

password length?

CONCEPT MAP

↓

Submit

58. Form action and method

Important backend concept.

HTML FOUNDATIONS TO ADVANCED

```
<form
    action="/register"
    method="POST"
```

>

action

Where the form data goes.

method

How data is sent.

Common:

GET

POST

GET vs POST

GET	POST
Retrieve data	Send/create data
Parameters commonly in URL	Data commonly in request body
Bookmarkable	Usually not represented by URL
Should not be used for passwords	Suitable for sensitive form submission when HTTPS is used

Example:

HTML FOUNDATIONS TO ADVANCED

```
<form action="/search" method="GET">
```

versus:

HTML FOUNDATIONS TO ADVANCED

```
<form action="/register" method="POST">
```

59. HTML + FastAPI

A form can communicate with your backend.

HTML

CONCEPT MAP

```
  │
  │ HTTP Request
  ▼
```

FastAPI

CONCEPT MAP

```
  │
  ├── Validate
  ├── Business logic
  ├── Database
  └── ML model
  │
  ▼
```

Response

CONCEPT MAP

```
  │
  ▼
```

HTML

Example HTML:

HTML FOUNDATIONS TO ADVANCED

```
<form
    action="/predict"
    method="POST"
```

>

HTML FOUNDATIONS TO ADVANCED

```
    <label for="age">
        Age
    </label>

    <input
        type="number"
        id="age"
        name="age"
        required
    >

    <button type="submit">
        Predict
    </button>

</form>
```

60. Audio

HTML FOUNDATIONS TO ADVANCED

```
<audio controls>

    <source
        src="music.mp3"
        type="audio/mpeg"
    >

    Your browser does not support audio.

</audio>
```

61. Video

HTML FOUNDATIONS TO ADVANCED

```
<video
    controls
    width="600"
```

>

HTML FOUNDATIONS TO ADVANCED

```
    <source
        src="video.mp4"
        type="video/mp4"
    >

    Your browser does not support video.

</video>
```

Useful attributes:

HTML FOUNDATIONS TO ADVANCED

```
<video
    controls
    autoplay
    muted
    loop
    poster="thumbnail.jpg"
```

>

Browser autoplay policies can restrict unmuted autoplay.

62. Figure and Caption

Semantic image structure:

HTML FOUNDATIONS TO ADVANCED

```
<figure>

    <img
        src="model.png"
        alt="Machine learning architecture"
    >

    <figcaption>
        Machine Learning Pipeline
    </figcaption>

</figure>
```

63. iframe

Used to embed another document/page or supported external content.

HTML FOUNDATIONS TO ADVANCED

```
<iframe
    src="https://example.com"
    title="Example website"
```

>

HTML FOUNDATIONS TO ADVANCED

```
</iframe>
```

For maps/videos, providers usually give embed code.

64. <details> and <summary>

Very useful for FAQs.

HTML FOUNDATIONS TO ADVANCED

```
<details>

    <summary>
        What is HTML?
    </summary>

    <p>
        HTML structures web pages.
    </p>

</details>
```

Result concept:

 What is HTML?

click

CONCEPT MAP

```
▼ What is HTML?
```

HTML structures web pages.

65. Dialog

Modern HTML has a native dialog element.

HTML FOUNDATIONS TO ADVANCED

```
<dialog open>

    <h2>Hello</h2>

    <p>
        This is a dialog.
    </p>

</dialog>
```

JavaScript is generally used to open and close dialogs dynamically.

66. Data Attributes

Custom data can be stored using data-*.

HTML FOUNDATIONS TO ADVANCED

```
<button
    data-product-id="101"
    data-category="laptop"
```

>

HTML FOUNDATIONS TO ADVANCED

```
    Buy
</button>
```

JavaScript can later read:

data-product-id

data-category

> Important

Use data-* for custom metadata, not arbitrary attributes.

67. Meta Tags

Meta tags provide information about the document.

Character encoding

HTML FOUNDATIONS TO ADVANCED

```
<meta charset="UTF-8">
```

Responsive viewport

HTML FOUNDATIONS TO ADVANCED

```
<meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
```

>

Description

HTML FOUNDATIONS TO ADVANCED

```
<meta
    name="description"
    content="AI/ML Engineer portfolio"
```

>

68. Responsive Viewport

This is extremely important.

HTML FOUNDATIONS TO ADVANCED

```
<meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
```

>

Meaning:

Mobile screen

CONCEPT MAP

```
      ↓
```

Use device width

CONCEPT MAP

```
      ↓
```

Page scales correctly

Without a proper viewport configuration, mobile layouts can behave incorrectly.

69. SEO Basics

HTML FOUNDATIONS TO ADVANCED

```
SEO = Search Engine Optimization.
```

> Easy meaning:

Making your website easier for search engines and users to understand.

Important HTML elements:

HTML FOUNDATIONS TO ADVANCED

```
<title>
<meta description>
<h1>
<h2>
```

semantic HTML

HTML FOUNDATIONS TO ADVANCED

```
<img alt="">
```

links

> Example:

HTML FOUNDATIONS TO ADVANCED

```
<head>

    <title>
        Aamir Hussain Azad | AI/ML Engineer
    </title>

    <meta
        name="description"
        content="AI/ML Engineer portfolio featuring Python, machine learning and AI projects."
    >

</head>
```

70. Accessibility

Accessibility means:

Making websites usable by people with different abilities.

Important practices:

Use labels

HTML FOUNDATIONS TO ADVANCED

```
<label for="email">
    Email
</label>

<input
    id="email"
    type="email"
```

>

Use alt text

HTML FOUNDATIONS TO ADVANCED

```
<img
    src="profile.jpg"
    alt="Profile photograph"
```

>

Use semantic HTML

HTML FOUNDATIONS TO ADVANCED

```
<nav>
<main>
<section>
<article>
<footer>
```

Use buttons for actions

Good:

HTML FOUNDATIONS TO ADVANCED

```
<button>
    Submit
</button>
```

Don't use a random as a button unless you have a strong reason and implement proper keyboard/accessibility behavior.

71. HTML Entities

Some characters have special meanings.

> Example:

<

>

&

"

Meaning:

Entity	Character
<	<
>	>
&	&
"	"
	non-breaking space

> Example:

HTML FOUNDATIONS TO ADVANCED

```
<p>
    5 &lt; 10
</p>
```

72. Character Encoding

Use:

HTML FOUNDATIONS TO ADVANCED

```
<meta charset="UTF-8">
```

UTF-8 allows the document to represent a very large range of characters.

> Example:

HTML FOUNDATIONS TO ADVANCED

```
<p>Hello        </p>
```

73. Block vs Inline

Block-level behavior

Examples:

div

p

h1

section

article

header

footer

Concept:

CONCEPT MAP



Inline behavior

Examples:

span

a

strong

em

Concept:

Text [link] [important] continues

Modern CSS layout can change how elements participate in layout, so don't treat “block vs inline” as an absolute visual rule.

74. DOM — Very Important

HTML FOUNDATIONS TO ADVANCED

```
DOM = Document Object Model
```

> Easy meaning:

Browser converts HTML into a tree of objects that JavaScript can work with.

HTML:

HTML FOUNDATIONS TO ADVANCED

```
<body>

    <h1>Hello</h1>

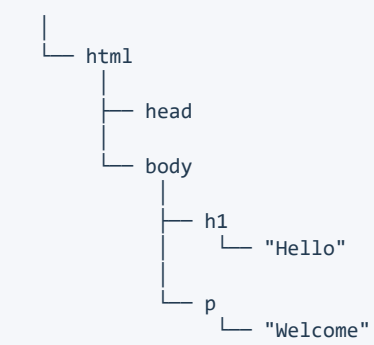
    <p>Welcome</p>

</body>
```

DOM:

Document

CONCEPT MAP



75. DOM Parent-Child Relationship

HTML FOUNDATIONS TO ADVANCED

```
<div>

    <p>
        Hello
    </p>

</div>
```

Tree:

div

CONCEPT MAP

```
  │
  └── p
       │
       └── Hello
```

Therefore:

HTML FOUNDATIONS TO ADVANCED

```
div = parent
p   = child
Hello = text node
```

76. HTML Parsing

Browser process:

HTML file

CONCEPT MAP

```
    ↓
```

Parser

CONCEPT MAP

```
    ↓
```

DOM Tree

CONCEPT MAP

```
    ↓
```

CSS + layout

CONCEPT MAP

```
    ↓
```

Rendering

CONCEPT MAP

```
    ↓
```

Web page

Very simplified mental model:

HTML

CONCEPT MAP

↓

DOM

CSS

CONCEPT MAP

↓

CSSOM

DOM + CSSOM

CONCEPT MAP

↓

Render information

CONCEPT MAP

↓

Pixels on screen

77. Global Attributes

Many HTML elements can use common attributes.

Examples:

id

HTML FOUNDATIONS TO ADVANCED

```
class
```

title

style

hidden

lang

dir

data-*

78. id

Unique identifier within a document.

HTML FOUNDATIONS TO ADVANCED

```
<section id="about">
```

Use for:

anchors

JavaScript targeting

CSS targeting

> Example:

HTML FOUNDATIONS TO ADVANCED

```
<a href="#about">
    About
</a>

<section id="about">
    <h2>About Me</h2>
</section>
```

79. class

Used to group elements.

HTML FOUNDATIONS TO ADVANCED

```
<p class="highlight">
    Hello
</p>

<p class="highlight">
    World
</p>
```

Unlike id, the same class can be used on many elements.

> Remember

HTML FOUNDATIONS TO ADVANCED

```
id = one unique identity
class = reusable group
```

80. title

HTML FOUNDATIONS TO ADVANCED

```
<button title="Submit your form">
    Submit
</button>
```

Provides advisory information, often shown as a tooltip by browsers.

Don't rely on title as the only accessibility label.

81. lang

HTML FOUNDATIONS TO ADVANCED

```
<html lang="en">
```

For Hindi content:

HTML FOUNDATIONS TO ADVANCED

```
<html lang="hi">
```

This helps browsers, screen readers, search engines, and other tools understand the document language.

82. HTML Security Concepts

HTML itself does not make an application secure.

Important concepts:

XSS

Cross-Site Scripting.

> Easy meaning:

An attacker gets unwanted JavaScript executed in another user's browser.

Example danger:

User input

CONCEPT MAP

```
↓
```

HTML output

CONCEPT MAP

```
↓
```

Unsafe script

CONCEPT MAP

```
↓
```

Browser executes it

Prevent with proper output encoding/sanitization and secure application architecture.

83. Never Trust Client-Side Validation

Suppose:

HTML FOUNDATIONS TO ADVANCED

```
<input
    type="number"
    min="18"
```

>

This is useful.

But an attacker can bypass browser validation.

Therefore:

HTML validation

CONCEPT MAP

```
        ↓
```

User experience

Backend validation

CONCEPT MAP

```
        ↓
```

Actual security

 Remember

Frontend validation improves UX. Backend validation protects the application.

84. File Upload

HTML FOUNDATIONS TO ADVANCED

```
<form
    action="/upload"
    method="POST"
    enctype="multipart/form-data"
```

>

HTML FOUNDATIONS TO ADVANCED

```
    <label for="file">
        Upload file
    </label>

    <input
        type="file"
        id="file"
        name="file"
    >

    <button type="submit">
        Upload
    </button>

</form>
```

> Important:

HTML FOUNDATIONS TO ADVANCED

```
enctype="multipart/form-data"
```

is required for traditional multipart file uploads.

85. HTML Comments vs Hidden Input

A comment:

HTML FOUNDATIONS TO ADVANCED

```
<!-- secret -->
```

is still sent as part of the HTML source.

A hidden input:

HTML FOUNDATIONS TO ADVANCED

```
<input
    type="hidden"
    name="user_id"
    value="123"
```

>

is also visible to the client.

> Important

Never put secrets, passwords, private API keys, or authorization decisions in HTML.

86. Complete Portfolio HTML Project

Here's a strong HTML-only foundation for a portfolio.

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta
        name="viewport"
        content="width=device-width, initial-scale=1.0"
    >

    <meta
        name="description"
        content="AI/ML Engineer portfolio"
    >

    <title>
        AI/ML Engineer Portfolio
    </title>

</head>

<body>

    <!-- ================= HEADER ================= -->

    <header>

        <h1>
            Aamir Hussain Azad
        </h1>
```

HTML FOUNDATIONS TO ADVANCED

```
        <p>
            AI/ML Engineer
        </p>

        <!-- ================= NAVIGATION ================= -->

        <nav aria-label="Main navigation">

            <a href="#home">
                Home
            </a>

            <a href="#about">
                About
            </a>

            <a href="#skills">
                Skills
            </a>

            <a href="#projects">
                Projects
            </a>

            <a href="#contact">
                Contact
            </a>

        </nav>

    </header>
```

```
    <!-- ================= MAIN ================= -->

    <main>

        <!-- ================= HOME ================= -->

        <section id="home">

            <h2>
                Hello, I'm Aamir
            </h2>

            <p>
                I build AI and machine learning applications.
            </p>

            <a href="#projects">
                View My Projects
            </a>

        </section>
```



```
        <!-- ================= ABOUT ================= -->

        <section id="about">

            <h2>
                About Me
            </h2>

            <figure>

                <img
                    src="profile.jpg"
                    alt="Profile photograph"
                    width="250"
                >

                <figcaption>
                    AI/ML Engineer
                </figcaption>

            </figure>

            <p>
                I work with Python, machine learning,
                data analysis and AI technologies.
            </p>

        </section>
```

```
        <!-- ================= SKILLS ================= -->

        <section id="skills">

            <h2>
                Skills
            </h2>

            <ul>

                <li>Python</li>

                <li>SQL</li>

                <li>NumPy</li>

                <li>Pandas</li>

                <li>Scikit-learn</li>

                <li>Machine Learning</li>

                <li>FastAPI</li>

                <li>Streamlit</li>

            </ul>

        </section>
```

HTML FOUNDATIONS TO ADVANCED

```
        <!-- ================= PROJECTS ================= -->

        <section id="projects">

            <h2>
                Projects
            </h2>
```

HTML FOUNDATIONS TO ADVANCED

```
            <article>

                <h3>
                    Crop Yield Prediction
                </h3>

                <p>
                    Machine learning application
                    for crop yield prediction.
                </p>

                <a href="#">
                    View Project
                </a>

            </article>
```

HTML FOUNDATIONS TO ADVANCED

```
            <article>

                <h3>
                    AI RAG Chatbot
                </h3>

                <p>
                    Retrieval-Augmented Generation
                    based AI chatbot.
                </p>

                <a href="#">
                    View Project
                </a>

            </article>
```

HTML FOUNDATIONS TO ADVANCED

```
            <article>

                <h3>
                    Sales Analytics Dashboard
                </h3>

                <p>
                    Interactive data analytics
                    dashboard.
                </p>

                <a href="#">
                    View Dashboard
                </a>

            </article>

        </section>
```

HTML FOUNDATIONS TO ADVANCED

```
        <!-- ================= EXPERIENCE ================= -->

        <section id="experience">

            <h2>
                Experience
            </h2>

            <article>

                <h3>
                    Frontend Developer Intern
                </h3>

                <p>
                    Worked on frontend development
                    using modern web technologies.
                </p>

            </article>

        </section>
```

HTML FOUNDATIONS TO ADVANCED

```
        <!-- ================= CONTACT ================= -->

        <section id="contact">

            <h2>
                Contact Me
            </h2>

            <form
                action="/contact"
                method="POST"
            >

                <div>

                    <label for="name">
                        Name
                    </label>

                    <input
                        type="text"
                        id="name"
                        name="name"
                        required
                    >

                </div>

                <br>

                <div>

                    <label for="email">
                        Email
```



```
                    </label>

                    <input
                        type="email"
                        id="email"
                        name="email"
                        required
                    >

                </div>

                <br>

                <div>

                    <label for="message">
                        Message
                    </label>

                    <textarea
                        id="message"
                        name="message"
                        rows="5"
                        required
                    ></textarea>

                </div>

                <br>

                <button type="submit">
                    Send Message
                </button>
```

HTML FOUNDATIONS TO ADVANCED

```
            </form>

        </section>

    </main>
```

HTML FOUNDATIONS TO ADVANCED

```
    <!-- ================= FOOTER ================= -->

    <footer>

        <p>
            © 2026 Aamir Hussain Azad
        </p>

        <nav aria-label="Social links">

            <a href="#">
                GitHub
            </a>

            <a href="#">
                LinkedIn
            </a>

        </nav>

    </footer>

</body>

</html>
```

87. Complete Portfolio Architecture

HTML PORTFOLIO

CONCEPT MAP

```
│
├── HEADER
│    ├── Name
│    └── Navigation
│
├── MAIN
│    │
│    ├── HOME
│    │
│    ├── ABOUT
│    │    └── Image
│    │
│    ├── SKILLS
│    │    └── List
│    │
│    ├── PROJECTS
│    │    └── Articles
│    │
│    ├── EXPERIENCE
│    │
│    └── CONTACT
│         └── Form
│
└── FOOTER
     ├── Copyright
     └── Social links
```

This is a very good structure to memorize.

88. HTML Attribute Cheat Sheet

Attribute	Easy meaning
id	Unique identity
class	Group/category
src	Resource location
href	Link destination
alt	Image alternative text
title	Advisory information
name	Form field name
value	Form value
placeholder	Example hint
required	Must provide
disabled	Cannot interact
readonly	Cannot edit
checked	Checkbox/radio selected
selected	Option selected
min	Minimum
max	Maximum
step	Increment
pattern	Validation pattern
target	Link browsing target
loading	Loading behavior
width	Width
height	Height

CONCEPT MAP

```
89. Difficult Words → Easy Meaning
```

Difficult	Easy
Markup	Tags describing content
Semantic	Meaningful
Attribute	Extra information
Element	Complete HTML unit
DOM	HTML represented as a tree
Accessibility	Usable by different people
Responsive	Works across screen sizes
Metadata	Information about information
Validation	Checking input
Encoding	Representing data correctly

Difficult	Easy
Rendering	Turning code into visible page
Parser	Software that understands HTML
SEO	Helping search engines understand content
Endpoint	Destination of an API request
Client	User/browser/application making request
Server	Computer/application responding
Multipart	Request containing multiple data parts
Entity	A meaningful thing/content object
Semantic HTML	HTML whose tags explain meaning

90. Colorful Revision Notes

BASIC

HTML creates structure.

> IMPORTANT

id should identify one unique element; class is reusable.

INTERVIEW

Semantic HTML improves structure, accessibility, and machine understanding.

SECURITY

Never trust HTML/JavaScript validation alone.

AI/ML

HTML can provide the frontend while FastAPI handles prediction/API logic.

PERFORMANCE

Use appropriate image dimensions, lazy loading where useful, and avoid unnecessary resources.

91. HTML Memory Tricks

Trick 1 — Web Stack

H-C-J

CONCEPT MAP

```
H = HTML → Structure
C = CSS → Style
J = JavaScript → Behavior
```

Trick 2 — Page Structure

> Remember:

H-N-M-S-A-F

CONCEPT MAP

```
H → Header
N → Nav
M → Main
S → Section
A → Article/Aside
F → Footer
```

Trick 3 — Form

> Remember:

L-I-B

```
HTML FOUNDATIONS TO ADVANCED
```

```
L = Label
I = Input
B = Button
```

Trick 4 — Image

> Remember:

S-A

```
HTML FOUNDATIONS TO ADVANCED
```

```
S = src
A = alt
```

Trick 5 — Table

> Remember:

T-R-C

```
HTML FOUNDATIONS TO ADVANCED
```

```
T = Table
R = Row
C = Cell
```

More accurately:

table

```
CONCEPT MAP
```

```
  ↓
```

tr

```
CONCEPT MAP
```

```
  ↓
```

th / td

92. Common Mistakes

Mistake 1

```
HTML FOUNDATIONS TO ADVANCED
```

```
<img src="photo.jpg">
```

Better:

```
HTML FOUNDATIONS TO ADVANCED
```

```
<img
    src="photo.jpg"
    alt="Profile photograph"
```

>

Mistake 2

Using many
 tags for layout.

HTML FOUNDATIONS TO ADVANCED

```
<br>
<br>
<br>
<br>
```

Use CSS for spacing.

Mistake 3

Using everywhere.

HTML FOUNDATIONS TO ADVANCED

```
<div class="header"></div>
<div class="nav"></div>
<div class="footer"></div>
```

Prefer semantic elements where appropriate:

HTML FOUNDATIONS TO ADVANCED

```
<header></header>
<nav></nav>
<footer></footer>
```

Mistake 4

Duplicate IDs.

HTML FOUNDATIONS TO ADVANCED

```
<div id="card"></div>
<div id="card"></div>
```

Use classes for repeated elements:

HTML FOUNDATIONS TO ADVANCED

```
<div class="card"></div>
<div class="card"></div>
```

Mistake 5

No labels.

HTML FOUNDATIONS TO ADVANCED

```
<input type="email">
```

Better:

HTML FOUNDATIONS TO ADVANCED

```
<label for="email">
    Email
</label>

<input
    id="email"
    type="email"
```

>

Mistake 6

Putting secrets in HTML.

Never put:

API keys

Passwords

Private tokens

Database credentials

into HTML.

93. HTML Interview Questions

Q1. What is HTML?

HTML is a markup language used to structure content on web pages.

Q2. Is HTML a programming language?

No. HTML is a markup language. It describes document structure rather than implementing general-purpose programming logic.

Q3. What is an HTML element?

An HTML element is a complete unit consisting of tags and content when applicable.

> Example:

HTML FOUNDATIONS TO ADVANCED

```
<p>Hello</p>
```

Q4. What is an attribute?

An attribute provides additional information about an HTML element.

HTML FOUNDATIONS TO ADVANCED

```
<a href="page.html">
```

href is an attribute.

Q5. Difference between id and class?

id identifies a specific element; class represents a reusable group of elements.

Q6. What is semantic HTML?

HTML elements that communicate the meaning or role of their content.

Examples:

HTML FOUNDATIONS TO ADVANCED

```
<header>
<nav>
<main>
<section>
<article>
<footer>
```

Q7. What is the DOM?

DOM stands for Document Object Model. It represents an HTML document as a tree of objects that scripts can interact with.

Q8. Difference between and ?

Traditionally, is a generic block container and is a generic inline container. Their actual layout can be changed with CSS.

Q9. Difference between and <b>?

HTML FOUNDATIONS TO ADVANCED

```
<strong> communicates strong importance semantically; <b> is primarily used to draw attention
without that same semantic meaning.
```

Q10. Difference between and <i>?

HTML FOUNDATIONS TO ADVANCED

```
<em> represents emphasis; <i> represents text set apart from normal prose for a different voice or
convention.
```

Q11. Why is alt important?

It provides alternative text for images, helping when images cannot be displayed and improving accessibility.

Q12. What does DOCTYPE do?

It tells the browser which document mode/HTML standard expectations to use. In modern HTML:

HTML FOUNDATIONS TO ADVANCED

```
<!DOCTYPE html>
```

Q13. What is responsive design?

Design that adapts appropriately to different screen sizes and devices.

Q14. What is the viewport meta tag?

HTML FOUNDATIONS TO ADVANCED

```
<meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
```

>

It helps control the viewport used for responsive layouts on mobile devices.

Q15. GET vs POST?

GET is generally used to retrieve data and POST is generally used to submit/create/process data. Security does not come from choosing POST alone; HTTPS and server-side controls are required.

Q16. What is required?

It tells the browser that the input must have a value before the form can be submitted normally.

Q17. What is placeholder?

HTML FOUNDATIONS TO ADVANCED

```
<input
    placeholder="Enter your name"
```

>

It provides a temporary hint. It should not replace a proper label.

Q18. What is readonly?

HTML FOUNDATIONS TO ADVANCED

```
<input
    value="Aamir"
    readonly
```

>

The user generally cannot edit the field through normal UI, but its value can still be submitted depending on the form.

Q19. Difference between disabled and readonly?

readonly	disabled
User cannot normally edit	User cannot interact
Can generally be submitted	Disabled controls are generally not submitted
Focus behavior differs	Usually excluded from normal interaction

Q20. What is accessibility?

Making a website usable by people with different abilities and assistive technologies.

94. Most Conceptual Interview Question

Why should we use semantic HTML instead of everywhere?

Answer:

Semantic HTML communicates the purpose of content to browsers, developers, search engines, and assistive technologies. It improves document structure, accessibility, maintainability, and can support SEO.

> Example:

HTML FOUNDATIONS TO ADVANCED

```
<div class="header">
```

Better:

HTML FOUNDATIONS TO ADVANCED

```
<header>
```

95. HTML Mastery Roadmap

LEVEL 1

HTML BASICS

CONCEPT MAP

```
    ↓
```

Tags

Elements

Attributes

Document structure

CONCEPT MAP

```
    ↓
```

LEVEL 2

TEXT + LINKS + IMAGES

CONCEPT MAP

```
    ↓
```

LEVEL 3

LISTS + TABLES

CONCEPT MAP

↓

LEVEL 4

FORMS

CONCEPT MAP

↓

LEVEL 5

SEMANTIC HTML

CONCEPT MAP

↓

LEVEL 6

MEDIA + EMBEDS

CONCEPT MAP

↓

LEVEL 7

ACCESSIBILITY + SEO

CONCEPT MAP

↓

LEVEL 8

DOM CONCEPT

CONCEPT MAP

↓

LEVEL 9

HTML + CSS

CONCEPT MAP

↓

LEVEL 10

HTML + JavaScript

CONCEPT MAP

↓

LEVEL 11

HTML + FastAPI

CONCEPT MAP

↓

LEVEL 12

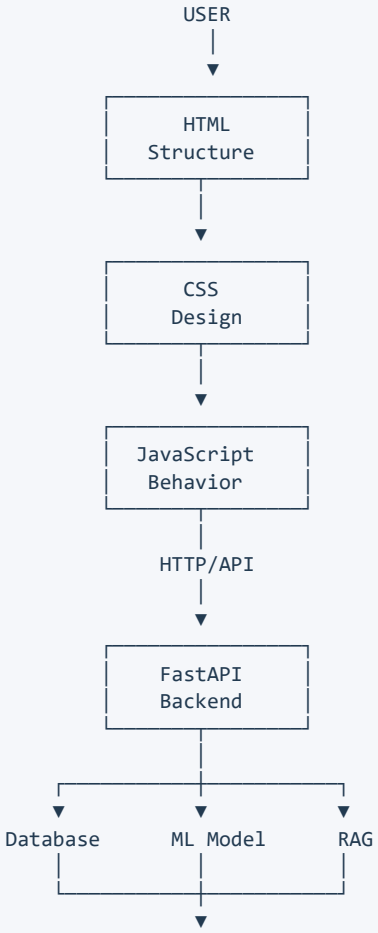
FULL PROJECT

CONCEPT MAP

```
96. HTML → CSS → JavaScript → FastAPI → AI/ML
```

For your broader AI/ML engineering path, understand this architecture:

CONCEPT MAP



CONCEPT MAP



97. The Ultimate HTML Cheat Sheet

HTML

CONCEPT MAP

```
│
├── Document
│   ├── <!DOCTYPE html>
│   ├── html
│   ├── head
│   └── body
│
├── Text
│   ├── h1-h6
│   ├── p
│   ├── strong
│   ├── em
│   ├── mark
│   ├── del
│   ├── ins
│   ├── sub
│   └── sup
│
├── Links
│   └── a
│
├── Media
│   ├── img
│   ├── audio
│   ├── video
│   ├── figure
│   └── iframe
│
├── Lists
│   ├── ul
│   ├── ol
│   ├── li
│   └── dl
│
```

CONCEPT MAP

```
├── Tables
│   ├── table
│   ├── tr
│   ├── th
│   ├── td
│   ├── thead
│   ├── tbody
│   └── tfoot
│
├── Forms
│   ├── form
│   ├── label
│   ├── input
│   ├── textarea
│   ├── select
│   ├── option
│   └── button
│
├── Semantic
│   ├── header
│   ├── nav
│   ├── main
│   ├── section
│   ├── article
│   ├── aside
│   └── footer
│
├── Advanced
│   ├── meta
│   ├── data-*
│   ├── details
│   ├── summary
│   ├── dialog
│   └── accessibility
```

CONCEPT MAP

```
│
└── Engineering
    ├── SEO
    ├── Security
    ├── Forms
    ├── APIs
    ├── DOM
    └── Deployment
```

□ Final Memory Formula

CONCEPT MAP

```
HTML = S-T-L-I-F-T-S
S → Structure
T → Text
L → Links
I → Images
F → Forms
T → Tables
S → Semantic HTML
```

Then master:

HTML

CONCEPT MAP

```
↓
```

CSS

CONCEPT MAP

```
↓
```

JavaScript

CONCEPT MAP

```
↓
```

HTTP/API

CONCEPT MAP

```
↓
```

FastAPI

CONCEPT MAP

```
↓
```

Database

CONCEPT MAP

```
↓
```

ML Model

CONCEPT MAP

```
↓
```

AI Application

To become genuinely strong in HTML, don't just memorize tags.

Build these 7 projects in order:

Personal Portfolio

Registration + Login UI

Product/Product Listing Page

Student Management Page

Admin Dashboard

AI/ML Prediction Frontend

HTML + JavaScript + FastAPI + ML full-stack application

CONCEPT MAP

```
If you can explain the DOM, semantic HTML, forms, validation, accessibility, attributes, HTTP form
submission, and HTML→CSS→JS→FastAPI flow, you have the conceptual foundation needed for serious
frontend/full-stack work.
```

PART 2 | CSS MASTERY — COMPLETE STUDY GUIDE

CONCEPT MAP

```
Beginner → Advanced | Concepts + Code + Diagrams + Practice + Interview
```

CSS is the language that controls how HTML looks and how elements are laid out.

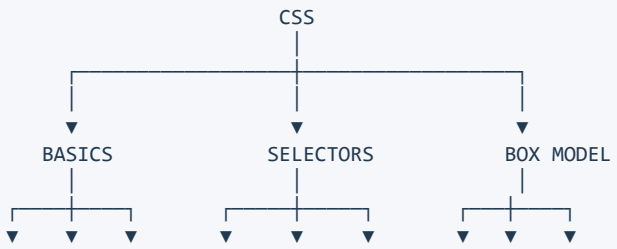
> Master formula:

CONCEPT MAP

```
HTML = Structure → CSS = Appearance/Layout → JavaScript = Behavior
```

1. COMPLETE CSS MIND MAP

CONCEPT MAP



Syntax Colors Element Class ID Margin Border Padding

Units Fonts Attribute Pseudo Width Height

CONCEPT MAP



Font / Text / Line

CONCEPT MAP



block / inline

inline-block

none

CONCEPT MAP



static / relative

absolute / fixed

sticky

CONCEPT MAP

```
        |
        ▼
     FLEXBOX
        |
```

container / items

direction / justify

align / gap

CONCEPT MAP

```
        |
        ▼
      GRID
        |
```

rows / columns

areas / gap

CONCEPT MAP

```
        |
        ▼
   RESPONSIVE
        |
```

media queries

mobile-first

CONCEPT MAP

```
        |
        ▼
    ADVANCED
        |
```

variables

CSS FOUNDATIONS TO ADVANCED

```
    calc()
    clamp()
```

transitions

transforms

animations

pseudo-elements

functions

CONCEPT MAP

```
        |
        ▼
   REAL WORLD
        |
```

Navbar

Cards

Forms

Dashboard

Portfolio

Responsive UI

2. What is CSS?

CSS FOUNDATIONS TO ADVANCED

```
CSS = Cascading Style Sheets
```

> Easy meaning

CSS tells the browser how HTML elements should look and where they should appear.

> Example:

HTML

CSS FOUNDATIONS TO ADVANCED

```
<h1>Hello Aamir</h1>
```

CSS

h1 {

CSS FOUNDATIONS TO ADVANCED

```
    color: blue;
    font-size: 40px;
```

}

Result:

CONCEPT MAP

```
      HTML
       ↓
  ┌─────────────┐
  │ Hello Aamir │
  └─────────────┘
       ↑
      CSS
```

color + size

3. CSS Syntax

Basic CSS:

selector {

CSS FOUNDATIONS TO ADVANCED

```
    property: value;
```

}

> Example:

p {

CSS FOUNDATIONS TO ADVANCED

```
    color: red;
    font-size: 20px;
```

}

Break it:

p

CONCEPT MAP

```
│
└── selector
```

color

CONCEPT MAP

```
│
└── property
```

red

CONCEPT MAP

```
│
└── value
```

A complete line:

CSS FOUNDATIONS TO ADVANCED

```
color: red;
```

is called a declaration.

4. Important CSS Vocabulary

Difficult word	Easy meaning
Selector	Element you want to style
Property	What you want to change
Value	Setting of that property
Declaration	Property + value
Rule	Selector + declarations
Cascade	Rules used to decide final style
Specificity	How strongly a selector targets something
Inheritance	Some properties pass from parent to child
Responsive	Works on different screen sizes
Pseudo-class	Special state of an element
Pseudo-element	Virtual part of an element
Viewport	Visible browser area
Layout	Arrangement of elements
Flexbox	One-dimensional layout system
Grid	Two-dimensional layout system

5. Three Ways to Add CSS

1. Inline CSS

CSS FOUNDATIONS TO ADVANCED

```
<p style="color: red;">
    Hello
</p>
```

Avoid for large applications.

2. Internal CSS

CSS FOUNDATIONS TO ADVANCED

```
<head>

<style>
```

p {

CSS FOUNDATIONS TO ADVANCED

```
    color: red;
```

}

CSS FOUNDATIONS TO ADVANCED

```
</style>

</head>
```

3. External CSS □

HTML:

CSS FOUNDATIONS TO ADVANCED

```
<link
    rel="stylesheet"
    href="style.css"
```

>

CSS:

p {

CSS FOUNDATIONS TO ADVANCED

```
    color: red;
```

}

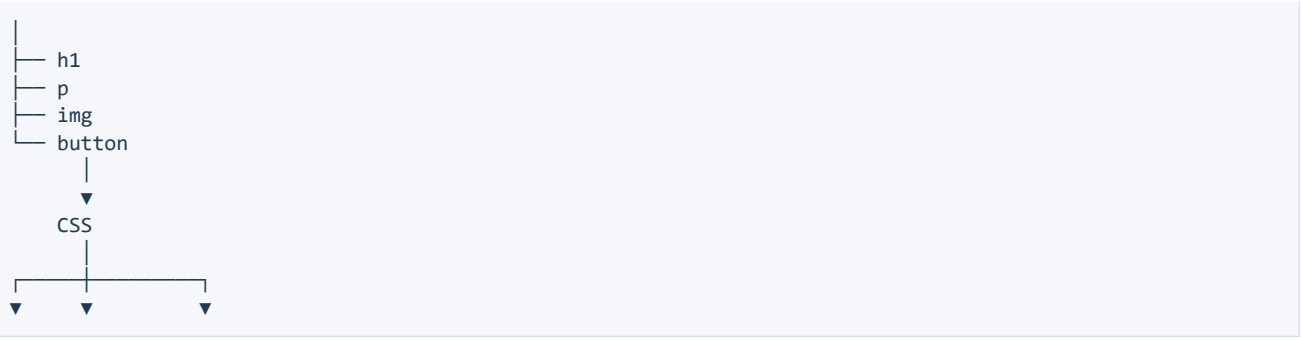
> Best practice

Use external CSS for maintainable projects.

6. CSS Relationship With HTML

HTML

CONCEPT MAP



Color Size Layout

7. Selectors

Selectors are one of the most important CSS topics.

Element Selector

p {

CSS FOUNDATIONS TO ADVANCED

```
    color: blue;
```

}

Selects all <p> elements.

Class Selector

HTML:

CSS FOUNDATIONS TO ADVANCED

```
<p class="important">
    Hello
</p>
```

CSS:

CSS FOUNDATIONS TO ADVANCED

```
.important {
    color: red;
```

}

. means class.

ID Selector

CSS FOUNDATIONS TO ADVANCED

```
<p id="main-text">
    Hello
</p>
```

CSS:

CSS FOUNDATIONS TO ADVANCED

```
#main-text {
    color: green;
```

}

means ID.

8. Selector Memory Trick

> Remember:

CONCEPT MAP

```
tag     → p
class   → .
id      → #
```

> Example:

p {}

CSS FOUNDATIONS TO ADVANCED

```
.card {}
#header {}
```

9. Universal Selector

- {

CSS FOUNDATIONS TO ADVANCED

```
    margin: 0;
    padding: 0;
```

}

- means all elements.

10. Group Selector

h1,

h2,

p {

CSS FOUNDATIONS TO ADVANCED

```
    color: blue;
```

}

Means:

CONCEPT MAP

```
h1 → blue
h2 → blue
p  → blue
```

11. Descendant Selector

HTML:

CSS FOUNDATIONS TO ADVANCED

```
<div class="card">

    <p>Hello</p>

</div>
```

CSS:

.card p {

CSS FOUNDATIONS TO ADVANCED

```
    color: red;
```

}

Meaning:

Select <p> elements inside .card.

Diagram:

.card

CONCEPT MAP

```
  │
  └── p
      ↑
    selected
```

12. Child Selector

.card > p {

CSS FOUNDATIONS TO ADVANCED

```
    color: red;
```

}

> means direct child.

.card

CONCEPT MAP

```
  │
  ├── p       ← selected
  │
  └── div
       │
       └── p ← not selected
```

13. Adjacent Sibling

h2 + p {

CSS FOUNDATIONS TO ADVANCED

```
    color: red;
```

}

Means:

Select the first <p> immediately after <h2>.

14. General Sibling

h2 ~ p {

CSS FOUNDATIONS TO ADVANCED

```
    color: red;
```

}

Means:

Select later sibling <p> elements after <h2>.

15. Attribute Selectors

input[type="email"] {

CSS FOUNDATIONS TO ADVANCED

```
    border: 2px solid blue;
```

}

Other examples:

input[required] {}

a[href] {}

input[type="text"] {}

16. Pseudo-Classes

Pseudo-class represents a state or condition.

Examples:

button:hover {

CSS FOUNDATIONS TO ADVANCED

```
    background: black;
```

}

Common:

:hover

:focus

:active

:visited

:checked

:disabled

:first-child

:last-child

:nth-child()

:not()

17. :hover

button:hover {

CSS FOUNDATIONS TO ADVANCED

```
    background-color: black;
    color: white;
```

}

Flow:

Normal

CONCEPT MAP

```
    ↓
```

Mouse enters

CONCEPT MAP

```
    ↓
```

:hover

CONCEPT MAP

```
    ↓
```

New style

 18. :focus

input:focus {

CSS FOUNDATIONS TO ADVANCED

```
    border-color: blue;
    outline: 2px solid lightblue;
```

}

Useful for keyboard and form interaction.

19. :nth-child()

li:nth-child(2) {

CSS FOUNDATIONS TO ADVANCED

```
    color: red;
```

}

Selects the second child.

> Example:

CONCEPT MAP

```
1 → normal
2 →  selected
3 → normal
4 → normal
```

20. :not()

p:not(.special) {

CSS FOUNDATIONS TO ADVANCED

```
    color: gray;
```

}

Means:

Select paragraphs that do NOT have .special.

21. Pseudo-Elements

Pseudo-elements represent a specific virtual part of an element.

Common:

::before

::after

::first-letter

::first-line

::selection

> Example:

h1::before {

```
CSS FOUNDATIONS TO ADVANCED
    content: " ";
```

}

22. ::before and ::after

.card::before {

```
CSS FOUNDATIONS TO ADVANCED
    content: "";
```

}

> Important:

content is normally required for generated content with ::before and ::after.

23. Colors

CSS supports many color formats.

Named color

```
CSS FOUNDATIONS TO ADVANCED
color: red;
```

HEX

```
CSS FOUNDATIONS TO ADVANCED
color: #ff0000;
```

RGB

```
CSS FOUNDATIONS TO ADVANCED
color: rgb(255, 0, 0);
```

RGBA

```
CSS FOUNDATIONS TO ADVANCED
color: rgba(255, 0, 0, 0.5);
```

HSL

```
CSS FOUNDATIONS TO ADVANCED
color: hsl(0, 100%, 50%);
```

24. HEX

Format:

RRGGBB

Example:

FF0000

CONCEPT MAP

```
│ │ │
│ │ └── Blue
│ └──── Green
└────── Red
```

25. CSS Units

Important units:

px

%

em

rem

vw

vh

vmin

vmax

26. px

Fixed CSS pixel unit.

CSS FOUNDATIONS TO ADVANCED

```
font-size: 20px;
```

27. %

Relative to an appropriate containing/reference size.

CSS FOUNDATIONS TO ADVANCED

```
width: 50%;
```

If parent content width is 1000px:

50% = 500px

28. rem

Relative to the root element's font size.

If:

html {

CSS FOUNDATIONS TO ADVANCED

```
    font-size: 16px;
```

}

Then:

2rem

means:

2 × 16px = 32px

> Formula

rem value = rem number × root font size

29. em

Relative to the relevant inherited/computed font size depending on the property.

For simple font-size examples:

CSS FOUNDATIONS TO ADVANCED

```
.parent {
    font-size: 20px;
```

}

CSS FOUNDATIONS TO ADVANCED

```
.child {
    font-size: 1.5em;
```

}

Then:

1.5 × 20px = 30px

Nested em calculations can compound, so rem is often easier for predictable typography.

30. vw and vh

CSS FOUNDATIONS TO ADVANCED

```
vw = viewport width.

width: 50vw;
vh = viewport height.

height: 50vh;
```

> Formula:

1vw = 1% of viewport width

1vh = 1% of viewport height

If viewport width = 1200px:

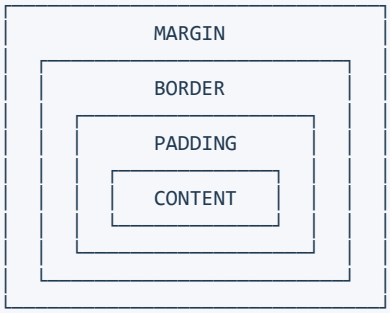
50vw = 600px

31. CSS Box Model □□□□□

One of the most important CSS concepts.

Every normal box can be understood as:

CONCEPT MAP



> Remember:

CONCEPT MAP

```
Content → Padding → Border → Margin
```

32. Box Model Formula

With default content-box:

Total width

Total Width =

Content Width

+ Left Padding

+ Right Padding

+ Left Border

+ Right Border

+ Left Margin

+ Right Margin

More precisely, the border-box width excludes margins.

If:

CSS FOUNDATIONS TO ADVANCED

```
content = 300px
```

padding-left = 20px

padding-right = 20px

border-left = 2px

border-right = 2px

Then border-box width:

300 + 20 + 20 + 2 + 2

= 344px

Margin is outside that box.

 33. box-sizing

Most modern projects use:

- {

CSS FOUNDATIONS TO ADVANCED

```
    box-sizing: border-box;
```

}

With:

box-sizing: border-box;

the declared width includes:

content + padding + border

> Example:

CSS FOUNDATIONS TO ADVANCED

```
.card {
    width: 300px;
    padding: 20px;
    border: 2px solid black;
    box-sizing: border-box;
```

}

The border-box remains 300px wide.

34. Margin vs Padding

Padding

Space inside the border.

Border

CONCEPT MAP

```
┌──────────────┐
│   PADDING    │
│   Content    │
└──────────────┘
```

Margin

Space outside the border.

CONCEPT MAP

```
        MARGIN
   ┌──────────────┐
   │   Border     │
   │   Content    │
   └──────────────┘
        MARGIN
```

Memory trick

CSS FOUNDATIONS TO ADVANCED

```
Padding = inside
Margin = outside
```

35. Margin Shorthand

CSS FOUNDATIONS TO ADVANCED

```
margin: 10px;
```

All sides.

CSS FOUNDATIONS TO ADVANCED

```
margin: 10px 20px;
```

Meaning:

top/bottom = 10px

left/right = 20px

CSS FOUNDATIONS TO ADVANCED

```
margin: 10px 20px 30px;
```

Meaning:

CSS FOUNDATIONS TO ADVANCED

```
top    = 10px
```

left/right = 20px

CSS FOUNDATIONS TO ADVANCED

```
bottom = 30px
margin: 10px 20px 30px 40px;
```

Clockwise:

TOP

RIGHT

BOTTOM

LEFT

Trick

CSS FOUNDATIONS TO ADVANCED

```
TRBL = Top Right Bottom Left
```

36. Width and Height

CSS FOUNDATIONS TO ADVANCED

```
.box {
    width: 300px;
    height: 200px;
```

}

Responsive:

CSS FOUNDATIONS TO ADVANCED

```
.box {
    width: 100%;
    max-width: 600px;
```

}

37. min-width, max-width

CSS FOUNDATIONS TO ADVANCED

```
.container {
    width: 100%;
    max-width: 1200px;
```

}

This is extremely common for responsive layouts.

38. Display Property

Important values:

block

inline

inline-block

none

flex

grid

39. display: block

div {

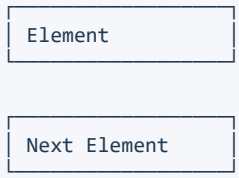
CSS FOUNDATIONS TO ADVANCED

```
    display: block;
```

}

Conceptually:

CONCEPT MAP



40. display: inline

span {

CSS FOUNDATIONS TO ADVANCED

```
    display: inline;
```

}

Conceptually:

Text [element] [element] continues

41. inline-block

Allows inline flow while permitting box dimensions.

button {

CSS FOUNDATIONS TO ADVANCED

```
    display: inline-block;
    width: 150px;
    padding: 10px;
```

}

42. display: none

CSS FOUNDATIONS TO ADVANCED

```
.hidden {
    display: none;
```

}

The element is removed from the layout.

Compare:

visibility: hidden;

This normally hides the element but preserves its layout space.

43. Position

CSS position values:

static

relative

absolute

fixed

sticky

44. position: static

Default positioning.

CSS FOUNDATIONS TO ADVANCED

```
.box {
    position: static;
```

}

Normal document flow.

45. position: relative

CSS FOUNDATIONS TO ADVANCED

```
.box {
    position: relative;
    top: 20px;
```

}

The element is positioned relative to its normal position.

It can also establish a containing block for absolutely positioned descendants.

46. position: absolute

CSS FOUNDATIONS TO ADVANCED

```
.child {
    position: absolute;
    top: 0;
    right: 0;
```

}

Usually positioned relative to the nearest positioned ancestor.

Example:

CSS FOUNDATIONS TO ADVANCED

```
.parent {
    position: relative;
```

}

CSS FOUNDATIONS TO ADVANCED

```
.child {
    position: absolute;
    top: 10px;
    right: 10px;
```

}

Diagram:

Parent

CONCEPT MAP



47. position: fixed

Fixed relative to the viewport.

CSS FOUNDATIONS TO ADVANCED

```
.navbar {
    position: fixed;
    top: 0;
    width: 100%;
```

}

Common for:

fixed navigation

floating buttons

support widgets

48. position: sticky

header {

CSS FOUNDATIONS TO ADVANCED

```
    position: sticky;
    top: 0;
```

}

It behaves like normal flow until a scroll threshold is reached, then sticks within its containing scroll context.

Common for:

sticky headers

table headings

sidebars

49. Position Memory Trick

CONCEPT MAP

```
static   → normal
relative → move from itself
absolute → position within containing block
fixed    → viewport
sticky   → scroll + stick
```

50. Flexbox □□□□□

Flexbox is a one-dimensional layout system.

> Easy meaning:

Flexbox arranges items along one main axis at a time.

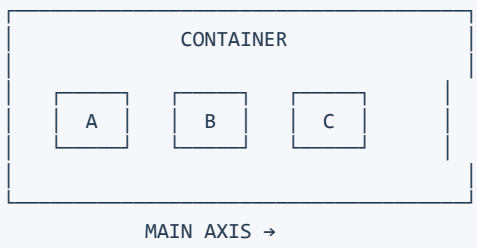
CSS FOUNDATIONS TO ADVANCED

```
.container {
    display: flex;
```

}

51. Flexbox Diagram

CONCEPT MAP



52. flex-direction

CSS FOUNDATIONS TO ADVANCED

```
.container {
    display: flex;
    flex-direction: row;
```

}

Options:

row

row-reverse

column

column-reverse

↔ 53. justify-content

Controls distribution along the main axis.

CSS FOUNDATIONS TO ADVANCED

```
.container {
    display: flex;
    justify-content: center;
```

}

Values:

flex-start

center

flex-end

space-between

space-around

space-evenly

↕ 54. align-items

Controls alignment along the cross axis.

CSS FOUNDATIONS TO ADVANCED

```
.container {
    align-items: center;
```

}

Memory trick

CSS FOUNDATIONS TO ADVANCED

```
justify = main axis
align = cross axis
```

55. gap

CSS FOUNDATIONS TO ADVANCED

```
.container {
    display: flex;
    gap: 20px;
```

}

Creates spacing between flex/grid items.

56. Perfect Centering

One of the most asked CSS problems.

CSS FOUNDATIONS TO ADVANCED

```
.container {
    display: flex;
    justify-content: center;
    align-items: center;
```

}

For a fixed-height container:

CSS FOUNDATIONS TO ADVANCED

```
.container {
    min-height: 100vh;

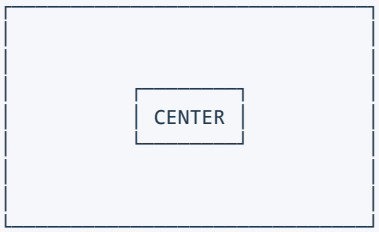
    display: flex;

    justify-content: center;
    align-items: center;
```

}

Diagram:

CONCEPT MAP



57. Flex Item Properties

Important:

flex-grow

flex-shrink

flex-basis

flex

order

align-self

Example:

CSS FOUNDATIONS TO ADVANCED

```
.item {
    flex: 1;
```

}

This commonly allows items to share available free space.

58. flex

Shorthand:

flex: grow shrink basis;

Example:

CSS FOUNDATIONS TO ADVANCED

```
.item {
    flex: 1 1 200px;
```

}

Meaning:

CSS FOUNDATIONS TO ADVANCED

```
grow   = 1
shrink = 1
basis  = 200px
```

59. CSS Grid □□□□□

Grid is a two-dimensional layout system.

Use it for:

rows + columns

> Example:

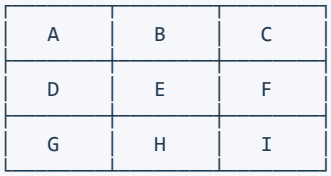
CSS FOUNDATIONS TO ADVANCED

```
.container {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 20px;
```

}

Diagram:

CONCEPT MAP



60. Flexbox vs Grid

Flexbox	Grid
One-dimensional	Two-dimensional
Row OR column	Rows AND columns
Components	Page/layout structure
Content-driven layouts	Structured layouts

Memory trick

CSS FOUNDATIONS TO ADVANCED

```
Flex = line
Grid = table
```

61. fr Unit

CSS FOUNDATIONS TO ADVANCED

```
grid-template-columns: 1fr 1fr 1fr;
```

Means three flexible tracks sharing available space.

> Example:

Available width = 900px

1fr + 1fr + 1fr

Each ≈ 300px

62. Responsive Grid

Very useful:

CSS FOUNDATIONS TO ADVANCED

```
.grid {
    display: grid;

    grid-template-columns:
        repeat(
            auto-fit,
            minmax(250px, 1fr)
        );

    gap: 20px;
```

}

Meaning:

Create as many columns as fit, while keeping each item at least about 250px wide and allowing it to grow.

This is a very powerful responsive pattern.

63. Media Queries

Responsive design:

@media (max-width: 768px) {

CSS FOUNDATIONS TO ADVANCED

```
    .container {
        flex-direction: column;
    }
```

}

Meaning:

Screen <= 768px

CONCEPT MAP

```
        ↓
```

Apply these CSS rules

64. Mobile-First CSS

Recommended strategy:

Mobile

CONCEPT MAP

```
    ↓
```

Tablet

CONCEPT MAP

```
    ↓
```

Desktop

Base:

CSS FOUNDATIONS TO ADVANCED

```
.container {
    display: block;
```

}

Then:

@media (min-width: 768px) {

CSS FOUNDATIONS TO ADVANCED

```
    .container {
        display: flex;
    }
```

}

65. Responsive Formula

A useful mental model:

Responsive UI

=

Flexible widths

+

Relative units

+

Flexible layout

+

Media queries

+

Responsive images

66. Responsive Images

img {

CSS FOUNDATIONS TO ADVANCED

```
    max-width: 100%;
    height: auto;
```

}

Meaning:

Image should not normally overflow its container.

67. Fonts

body {

CSS FOUNDATIONS TO ADVANCED

```
    font-family: Arial, sans-serif;
```

}

Common properties:

font-family

font-size

font-weight

font-style

line-height

letter-spacing

68. font-size

h1 {

CSS FOUNDATIONS TO ADVANCED

```
    font-size: 3rem;
```

}

69. font-weight

h1 {

CSS FOUNDATIONS TO ADVANCED

```
    font-weight: 700;
```

}

Common values:

CONCEPT MAP

```
400 → normal
700 → bold
```

↕ 70. line-height

p {

CSS FOUNDATIONS TO ADVANCED

```
    line-height: 1.6;
```

}

> Easy meaning:

Controls vertical spacing between lines of text.

↔ 71. letter-spacing

h1 {

CSS FOUNDATIONS TO ADVANCED

```
    letter-spacing: 2px;
```

}

72. Text Alignment

h1 {

CSS FOUNDATIONS TO ADVANCED

```
    text-align: center;
```

}

Options:

left

center

right

justify

73. Text Decoration

a {

CSS FOUNDATIONS TO ADVANCED

```
    text-decoration: none;
```

}

Other values:

underline

overline

line-through

none

74. Backgrounds

CSS FOUNDATIONS TO ADVANCED

```
.card {
    background-color: lightblue;
```

}

Image:

CSS FOUNDATIONS TO ADVANCED

```
.hero {
    background-image: url("hero.jpg");
```

}

75. Background Properties

CSS FOUNDATIONS TO ADVANCED

```
.hero {
    background-image: url("hero.jpg");

    background-size: cover;

    background-position: center;

    background-repeat: no-repeat;
```

}

cover

The image covers the area while preserving its aspect ratio; some cropping may occur.

76. Gradients

Linear gradient:

CSS FOUNDATIONS TO ADVANCED

```
.hero {
    background:
        linear-gradient(
            90deg,
            blue,
            purple
        );
```

}

Radial:

CSS FOUNDATIONS TO ADVANCED

```
.hero {
    background:
        radial-gradient(
            circle,
            white,
            blue
        );
```

}

77. Borders

CSS FOUNDATIONS TO ADVANCED

```
.card {
    border: 1px solid black;
```

}

Shorthand:

CSS FOUNDATIONS TO ADVANCED

```
border: width style color
```

Example:

CSS FOUNDATIONS TO ADVANCED

```
border: 2px solid red;
```

78. Border Radius

CSS FOUNDATIONS TO ADVANCED

```
.card {
    border-radius: 12px;
```

}

Circle:

CSS FOUNDATIONS TO ADVANCED

```
.avatar {
    border-radius: 50%;
```

}

79. Box Shadow

CSS FOUNDATIONS TO ADVANCED

```
.card {
    box-shadow:
        0 4px 20px rgba(0, 0, 0, 0.15);
```

}

General conceptual format:

offset-x

offset-y

blur-radius

spread-radius

color

80. CSS Variables

Very important for professional CSS.

:root {

CSS FOUNDATIONS TO ADVANCED

```
    --primary-color: #2563eb;

    --text-color: #111827;

    --radius: 12px;
```

}

Use:

button {

CSS FOUNDATIONS TO ADVANCED

```
    background: var(--primary-color);

    color: white;

    border-radius: var(--radius);
```

}

81. Why CSS Variables?

Without variables:

button {

CSS FOUNDATIONS TO ADVANCED

```
    background: blue;
```

}

CSS FOUNDATIONS TO ADVANCED

```
.card {
    border-color: blue;
```

}

CSS FOUNDATIONS TO ADVANCED

```
.link {
    color: blue;
```

}

With variables:

:root {

CSS FOUNDATIONS TO ADVANCED

```
    --primary: blue;
```

}

button {

CSS FOUNDATIONS TO ADVANCED

```
    background: var(--primary);
```

}

CSS FOUNDATIONS TO ADVANCED

```
.card {
    border-color: var(--primary);
```

}

CSS FOUNDATIONS TO ADVANCED

```
.link {
    color: var(--primary);
```

}

CONCEPT MAP

```
Change once → many places update.
```

82. calc()

CSS can calculate values.

CSS FOUNDATIONS TO ADVANCED

```
width: calc(100% - 40px);
```

> Example:

Parent width = 1000px

100% - 40px

= 960px

83. min(), max(), clamp()

CSS FOUNDATIONS TO ADVANCED

```
min()
width: min(90%, 1200px);
max()
width: max(300px, 50%);
clamp()
font-size: clamp(
    1.5rem,
    4vw,
    3rem
```

);

Concept:

minimum

CONCEPT MAP

```
    ↓
```

preferred

CONCEPT MAP

↓

maximum

84. CSS Cascade □□□□□

"Cascading" means CSS uses rules to determine which declarations win when multiple rules apply.

Important concepts:

Origin

Importance

Specificity

Source order

Inheritance

A simplified mental model for normal author CSS:

Which rules apply?

CONCEPT MAP

↓

!important?

CONCEPT MAP

↓

Specificity

CONCEPT MAP

↓

```
If tied → later declaration wins
```

85. Specificity

A useful conceptual ranking:

Inline style

CONCEPT MAP

↓

ID

CONCEPT MAP

↓

Class / attribute / pseudo-class

CONCEPT MAP

↓

Element / pseudo-element

> Example:

p {

```
    color: blue;
```

}


```
.text {
    color: green;
```

}


```
#main {
    color: red;
```

}

HTML:


```
<p
    id="main"
    class="text"
```

>


```
    Hello
</p>
```

The ID selector is more specific than the class and element selector, so red wins among these rules.

86. Specificity Score

A common teaching model is:

(a, b, c, d)

where:


```
a = inline styles
b = ID selectors
c = class/attribute/pseudo-class selectors
d = element/pseudo-element selectors
```

> Example:

box.card p

Score conceptually:


```
ID       = 1
class    = 1
element  = 1
```

(0,1,1,1)

Don't think of specificity as simple decimal arithmetic.

Compare the categories lexicographically.

87. !important

p {

CSS FOUNDATIONS TO ADVANCED

```
    color: red !important;
```

}

It changes the normal priority rules.

Avoid overusing it.

Bad:

CSS FOUNDATIONS TO ADVANCED

```
color: red !important;
margin: 10px !important;
padding: 20px !important;
```

Too much !important usually means the CSS architecture needs improvement.

88. Inheritance

Some properties naturally inherit from parents.

body {

CSS FOUNDATIONS TO ADVANCED

```
    color: #222;
    font-family: Arial;
```

}

Many descendant elements can inherit these properties.

Concept:

body

CONCEPT MAP

```
  │
  ├── h1 ← inherits applicable properties
  ├── p  ← inherits applicable properties
  └── a  ← inheritance depends on property/default styles
```

Not every property inherits.

89. CSS Transitions

Transition makes changes smoother.

button {

CSS FOUNDATIONS TO ADVANCED

```
    background: blue;

    transition:
        background-color 0.3s ease;
```

}

button:hover {

CSS FOUNDATIONS TO ADVANCED

```
    background: purple;
```

}

Without transition:

CONCEPT MAP

```
blue → purple
```

With transition:

CONCEPT MAP

```
blue → intermediate states → purple
```

90. Transform

.card:hover {

CSS FOUNDATIONS TO ADVANCED

```
    transform: translateY(-5px);
```

}

Common transforms:

CSS FOUNDATIONS TO ADVANCED

```
translate()
scale()
rotate()
skew()
```

> Example:

.card:hover {

CSS FOUNDATIONS TO ADVANCED

```
    transform: scale(1.05);
```

}

91. CSS Animation

@keyframes move {

CSS FOUNDATIONS TO ADVANCED

```
    from {
        transform: translateX(0);
    }

    to {
        transform: translateX(200px);
    }
```

}

CSS FOUNDATIONS TO ADVANCED

```
.box {
    animation:
        move 2s ease-in-out;
```

}

Flow:

CONCEPT MAP

```
from
 ↓
```

animation

CONCEPT MAP

```
 ↓
```

to

92. Repeating Animation

CSS FOUNDATIONS TO ADVANCED

```
.box {
    animation:
        move 2s infinite alternate;
```

}

Useful properties:

animation-name

animation-duration

animation-delay

animation-iteration-count

animation-direction

animation-timing-function

animation-fill-mode

93. CSS Functions

> Important:

CSS FOUNDATIONS TO ADVANCED

```
var()
calc()
min()
max()
clamp()
rgb()
rgba()
hsl()
url()
```

> Example:

CSS FOUNDATIONS TO ADVANCED

```
width: min(
    90%,
    1200px
```

);

94. Overflow

CSS FOUNDATIONS TO ADVANCED

```
.box {
    overflow: hidden;
```

}

Other values:

visible

hidden

scroll

auto

clip

> Example:

CSS FOUNDATIONS TO ADVANCED

```
.container {
    overflow-x: auto;
```

}

Useful for wide tables.

95. Opacity

CSS FOUNDATIONS TO ADVANCED

```
.card {
    opacity: 0.5;
```

}

Range concept:

CONCEPT MAP

```
0 → invisible
1 → fully opaque
```

96. visibility vs display

CSS FOUNDATIONS TO ADVANCED

```
display: none;
```

Usually removes the element from layout.

visibility: hidden;

Usually keeps its layout space while hiding it.

97. z-index

Controls stacking order when elements participate in stacking contexts where z-index applies.

CSS FOUNDATIONS TO ADVANCED

```
.modal {
    position: fixed;
    z-index: 1000;
```

}

Concept:

CONCEPT MAP

```
Layer 3 → Modal
Layer 2 → Navbar
Layer 1 → Page
```

> Important

z-index is not simply "bigger number always wins"; stacking contexts matter.

98. Cursor

button {

CSS FOUNDATIONS TO ADVANCED

```
    cursor: pointer;
```

}

Other examples:

default

pointer

not-allowed

grab

text

99. Object Fit

For images/video:

img {

CSS FOUNDATIONS TO ADVANCED

```
    width: 100%;
    height: 250px;
    object-fit: cover;
```

}

Useful values:

cover

contain

fill

none

scale-down

100. Real-World Card Component

HTML:

CSS FOUNDATIONS TO ADVANCED

```
<div class="card">

    <img
        src="product.jpg"
        alt="Laptop"
    >

    <div class="card-content">

        <h2>Laptop</h2>

        <p>
            Powerful laptop for developers.
        </p>

        <button>
            Buy Now
        </button>

    </div>

</div>
```

CSS:

CSS FOUNDATIONS TO ADVANCED

```
.card {

    width: 320px;

    border: 1px solid #ddd;

    border-radius: 16px;

    overflow: hidden;

    background: white;

    box-shadow:
        0 8px 25px rgba(0, 0, 0, 0.1);

    transition:
        transform 0.3s ease;
```

}

.card:hover {

CSS FOUNDATIONS TO ADVANCED

```
    transform:
        translateY(-8px);
```

}

.card img {

CSS FOUNDATIONS TO ADVANCED

```
    width: 100%;

    height: 200px;

    object-fit: cover;
```

}

CSS FOUNDATIONS TO ADVANCED

```
.card-content {

    padding: 20px;
```

}

.card button {

CSS FOUNDATIONS TO ADVANCED

```
    width: 100%;

    padding: 12px;

    border: none;

    border-radius: 8px;

    cursor: pointer;
```

}

101. Card Dry Run

.card

CONCEPT MAP

```
  │
  ├── image
  │     ↓
  │   width: 100%
  │   height: 200px
  │
  └── card-content
        ↓
      padding: 20px
        ↓
      heading
      paragraph
      button
```

When mouse enters:

.card

CONCEPT MAP

```
  ↓
```

:hover

CONCEPT MAP

```
  ↓
translateY(-8px)
  ↓
```

card moves upward

102. Professional Navbar

HTML:

CSS FOUNDATIONS TO ADVANCED

```
<header class="navbar">

    <a
        href="#"
        class="logo"
    >
        AI Portfolio
    </a>

    <nav>

        <a href="#home">
            Home
        </a>

        <a href="#about">
            About
        </a>

        <a href="#projects">
            Projects
        </a>

        <a href="#contact">
            Contact
        </a>

    </nav>

</header>
```

CSS:

CSS FOUNDATIONS TO ADVANCED

```
.navbar {

    display: flex;

    justify-content: space-between;

    align-items: center;

    padding: 16px 5%;

    background: white;

    border-bottom:
        1px solid #e5e7eb;

    position: sticky;

    top: 0;

    z-index: 100;
```

}

CSS FOUNDATIONS TO ADVANCED

```
.logo {

    font-size: 1.5rem;

    font-weight: 700;

    text-decoration: none;
```

}

.navbar nav {

CSS FOUNDATIONS TO ADVANCED

```
    display: flex;
    gap: 24px;
```

}

.navbar nav a {

CSS FOUNDATIONS TO ADVANCED

```
    text-decoration: none;
    color: #333;
```

}

.navbar nav a:hover {

CSS FOUNDATIONS TO ADVANCED

```
    color: blue;
```

}

103. Responsive Navbar

@media (max-width: 700px) {

CSS FOUNDATIONS TO ADVANCED

```
    .navbar {
        flex-direction: column;
        gap: 15px;
    }
    .navbar nav {
        flex-direction: column;
        align-items: center;
        gap: 10px;
    }
```

}

104. Professional Form

CSS FOUNDATIONS TO ADVANCED

```
<form class="form">

    <div class="form-group">

        <label for="name">
            Name
        </label>

        <input
            id="name"
            type="text"
            placeholder="Enter your name"
        >

    </div>

    <div class="form-group">

        <label for="email">
            Email
        </label>

        <input
            id="email"
            type="email"
            placeholder="Enter your email"
        >

    </div>

    <div class="form-group">

        <label for="message">
            Message
```

CSS FOUNDATIONS TO ADVANCED

```
        </label>

        <textarea
            id="message"
            rows="5"
            placeholder="Write your message"
        ></textarea>

    </div>

    <button
        type="submit"
        class="submit-btn"
    >
        Send
    </button>

</form>
```

CSS:

CSS FOUNDATIONS TO ADVANCED

```
.form {

    max-width: 600px;

    margin: 40px auto;

    padding: 30px;

    border-radius: 16px;

    box-shadow:
        0 10px 30px rgba(0, 0, 0, 0.1);
```

}

CSS FOUNDATIONS TO ADVANCED

```
.form-group {

    display: flex;

    flex-direction: column;

    gap: 8px;

    margin-bottom: 20px;
```

}

.form-group input,

.form-group textarea {

CSS FOUNDATIONS TO ADVANCED

```
    width: 100%;

    padding: 12px;

    border:
        1px solid #ccc;

    border-radius: 8px;

    font: inherit;
```

}

.form-group input:focus,

.form-group textarea:focus {

CSS FOUNDATIONS TO ADVANCED

```
    outline: 2px solid blue;

    border-color: blue;
```

}

CSS FOUNDATIONS TO ADVANCED

```
.submit-btn {

    width: 100%;

    padding: 12px;

    border: none;

    border-radius: 8px;

    cursor: pointer;
```

}

105. Dashboard Grid

CSS FOUNDATIONS TO ADVANCED

```
.dashboard {

    display: grid;

    grid-template-columns:
        repeat(
            auto-fit,
            minmax(220px, 1fr)
        );

    gap: 20px;

    padding: 30px;
```

}

CSS FOUNDATIONS TO ADVANCED

```
.metric-card {

    padding: 25px;

    border-radius: 15px;

    background: white;

    box-shadow:
        0 5px 20px rgba(0, 0, 0, 0.08);
```

}

Architecture:

Dashboard

CONCEPT MAP

```
│
├── Revenue
├── Profit
├── Customers
└── Orders
```

106. AI/ML Dashboard UI

For an AI/ML portfolio:

CONCEPT MAP

```
┌──────────────────────────────────────────┐
│              AI Dashboard                │
├──────────────────────────────────────────┤
│                                          │
│  Model Accuracy    Predictions           │
│     94%               1,250              │
│                                          │
├──────────────────────────────────────────┤
│                                          │
│        Prediction Form                   │
│                                          │
│    Feature 1 [____________]              │
│    Feature 2 [____________]              │
│    Feature 3 [____________]              │
│                                          │
│             [ Predict ]                  │
│                                          │
├──────────────────────────────────────────┤
│                Result                    │
└──────────────────────────────────────────┘
```

CSS can create the complete layout, while JavaScript/FastAPI handles interaction and ML inference.

107. CSS Architecture

For bigger applications, avoid putting everything into one huge file.

> Example:

css/

CONCEPT MAP

```
│
├── reset.css
├── variables.css
├── base.css
├── layout.css
├── components.css
├── utilities.css
└── responsive.css
```

Or a component-oriented structure:

styles/

CONCEPT MAP

```
├── globals.css
├── navbar.css
├── card.css
├── form.css
├── dashboard.css
└── responsive.css
```

108. CSS Reset

A simple reset:

- ,
- ::before,
- ::after {

CSS FOUNDATIONS TO ADVANCED

```
    box-sizing: border-box;
```

}

html {

CSS FOUNDATIONS TO ADVANCED

```
    scroll-behavior: smooth;
```

}

body {

CSS FOUNDATIONS TO ADVANCED

```
    margin: 0;
```

}

img,

picture,

video,

canvas,

svg {

CSS FOUNDATIONS TO ADVANCED

```
    display: block;
    max-width: 100%;
```

}

button,

input,

textarea,

select {

CSS FOUNDATIONS TO ADVANCED

```
    font: inherit;
```

}

This establishes more predictable defaults.

109. Design System With Variables

:root {

CSS FOUNDATIONS TO ADVANCED

```
    --primary: #2563eb;

    --secondary: #7c3aed;

    --text: #111827;

    --muted: #6b7280;

    --background: #f9fafb;

    --surface: #ffffff;

    --border: #e5e7eb;

    --radius-sm: 6px;

    --radius-md: 12px;

    --radius-lg: 20px;

    --shadow:
        0 10px 30px rgba(0, 0, 0, 0.08);
```

}

Then:

CSS FOUNDATIONS TO ADVANCED

```
.card {

    background: var(--surface);

    color: var(--text);

    border:
        1px solid var(--border);

    border-radius:
        var(--radius-md);

    box-shadow:
        var(--shadow);
```

}

110. Advanced CSS — Container Queries

Media queries usually react to viewport size.

Container queries can react to the size of a component's containing box.

Concept:

CSS FOUNDATIONS TO ADVANCED

```
.card-container {
    container-type: inline-size;
```

}

@container (min-width: 500px) {

CSS FOUNDATIONS TO ADVANCED

```
    .card {
        display: grid;
        grid-template-columns: 1fr 1fr;
    }
```

}

> Easy meaning:

Media query asks: How wide is the screen?

Container query asks: How wide is my container?

Very useful for reusable components.

111. Advanced CSS — aspect-ratio

CSS FOUNDATIONS TO ADVANCED

```
.video {
    aspect-ratio: 16 / 9;
```

}

> Formula:

aspect ratio = width / height

For 16 / 9:

CSS FOUNDATIONS TO ADVANCED

```
width : height
```

16 : 9

Useful for:

videos

cards

image containers

thumbnails

112. Advanced CSS — object-position

img {

CSS FOUNDATIONS TO ADVANCED

```
    width: 100%;
    height: 300px;

    object-fit: cover;

    object-position: center;
```

}

You can adjust the visible focal position.

113. Advanced CSS — Logical Properties

Instead of:

margin-left: 20px;

you can use:

margin-inline-start: 20px;

Other useful properties:

margin-inline

margin-block

padding-inline

padding-block

inset-inline

inset-block

These are useful for different writing directions and internationalization.

114. Accessibility in CSS

Don't remove focus without replacement

Avoid:

button {

CSS FOUNDATIONS TO ADVANCED

```
    outline: none;
```

}

unless you provide an accessible alternative.

Better:

button:focus-visible {

CSS FOUNDATIONS TO ADVANCED

```
    outline: 3px solid blue;
    outline-offset: 3px;
```

}

115. Reduced Motion

Some users prefer less animation.

Use:

@media (prefers-reduced-motion: reduce) {

CSS FOUNDATIONS TO ADVANCED

```
    *,
    *::before,
    *::after {

        animation-duration: 0.01ms;
        animation-iteration-count: 1;

        transition-duration: 0.01ms;

        scroll-behavior: auto;

    }
```

}

116. CSS Performance

Good practices:

Use efficient selectors

CONCEPT MAP

```
↓
```

Avoid unnecessary animations

CONCEPT MAP

↓

Animate transform/opacity when appropriate

CONCEPT MAP

↓

Compress images

CONCEPT MAP

↓

Remove unused CSS

CONCEPT MAP

↓

Use caching

For animations, properties such as:

transform

opacity

are often preferable to repeatedly animating expensive layout properties.

117. Common CSS Mistakes

Mistake 1

Using excessive absolute positioning for layouts.

CSS FOUNDATIONS TO ADVANCED

```
left: 200px;
top: 100px;
```

This can break on different screen sizes.

Better

Use:

Flexbox

Grid

responsive sizing

Mistake 2

Using fixed widths everywhere.

Bad:

CSS FOUNDATIONS TO ADVANCED

```
.container {
    width: 1200px;
```

}

Better:

CSS FOUNDATIONS TO ADVANCED

```
.container {
    width: 100%;
    max-width: 1200px;
    margin-inline: auto;
    padding-inline: 20px;
```

}

Mistake 3

Using too many !important.

Mistake 4

No mobile design.

Mistake 5

Removing keyboard focus.

Mistake 6

Using CSS as a substitute for semantic HTML.

CSS controls presentation; HTML should communicate structure and meaning.

118. CSS Dry Run — Box Model

Given:

CSS FOUNDATIONS TO ADVANCED

```
.box {

    width: 300px;

    padding: 20px;

    border: 5px solid black;

    margin: 10px;
```

}

With default content-box:

Property	Value
Content width	300px
Left padding	20px
Right padding	20px
Left border	5px
Right border	5px
Border-box width	350px
Left margin	10px
Right margin	10px
Outer occupied width	370px

Calculation:

300 + 20 + 20 + 5 + 5

= 350px

350 + 10 + 10

= 370px

119. Flexbox Dry Run

CSS FOUNDATIONS TO ADVANCED

```
.container {
    display: flex;
    justify-content: center;
    align-items: center;
```

}

Suppose:

Container width = 1000px

Container height = 500px

Item width = 200px

Item height = 100px

Concept:

Horizontal:

(1000 - 200) / 2

= 400px left space

Vertical:

(500 - 100) / 2

= 200px top space

Result:

CONCEPT MAP



120. Grid Dry Run

grid-template-columns:

CSS FOUNDATIONS TO ADVANCED

```
    repeat(3, 1fr);
```

Container:

CSS FOUNDATIONS TO ADVANCED

```
width = 900px
```

Ignoring gaps for simplicity:

900 / 3

= 300px

So:

Column 1 ≈ 300px

Column 2 ≈ 300px

Column 3 ≈ 300px

121. Responsive Dry Run

CSS FOUNDATIONS TO ADVANCED

```
.card {
    width: 100%;
```

}

@media (min-width: 768px) {

CSS FOUNDATIONS TO ADVANCED

```
    .card {
        width: 50%;
    }
```

}

Screen	Applied rule
400px	width: 100%
600px	width: 100%
768px	width: 50%
1000px	width: 50%

122. Most Asked CSS Interview Questions

Q1. What is CSS?

CSS is a stylesheet language used to control the presentation, layout, and visual appearance of HTML/XML documents.

Q2. What is the box model?

The CSS box model describes an element as:

Content

Padding

Border

Margin

Q3. Difference between margin and padding?

CSS FOUNDATIONS TO ADVANCED

```
Margin = outside the border.
Padding = inside the border.
```

Q4. What is box-sizing: border-box?

It makes the declared width/height include the element's padding and border.

Q5. What is Flexbox?

Flexbox is a one-dimensional CSS layout system for arranging items along a main axis with cross-axis alignment.

Q6. What is CSS Grid?

Grid is a two-dimensional layout system for rows and columns.

Q7. Flexbox vs Grid?

CSS FOUNDATIONS TO ADVANCED

```
Flexbox = one dimension.
Grid = two dimensions.
```

Q8. What is specificity?

Specificity is the selector-priority mechanism used when multiple applicable declarations compete for the same property.

Q9. Which is more specific?

p {}

CSS FOUNDATIONS TO ADVANCED

```
.card {}
#main {}
```

Generally:

main

CONCEPT MAP

```
   ↓
```

.card

CONCEPT MAP

```
   ↓
```

p

Q10. What is inheritance?

Some CSS properties can take their value from the parent element when no overriding value is specified.

Q11. What is a pseudo-class?

It represents a state or condition.

> Example:

button:hover {}

Q12. What is a pseudo-element?

It represents a conceptual part of an element.

> Example:

p::first-letter {}

Q13. Difference between display: none and visibility: hidden?

display: none removes the element from normal layout; visibility: hidden generally preserves its layout space.

Q14. What is position: absolute?

It removes the element from normal flow and positions it relative to its containing block.

Q15. What is position: fixed?

It positions an element relative to the viewport in the common case and keeps it fixed during normal page scrolling.

Q16. What is position: sticky?

It behaves like a relatively positioned element until a scroll threshold is reached, then sticks within its scroll container.

Q17. What is z-index?

It controls stacking order within the relevant stacking contexts.

Q18. What is responsive design?

Designing a UI so it adapts appropriately to different viewport sizes and devices.

Q19. What is a media query?

A CSS condition that applies styles when specified environmental conditions are true.

> Example:

@media (max-width: 768px) {}

Q20. What are CSS variables?

Reusable custom properties:

:root {

CSS FOUNDATIONS TO ADVANCED

```
    --primary: blue;
```

}

Use:

CSS FOUNDATIONS TO ADVANCED

```
color: var(--primary);
```

123. Most Conceptual Interview Question

Why does this not necessarily work?

CSS FOUNDATIONS TO ADVANCED

```
.parent {
    display: flex;
    align-items: center;
```

}

CSS FOUNDATIONS TO ADVANCED

```
.child {
    height: 100%;
```

}

Because height: 100% needs a definite containing height in the relevant context.

Better:

CSS FOUNDATIONS TO ADVANCED

```
.parent {
    min-height: 500px;

    display: flex;

    align-items: center;
```

}

Lesson

Percentage dimensions depend on the relevant containing block and whether the reference dimension is definite.

124. Another Important Question

How do you center something?

Modern answer:

CSS FOUNDATIONS TO ADVANCED

```
.container {
    display: grid;
    place-items: center;
```

}

or:

CSS FOUNDATIONS TO ADVANCED

```
.container {
    display: flex;
    justify-content: center;
    align-items: center;
```

}

125. CSS Problem-Solving Patterns

Problem: Center an element

CSS FOUNDATIONS TO ADVANCED

```
display: flex;
justify-content: center;
align-items: center;
```

Problem: Create 3 columns

CSS FOUNDATIONS TO ADVANCED

```
display: grid;
grid-template-columns: repeat(3, 1fr);
```

Problem: Responsive cards

grid-template-columns:

CSS FOUNDATIONS TO ADVANCED

```
    repeat(
        auto-fit,
        minmax(250px, 1fr)
    );
```

Problem: Image fitting

img {

CSS FOUNDATIONS TO ADVANCED

```
    width: 100%;
    height: 100%;
    object-fit: cover;
```

}

Problem: Sticky header

header {

CSS FOUNDATIONS TO ADVANCED

```
    position: sticky;
    top: 0;
```

}

Problem: Overlay badge

CSS FOUNDATIONS TO ADVANCED

```
.card {
    position: relative;
```

}

CSS FOUNDATIONS TO ADVANCED

```
.badge {
    position: absolute;
    top: 10px;
    right: 10px;
```

}

Problem: Responsive typography

h1 {

CSS FOUNDATIONS TO ADVANCED

```
    font-size:
        clamp(
            2rem,
            5vw,
            4rem
        );
```

}

126. Ultimate CSS Memory System

> Remember:

S-B-D-P-F-G-R-A

CONCEPT MAP

```
S → Selectors
B → Box Model
D → Display
P → Position
F → Flexbox
G → Grid
R → Responsive
A → Advanced
```

Advanced:

Variables

Functions

Transitions

Transforms

Animations

Container Queries

Accessibility

Performance

127. CSS Mastery Levels

LEVEL 1

Syntax

Selectors

Colors

Units

CONCEPT MAP

↓

LEVEL 2

Box Model

Margin

Padding

Border

CONCEPT MAP

↓

LEVEL 3

Display

Position

Overflow

CONCEPT MAP

↓

LEVEL 4

Flexbox

CONCEPT MAP

↓

LEVEL 5

Grid

CONCEPT MAP

↓

LEVEL 6

Responsive Design

Media Queries

CONCEPT MAP

↓

LEVEL 7

Cascade

Specificity

Inheritance

CONCEPT MAP

↓

LEVEL 8

Variables

Functions

Transitions

Transforms

Animations

CONCEPT MAP

↓

LEVEL 9

Accessibility

Performance

Container Queries

Logical Properties

CONCEPT MAP

↓

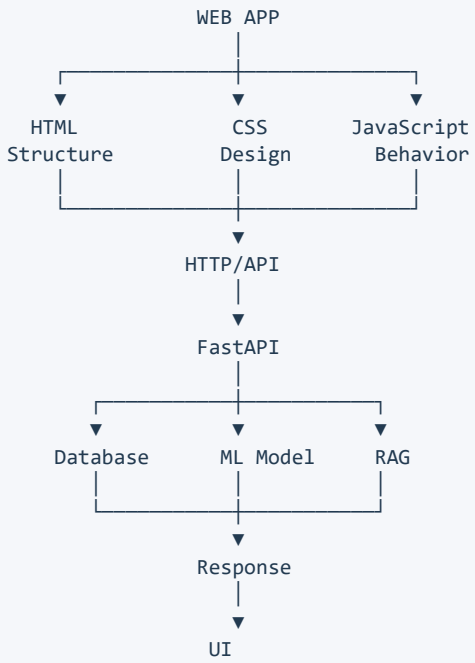
LEVEL 10

Real Projects

128. CSS Full-Stack Architecture

When you combine your HTML, CSS, JavaScript and backend:

CONCEPT MAP



129. Complete CSS Portfolio Starter

CSS FOUNDATIONS TO ADVANCED

```
/* ========================
```

GLOBAL

========================== */

:root {

CSS FOUNDATIONS TO ADVANCED

```
    --primary: #2563eb;

    --text: #111827;

    --muted: #6b7280;

    --background: #f8fafc;

    --surface: #ffffff;

    --border: #e5e7eb;

    --radius: 14px;

    --shadow:
        0 10px 30px rgba(0, 0, 0, 0.08);
```

}

- ,
- ::before,
- ::after {

CSS FOUNDATIONS TO ADVANCED

```
    box-sizing: border-box;
```

}

html {

CSS FOUNDATIONS TO ADVANCED

```
    scroll-behavior: smooth;
```

}

body {

CSS FOUNDATIONS TO ADVANCED

```
    margin: 0;

    font-family:
        Arial,
        sans-serif;

    color: var(--text);

    background:
        var(--background);

    line-height: 1.6;
```

}

img {

```
    display: block;

    max-width: 100%;
```

}

a {


```
    color: inherit;
```

}

button,

input,

textarea,

select {


```
    font: inherit;
```

}


```
/* ========================
```

CONTAINER

========================== */


```
.container {

    width: min(
        100% - 40px,
        1200px
    );

    margin-inline: auto;
```

}


```
/* ========================
```

NAVBAR

========================== */

```
.navbar {

    position: sticky;

    top: 0;

    z-index: 100;

    display: flex;

    justify-content:
        space-between;

    align-items: center;

    padding: 16px 5%;

    background:
        var(--surface);

    border-bottom:
        1px solid var(--border);
```

}

CSS FOUNDATIONS TO ADVANCED

```
.nav-links {

    display: flex;

    gap: 24px;
```

}

.nav-links a {

CSS FOUNDATIONS TO ADVANCED

```
    text-decoration: none;

    color: var(--text);

    transition:
        color 0.2s ease;
```

}

.nav-links a:hover {

CSS FOUNDATIONS TO ADVANCED

```
    color: var(--primary);
```

}

CSS FOUNDATIONS TO ADVANCED

```
/* ========================
```

HERO

========================= */

CSS FOUNDATIONS TO ADVANCED

```
.hero {

    min-height: 80vh;

    display: grid;

    place-items: center;

    text-align: center;

    padding: 80px 20px;
```

}

.hero h1 {

```
CSS FOUNDATIONS TO ADVANCED
    font-size:
        clamp(
            2.2rem,
            6vw,
            5rem
        );

    margin-bottom: 15px;
```

}

.hero p {

```
CSS FOUNDATIONS TO ADVANCED
    max-width: 700px;

    margin-inline: auto;

    color: var(--muted);
```

}

```
CSS FOUNDATIONS TO ADVANCED
/* ========================
```

SKILLS

========================= */

```
CSS FOUNDATIONS TO ADVANCED
.skills {

    display: flex;

    flex-wrap: wrap;

    gap: 12px;
```

}

```
CSS FOUNDATIONS TO ADVANCED
```

```
.skill {

    padding: 8px 14px;

    border:
        1px solid var(--border);

    border-radius:
        999px;

    background:
        var(--surface);
```

}

CSS FOUNDATIONS TO ADVANCED

```
/* ========================
```

PROJECT GRID

========================== */

CSS FOUNDATIONS TO ADVANCED

```
.projects {

    display: grid;

    grid-template-columns:
        repeat(
            auto-fit,
            minmax(280px, 1fr)
        );

    gap: 24px;
```

}

CSS FOUNDATIONS TO ADVANCED

```
.project-card {

    background:
        var(--surface);

    border:
        1px solid var(--border);

    border-radius:
        var(--radius);

    overflow: hidden;

    box-shadow:
        var(--shadow);

    transition:
        transform 0.3s ease;
```

}

.project-card:hover {

CSS FOUNDATIONS TO ADVANCED

```
    transform:
        translateY(-6px);
```

}

CSS FOUNDATIONS TO ADVANCED

```
.project-content {

    padding: 24px;
```

}

CSS FOUNDATIONS TO ADVANCED

```
/* ========================
```

CONTACT FORM

========================= */

CSS FOUNDATIONS TO ADVANCED

```
.contact-form {

    max-width: 650px;

    margin-inline: auto;

    padding: 30px;

    background:
        var(--surface);

    border-radius:
        var(--radius);

    box-shadow:
        var(--shadow);
```

}

CSS FOUNDATIONS TO ADVANCED

```
.form-group {

    display: flex;

    flex-direction: column;

    gap: 8px;

    margin-bottom: 20px;
```

}

.form-group input,

.form-group textarea {

CSS FOUNDATIONS TO ADVANCED

```
    width: 100%;

    padding: 12px 14px;

    border:
        1px solid var(--border);

    border-radius: 8px;
```

}

.form-group input:focus,

.form-group textarea:focus {

CSS FOUNDATIONS TO ADVANCED

```
    outline:
        3px solid
        rgba(37, 99, 235, 0.2);

    border-color:
        var(--primary);
```

}

CSS FOUNDATIONS TO ADVANCED

```
/* ========================
```

BUTTON

========================== */

CSS FOUNDATIONS TO ADVANCED

```
.btn {

    display: inline-flex;

    align-items: center;

    justify-content: center;

    padding: 12px 20px;

    border: none;

    border-radius: 8px;

    background:
        var(--primary);

    color: white;

    cursor: pointer;

    transition:
        transform 0.2s ease,
        opacity 0.2s ease;
```

}

.btn:hover {

CSS FOUNDATIONS TO ADVANCED

```
    transform:
        translateY(-2px);
```

}

.btn:focus-visible {

CSS FOUNDATIONS TO ADVANCED

```
    outline:
        3px solid
        rgba(37, 99, 235, 0.4);

    outline-offset: 3px;
```

}

CSS FOUNDATIONS TO ADVANCED

```
/* ========================
```

MOBILE

========================== */

@media (max-width: 700px) {

CSS FOUNDATIONS TO ADVANCED

```
    .navbar {
        flex-direction: column;
        gap: 15px;
    }
    .nav-links {
        flex-direction: column;
        align-items: center;
        gap: 10px;
    }
    .hero {
        min-height: 70vh;
    }
```

}

130. Final CSS Cheat Sheet

CSS

CONCEPT MAP

```
│
├── SELECTORS
│   ├── element
│   ├── class
│   ├── id
│   ├── attribute
│   ├── descendant
│   ├── child
│   ├── sibling
│   ├── pseudo-class
│   └── pseudo-element
│
├── BOX MODEL
│   ├── content
│   ├── padding
│   ├── border
│   ├── margin
│   └── box-sizing
│
├── LAYOUT
│   ├── display
│   ├── position
│   ├── overflow
│   └── z-index
│
├── FLEXBOX
│   ├── direction
│   ├── justify
│   ├── align
│   ├── gap
│   └── flex
│
├── GRID
│   ├── columns
```

CONCEPT MAP

```
│   ├── rows
│   ├── gap
│   ├── areas
│   └── fr
│
├── RESPONSIVE
│   ├── %
│   ├── rem
│   ├── vw/vh
│   ├── media queries
│   ├── clamp
│   └── container queries
│
├── VISUAL
│   ├── color
│   ├── background
│   ├── border
│   ├── shadow
│   └── gradient
│
├── TYPOGRAPHY
│   ├── font
│   ├── size
│   ├── weight
│   ├── line-height
│   └── letter-spacing
│
└── ADVANCED
    ├── variables
    ├── calc
    ├── min/max
    ├── transitions
    ├── transforms
    ├── animations
```

CONCEPT MAP

```
    ├── accessibility
    └── performance
```

CSS Master Formula

CONCEPT MAP

```
S → B → L → F → G → R → A → P
S = Selectors
B = Box Model
L = Layout
F = Flexbox
G = Grid
R = Responsive
A = Advanced CSS
P = Projects
```

And the most important practical rule:

Don't solve layout problems with random margin, top, left, or position: absolute. First ask: “Should this be Flexbox, Grid, normal flow, or responsive sizing?”

CONCEPT MAP

```
Once you master Box Model → Cascade/Specificity → Flexbox → Grid → Responsive Design → Variables →
Animations → Accessibility, you have the core CSS foundation needed to move from HTML into
JavaScript, React, FastAPI, and full-stack AI/ML applications.
```

PART 3 | JavaScript — Complete Mastery Guide

CONCEPT MAP

```
Basics → Advanced → DSA/Problem Solving → Browser → Async → Modern JavaScript → Projects → Interview
```

> Goal: By the end, you should be able to understand JavaScript concepts, solve coding problems, build browser applications, work with APIs, and explain JavaScript confidently in interviews.

> Important: You asked for Python examples too, but this chapter is JavaScript mastery. JavaScript and Python are different languages, so the main examples below are JavaScript. I'll include a small Python-vs-JavaScript comparison where useful, but using Python for every JavaScript concept would make the learning confusing.

0. JavaScript in One Picture

CONCEPT MAP

```
                    JAVASCRIPT
                        |
        ┌───────────────┼───────────────┐
        ↓               ↓               ↓
    BASICS          BROWSER         ADVANCED
      |               |               |
```

Variables DOM Closures

Data Types Events this

Operators Forms Prototype

Conditions Storage Classes

Loops Fetch Modules

Functions BOM Async

Arrays Web APIs Promises

Objects Event Loop

CONCEPT MAP

```
        |
        ↓
```

PROBLEM SOLVING

CONCEPT MAP

```
        |
  Arrays → Strings → Objects → Recursion
  Searching → Sorting → Hashing → DSA
```

1. What Is JavaScript?

> Simple definition

JavaScript is a programming language used to make websites interactive and dynamic.

CONCEPT MAP

```
HTML → structure
CSS → appearance
JavaScript → behavior
```

HTML

CONCEPT MAP

↓

What exists?

CSS

CONCEPT MAP

↓

How does it look?

JavaScript

CONCEPT MAP

↓

What does it do?

> Real-world example

A button:

JAVASCRIPT: COMPLETE MASTERY

```
<button id="btn">Click Me</button>
```

JavaScript:

JAVASCRIPT: COMPLETE MASTERY

```
document.getElementById("btn").addEventListener("click", function () {
    alert("Hello Aamir!");
```

});

When the user clicks:

User

CONCEPT MAP

↓

Click

CONCEPT MAP

↓

Event

CONCEPT MAP

↓

JavaScript function

CONCEPT MAP

↓
alert()

2. JavaScript Execution Model

A very important interview concept.

JavaScript Code

CONCEPT MAP

```
        ↓
```

JavaScript Engine

CONCEPT MAP

```
        ↓
```

Parsing

CONCEPT MAP

```
        ↓
```

Execution

CONCEPT MAP

```
        ↓
```

Result

Popular engines:

CONCEPT MAP

```
Chrome → V8
Firefox → SpiderMonkey
Safari → JavaScriptCore
```

3. Variables

Variables store values.

JAVASCRIPT: COMPLETE MASTERY

```
let name = "Aamir";
let age = 25;
let isStudent = true;
```

Think:

CONCEPT MAP

```
name ————————→ "Aamir"
age ————————→ 25
isStudent ——→ true
var, let, const
var
```

Old style.

JAVASCRIPT: COMPLETE MASTERY

```
var age = 25;
age = 26;
let
```

Use when value can change.

JAVASCRIPT: COMPLETE MASTERY

```
let score = 50;
score = 80;
const
```

Use when variable should not be reassigned.

JAVASCRIPT: COMPLETE MASTERY

```
const pi = 3.14159;
```

This is invalid:

JAVASCRIPT: COMPLETE MASTERY

```
const x = 10;
x = 20;
```

4. Scope

JAVASCRIPT: COMPLETE MASTERY

```
Scope = where a variable can be accessed.
```

Global scope

JAVASCRIPT: COMPLETE MASTERY

```
let name = "Aamir";

function greet() {
    console.log(name);
```

}

Block scope

JAVASCRIPT: COMPLETE MASTERY

```
if (true) {
    let x = 10;
    const y = 20;
```

}

x and y cannot normally be accessed outside the block.

5. Data Types

JavaScript has primitive and non-primitive/reference values.

Data Types

CONCEPT MAP

```
│
├── Primitive
│   ├── String
│   ├── Number
│   ├── BigInt
│   ├── Boolean
│   ├── Undefined
│   ├── Null
│   └── Symbol
│
└── Reference
    ├── Object
    ├── Array
    └── Function
```

String

JAVASCRIPT: COMPLETE MASTERY

```
let name = "Aamir";
```

Number

JAVASCRIPT: COMPLETE MASTERY

```
let age = 25;
let price = 99.99;
```

Boolean

JAVASCRIPT: COMPLETE MASTERY

```
let isLoggedIn = true;
```

Undefined

JAVASCRIPT: COMPLETE MASTERY

```
let x;
console.log(x);
```

Output:

undefined

Null

JAVASCRIPT: COMPLETE MASTERY

```
let user = null;
```

Meaning:

Intentionally no value.

6. typeof

JAVASCRIPT: COMPLETE MASTERY

```
console.log(typeof "Hello");
console.log(typeof 10);
console.log(typeof true);
console.log(typeof undefined);
```

Output:

string

number

boolean

undefined

Famous interview trap

typeof null

returns:

"object"

This is an old JavaScript behavior.

7. Operators

Operators

CONCEPT MAP

```
│
├── Arithmetic
├── Assignment
├── Comparison
├── Logical
├── Increment/Decrement
├── Ternary
└── Nullish
```

Arithmetic

JAVASCRIPT: COMPLETE MASTERY

```
let a = 10;
let b = 3;

console.log(a + b); // 13
console.log(a - b); // 7
console.log(a * b); // 30
console.log(a / b); // 3.333...
console.log(a % b); // 1
console.log(a ** b); // 1000
```

> Important formula

a % b = remainder after a ÷ b

> Example:

10 ÷ 3

JAVASCRIPT: COMPLETE MASTERY

```
quotient = 3
remainder = 1
```

10 % 3 = 1

8. Comparison Operators

5 == "5"; // true

5 === "5"; // false

==

Loose comparison.

===

Strict comparison.

 Interview rule: Prefer ===.

5 === 5 // true

5 === "5" // false

9. Logical Operators

AND &&

Both must be true.

true && true

CONCEPT MAP

```
→ true
```

OR ||

At least one true.

true || false

CONCEPT MAP

```
→ true
```

NOT !

Reverses Boolean.

!true

CONCEPT MAP

```
→ false
```

Truth table

A	B	A && B	A || B
T	T	T	T
T	F	F	T
F	T	F	T
F	F	F	F

10. Conditional Statements

JAVASCRIPT: COMPLETE MASTERY

```
if
let age = 20;

if (age >= 18) {
    console.log("Adult");
```

}

JAVASCRIPT: COMPLETE MASTERY

```
if-else
if (age >= 18) {
    console.log("Adult");
```

} else {

JAVASCRIPT: COMPLETE MASTERY

```
    console.log("Minor");
```

}

JAVASCRIPT: COMPLETE MASTERY

```
else-if
let marks = 85;

if (marks >= 90) {
    console.log("A+");
```

} else if (marks >= 80) {

JAVASCRIPT: COMPLETE MASTERY

```
    console.log("A");
```

} else if (marks >= 70) {

JAVASCRIPT: COMPLETE MASTERY

```
    console.log("B");
```

} else {

JAVASCRIPT: COMPLETE MASTERY

```
    console.log("C");
```

}

11. Ternary Operator

Short form of if-else.

JAVASCRIPT: COMPLETE MASTERY

```
let age = 20;

let result = age >= 18 ? "Adult" : "Minor";
```

Diagram:

CONCEPT MAP

```
          age >= 18?
          /       \
       YES          NO
        ↓            ↓
    "Adult"      "Minor"
```

12. Switch

Useful when comparing one value with multiple cases.

JAVASCRIPT: COMPLETE MASTERY

```
let day = 2;

switch (day) {
    case 1:
        console.log("Monday");
        break;

    case 2:
        console.log("Tuesday");
        break;

    default:
        console.log("Invalid");
```

}

13. Loops

Loops repeat code.

Loop

CONCEPT MAP

```
  │
  ├── for
  ├── while
  ├── do...while
  ├── for...of
  └── for...in
```

For loop

JAVASCRIPT: COMPLETE MASTERY

```
for (let i = 1; i <= 5; i++) {
    console.log(i);
```

}

Dry run

i	condition	output
1	1 <= 5	1
2	2 <= 5	2
3	3 <= 5	3
4	4 <= 5	4
5	5 <= 5	5
6	6 <= 5	stop

14. While Loop

JAVASCRIPT: COMPLETE MASTERY

```
let i = 1;

while (i <= 5) {
    console.log(i);
    i++;
```

}

15. do...while

Runs at least once.

JAVASCRIPT: COMPLETE MASTERY

```
let i = 10;
```

do {

JAVASCRIPT: COMPLETE MASTERY

```
    console.log(i);
    i++;
```

} while (i < 5);

Output:

10

16. break and continue

break

Stops loop.

JAVASCRIPT: COMPLETE MASTERY

```
for (let i = 1; i <= 10; i++) {
    if (i === 5) {
        break;
    }

    console.log(i);
```

}

Output:

```
1 2 3 4
```

continue

Skips current iteration.

JAVASCRIPT: COMPLETE MASTERY

```
for (let i = 1; i <= 5; i++) {
    if (i === 3) {
        continue;
    }

    console.log(i);
```

}

Output:

1

2

4

5

17. Functions

JAVASCRIPT: COMPLETE MASTERY

```
Function = reusable block of code.

function greet() {
    console.log("Hello");
```

}

JAVASCRIPT: COMPLETE MASTERY

```
greet();
```

Parameters and arguments

JAVASCRIPT: COMPLETE MASTERY

```
function add(a, b) {
    return a + b;
```

}

JAVASCRIPT: COMPLETE MASTERY

```
console.log(add(10, 20));
```

Here:

a, b = parameters

10, 20 = arguments

> Formula

JAVASCRIPT: COMPLETE MASTERY

```
output = function(input)
```

For addition:

JAVASCRIPT: COMPLETE MASTERY

```
sum = a + b
```

18. Arrow Functions

Traditional:

JAVASCRIPT: COMPLETE MASTERY

```
function add(a, b) {
    return a + b;
```

}

Arrow:

JAVASCRIPT: COMPLETE MASTERY

```
const add = (a, b) => a + b;
```

Multiple lines:

JAVASCRIPT: COMPLETE MASTERY

```
const multiply = (a, b) => {
    return a * b;
```

};

19. Default Parameters

JAVASCRIPT: COMPLETE MASTERY

```
function greet(name = "Guest") {
    return `Hello ${name}`;
```

}

JAVASCRIPT: COMPLETE MASTERY

```
console.log(greet());
```

Output:

Hello Guest

20. Rest Parameters

Collect many values.

JAVASCRIPT: COMPLETE MASTERY

```
function sum(...numbers) {
    let total = 0;

    for (const num of numbers) {
        total += num;
    }

    return total;
```

}

JAVASCRIPT: COMPLETE MASTERY

```
console.log(sum(1, 2, 3, 4));
```

Output:

10

Diagram:

1 2 3 4

\ | | /

...numbers

CONCEPT MAP

```
      ↓
```

[1,2,3,4]

 21. Strings

JAVASCRIPT: COMPLETE MASTERY

```
let text = "JavaScript";
```

Important methods:

text.length

JAVASCRIPT: COMPLETE MASTERY

```
text.toUpperCase()
text.toLowerCase()
text.includes("Script")
text.startsWith("Java")
text.endsWith("Script")
text.slice(0, 4)
text.substring(0, 4)
text.replace("Java", "Type")
text.trim()
text.split("")
```

String problem

Reverse a string:

JAVASCRIPT: COMPLETE MASTERY

```
function reverseString(str) {
    return str.split("").reverse().join("");
```

}

JAVASCRIPT: COMPLETE MASTERY

```
console.log(reverseString("hello"));
```

Output:

olleh

22. Arrays

Array stores multiple values.

JAVASCRIPT: COMPLETE MASTERY

```
const numbers = [10, 20, 30, 40];
```

Diagram:

Index: 0 1 2 3

CONCEPT MAP

```
          ↓    ↓    ↓    ↓
```

Array: 10 20 30 40

Access:

numbers[0]; // 10

numbers[2]; // 30

23. Array Methods

Add

JAVASCRIPT: COMPLETE MASTERY

```
arr.push(50);
```

Adds at end.

Remove from end

JAVASCRIPT: COMPLETE MASTERY

```
arr.pop();
```

Add at beginning

JAVASCRIPT: COMPLETE MASTERY

```
arr.unshift(5);
```

Remove beginning

JAVASCRIPT: COMPLETE MASTERY

```
arr.shift();
```

24. map()

Creates a new array by transforming every item.

JAVASCRIPT: COMPLETE MASTERY

```
const nums = [1, 2, 3];

const result = nums.map(x => x * 2);

console.log(result);
```

Output:

[2, 4, 6]

Diagram:

CONCEPT MAP

```
1 → ×2 → 2
2 → ×2 → 4
3 → ×2 → 6
```

25. filter()

Keeps elements that satisfy a condition.

JAVASCRIPT: COMPLETE MASTERY

```
const nums = [1, 2, 3, 4, 5];

const even = nums.filter(x => x % 2 === 0);

console.log(even);
```

Output:

[2, 4]

26. reduce()

Converts many values into one result.

JAVASCRIPT: COMPLETE MASTERY

```
const nums = [1, 2, 3, 4];

const sum = nums.reduce((total, x) => total + x, 0);

console.log(sum);
```

> Dry run

total	x	new total
0	1	1
1	2	3
3	3	6
6	4	10

27. find, findIndex, some, every

CONCEPT MAP

```
const nums = [10, 20, 30, 40];

nums.find(x => x > 25);
→ 30

nums.findIndex(x => x > 25);
→ 2

nums.some(x => x > 35);
→ true

nums.every(x => x > 5);
→ true
```

28. Objects

Object stores data using key-value pairs.

JAVASCRIPT: COMPLETE MASTERY

```
const user = {
    name: "Aamir",
    age: 25,
    role: "AI/ML Engineer"
```

};

Diagram:

user

CONCEPT MAP

```
  │
  ├── name → Aamir
  ├── age  → 25
  └── role → AI/ML Engineer
```

Access:

user.name;

user["age"];

29. Object Methods

JAVASCRIPT: COMPLETE MASTERY

```
Object.keys(user);
Object.values(user);
Object.entries(user);
```

> Example:

JAVASCRIPT: COMPLETE MASTERY

```
console.log(Object.keys(user));
```

Output:

["name", "age", "role"]

30. Destructuring

Instead of:

JAVASCRIPT: COMPLETE MASTERY

```
const name = user.name;
const age = user.age;
```

Use:

JAVASCRIPT: COMPLETE MASTERY

```
const { name, age } = user;
```

Array:

JAVASCRIPT: COMPLETE MASTERY

```
const [first, second] = [10, 20];
```

31. Spread Operator

Copies/expands values.

JAVASCRIPT: COMPLETE MASTERY

```
const a = [1, 2, 3];

const b = [...a, 4, 5];

console.log(b);
```

Output:

[1, 2, 3, 4, 5]

Object:

JAVASCRIPT: COMPLETE MASTERY

```
const user2 = {
    ...user,
    age: 26
```

};

32. Shallow Copy vs Deep Copy

Shallow copy

JAVASCRIPT: COMPLETE MASTERY

```
const copy = { ...user };
```

Nested objects can still share references.

Deep copy

For data that can be safely represented as JSON:

JAVASCRIPT: COMPLETE MASTERY

```
const copy = JSON.parse(JSON.stringify(user));
```

Modern option:

JAVASCRIPT: COMPLETE MASTERY

```
const copy = structuredClone(user);
```

33. Primitive vs Reference

Very important.

JAVASCRIPT: COMPLETE MASTERY

```
let a = 10;
let b = a;

b = 20;

console.log(a); // 10
```

But:

JAVASCRIPT: COMPLETE MASTERY

```
let obj1 = { name: "Aamir" };
let obj2 = obj1;

obj2.name = "Rahul";

console.log(obj1.name);
```

Output:

Rahul

Why?

CONCEPT MAP

```
obj1 ──────┐
           ↓
        Object
           ↑
obj2 ──────┘
```

Both point to the same object.

34. null, undefined, NaN

undefined

Value hasn't been assigned.

null

Intentional empty value.

NaN

Not a valid numeric result.

CONCEPT MAP

```
console.log("abc" * 2);
→ NaN
```

Check:

JAVASCRIPT: COMPLETE MASTERY

```
Number.isNaN(value);
```

35. Type Conversion

CONCEPT MAP

```
String → Number:

Number("100");
Number → String:

String(100);
```

Boolean:

JAVASCRIPT: COMPLETE MASTERY

```
Boolean(1); // true
Boolean(0); // false
```

36. Truthy and Falsy

Falsy values include:

false

0

• 0

0n

""

null

undefined

NaN

Most other values are truthy.

□ 37. Optional Chaining

Prevents errors when something doesn't exist.

JAVASCRIPT: COMPLETE MASTERY

```
const user = {};

console.log(user.address?.city);
```

Instead of crashing, result is:

undefined

□ 38. Nullish Coalescing

JAVASCRIPT: COMPLETE MASTERY

```
const name = null ?? "Guest";
```

Result:

Guest

Important difference:

0 || 100

CONCEPT MAP

```
→ 100
```

But:

0 ?? 100

CONCEPT MAP

```
→ 0
```

?? checks specifically for:

null

undefined

39. Scope — Deep Concept

There are mainly:

Global Scope

Function Scope

Block Scope

Module Scope

Example:

JAVASCRIPT: COMPLETE MASTERY

```
let global = "G";

function test() {
    let local = "L";

    if (true) {
        let block = "B";
        console.log(global);
        console.log(local);
        console.log(block);
    }
```

}

40. Hoisting

JavaScript processes declarations before normal execution.

Example:

JAVASCRIPT: COMPLETE MASTERY

```
console.log(x);

var x = 10;
```

Output:

undefined

But:

JAVASCRIPT: COMPLETE MASTERY

```
console.log(y);

let y = 10;
```

This causes an error because y is in the Temporal Dead Zone until initialization.

Easy meaning

Temporal Dead Zone = the period where a let/`const` variable exists in its scope but cannot yet be accessed.

41. Closures

One of the most important JavaScript interview concepts.

Simple definition

A closure happens when an inner function remembers variables from its outer function even after the outer function has finished.

JAVASCRIPT: COMPLETE MASTERY

```
function counter() {
    let count = 0;

    return function () {
        count++;
        return count;
    };
```

}

JAVASCRIPT: COMPLETE MASTERY

```
const increment = counter();

console.log(increment());
console.log(increment());
console.log(increment());
```

Output:

1

2

3

Diagram:

CONCEPT MAP

```
counter()
   │
   ├── count = 0
   │
   └── returns function
            │
            ↓
       remembers count
            │
       ┌────┼────┐
       ↓    ↓    ↓
       1    2    3
```

Real-world use

Private variables

Counters

Function factories

Callbacks

Event handlers

Memoization

42. Callback Functions

A function passed into another function.

JAVASCRIPT: COMPLETE MASTERY

```
function greet(name, callback) {
    console.log("Hello " + name);
    callback();
```

}

JAVASCRIPT: COMPLETE MASTERY

```
greet("Aamir", function () {
    console.log("Welcome!");
```

});

Diagram:

CONCEPT MAP

```
greet()
  │
  ├── name
  │
  └── callback()
         ↓
      function
```

43. Higher-Order Functions

A function that:

accepts another function, or

returns another function.

Examples:

JAVASCRIPT: COMPLETE MASTERY

```
map()
filter()
reduce()
```

44. IIFE

Immediately Invoked Function Expression.

(function () {

JAVASCRIPT: COMPLETE MASTERY

```
    console.log("Runs immediately");
```

})();

45. this

this refers to an object/context depending on how a function is called.

JAVASCRIPT: COMPLETE MASTERY

```
const user = {
    name: "Aamir",

    greet() {
        console.log(this.name);
    }
```

};

JAVASCRIPT: COMPLETE MASTERY

```
user.greet();
```

Output:

Aamir

46. Arrow Function and this

Arrow functions do not create their own this.

JAVASCRIPT: COMPLETE MASTERY

```
const user = {
    name: "Aamir",

    greet: () => {
        console.log(this.name);
    }
```

};

This usually does not behave like the regular method version.

Memory trick

Normal function: this depends on how it is called.

Arrow function: this comes from surrounding scope.

47. call, apply, bind

JAVASCRIPT: COMPLETE MASTERY

```
function greet(city) {
    console.log(this.name, city);
```

}

JAVASCRIPT: COMPLETE MASTERY

```
const user = {
    name: "Aamir"
```

};

call

JAVASCRIPT: COMPLETE MASTERY

```
greet.call(user, "Delhi");
```

apply

JAVASCRIPT: COMPLETE MASTERY

```
greet.apply(user, ["Delhi"]);
```

bind

JAVASCRIPT: COMPLETE MASTERY

```
const fn = greet.bind(user);
fn("Delhi");
```

Memory trick:

CONCEPT MAP

```
call  → arguments individually
apply → arguments in array
bind  → returns new function
```

48. Prototypes

JavaScript uses prototype-based inheritance.

Simple meaning:

Objects can get properties and methods from another object through the prototype chain.

object

CONCEPT MAP

```
    ↓
```

prototype

CONCEPT MAP

```
    ↓
```

prototype

CONCEPT MAP

```
    ↓
```

null

49. Classes

Classes provide a convenient syntax for creating objects.

JAVASCRIPT: COMPLETE MASTERY

```
class Person {
    constructor(name, age) {
        this.name = name;
        this.age = age;
    }

    greet() {
        console.log(`Hello ${this.name}`);
    }
```

}

JAVASCRIPT: COMPLETE MASTERY

```
const p1 = new Person("Aamir", 25);

p1.greet();
```

50. Inheritance

JAVASCRIPT: COMPLETE MASTERY

```
class Animal {
    speak() {
        console.log("Animal speaks");
    }
```

}

JAVASCRIPT: COMPLETE MASTERY

```
class Dog extends Animal {
    bark() {
        console.log("Dog barks");
    }
```

}

JAVASCRIPT: COMPLETE MASTERY

```
const dog = new Dog();

dog.speak();
dog.bark();
```

Diagram:

Animal

CONCEPT MAP

```
    |
```

extends

CONCEPT MAP

```
    ↓
```

Dog

51. Encapsulation

Hide internal details and expose only what is needed.

Modern JavaScript private fields:

JAVASCRIPT: COMPLETE MASTERY

```
class BankAccount {
    #balance = 0;

    deposit(amount) {
        this.#balance += amount;
    }

    getBalance() {
        return this.#balance;
    }
```

}

52. Getters and Setters

JAVASCRIPT: COMPLETE MASTERY

```
class User {
    constructor(name) {
        this._name = name;
    }

    get name() {
        return this._name;
    }

    set name(value) {
        this._name = value;
    }
```

}

JAVASCRIPT: COMPLETE MASTERY

```
const user = new User("Aamir");

console.log(user.name);

user.name = "Azad";
```

53. DOM

JAVASCRIPT: COMPLETE MASTERY

```
DOM = Document Object Model
```

> Easy meaning:

Browser converts HTML into a tree of objects that JavaScript can access and modify.

HTML

CONCEPT MAP

↓

DOM Tree

CONCEPT MAP

↓

JavaScript

CONCEPT MAP

↓

Change page

> Example:

JAVASCRIPT: COMPLETE MASTERY

```
<h1 id="title">Hello</h1>
const title = document.getElementById("title");

title.textContent = "Hello Aamir";
```

54. DOM Tree

document

CONCEPT MAP

```
    │
    └── html
        │
        ├── head
        │
        └── body
            │
            ├── h1
            ├── p
            └── button
```

55. DOM Selectors

JAVASCRIPT: COMPLETE MASTERY

```
document.getElementById("title");

document.querySelector(".card");

document.querySelector("#title");

document.querySelectorAll(".item");
```

Most useful:

JAVASCRIPT: COMPLETE MASTERY

```
document.querySelector()
document.querySelectorAll()
```

56. Changing Content

JAVASCRIPT: COMPLETE MASTERY

```
element.textContent = "New text";
```

HTML:

JAVASCRIPT: COMPLETE MASTERY

```
element.innerHTML = "<b>Hello</b>";
```

Be careful with innerHTML when inserting untrusted user input because it can create security problems.

57. Changing CSS with JavaScript

JAVASCRIPT: COMPLETE MASTERY

```
element.style.color = "red";
element.style.backgroundColor = "black";
```

Better for larger changes:

JAVASCRIPT: COMPLETE MASTERY

```
element.classList.add("active");
element.classList.remove("active");
element.classList.toggle("active");
```

58. Events

An event is something that happens.

Examples:

click

submit

input

change

keydown

keyup

mouseover

scroll

load

> Example:

JAVASCRIPT: COMPLETE MASTERY

```
const button = document.querySelector("#btn");

button.addEventListener("click", () => {
    console.log("Clicked");
```

});

59. Event Flow

Window

CONCEPT MAP

↓

Document

CONCEPT MAP

↓

Parent

CONCEPT MAP

↓

Child

CONCEPT MAP

↓

Target

Event phases:

Capturing

CONCEPT MAP

↓

Target

CONCEPT MAP

↓

Bubbling

60. Event Bubbling

Suppose:

JAVASCRIPT: COMPLETE MASTERY

```
<div id="parent">
    <button id="child">Click</button>
</div>
```

Clicking button can trigger:

button

CONCEPT MAP

↓

parent

CONCEPT MAP

↓

body

CONCEPT MAP

↓

document

61. Event Delegation

Instead of attaching listeners to many children, attach one to parent.

JAVASCRIPT: COMPLETE MASTERY

```
const list = document.querySelector("#list");

list.addEventListener("click", event => {
    if (event.target.matches("li")) {
        console.log(event.target.textContent);
    }
```

});

Very useful for dynamic lists.

62. Browser Storage

localStorage

Data survives browser restart.

JAVASCRIPT: COMPLETE MASTERY

```
localStorage.setItem("name", "Aamir");

const name = localStorage.getItem("name");

console.log(name);
```

Remove:

JAVASCRIPT: COMPLETE MASTERY

```
localStorage.removeItem("name");
```

Clear:

JAVASCRIPT: COMPLETE MASTERY

```
localStorage.clear();
```

63. JSON

CONCEPT MAP

```
JSON = JavaScript Object Notation.

Convert object → JSON:

const user = {
    name: "Aamir",
    age: 25
```

};

CONCEPT MAP

```
const json = JSON.stringify(user);
JSON → object:

const obj = JSON.parse(json);
```

Diagram:

Object

CONCEPT MAP

```
↓ stringify()
```

JSON string

CONCEPT MAP

```
↓ parse()
```

Object

64. Asynchronous JavaScript

This is a huge topic.

Normally JavaScript executes sequentially:

1

CONCEPT MAP

```
↓
```

2

CONCEPT MAP

```
↓
```

3

But asynchronous operations allow waiting work to complete later.

Examples:

API calls

timers

file operations

database requests

 65. setTimeout

JAVASCRIPT: COMPLETE MASTERY

```
console.log("A");

setTimeout(() => {
    console.log("B");
```

}, 1000);

JAVASCRIPT: COMPLETE MASTERY

```
console.log("C");
```

Output:

A

C

B

Why?

Call Stack

CONCEPT MAP

```
↓
```

setTimeout

CONCEPT MAP

↓

Browser/Runtime Timer

CONCEPT MAP

↓

Callback Queue

CONCEPT MAP

↓

Event Loop

CONCEPT MAP

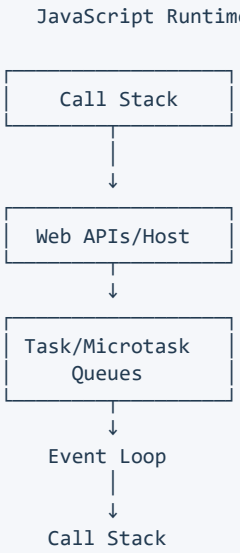
↓

Call Stack

66. Event Loop

One of the most important interview topics.

CONCEPT MAP



Important priority

In simplified terms:

Synchronous code

CONCEPT MAP

↓

Microtasks

CONCEPT MAP

↓

Tasks

Promises use the microtask queue.

67. Promise

Promise represents a future result.

States:

Promise

CONCEPT MAP

```
  │
  ├── Pending
  ├── Fulfilled
  └── Rejected
```

Example:

JAVASCRIPT: COMPLETE MASTERY

```
const promise = new Promise((resolve, reject) => {
    const success = true;

    if (success) {
        resolve("Success");
    } else {
        reject("Failed");
    }
```

});

68. .then() and .catch()

promise

JAVASCRIPT: COMPLETE MASTERY

```
    .then(result => {
        console.log(result);
    })
    .catch(error => {
        console.log(error);
    });
```

69. async/await

Cleaner way to work with promises.

JAVASCRIPT: COMPLETE MASTERY

```
async function getData() {
    try {
        const response = await fetch("https://example.com/data");
        const data = await response.json();

        console.log(data);
    } catch (error) {
        console.error(error);
    }
```

}

Diagram:

CONCEPT MAP

```
async function
      ↓
await Promise
      ↓
```

wait for result

CONCEPT MAP

```
      ↓
```

continue execution

JAVASCRIPT: COMPLETE MASTERY

```
await does not freeze the entire JavaScript runtime.
```

70. Fetch API

Used to make HTTP requests.

JAVASCRIPT: COMPLETE MASTERY

```
fetch("/api/users")
    .then(response => response.json())
    .then(data => console.log(data))
    .catch(error => console.error(error));
```

POST:

JAVASCRIPT: COMPLETE MASTERY

```
fetch("/api/users", {
    method: "POST",
    headers: {
        "Content-Type": "application/json"
    },
    body: JSON.stringify({
        name: "Aamir"
    })
```

});

71. HTTP Basics for JavaScript

Frontend

CONCEPT MAP

```
    |
    | HTTP Request
    ↓
```

Backend API

CONCEPT MAP

```
    |
    | Response
    ↓
```

Frontend

Common methods:

Method	Meaning
GET	Read

Method	Meaning
POST	Create
PUT	Replace
PATCH	Update partially
DELETE	Delete

72. Error Handling

JAVASCRIPT: COMPLETE MASTERY

```
try {
    riskyOperation();
```

} catch (error) {

JAVASCRIPT: COMPLETE MASTERY

```
    console.log(error.message);
```

} finally {

JAVASCRIPT: COMPLETE MASTERY

```
    console.log("Finished");
```

}

Create your own error:

throw new Error("Invalid input");

73. Modules

Modern JavaScript supports modules.

export

export function add(a, b) {

JAVASCRIPT: COMPLETE MASTERY

```
    return a + b;
```

}

JAVASCRIPT: COMPLETE MASTERY

```
import
import { add } from "./math.js";

console.log(add(2, 3));
```

Default export:

export default function greet() {

JAVASCRIPT: COMPLETE MASTERY

```
    console.log("Hello");
```

}

Import:

JAVASCRIPT: COMPLETE MASTERY

```
import greet from "./greet.js";
```

74. Map

Map stores key-value pairs.

JAVASCRIPT: COMPLETE MASTERY

```
const map = new Map();

map.set("name", "Aamir");
map.set("age", 25);

console.log(map.get("name"));
```

75. Set

Stores unique values.

JAVASCRIPT: COMPLETE MASTERY

```
const numbers = new Set([1, 2, 2, 3, 3]);

console.log(numbers);
```

Result conceptually:

{1, 2, 3}

Remove duplicates:

JAVASCRIPT: COMPLETE MASTERY

```
const unique = [...new Set([1, 2, 2, 3])];
```

76. WeakMap and WeakSet

Advanced structures useful when working with object references and memory-sensitive scenarios.

JAVASCRIPT: COMPLETE MASTERY

```
const weakMap = new WeakMap();

const obj = {};

weakMap.set(obj, "data");
```

You cannot iterate over a WeakMap like a normal Map.

77. Symbols

Symbol creates unique primitive values.

JAVASCRIPT: COMPLETE MASTERY

```
const id = Symbol("id");
```

Two symbols are unique:

CONCEPT MAP

```
Symbol("x") === Symbol("x")
→ false
```

78. BigInt

For integers larger than normal safe integer limits:

JAVASCRIPT: COMPLETE MASTERY

```
const big = 1234567890123456789012345678901234567890n;
```

Operations need compatible BigInt values.

79. Regular Expressions

JAVASCRIPT: COMPLETE MASTERY

```
Regex = pattern used to search/match text.

const pattern = /^[A-Za-z]+$/;

console.log(pattern.test("Aamir"));
```

Email-style example:

JAVASCRIPT: COMPLETE MASTERY

```
const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

80. Debouncing

Very important frontend interview concept.

Problem

User types:

A

Am

Aam

Aami

Aamir

Without debounce:

5 API calls

With debounce:

wait

CONCEPT MAP

```
↓
```

Aamir

CONCEPT MAP

```
↓
```

1 API call

> Example:

JAVASCRIPT: COMPLETE MASTERY

```
function debounce(fn, delay) {
    let timer;

    return function (...args) {
        clearTimeout(timer);

        timer = setTimeout(() => {
            fn.apply(this, args);
        }, delay);
    };
```

}

JAVASCRIPT: COMPLETE MASTERY

```
const search = debounce(value => {
    console.log("Searching:", value);
```

}, 500);

81. Throttling

Throttle limits how often a function can run.

Useful for:

scroll

resize

mouse movement

Concept:

Events:



Throttle:



82. Recursion

A function calling itself.

JAVASCRIPT: COMPLETE MASTERY

```
function countdown(n) {
    if (n === 0) {
        return;
    }

    console.log(n);

    countdown(n - 1);
```

}

JAVASCRIPT: COMPLETE MASTERY

```
countdown(5);
```

Diagram:

CONCEPT MAP

```
countdown(5)
    ↓
countdown(4)
    ↓
countdown(3)
    ↓
countdown(2)
    ↓
countdown(1)
    ↓
countdown(0)
    ↓
```

stop

83. Factorial

Formula:

n! = n × (n-1) × (n-2) × ... × 1

Example:

5! = 5×4×3×2×1

= 120

Code:

JAVASCRIPT: COMPLETE MASTERY

```
function factorial(n) {
    if (n <= 1) {
        return 1;
    }

    return n * factorial(n - 1);
```

}

84. Time Complexity

Important for DSA interviews.

Common complexities:

JAVASCRIPT: COMPLETE MASTERY

```
O(1)       Constant
O(log n)   Logarithmic
O(n)       Linear
O(n log n)
O(n²)      Quadratic
O(2ⁿ)      Exponential
O(n!)      Factorial
```

Diagram:

Fast

CONCEPT MAP

```
│
├── O(1)
├── O(log n)
├── O(n)
├── O(n log n)
├── O(n²)
├── O(2ⁿ)
└── O(n!)
│
```

Slow

85. JavaScript DSA Problem 1 — Two Sum

Problem:

JAVASCRIPT: COMPLETE MASTERY

```
nums = [2,7,11,15]
target = 9
```

Need:

2 + 7 = 9

Efficient solution:

JAVASCRIPT: COMPLETE MASTERY

```
function twoSum(nums, target) {
    const map = new Map();

    for (let i = 0; i < nums.length; i++) {
        const complement = target - nums[i];

        if (map.has(complement)) {
            return [map.get(complement), i];
        }

        map.set(nums[i], i);
    }

    return [];
```

}

> Dry run

i	nums[i]	complement	map
0	2	7	{2:0}
1	7	2	{2:0,7:1}

Found:

[0,1]

Complexity:

Time: O(n)

Space: O(n)

86. Reverse Array

JAVASCRIPT: COMPLETE MASTERY

```
function reverseArray(arr) {
    let left = 0;
    let right = arr.length - 1;

    while (left < right) {
        [arr[left], arr[right]] =
            [arr[right], arr[left]];

        left++;
        right--;
    }

    return arr;
```

}

Concept:

[1, 2, 3, 4, 5]

CONCEPT MAP

```
  ↑           ↑
```

left right

[5, 2, 3, 4, 1]

CONCEPT MAP

```
    ↑       ↑
```

left right

87. Find Maximum

JAVASCRIPT: COMPLETE MASTERY

```
function findMax(arr) {
    let max = arr[0];

    for (const num of arr) {
        if (num > max) {
            max = num;
        }
    }

    return max;
```

}

Complexity:

JAVASCRIPT: COMPLETE MASTERY

```
O(n)
```

88. Palindrome

Palindrome reads same forward/backward.

madam

racecar

Code:

JAVASCRIPT: COMPLETE MASTERY

```
function isPalindrome(str) {
    let left = 0;
    let right = str.length - 1;

    while (left < right) {
        if (str[left] !== str[right]) {
            return false;
        }

        left++;
        right--;
    }

    return true;
```

}

89. Frequency Counter

JAVASCRIPT: COMPLETE MASTERY

```
function frequency(arr) {
    const freq = {};

    for (const item of arr) {
        freq[item] = (freq[item] || 0) + 1;
    }

    return freq;
```

}

JAVASCRIPT: COMPLETE MASTERY

```
console.log(frequency(["a", "b", "a", "c", "a"]));
```

Result:

{

JAVASCRIPT: COMPLETE MASTERY

```
    a: 3,
    b: 1,
    c: 1
```

}

90. Anagram

Two strings contain the same characters with the same frequencies.

JAVASCRIPT: COMPLETE MASTERY

```
function isAnagram(a, b) {
    if (a.length !== b.length) {
        return false;
    }

    const count = {};

    for (const char of a) {
        count[char] = (count[char] || 0) + 1;
    }

    for (const char of b) {
        if (!count[char]) {
            return false;
        }

        count[char]--;
    }

    return true;
```

}

Example:

listen

silent

CONCEPT MAP

```
→ true
```

91. Kadane's Algorithm

Find maximum subarray sum.

JAVASCRIPT: COMPLETE MASTERY

```
function maxSubArray(nums) {
    let current = nums[0];
    let best = nums[0];

    for (let i = 1; i < nums.length; i++) {
        current = Math.max(nums[i], current + nums[i]);

        best = Math.max(best, current);
    }

    return best;
```

}

Core formula:

JAVASCRIPT: COMPLETE MASTERY

```
current =
max(current element,
    previous current + current element)
```

92. Binary Search

Works on a sorted array.

JAVASCRIPT: COMPLETE MASTERY

```
function binarySearch(arr, target) {
    let left = 0;
    let right = arr.length - 1;

    while (left <= right) {
        const mid = Math.floor((left + right) / 2);

        if (arr[mid] === target) {
            return mid;
        }

        if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return -1;
```

}

Diagram:

[1, 3, 5, 7, 9, 11, 13]

CONCEPT MAP

```
          ↑
         mid

target = 11
```

search right half

CONCEPT MAP

```
        ↓
```

[9,11,13]

CONCEPT MAP

```
   ↑
```

mid

Complexity:

JAVASCRIPT: COMPLETE MASTERY

```
O(log n)
```

93. Sorting

Built-in sort

For numbers:

JAVASCRIPT: COMPLETE MASTERY

```
const nums = [10, 2, 30, 4];

nums.sort((a, b) => a - b);
```

Descending:

JAVASCRIPT: COMPLETE MASTERY

```
nums.sort((a, b) => b - a);
```

Why comparator?

Without comparator:

[10, 2, 30].sort()

JavaScript can compare values as strings, producing unexpected numeric ordering.

94. Common Sorting Algorithms

Sorting

CONCEPT MAP

```
│
├── Bubble Sort       O(n²)
├── Selection Sort    O(n²)
├── Insertion Sort    O(n²)
├── Merge Sort        O(n log n)
├── Quick Sort        Average O(n log n)
└── Heap Sort         O(n log n)
```

95. Bubble Sort

JAVASCRIPT: COMPLETE MASTERY

```
function bubbleSort(arr) {
    const result = [...arr];

    for (let i = 0; i < result.length; i++) {
        for (let j = 0; j < result.length - i - 1; j++) {
            if (result[j] > result[j + 1]) {
                [result[j], result[j + 1]] =
                    [result[j + 1], result[j]];
            }
        }
    }

    return result;
```

}

Concept:

5 3 2 1

CONCEPT MAP

↓

3 5 2 1

CONCEPT MAP

↓

3 2 5 1

CONCEPT MAP

↓

3 2 1 5

Largest value moves toward the end.

96. Sliding Window

Useful for contiguous subarray/string problems.

> Example:

[1,2,3,4,5]

CONCEPT MAP

```
  └──┘
```

window size = 3

Instead of recalculating every window:

1+2+3

2+3+4

3+4+5

Move:

new sum = old sum - outgoing + incoming

> Example:

JAVASCRIPT: COMPLETE MASTERY

```
function maxSum(arr, k) {
    let windowSum = 0;

    for (let i = 0; i < k; i++) {
        windowSum += arr[i];
    }

    let max = windowSum;

    for (let i = k; i < arr.length; i++) {
        windowSum += arr[i];
        windowSum -= arr[i - k];

        max = Math.max(max, windowSum);
    }

    return max;
```

}

97. Two Pointers

Useful when working from both ends.

[1, 2, 3, 4, 5]

CONCEPT MAP

```
  ↑          ↑
```

left right

Common problems:

palindrome

reverse array

pair sum

sorted array problems

98. Complete DOM Mini Project

HTML

JAVASCRIPT: COMPLETE MASTERY

```
<!DOCTYPE html>
<html>
<head>
    <title>JavaScript Todo</title>
</head>

<body>

    <h1>Todo App</h1>

    <input id="taskInput" placeholder="Enter task">

    <button id="addBtn">Add</button>

    <ul id="taskList"></ul>

    <script src="app.js"></script>
</body>
</html>
```

JavaScript

JAVASCRIPT: COMPLETE MASTERY

```
const input = document.querySelector("#taskInput");
const button = document.querySelector("#addBtn");
const list = document.querySelector("#taskList");

button.addEventListener("click", addTask);

function addTask() {
    const task = input.value.trim();

    if (task === "") {
        alert("Enter a task");
        return;
    }

    const li = document.createElement("li");

    li.innerHTML = `
        <span>${task}</span>
        <button class="delete">Delete</button>
    `;

    list.appendChild(li);

    input.value = "";
```

}

JAVASCRIPT: COMPLETE MASTERY

```
list.addEventListener("click", event => {
    if (event.target.classList.contains("delete")) {
        event.target.parentElement.remove();
    }
```

});

Concepts used:

DOM

CONCEPT MAP

```
  ↓
```

querySelector

CONCEPT MAP

```
  ↓
```

Event Listener

CONCEPT MAP

```
  ↓
```

Function

CONCEPT MAP

```
  ↓
```

Condition

CONCEPT MAP

```
  ↓
```

createElement

CONCEPT MAP

```
  ↓
```

innerHTML

CONCEPT MAP

```
  ↓
```

appendChild

CONCEPT MAP

```
  ↓
```

Event Delegation

CONCEPT MAP

```
  ↓
remove()
```

99. Advanced Todo App With LocalStorage

JAVASCRIPT: COMPLETE MASTERY

```
const input = document.querySelector("#taskInput");
const addBtn = document.querySelector("#addBtn");
const list = document.querySelector("#taskList");

let tasks = JSON.parse(localStorage.getItem("tasks")) || [];

function saveTasks() {
    localStorage.setItem("tasks", JSON.stringify(tasks));
```

}

JAVASCRIPT: COMPLETE MASTERY

```
function renderTasks() {
    list.innerHTML = "";

    tasks.forEach((task, index) => {
        const li = document.createElement("li");

        li.innerHTML = `
            <span>${task}</span>
            <button data-index="${index}">
                Delete
            </button>
        `;

        list.appendChild(li);
    });
```

}

JAVASCRIPT: COMPLETE MASTERY

```
addBtn.addEventListener("click", () => {
    const task = input.value.trim();

    if (!task) {
        return;
    }

    tasks.push(task);

    saveTasks();
    renderTasks();

    input.value = "";
```

});

JAVASCRIPT: COMPLETE MASTERY

```
list.addEventListener("click", event => {
    if (event.target.tagName === "BUTTON") {
        const index = Number(event.target.dataset.index);

        tasks.splice(index, 1);

        saveTasks();
        renderTasks();
    }
```

});

JAVASCRIPT: COMPLETE MASTERY

```
renderTasks();
```

This uses:

Arrays

Objects/data

JSON

localStorage

DOM

events

event delegation

functions

conditions

template literals

100. Complete API Project

JAVASCRIPT: COMPLETE MASTERY

```
async function loadUsers() {
    try {
        const response =
            await fetch("https://jsonplaceholder.typicode.com/users");

        if (!response.ok) {
            throw new Error("Request failed");
        }

        const users = await response.json();

        users.forEach(user => {
            console.log(user.name);
        });

    } catch (error) {
        console.error(error.message);
    }
```

}

JAVASCRIPT: COMPLETE MASTERY

```
loadUsers();
```

Flow:

Function

CONCEPT MAP

```
  ↓
fetch()
  ↓
```

Promise

CONCEPT MAP

```
  ↓
await
  ↓
```

HTTP response

CONCEPT MAP

```
  ↓
```

JSON

CONCEPT MAP

```
  ↓
```

JavaScript object

CONCEPT MAP

```
  ↓
```

display/process

101. Important JavaScript Concepts You MUST Master

- □ Variables
- □ Data types
- □ Type conversion
- □ Operators
- □ Conditions
- □ Loops
- □ Functions
- □ Scope
- □ Hoisting
- □ Closures
- □ this
- □ Objects
- □ Arrays
- □ Destructuring
- □ Spread/Rest
- □ Prototype
- □ Classes
- □ DOM
- □ Events
- □ Event bubbling
- □ Event delegation
- □ JSON
- □ localStorage
- □ Callbacks
- □ Promises
- □ async/await
- □ Event loop
- □ Fetch
- □ Modules
- □ Map/Set
- □ Regex
- □ Debounce
- □ Throttle
- □ Error handling
- □ Recursion
- □ DSA

102. Most Important Dry-Run Questions

Question 1

JAVASCRIPT: COMPLETE MASTERY

```
    console.log(1);

    setTimeout(() => {
        console.log(2);
```

}, 0);

JAVASCRIPT: COMPLETE MASTERY

```
    Promise.resolve().then(() => {
        console.log(3);
```

});

JAVASCRIPT: COMPLETE MASTERY

```
    console.log(4);
```

Output

1

4

3

2

Why?

Synchronous

1

4

Microtask

3

Task

2

Question 2

JAVASCRIPT: COMPLETE MASTERY

```
let x = 10;

function test() {
    let x = 20;

    if (true) {
        let x = 30;
        console.log(x);
    }

    console.log(x);
```

}

JAVASCRIPT: COMPLETE MASTERY

```
test();
console.log(x);
```

Output:

30

20

10

Reason:

Global x = 10

Function x = 20

Block x = 30

Question 3 — Closure

JAVASCRIPT: COMPLETE MASTERY

```
function outer() {
    let x = 10;

    return function inner() {
        x++;
        return x;
    };
```

}

JAVASCRIPT: COMPLETE MASTERY

```
const fn = outer();

console.log(fn());
console.log(fn());
```

Output:

11

12

Because inner() remembers x.

Question 4 — Reference

JAVASCRIPT: COMPLETE MASTERY

```
const a = {
    value: 10
```

};

JAVASCRIPT: COMPLETE MASTERY

```
const b = a;

b.value = 20;

console.log(a.value);
```

Output:

20

Because:

CONCEPT MAP

```
a ──┐
    ↓
```

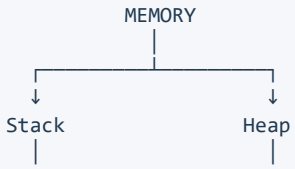
object

CONCEPT MAP

```
    ↑
b ──┘
```

103. JavaScript Memory Model — Simple View

CONCEPT MAP



primitives objects/arrays

references functions

A useful interview-level mental model:

CONCEPT MAP

```
Stack → execution/context information
Heap  → dynamically allocated objects
```

The exact engine implementation is more complex, but this model is useful for understanding references.

104. Garbage Collection

JavaScript automatically removes objects that can no longer be reached.

Simple idea:

Reachable object

CONCEPT MAP

```
      ↓
```

keep

Unreachable object

CONCEPT MAP

```
      ↓
```

Garbage Collector

CONCEPT MAP

```
      ↓
```

memory reclaimed

105. JavaScript Architecture

For a serious project:

project/

CONCEPT MAP

```
│
├── index.html
│
├── css/
│   └── style.css
│
├── js/
│   ├── main.js
│   ├── api.js
│   ├── utils.js
│   └── components.js
│
└── assets/
```

106. JavaScript Design Patterns

Important patterns:

Module Pattern

Factory Pattern

Singleton Pattern

Observer Pattern

Strategy Pattern

Adapter Pattern

MVC

Factory example

JAVASCRIPT: COMPLETE MASTERY

```
function createUser(name, role) {
    return {
        name,
        role,

        greet() {
            console.log(`Hello ${name}`);
        }
    };
```

}

JAVASCRIPT: COMPLETE MASTERY

```
const user1 = createUser("Aamir", "Engineer");
```

107. JavaScript Security Basics

Never blindly trust user input.

Avoid unsafe patterns like:

JAVASCRIPT: COMPLETE MASTERY

```
element.innerHTML = userInput;
```

Prefer:

JAVASCRIPT: COMPLETE MASTERY

```
element.textContent = userInput;
```

Other important concepts:

XSS

CSRF

CORS

Input validation

Authentication

Authorization

Secure cookies

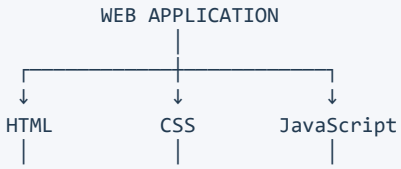
HTTPS

Easy meanings

Difficult word	Easy meaning
XSS	Attacker puts harmful script into a page
CORS	Browser rule controlling cross-origin requests
Authentication	Who are you?
Authorization	What are you allowed to do?
Validation	Is input acceptable?
Sanitization	Remove dangerous content

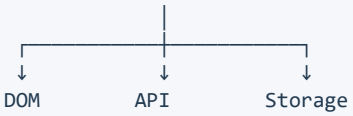
108. JavaScript + CSS + HTML Architecture

CONCEPT MAP



Structure Design Logic

CONCEPT MAP



109. Difficult Words → Easy Meaning

Word	Easy meaning
Variable	Named storage
Scope	Where a variable works
Hoisting	Declaration processing before normal execution
Closure	Function remembering outer variables
Callback	Function passed to another function
Promise	Future result
Async	Work that finishes later
Await	Wait for a Promise inside async function
Event Loop	Mechanism that coordinates async callbacks
DOM	HTML represented as objects

Word	Easy meaning
Event	Something that happens
Bubbling	Event moving from child toward parent
Prototype	Object used as another object's inheritance source
Encapsulation	Hiding internal details
Recursion	Function calling itself
API	Way programs communicate
JSON	Text format for structured data
Debounce	Wait until activity stops
Throttle	Limit how often something runs
Mutation	Changing existing data
Immutable	Don't directly change original data
Callback Queue	Waiting area for callbacks
Microtask	High-priority async job such as Promise reactions
Garbage Collection	Automatic cleanup of unreachable memory

110. Colorful Revision Notes

MUST KNOW

JAVASCRIPT: COMPLETE MASTERY

```
let / const
```

data types

operators

conditions

loops

functions

arrays

objects

DOM

events

fetch

promises

JAVASCRIPT: COMPLETE MASTERY

```
async/await
```

INTERVIEW TRAPS

== vs ===

null vs undefined

JAVASCRIPT: COMPLETE MASTERY

```
var vs let
```

this

closure

hoisting

event loop

Promise microtasks

reference vs value

DSA

Two pointers

Sliding window

Hash Map

Stack

Queue

Recursion

Binary search

Sorting

Trees

Graphs

Dynamic programming

ADVANCED

Prototype

Generators

Iterators

Symbols

WeakMap

WeakSet

Modules

Debouncing

Throttling

Memory

Performance

Design patterns

> COMMON MISTAKE

Don't use == unnecessarily

Don't overuse var

Don't blindly use innerHTML

Don't mutate objects accidentally

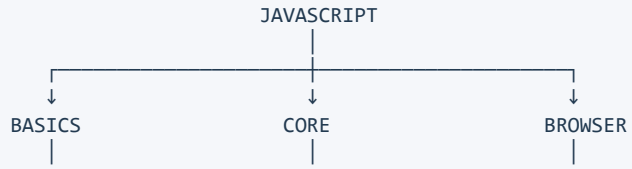
Don't forget error handling

Don't create unnecessary API calls

Don't ignore accessibility/security

111. JavaScript Master Mind Map

CONCEPT MAP



Variables Functions DOM

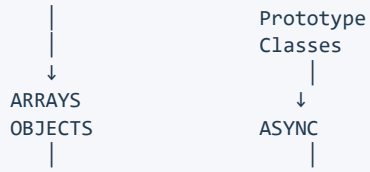
Types Scope Events

Operators Closure Forms

Conditions Hoisting Storage

Loops this BOM

CONCEPT MAP



map/filter/reduce Callback

destructuring Promise

spread/rest async/await

CONCEPT MAP



Arrays Fetch

Strings JSON

Hashing HTTP

Searching

Sorting

Recursion

CONCEPT MAP



Modules

Regex

Map/Set

Generators

Iterators

Debounce

Throttle

Security

Performance

Design Patterns

112. Memory Tricks

Variables

JAVASCRIPT: COMPLETE MASTERY

```
LET = Let it change
CONST = Constant assignment
VAR = Very old style
```

Array methods

CONCEPT MAP

```
MAP    → Modify
FILTER → Find matching
REDUCE → Reduce to one
FIND   → Find one
SOME   → At least one
EVERY  → All
```

Flex-style conceptual memory isn't relevant here; for JS async:

CALLBACK

CONCEPT MAP

```
    ↓
```

PROMISE

CONCEPT MAP

```
    ↓
```

ASYNC/AWAIT

Think of it as:

CONCEPT MAP

```
Old → Better → Cleaner
```

113. Interview Questions

1. What is JavaScript?

JavaScript is a programming language commonly used to add logic and interactivity to web applications.

2. let vs const?

JAVASCRIPT: COMPLETE MASTERY

```
let allows reassignment. const does not allow reassignment.
```

3. == vs ===?

== allows type coercion. === checks value and type strictly.

4. What is scope?

Scope defines where a variable can be accessed.

5. What is closure?

A function retaining access to variables from its outer lexical environment.

6. What is hoisting?

JavaScript's handling of declarations before normal execution reaches them.

7. What is the DOM?

An object representation of an HTML document.

8. What is event bubbling?

An event propagating from the target toward its ancestors.

9. What is event delegation?

Handling events for child elements through a parent listener.

10. What is a Promise?

An object representing the eventual success or failure of an asynchronous operation.

11. What is async/await?

Syntax that makes Promise-based asynchronous code easier to read.

12. What is the event loop?

A mechanism that coordinates execution between the call stack and asynchronous queues.

13. What is this?

A context-dependent reference determined by how a function is invoked, with special rules for arrow functions.

14. What is prototype inheritance?

Objects can access properties and methods through their prototype chain.

15. What is destructuring?

A syntax for extracting values from arrays or properties from objects.

16. Spread vs rest?

Same ... syntax, different purpose.

CONCEPT MAP

```
Spread → expand
Rest   → collect
```

17. map vs forEach?

map creates a new transformed array; forEach is generally used to perform an action for each element.

18. filter?

Creates a new array containing elements that pass a condition.

19. What is debouncing?

Delaying execution until a burst of activity stops for a specified period.

20. What is throttling?

Restricting execution to at most a controlled frequency.

114. Most Asked Coding Problems

Master these in order:

Level 1 — Basics

1. Reverse number
2. Reverse string
3. Palindrome
4. Factorial
5. Fibonacci
6. Prime number
7. Count digits
8. Sum of digits
9. Find maximum

10. Find minimum

Level 2 — Arrays

11. Two Sum
12. Remove duplicates
13. Move zeroes
14. Find missing number
15. Find duplicate
16. Maximum subarray
17. Rotate array
18. Merge sorted arrays
19. Intersection

20. Union

Level 3 — Strings

21. Anagram
22. First non-repeating character
23. Character frequency
24. Longest substring
25. Reverse words
26. Valid parentheses

27. String compression

Level 4 — Patterns

28. Two pointers
29. Sliding window
30. Prefix sum
31. Hash map
32. Binary search
33. Recursion

34. Backtracking

Level 5 — Advanced DSA

35. Linked list
36. Stack
37. Queue
38. Binary tree
39. BST
40. Heap
41. Graph BFS
42. Graph DFS
43. Dynamic programming
44. Greedy algorithms

45. Trie

115. Complete JavaScript Mastery Roadmap

PHASE 1

CONCEPT MAP

↓

Syntax

Variables

Types

Operators

Conditions

Loops

CONCEPT MAP

↓

PHASE 2

CONCEPT MAP

↓

Functions

Arrays

Strings

Objects

Destructuring

Spread/Rest

CONCEPT MAP

↓

PHASE 3

CONCEPT MAP

```
↓
```

Scope

Hoisting

Closure

this

Prototype

Classes

Inheritance

CONCEPT MAP

```
↓
```

PHASE 4

CONCEPT MAP

```
↓
```

DOM

Events

Forms

Storage

BOM

CONCEPT MAP

```
↓
```

PHASE 5

CONCEPT MAP

```
↓
```

Callbacks

Promises

JAVASCRIPT: COMPLETE MASTERY

```
async/await
```

Fetch

Event Loop

CONCEPT MAP

```
↓
```

PHASE 6

CONCEPT MAP

```
↓
```

ES Modules

Map

Set

Regex

Generators

Iterators

Symbols

CONCEPT MAP

↓

PHASE 7

CONCEPT MAP

↓

DSA

Arrays

Strings

Hashing

Stack

Queue

Linked List

Trees

Graphs

DP

CONCEPT MAP

↓

PHASE 8

CONCEPT MAP

↓

Frontend

HTML

CSS

JavaScript

API

Authentication

State Management

CONCEPT MAP

↓

PHASE 9

CONCEPT MAP

↓

Projects

Todo

Weather

Dashboard

E-commerce

Chat App

AI Application

CONCEPT MAP

```
        ↓
```

PHASE 10

CONCEPT MAP

```
↓
```

Interview

DSA

JavaScript internals

Projects

System design basics

Final JavaScript Cheat Sheet

VARIABLE

JAVASCRIPT: COMPLETE MASTERY

```
let / const
```

TYPE

string / number / boolean / null / undefined

object / bigint / symbol

CONTROL

JAVASCRIPT: COMPLETE MASTERY

```
if / else / switch
```

LOOPS

JAVASCRIPT: COMPLETE MASTERY

```
for / while / do while
for...of / for...in
```

FUNCTION

JAVASCRIPT: COMPLETE MASTERY

```
function
```

arrow function

callback

higher-order function

ARRAY

map

filter

reduce

find
some
every
sort
slice
splice
OBJECT
keys
values
entries
destructuring
spread
CORE
scope
closure
hoisting
this
prototype

JAVASCRIPT: COMPLETE MASTERY

```
class
```

inheritance
DOM
querySelector
createElement
append
remove
classList
EVENTS
click
input
submit
keydown
change
bubbling
delegation
ASYNC
callback
Promise
then
catch
finally

JAVASCRIPT: COMPLETE MASTERY

```
async
await
```

fetch

ADVANCED

Map

Set

WeakMap

WeakSet

Symbol

BigInt

Regex

Modules

Generators

Iterators

Debounce

Throttle

DSA

Two pointers

Sliding window

Hashing

Binary search

Recursion

Sorting

Trees

Graphs

DP

 The one-line memory formula

JAVASCRIPT MASTERY

=

Syntax

+ Logic

+ Functions

+ Arrays/Objects

+ Scope/Closure/this

+ DOM/Events

+ Async/Event Loop

+ APIs

+ DSA

+ Projects

+ Interview Practice

The most important concepts to deeply understand

If you have limited time, do not memorize everything equally. Focus especially on:

CONCEPT MAP

```
let/const → types → functions → arrays/objects → scope → closure → hoisting → this → prototype →
DOM → events → promises → async/await → event loop → fetch → DSA patterns.
```

That sequence takes you from beginner JavaScript to interview-level JavaScript.

PART 4 | Chapter 11: Object-Oriented Programming

Core concepts

Class and object

Attributes and methods

Constructor

self

Instance variables

Class variables

Instance methods

Class methods

Static methods

Four pillars

Encapsulation

Inheritance

Polymorphism

Abstraction

Advanced OOP

Single inheritance

Multiple inheritance

Multilevel inheritance

Hierarchical inheritance

Method overriding

Method overloading alternatives

Operator overloading

Abstract classes

Interfaces through abstract base classes

PYTHON OOP: CHAPTER 11

```
super()
```

Method Resolution Order

Composition

Aggregation

Magic methods

Properties

Getters and setters

Dataclasses

Chapter 11 — Object-Oriented Programming (OOP) in Python

CONCEPT MAP

```
Basics → Four Pillars → Advanced OOP → Design → Interview → Coding Problems
```

> Goal: After this chapter, you should be able to design real software using classes and objects, explain all major OOP concepts in interviews, and solve OOP-based coding problems confidently.

1. What is OOP?

PYTHON OOP: CHAPTER 11

```
OOP = Object-Oriented Programming
```

It is a programming style where we organize software around objects.

> Easy meaning

Instead of writing one huge program:

Data + Functions + Data + Functions + Data + Functions

we group related data and behavior together:

PYTHON OOP: CHAPTER 11

```
                OBJECT
              /          \
          DATA           BEHAVIOR
       attributes       methods
```

> Real-world example

Think about a Bank Account.

It has:

Data:

PYTHON OOP: CHAPTER 11

```
    account_holder
    balance
    account_number
```

Behavior:

PYTHON OOP: CHAPTER 11

```
    deposit()
    withdraw()
    check_balance()
```

In Python:

PYTHON OOP: CHAPTER 11

```
class BankAccount:

    def deposit(self):
        pass

    def withdraw(self):
        pass
```

An actual account becomes an object.

2. Class vs Object

Class

A class is a blueprint/template for creating objects.

Think:

PYTHON OOP: CHAPTER 11

```
Class = Building Blueprint
Object = Actual Building
```

> Example:

PYTHON OOP: CHAPTER 11

```
class Student:
    pass
```

The class describes what a student object can contain.

Object

An object is an actual instance created from a class.

PYTHON OOP: CHAPTER 11

```
class Student:
    pass
```

PYTHON OOP: CHAPTER 11

```
student1 = Student()
student2 = Student()
```

Diagram:

CONCEPT MAP

```
                CLASS
               Student
                  |
        ┌─────────┴─────────┐
        ↓                   ↓
   Object 1            Object 2
   student1            student2
```

Important interview definition

Class defines the structure and behavior; object is an actual instance of that class.

3. Attributes and Methods

A class normally contains:

Class

CONCEPT MAP

```
  |
  ├── Attributes → data/state
  |
  └── Methods → behavior/actions
```

> Example:

PYTHON OOP: CHAPTER 11

```
class Car:

    def __init__(self, brand, color):
        self.brand = brand
        self.color = color

    def drive(self):
        print("Car is driving")
```

Here:

CONCEPT MAP

```
brand → attribute
color → attribute
drive() → method
```

4. Constructor

A constructor is commonly used to initialize an object.

In Python:

PYTHON OOP: CHAPTER 11

```
__init__()
```

> Example:

PYTHON OOP: CHAPTER 11

```
class Student:

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Creating object:

PYTHON OOP: CHAPTER 11

```
student = Student("Aamir", 25)
```

What happens?

CONCEPT MAP

```
Student("Aamir", 25)
        ↓
__init__()
        ↓
self.name = "Aamir"
self.age = 25
        ↓
```

object initialized

> Important

Strictly speaking, __init__() initializes an already-created instance; __new__() is responsible for creating the instance.

For beginner/interview purposes, people often call __init__() the constructor.

5. self

self is one of the most important Python OOP concepts.

> Easy meaning

self means the current object.

> Example:

PYTHON OOP: CHAPTER 11

```
class Student:

    def __init__(self, name):
        self.name = name

    def show(self):
        print(self.name)
```

Create:

PYTHON OOP: CHAPTER 11

```
s1 = Student("Aamir")
s2 = Student("Rahul")
```

Think:

s1

CONCEPT MAP

```
  |
  └── name → Aamir
```

s2

CONCEPT MAP

```
  |
  └── name → Rahul
```

When:

PYTHON OOP: CHAPTER 11

```
s1.show()
```

Python conceptually passes s1 as the first argument:

PYTHON OOP: CHAPTER 11

```
Student.show(s1)
```

Therefore:

PYTHON OOP: CHAPTER 11

```
self == s1
```

For:

PYTHON OOP: CHAPTER 11

```
s2.show()
self == s2
```

Dry Run — self

PYTHON OOP: CHAPTER 11

```
class Person:

    def __init__(self, name):
        self.name = name

    def greet(self):
        print("Hello", self.name)
```

PYTHON OOP: CHAPTER 11

```
p1 = Person("Aamir")
p2 = Person("Azad")

p1.greet()
p2.greet()
```

> Dry run

Step	Object	self	self.name	Output
1	p1	p1	Aamir	—
2	p2	p2	Azad	—
3	p1.greet()	p1	Aamir	Hello Aamir
4	p2.greet()	p2	Azad	Hello Azad

6. Instance Variables

An instance variable belongs to a particular object.

PYTHON OOP: CHAPTER 11

```
class Student:

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Here:

student1.name

student1.age

student2.name

student2.age

can contain different values.

PYTHON OOP: CHAPTER 11

```
s1 = Student("Aamir", 25)
s2 = Student("Rahul", 22)
```

Diagram:

Student class

CONCEPT MAP



s1 s2

CONCEPT MAP

```
|          |
name=Aamir  name=Rahul
age=25      age=22
```

7. Class Variables

A class variable is shared by objects of the class.

PYTHON OOP: CHAPTER 11

```
class Student:

    school = "ABC School"

    def __init__(self, name):
        self.name = name
```

Now:

PYTHON OOP: CHAPTER 11

```
s1 = Student("Aamir")
s2 = Student("Rahul")

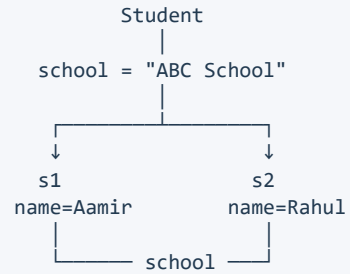
print(s1.school)
print(s2.school)
```

Both can access:

ABC School

Diagram:

CONCEPT MAP



Instance vs class variable

Instance variable	Class variable
Belongs to object	Belongs to class
Usually self.x	Usually Class.x
Different for each object	Shared
Created per instance	One class-level value

8. Instance Methods

Normal methods that work with object data.

PYTHON OOP: CHAPTER 11

```
class BankAccount:

    def __init__(self, balance):
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
```

Usage:

PYTHON OOP: CHAPTER 11

```
account = BankAccount(1000)

account.deposit(500)

print(account.balance)
```

Output:

1500

9. Class Methods

A class method works with the class, not primarily one individual object.

Use:

@classmethod

and cls.

PYTHON OOP: CHAPTER 11

```
class Student:

    school = "ABC"

    @classmethod
    def change_school(cls, name):
        cls.school = name
```

Usage:

PYTHON OOP: CHAPTER 11

```
Student.change_school("XYZ")

print(Student.school)
```

Output:

XYZ

self vs cls

CONCEPT MAP

```
self → current object
cls  → current class
```

10. Static Methods

A static method does not automatically receive self or cls.

PYTHON OOP: CHAPTER 11

```
class MathUtils:

    @staticmethod
    def add(a, b):
        return a + b
```

Usage:

PYTHON OOP: CHAPTER 11

```
print(MathUtils.add(10, 20))
```

Output:

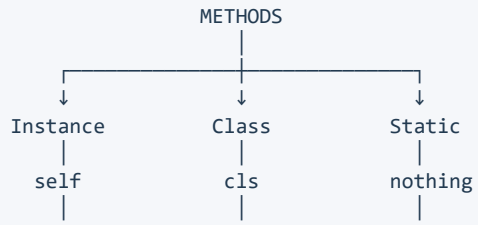
30

When to use?

When a function logically belongs to the class but doesn't need object/class state.

11. Three Types of Methods

CONCEPT MAP



object data class data utility logic

Method	First parameter	Used for
Instance	self	Object data
Class	cls	Class data
Static	None automatically	Utility logic

12. Four Pillars of OOP

This is extremely important for interviews.

CONCEPT MAP



Encapsulation Inheritance Polymorphism

CONCEPT MAP



Memory trick:

EIPA

CONCEPT MAP

```
E → Encapsulation
I → Inheritance
P → Polymorphism
A → Abstraction
```

Think:

PYTHON OOP: CHAPTER 11

```
EIPA = "Encapsulate Inheritance, Polymorphism, Abstraction."
```

13. Encapsulation

> Easy meaning

Encapsulation means keeping data and the methods that work on that data together, while controlling direct access to internal data.

> Real-world example:

Bank account:

Bank Account

CONCEPT MAP

```
│
├── balance
├── deposit()
└── withdraw()
```

You should not allow anyone to directly modify balance however they want.

14. Encapsulation in Python

Python does not enforce private fields exactly like some languages.

Conventions:

CONCEPT MAP

```
name       → public
_name      → internal/protected convention
__name     → name-mangled/private-style
```

> Example:

PYTHON OOP: CHAPTER 11

```
class BankAccount:

    def __init__(self, balance):
        self.__balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):
        return self.__balance
```

Usage:

PYTHON OOP: CHAPTER 11

```
account = BankAccount(1000)

account.deposit(500)

print(account.get_balance())
```

Output:

1500

15. Name Mangling

With:

PYTHON OOP: CHAPTER 11

```
self.__balance
```

Python internally changes the name roughly to:

_BankAccount__balance

This is called name mangling.

It is not absolute security/privacy. It mainly prevents accidental access/name collisions.

16. Inheritance

> Easy meaning

Inheritance allows one class to reuse or extend another class.

> Real-world example:

Vehicle

CONCEPT MAP

```
   │
   ├── Car
   ├── Bike
   └── Truck
```

Parent:

PYTHON OOP: CHAPTER 11

```
class Vehicle:

    def move(self):
        print("Vehicle moves")
```

Child:

PYTHON OOP: CHAPTER 11

```
class Car(Vehicle):
    pass
```

Usage:

PYTHON OOP: CHAPTER 11

```
car = Car()
car.move()
```

Output:

Vehicle moves

17. Parent and Child

Parent Class

CONCEPT MAP

```
      |
      | inheritance
      ↓
```

Child Class

Terminology:

Term	Meaning
Parent	Base/Super class
Child	Derived/Sub class
Inherit	Receive/reuse behavior

18. Single Inheritance

CONCEPT MAP

```
One parent → one child.
```

Animal

CONCEPT MAP

```
    ↓
```

Dog

PYTHON OOP: CHAPTER 11

```
class Animal:

    def eat(self):
        print("Eating")
```

PYTHON OOP: CHAPTER 11

```
class Dog(Animal):

    def bark(self):
        print("Barking")
```

19. Multilevel Inheritance

Inheritance chain.

Animal

CONCEPT MAP

```
    ↓
```

Mammal

CONCEPT MAP

```
    ↓
```

Dog

PYTHON OOP: CHAPTER 11

```
class Animal:
    def eat(self):
        print("Eating")
```

PYTHON OOP: CHAPTER 11

```
class Mammal(Animal):
    def walk(self):
        print("Walking")
```

PYTHON OOP: CHAPTER 11

```
class Dog(Mammal):
    def bark(self):
        print("Barking")
```

Dog gets:

PYTHON OOP: CHAPTER 11

```
eat()
walk()
bark()
```

20. Hierarchical Inheritance

CONCEPT MAP

```
One parent → multiple children.

        Animal
       /      \
     Dog      Cat
class Animal:
    def eat(self):
        print("Eating")
```

PYTHON OOP: CHAPTER 11

```
class Dog(Animal):
    def bark(self):
        print("Barking")
```

PYTHON OOP: CHAPTER 11

```
class Cat(Animal):
    def meow(self):
        print("Meowing")
```

21. Multiple Inheritance

One child inherits from multiple parents.

Father Mother

\ /

PYTHON OOP: CHAPTER 11

```
    \        /
      Child
```

Python supports multiple inheritance.

PYTHON OOP: CHAPTER 11

```
class Father:

    def skills(self):
        print("Driving")
```

PYTHON OOP: CHAPTER 11

```
class Mother:

    def cooking(self):
        print("Cooking")
```

PYTHON OOP: CHAPTER 11

```
class Child(Father, Mother):
    pass
```

Usage:

PYTHON OOP: CHAPTER 11

```
c = Child()

c.skills()
c.cooking()
```

22. Diamond Problem

Multiple inheritance can create ambiguity.

PYTHON OOP: CHAPTER 11

```
        A
       / \
      B   C
       \ /
        D
```

Question:

If B and C both override the same method, which one should D use?

Python solves this using:

Method Resolution Order — MRO

23. MRO

PYTHON OOP: CHAPTER 11

```
MRO = Method Resolution Order
```

It determines the order Python searches for methods.

> Example:

PYTHON OOP: CHAPTER 11

```
class A:
    def show(self):
        print("A")
```

PYTHON OOP: CHAPTER 11

```
class B(A):
    def show(self):
        print("B")
```

PYTHON OOP: CHAPTER 11

```
class C(A):
    def show(self):
        print("C")
```

PYTHON OOP: CHAPTER 11

```
class D(B, C):
    pass
```

Check:

PYTHON OOP: CHAPTER 11

```
print(D.mro())
```

Conceptually:

CONCEPT MAP

```
D → B → C → A → object
```

Python uses C3 linearization to calculate MRO.

Memory Trick

For:

PYTHON OOP: CHAPTER 11

```
class D(B, C):
```

think:

D

CONCEPT MAP

```
↓
```

B

CONCEPT MAP

```
↓
```

C

CONCEPT MAP

```
↓
```

A

CONCEPT MAP

```
↓
```

object

Python searches according to the MRO.

24. Method Overriding

Child provides a different implementation of a parent method.

PYTHON OOP: CHAPTER 11

```
class Animal:

    def sound(self):
        print("Some sound")
```

PYTHON OOP: CHAPTER 11

```
class Dog(Animal):

    def sound(self):
        print("Bark")

dog = Dog()
dog.sound()
```

Output:

Bark

Diagram:

CONCEPT MAP

```
Animal.sound()
       ↓
```

overridden

CONCEPT MAP

```
       ↓
Dog.sound()
```

25. super()

PYTHON OOP: CHAPTER 11

```
super() allows a child class to access parent behavior.

class Animal:

    def __init__(self, name):
        self.name = name
```

PYTHON OOP: CHAPTER 11

```
class Dog(Animal):

    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed
```

Diagram:

CONCEPT MAP

```
Dog.__init__()
     |
     ├── super().__init__()
     |        ↓
     |    Animal.__init__()
     |
     └── self.breed
```

26. Polymorphism

> Easy meaning

One interface, many forms.

> Example:

Animal

CONCEPT MAP

```
  |
  ├── Dog → bark
  ├── Cat → meow
  └── Cow → moo
```

All can have:

PYTHON OOP: CHAPTER 11

```
sound()
```

but behave differently.

PYTHON OOP: CHAPTER 11

```
class Dog:

    def sound(self):
        return "Bark"
```

PYTHON OOP: CHAPTER 11

```
class Cat:

    def sound(self):
        return "Meow"
```

PYTHON OOP: CHAPTER 11

```
animals = [Dog(), Cat()]

for animal in animals:
    print(animal.sound())
```

Output:

Bark

Meow

27. Duck Typing

Python uses a powerful form of runtime polymorphism called duck typing.

Famous idea:

If it behaves like the required thing, Python can use it.

PYTHON OOP: CHAPTER 11

```
class Dog:

    def speak(self):
        print("Bark")
```

PYTHON OOP: CHAPTER 11

```
class Robot:

    def speak(self):
        print("Hello")
```

PYTHON OOP: CHAPTER 11

```
def make_speak(obj):
    obj.speak()
```

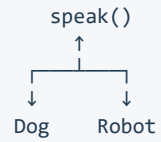
PYTHON OOP: CHAPTER 11

```
make_speak(Dog())
make_speak(Robot())
```

No common parent is required.

Diagram:

CONCEPT MAP



28. Method Overloading

Languages like Java/C++ commonly support multiple methods with the same name but different parameter lists.

Python does not support traditional compile-time method overloading by signature.

This will simply replace the earlier definition:

PYTHON OOP: CHAPTER 11

```
class Test:

    def add(self, a):
        pass

    def add(self, a, b):
        pass
```

Only the second add() remains.

29. 29. Python Alternatives to Overloading

Default arguments

PYTHON OOP: CHAPTER 11

```
class Calculator:

    def add(self, a, b=0):
        return a + b
```

Works:

PYTHON OOP: CHAPTER 11

```
c = Calculator()

print(c.add(5))
print(c.add(5, 10))
```

- args

PYTHON OOP: CHAPTER 11

```
class Calculator:

    def add(self, *numbers):
        return sum(numbers)
```

Now:

PYTHON OOP: CHAPTER 11

```
c.add(1)
c.add(1, 2)
c.add(1, 2, 3, 4)
```

30. Operator Overloading

Python allows operators to work with your own classes using special methods.

> Example:

PYTHON OOP: CHAPTER 11

```
class Point:

    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(
            self.x + other.x,
            self.y + other.y
        )
```

Usage:

PYTHON OOP: CHAPTER 11

```
p1 = Point(2, 3)
p2 = Point(4, 5)

p3 = p1 + p2

print(p3.x, p3.y)
```

Output:

6 8

What happened?

p1 + p2

CONCEPT MAP

```
   ↓
p1.__add__(p2)
   ↓
```

new Point

31. Magic Methods

Magic methods usually have names like:

__name__

They allow Python objects to interact naturally with language features.

Examples:

__init__

__str__

__repr__

__len__

__add__

__eq__

__lt__

__getitem__

__iter__

__next__

__enter__

__exit__

32. __str__

Controls a user-friendly string representation.

PYTHON OOP: CHAPTER 11

```
class Student:

    def __init__(self, name):
        self.name = name

    def __str__(self):
        return f"Student: {self.name}"

s = Student("Aamir")

print(s)
```

Output:

Student: Aamir

33. __repr__

Used for a more developer-focused representation.

PYTHON OOP: CHAPTER 11

```
class Student:

    def __init__(self, name):
        self.name = name

    def __repr__(self):
        return f"Student(name={self.name!r})"
```

34. __len__

PYTHON OOP: CHAPTER 11

```
class Team:

    def __init__(self, members):
        self.members = members

    def __len__(self):
        return len(self.members)
```

Now:

PYTHON OOP: CHAPTER 11

```
team = Team(["A", "B", "C"])

print(len(team))
```

Output:

3

35. __eq__

Controls equality.

PYTHON OOP: CHAPTER 11

```
class Person:

    def __init__(self, name):
        self.name = name

    def __eq__(self, other):
        return self.name == other.name
```

36. Abstract Classes

> Easy meaning

An abstract class is a class designed as a base/template where certain methods are required from child classes.

Python uses:

PYTHON OOP: CHAPTER 11

```
from abc import ABC, abstractmethod
```

> Example:

PYTHON OOP: CHAPTER 11

```
from abc import ABC, abstractmethod
```

PYTHON OOP: CHAPTER 11

```
class Animal(ABC):

    @abstractmethod
    def sound(self):
        pass
```

Child:

PYTHON OOP: CHAPTER 11

```
class Dog(Animal):

    def sound(self):
        print("Bark")
```

37. Why Abstraction?

Suppose you build a payment system.

You know every payment method must:

PYTHON OOP: CHAPTER 11

```
pay()
```

But implementation differs.

Payment

CONCEPT MAP

```
    │
    ├── CreditCard
    ├── UPI
    └── PayPal
```

Each must implement:

PYTHON OOP: CHAPTER 11

```
pay()
```

38. Interface Through ABC

Python doesn't have a separate interface keyword like Java.

A common Python approach is to use an abstract base class.

PYTHON OOP: CHAPTER 11

```
from abc import ABC, abstractmethod
```

PYTHON OOP: CHAPTER 11

```
class Payment(ABC):

    @abstractmethod
    def pay(self, amount):
        pass
```

Implementations:

PYTHON OOP: CHAPTER 11

```
class UPI(Payment):

    def pay(self, amount):
        print(f"Paid ₹{amount} using UPI")
```

PYTHON OOP: CHAPTER 11

```
class Card(Payment):

    def pay(self, amount):
        print(f"Paid ₹{amount} using Card")
```

39. Abstraction vs Encapsulation

Very common interview question.

Abstraction

Hide unnecessary implementation details.

Encapsulation

Bundle data/behavior and control access to internal state.

Memory:

ABSTRACTION

"What should the user see?"

ENCAPSULATION

"How do I protect/manage the internal data?"

> Example:

ATM:

User sees:

PYTHON OOP: CHAPTER 11

```
    Withdraw
    Deposit
    Balance
```

User doesn't see:

PYTHON OOP: CHAPTER 11

```
    internal banking logic
    database operations
    transaction processing
```

That's abstraction.

40. Composition

Very important software design concept.

Instead of:

Car IS-A Vehicle

inheritance represents an IS-A relationship.

Composition represents:

Car HAS-A Engine

> Example:

PYTHON OOP: CHAPTER 11

```
class Engine:

    def start(self):
        print("Engine started")
```

PYTHON OOP: CHAPTER 11

```
class Car:

    def __init__(self):
        self.engine = Engine()

    def start(self):
        self.engine.start()
        print("Car started")
```

Diagram:

Car

CONCEPT MAP

```
  │
  └── has-a
        ↓
     Engine
```

41. Inheritance vs Composition

Inheritance	Composition
IS-A	HAS-A
Parent-child relationship	Object contains another object
Reuse through hierarchy	Reuse through objects
Can create tight coupling	Often more flexible
class Dog(Animal)	Car contains Engine

Memory trick

CONCEPT MAP

```
IS-A  → Inheritance
HAS-A → Composition
```

42. Aggregation

Aggregation is also a HAS-A relationship, but the contained object can exist independently.

> Example:

Department

CONCEPT MAP

```
      │
      ├── Teacher
      └── Teacher
```

A teacher can exist without that particular department.

PYTHON OOP: CHAPTER 11

```
class Teacher:

    def __init__(self, name):
        self.name = name
```

PYTHON OOP: CHAPTER 11

```
class Department:

    def __init__(self, teachers):
        self.teachers = teachers
```

43. Composition vs Aggregation

Composition

Strong ownership.

House

CONCEPT MAP

```
   │
   └── Room
```

The room is conceptually part of that house.

Aggregation

Loose relationship.

University

CONCEPT MAP

```
    │
    └── Student
```

A student can exist separately from one particular university.

In real Python code, the difference is mainly a design relationship, not a special language feature.

44. Properties

Properties allow controlled attribute access using methods behind normal attribute syntax.

PYTHON OOP: CHAPTER 11

```
class Person:

    def __init__(self, age):
        self._age = age

    @property
    def age(self):
        return self._age
```

Usage:

PYTHON OOP: CHAPTER 11

```
p = Person(25)

print(p.age)
```

Notice:

p.age

not:

PYTHON OOP: CHAPTER 11

```
p.age()
```

45. Getter and Setter

PYTHON OOP: CHAPTER 11

```
class Person:

    def __init__(self, age):
        self._age = age

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):

        if value < 0:
            raise ValueError("Age cannot be negative")

        self._age = value
```

Usage:

PYTHON OOP: CHAPTER 11

```
p = Person(25)

print(p.age)

p.age = 30

print(p.age)
```

46. Why Use Properties?

Without validation:

PYTHON OOP: CHAPTER 11

```
p.age = -500
```

Bad data.

With setter:

@age.setter

PYTHON OOP: CHAPTER 11

```
def age(self, value):
    if value < 0:
        raise ValueError(...)
```

Diagram:

User

CONCEPT MAP

```
  ↓
p.age = -5
  ↓
```

setter

CONCEPT MAP

```
  ↓
```

validation

CONCEPT MAP

```
  ↓
```

reject

47. Dataclasses

Dataclasses reduce repetitive code for classes mainly used to store data.

PYTHON OOP: CHAPTER 11

```
from dataclasses import dataclass
```

@dataclass

PYTHON OOP: CHAPTER 11

```
class Student:
    name: str
    age: int
    course: str
```

Now:

PYTHON OOP: CHAPTER 11

```
s = Student("Aamir", 25, "AI/ML")

print(s)
```

Dataclasses automatically provide useful methods such as:

__init__

__repr__

__eq__

depending on configuration.

48. Dataclass Example

PYTHON OOP: CHAPTER 11

```
from dataclasses import dataclass
```

@dataclass

PYTHON OOP: CHAPTER 11

```
class Product:
    name: str
    price: float
    quantity: int = 1

    @property
    def total(self):
        return self.price * self.quantity
```

PYTHON OOP: CHAPTER 11

```
p = Product("Laptop", 60000, 2)

print(p.total)
```

Formula:

PYTHON OOP: CHAPTER 11

```
total_price = price × quantity
```

For:

PYTHON OOP: CHAPTER 11

```
price = ₹60,000
quantity = 2
total = ₹120,000
```

49. Complete OOP Example — Bank System

This combines many concepts.

PYTHON OOP: CHAPTER 11

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
```

@dataclass

PYTHON OOP: CHAPTER 11

```
class Customer:
    name: str
    customer_id: int
```

PYTHON OOP: CHAPTER 11

```
class Account(ABC):

    bank_name = "ABC Bank"

    def __init__(self, customer, balance=0):
        self.customer = customer
        self._balance = balance

    @property
    def balance(self):
        return self._balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Amount must be positive")

        self._balance += amount

    @abstractmethod
    def withdraw(self, amount):
        pass

    @classmethod
    def change_bank_name(cls, name):
        cls.bank_name = name
```

Output:

12000

Dry Run — Bank System

Initial:

PYTHON OOP: CHAPTER 11

```
balance = 10000
```

Deposit

PYTHON OOP: CHAPTER 11

```
amount = 5000
```

> Formula:

PYTHON OOP: CHAPTER 11

```
new_balance = old_balance + amount
```

10000 + 5000 = 15000

Withdraw

PYTHON OOP: CHAPTER 11

```
amount = 3000
```

> Formula:

PYTHON OOP: CHAPTER 11

```
new_balance = old_balance - amount
```

15000 - 3000 = 12000

Operation	Amount	Balance
Initial	—	₹10,000
Deposit	+₹5,000	₹15,000
Withdraw	-₹3,000	₹12,000

50. Complete OOP Project — E-Commerce

Let's combine:

classes

objects

inheritance

encapsulation

abstraction

polymorphism

composition

properties

dataclasses

PYTHON OOP: CHAPTER 11

```
class methods
```

static methods

magic methods

PYTHON OOP: CHAPTER 11

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
```

@dataclass

PYTHON OOP: CHAPTER 11

```
class Product:
    name: str
    price: float
    quantity: int = 1

    @property
    def total_price(self):
        return self.price * self.quantity

    def __str__(self):
        return f"{self.name} - ₹{self.price}"
```

PYTHON OOP: CHAPTER 11

```
class Payment(ABC):

    @abstractmethod
    def pay(self, amount):
        pass
```

PYTHON OOP: CHAPTER 11

```
class UPI(Payment):

    def pay(self, amount):
        return f"Paid ₹{amount} using UPI"
```

PYTHON OOP: CHAPTER 11

```
class Card(Payment):

    def pay(self, amount):
        return f"Paid ₹{amount} using Card"
```

Dry Run — E-Commerce

Products:

PYTHON OOP: CHAPTER 11

```
Laptop = 60,000 × 1
Mouse  = 1,000 × 2
```

Formula:

PYTHON OOP: CHAPTER 11

```
product_total = price × quantity
```

Laptop:

60,000 × 1 = 60,000

Mouse:

1,000 × 2 = 2,000

Subtotal:

60,000 + 2,000 = 62,000

Tax:

PYTHON OOP: CHAPTER 11

```
tax = subtotal × tax_rate
```

62,000 × 0.18 = 11,160

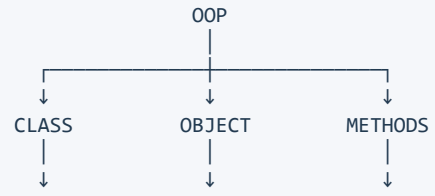
Final:

PYTHON OOP: CHAPTER 11

```
total = subtotal + tax
      = 62,000 + 11,160
      = 73,160
```

51. OOP Relationship Diagram

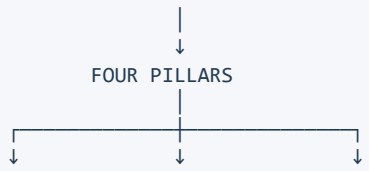
CONCEPT MAP



Attributes Instance Behavior

Constructor State Actions

CONCEPT MAP



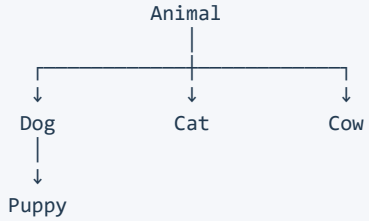
Encapsulation Inheritance Polymorphism

CONCEPT MAP



52. Inheritance Diagram

CONCEPT MAP



This demonstrates:

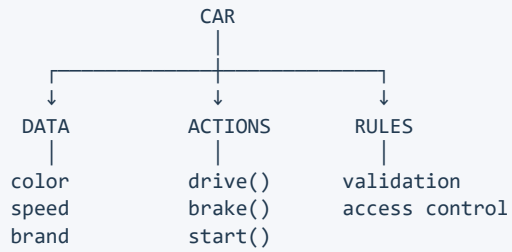
CONCEPT MAP

```
Animal → hierarchical inheritance
Dog → Puppy → multilevel inheritance
```

53. Complete OOP Mental Model

Think about a car.

CONCEPT MAP



Then:

Encapsulation

CONCEPT MAP

↓

keep car data + methods together

Inheritance

CONCEPT MAP

```
↓
```

ElectricCar extends Car

Polymorphism

CONCEPT MAP

```
↓
```

different cars implement start() differently

Abstraction

CONCEPT MAP

```
↓
```

user only needs start(), not internal engine details

54. Common OOP Mistakes

Mistake 1 — Forgetting self

Wrong:

PYTHON OOP: CHAPTER 11

```
class Student:

    def show():
        print("Hello")
```

Correct:

PYTHON OOP: CHAPTER 11

```
class Student:

    def show(self):
        print("Hello")
```

Mistake 2 — Confusing class and instance variables

PYTHON OOP: CHAPTER 11

```
class Student:

    school = "ABC"

    def __init__(self, name):
        self.name = name
```

school is class-level.

name is instance-level.

Mistake 3 — Calling instance method incorrectly

PYTHON OOP: CHAPTER 11

```
Student.show()
```

If it needs an instance, this normally fails because no self was supplied.

Correct:

PYTHON OOP: CHAPTER 11

```
student = Student(...)
student.show()
```

Mistake 4 — Misusing @staticmethod

Don't use static methods simply because they look convenient.

Ask:

Does this function need object state?

CONCEPT MAP

```
If yes → instance method.
```

Does it need class state?

CONCEPT MAP

```
If yes → class method.
```

Does it need neither?

CONCEPT MAP

```
→ static method may be appropriate.
```

55. Common Inheritance Mistake

Don't create inheritance just to reuse a few lines.

Bad conceptual design:

Printer

CONCEPT MAP

```
    ↓
```

Database

If there is no real IS-A relationship, composition may be better.

56. Common Encapsulation Mistake

This:

PYTHON OOP: CHAPTER 11

```
self.__balance
```

doesn't mean cryptographically secure/private.

It mainly prevents accidental direct access and name collisions.

57. Common super() Mistake

When using multiple inheritance, don't assume:

PYTHON OOP: CHAPTER 11

```
super()
```

always means "my direct parent."

More accurately:

PYTHON OOP: CHAPTER 11

```
super() follows the class's MRO.
```

This becomes very important in multiple inheritance.

58. OOP Interview Questions

Q1. What is OOP?

OOP is a programming paradigm that organizes software around objects containing data and behavior.

Q2. What is a class?

A class is a blueprint for creating objects.

Q3. What is an object?

An object is an instance of a class.

Q4. What is self?

self refers to the current object when using instance methods.

Q5. What is __init__()?

It initializes an instance after it has been created.

Q6. Instance variable vs class variable?

Instance variables belong to individual objects; class variables are associated with the class and can be shared.

Q7. What is encapsulation?

Bundling related state and behavior together while controlling access to internal state.

Q8. What is inheritance?

A mechanism through which one class can reuse or extend another class.

Q9. What is polymorphism?

The ability to use a common interface while different objects provide different behavior.

Q10. What is abstraction?

Exposing essential behavior while hiding unnecessary implementation details.

Q11. What is method overriding?

A child class provides its own implementation of an inherited method.

Q12. Does Python support traditional method overloading?

No, not by defining multiple methods with the same name and different signatures. Later definitions replace earlier ones. Python commonly uses default parameters, *args, and other techniques instead.

Q13. What is multiple inheritance?

A class inheriting from more than one parent class.

PYTHON OOP: CHAPTER 11

```
class Child(Parent1, Parent2):
    pass
```

Q14. What is MRO?

MRO is the order in which Python searches classes for methods and attributes.

Q15. What is super()?

It provides access to the next class in the MRO, commonly used to cooperate with inherited implementations.

Q16. What is composition?

Building an object using other objects.

Car HAS-A Engine

Q17. Inheritance vs composition?

CONCEPT MAP

```
Inheritance → IS-A
Composition  → HAS-A
```

Q18. What is a static method?

A method that does not automatically receive self or cls.

Q19. What is a class method?

A method that receives the class as cls.

Q20. What is a magic method?

A specially named method such as __init__, __str__, or __add__ that integrates objects with Python's language features.

Q21. What is a property?

A mechanism that lets methods behave like managed attributes.

Q22. What is a dataclass?

A convenient way to create classes primarily intended to hold structured data with generated methods.

Q23. What is duck typing?

Python focuses on whether an object supports the required operation rather than requiring a specific class hierarchy.

Q24. What is an abstract class?

A class that can define required behavior for subclasses and may contain abstract methods.

Q25. Does Python have interfaces?

It has no separate interface keyword. Abstract base classes and protocols are common ways to express interface-like designs.

59. Most Conceptual OOP Coding Problems

Practice these:

Beginner

1. Create Student class
2. Create BankAccount
3. Create Employee
4. Create Rectangle
5. Create Circle
6. Create Book
7. Create Car
8. Create ShoppingCart
9. Create Library

10. Create ATM

Intermediate

11. Inheritance hierarchy
12. Employee salary system
13. Bank account inheritance
14. Shape polymorphism
15. Payment system
16. Notification system

17. Vehicle rental system
18. E-commerce cart
19. Library management

20. School management

Advanced

21. Multiple inheritance + MRO
22. Abstract payment system
23. Plugin architecture
24. Logging framework
25. Role-based authentication
26. Inventory management
27. Parking lot system
28. Food delivery system
29. Ride-sharing system

30. ATM machine design

60. OOP Problem — Shape System

PYTHON OOP: CHAPTER 11

```
from abc import ABC, abstractmethod
import math
```

PYTHON OOP: CHAPTER 11

```
class Shape(ABC):

    @abstractmethod
    def area(self):
        pass
```

PYTHON OOP: CHAPTER 11

```
class Circle(Shape):

    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return math.pi * self.radius ** 2
```

PYTHON OOP: CHAPTER 11

```
class Rectangle(Shape):

    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height
```

PYTHON OOP: CHAPTER 11

```
shapes = [
    Circle(5),
    Rectangle(10, 20)
```

]

PYTHON OOP: CHAPTER 11

```
for shape in shapes:
```

This demonstrates:

ABC

CONCEPT MAP

```
  ↓
```

Abstraction

CONCEPT MAP

```
  ↓
```

Circle / Rectangle

CONCEPT MAP

```
  ↓
```

Polymorphism

61. Important OOP Formulas

OOP itself doesn't have many mathematical formulas, but your domain classes often do.

Rectangle

PYTHON OOP: CHAPTER 11

```
Area = width × height
```

Where:

PYTHON OOP: CHAPTER 11

```
width  = horizontal size
height = vertical size
```

Circle

PYTHON OOP: CHAPTER 11

```
Area = π × radius²
```

Bank

PYTHON OOP: CHAPTER 11

```
new_balance = old_balance + deposit
new_balance = old_balance - withdrawal
```

Shopping

PYTHON OOP: CHAPTER 11

```
item_total = price × quantity
subtotal = Σ item_total
tax = subtotal × tax_rate
final_total = subtotal + tax
```

62. OOP Color Notes

MUST REMEMBER

PYTHON OOP: CHAPTER 11

```
class
```

object

self

__init__

instance variable

PYTHON OOP: CHAPTER 11

```
class variable
```

instance method

PYTHON OOP: CHAPTER 11

```
class method
```

staticmethod

FOUR PILLARS

CONCEPT MAP

```
E → Encapsulation
I → Inheritance
P → Polymorphism
A → Abstraction
```

ADVANCED

MRO

PYTHON OOP: CHAPTER 11

```
super()
```

composition

aggregation

ABC

magic methods

properties

dataclasses

duck typing

INTERVIEW TRAPS

Python doesn't traditionally overload methods by signature

__init__ initializes; __new__ creates

private-style __name__ uses name mangling

PYTHON OOP: CHAPTER 11

```
super() follows MRO
```

composition is often preferable to deep inheritance

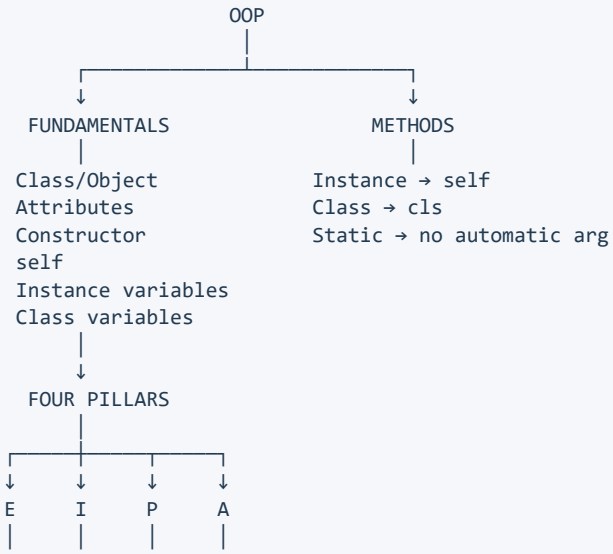
CONCEPT MAP

```
63. Difficult Words → Easy Meaning
```

Difficult word	Easy meaning
Paradigm	Programming style
Class	Blueprint
Object	Actual thing created from blueprint
Attribute	Object data
Method	Function inside class
Constructor	Initialization mechanism
Instance	Individual object
Encapsulation	Keep data + behavior together/control access
Inheritance	Reuse/extend another class
Polymorphism	Same interface, different behavior
Abstraction	Hide unnecessary details
Override	Replace inherited behavior
Overload	Same operation with different input forms
MRO	Method search order
Composition	One object contains another
Aggregation	Loose object relationship
Abstract	Incomplete/template-like
Interface	Required set of operations
Property	Managed attribute access
Getter	Reads/controls a value
Setter	Changes/validates a value
Dataclass	Easy data-holding class
Duck typing	If it supports the needed behavior, use it
Magic method	Special Python protocol method
Name mangling	Python changes __name internally
Coupling	How strongly components depend on each other
Cohesion	How closely related responsibilities are

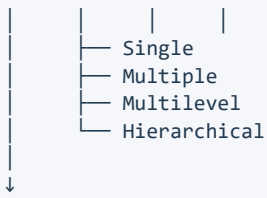
64. Ultimate OOP Mind Map

CONCEPT MAP



Encaps Inherit Poly Abstract

CONCEPT MAP



ADVANCED

CONCEPT MAP



MRO super() Composition

CONCEPT MAP

Magic Methods Aggregation

CONCEPT MAP

Operator Overloading

CONCEPT MAP

Properties

CONCEPT MAP

Getters/Setters

CONCEPT MAP

```
↓
```

ABC

CONCEPT MAP

```
↓
```

Dataclasses

CONCEPT MAP

```
↓
```

Duck Typing

65. The Ultimate Memory Trick

> Remember:

CO-MAP

CONCEPT MAP

```
C → Class
O → Object

M → Methods
A → Attributes

P → Four Pillars
```

Then remember the four pillars:

EIPA

CONCEPT MAP

```
E → Encapsulation
I → Inheritance
P → Polymorphism
A → Abstraction
```

For advanced:

M-S-C-P-D

CONCEPT MAP

```
M → MRO
S → super()
C → Composition
P → Properties
D → Dataclass
```

66. OOP Mastery Checklist

You are beginner when you can:

[ ] Create classes

[ ] Create objects

[ ] Use __init__

[ ] Understand self

[ ] Create attributes

[ ] Create methods

You are intermediate when you can:

[ ] Instance vs class variables
[ ] Class methods
[ ] Static methods
[ ] Encapsulation
[ ] Inheritance
[ ] Method overriding
[ ] Polymorphism
[ ] super()

You are advanced when you can:

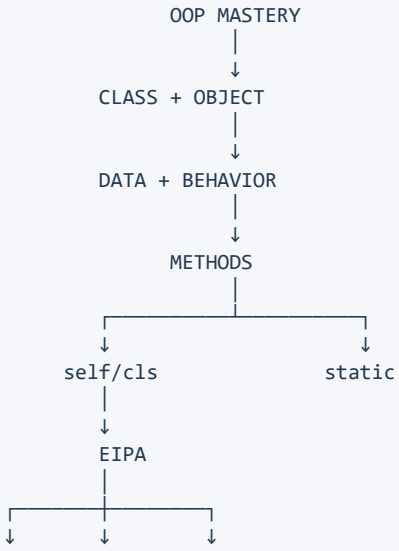
[ ] Multiple inheritance
[ ] MRO
[ ] Abstract classes
[ ] ABC interfaces
[ ] Composition
[ ] Aggregation
[ ] Operator overloading
[ ] Magic methods
[ ] Properties
[ ] Getters/setters
[ ] Dataclasses
[ ] Duck typing

You are software-development ready when you can design:

[ ] Bank system
[ ] E-commerce system
[ ] Library system
[ ] Parking system
[ ] Payment system
[ ] Employee management
[ ] Inventory system
[ ] Authentication system
[ ] Notification system

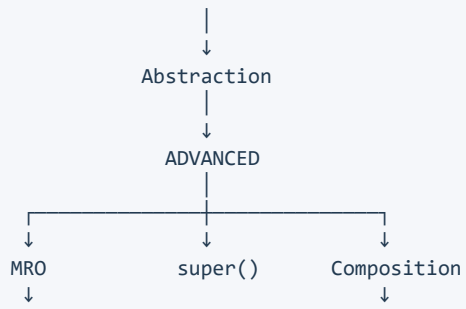
67. Final OOP Master Formula

CONCEPT MAP



Encaps Inherit Poly

CONCEPT MAP



Magic Methods Properties

CONCEPT MAP

Operator Overload Dataclasses

CONCEPT MAP



 One sentence to remember everything

Class is the blueprint, object is the real thing, attributes store data, methods perform actions, self represents the current object, the four pillars are Encapsulation–Inheritance–Polymorphism–Abstraction, and advanced OOP is about designing relationships and reusable behavior with MRO, super(), composition, properties, magic methods, ABCs, and dataclasses.

Best learning order

Class/Object

CONCEPT MAP

self + __init__

CONCEPT MAP

↓

Attributes + Methods

CONCEPT MAP

↓

Instance/Class/Static methods

CONCEPT MAP

↓

Encapsulation

CONCEPT MAP

↓

Inheritance

CONCEPT MAP

↓

Overriding + super()

CONCEPT MAP

↓

Polymorphism

CONCEPT MAP

↓

Abstraction + ABC

CONCEPT MAP

↓

Multiple Inheritance + MRO

CONCEPT MAP

↓

Composition/Aggregation

CONCEPT MAP

↓

Magic Methods

CONCEPT MAP

↓

Properties

CONCEPT MAP

↓

Dataclasses

CONCEPT MAP

↓

OOP Projects

CONCEPT MAP

↓

Interview Problems

If you can build the Bank System + E-commerce System + Shape/Payment System above without copying the code, you have moved from memorizing OOP to actually understanding OOP.

PART 5 | Complete Machine Learning for Interview

Concepts, Formulas, Parameters, Examples, Code and Interview Answers

Machine Learning is a branch of AI in which computers learn patterns from data and use those patterns to make predictions or decisions.

CONCEPT MAP

```
Historical data → Learn pattern → Trained model → New prediction
```

> Example:

Inputs: House area, rooms, location

Target: House price

The model learns the relationship between house features and price.

It then predicts the price of a new house.

1. Complete Machine Learning Workflow

1. Understand the business problem
2. Collect data
3. Explore the data
4. Clean the data
5. Select features and target
6. Split data
7. Preprocess features
8. Train a baseline model
9. Evaluate the model
10. Tune hyperparameters
11. Perform error analysis
12. Save the model
13. Deploy the model

14. Monitor and retrain it

> A common interview answer:

I first understand the business objective and define the target variable. Then I clean and explore the data, create useful features, split the data into training and testing sets, build preprocessing and model pipelines, evaluate the model using suitable metrics, tune it, perform error analysis, and finally deploy and monitor it.

2. Types of Machine Learning

2.1 Supervised Learning

The dataset contains input features and a known target.

CONCEPT MAP

```
Features → Model → Target prediction
```

Examples:

House price prediction

Customer churn prediction

Spam detection

Medical diagnosis

Supervised learning has two major types.

Regression

Predicts a continuous numerical value.

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
Salary = ₹50,000
```

House price = ₹80 lakh

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
Sales = 12,500 units
```

Classification

Predicts a category or class.

Spam / Not Spam

Fraud / Genuine

Disease / No Disease

2.2 Unsupervised Learning

The dataset does not contain a target variable.

The model tries to discover hidden patterns.

Examples:

Customer segmentation

Grouping similar products

Anomaly detection

Dimensionality reduction

Major types:

Clustering

Dimensionality Reduction

Association Rules

2.3 Reinforcement Learning

An agent learns through rewards and penalties.

CONCEPT MAP

```
State → Action → Reward → New State
```

Examples:

Robotics

Game playing

Dynamic pricing

Recommendation optimization

3. Important ML Terminology

Term	Meaning
Feature	Input column used for prediction
Target	Output column to predict
Observation	One row of data
Model	Learned mathematical relationship
Training	Learning from data
Inference	Predicting on new data
Parameter	Value learned by the model
Hyperparameter	Setting chosen before training
Epoch	One complete pass through training data
Batch	Small group of training observations
Loss function	Measures prediction error
Optimizer	Updates parameters to reduce loss

> Example:

Features: age, salary, experience

Target: employee_attrition

4. Python Libraries for Machine Learning

pip install numpy pandas matplotlib scikit-learn scipy joblib

Optional libraries:

pip install xgboost lightgbm catboost imbalanced-learn

pip install tensorflow torch

pip install mlflow streamlit fastapi uvicorn

Main uses:

Library	Use
NumPy	Numerical and matrix operations
Pandas	Data manipulation
Matplotlib	Visualization
Scikit-learn	ML algorithms
SciPy	Statistics and scientific computing
TensorFlow/PyTorch	Deep learning
Joblib	Save trained models

5. NumPy Basics

NumPy provides fast multidimensional arrays.

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

print("Mean:", np.mean(arr))
print("Median:", np.median(arr))
print("Variance:", np.var(arr))
print("Standard deviation:", np.std(arr))
```

Matrix multiplication

C

=

A

B

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
C=AB
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print(A @ B)
```

Dot product

a

□

b

=

∑

i

=

1

n

a

i

b

i

a□b=

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
i=1
```

∑

n

a

i

b

i

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(np.dot(a, b))
```

The dot product is used in linear regression, logistic regression, neural networks, SVM and attention mechanisms.

6. Pandas Basics

Pandas works with table-like data.

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
import pandas as pd

df = pd.DataFrame({
    "age": [22, 25, 28, 35],
    "salary": [25000, 35000, 45000, 70000],
    "purchased": [0, 0, 1, 1]
```

})

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
print(df.head())
print(df.info())
print(df.describe())
```

Filtering

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
high_salary = df[df["salary"] > 40000]
print(high_salary)
```

Grouping

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
summary = df.groupby("purchased")["salary"].mean()
print(summary)
```

Missing values

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
print(df.isnull().sum())
```

df["age"] = df["age"].fillna(df["age"].median())

7. Exploratory Data Analysis

EDA means understanding data before training a model.

Check:

Shape

Column names

Data types

Missing values

Duplicates

Outliers

Target balance

Distributions

Feature relationships

Correlation

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
print(df.shape)
print(df.columns)
print(df.dtypes)
print(df.isnull().sum())
print(df.duplicated().sum())
print(df.describe(include="all"))
```

Histogram

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
import matplotlib.pyplot as plt
```

df["salary"].hist()

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
plt.xlabel("Salary")
plt.ylabel("Frequency")
plt.title("Salary Distribution")
plt.show()
```

Scatter plot

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
plt.scatter(df["age"], df["salary"])
plt.xlabel("Age")
plt.ylabel("Salary")
plt.show()
```

Correlation

r

=

∑

(

x

i

–

x

-

)

(

y

i

–

y

-

)

∑

(
x
i
–
x
-
)
2
∑
(
y
i
–
y
-
)
2

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

r=

∑(x
i
–
x
-
)
2
∑(y
i
–
y
-
)
2
∑(x
i
–
x
-
)(y
i

–

y

-

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
print(df[["age", "salary"]].corr())
```

Correlation ranges from

–

1

−1 to

1

1.

+1 = strong positive linear relationship

0 = no linear relationship

- 1 = strong negative linear relationship

Correlation does not prove causation.

8. Data Cleaning

8.1 Missing Values

Remove missing rows

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
df = df.dropna()
```

Fill numerical values

df["age"] = df["age"].fillna(df["age"].median())

Fill categorical values

df["city"] = df["city"].fillna(df["city"].mode()[0])

Scikit-learn imputer

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median")
```

Common strategies:

mean

median

most_frequent

constant

8.2 Duplicate Records

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
print(df.duplicated().sum())

df = df.drop_duplicates()
```

8.3 Wrong Data Types

df["date"] = pd.to_datetime(df["date"])

df["salary"] = pd.to_numeric(df["salary"], errors="coerce")

8.4 Text Standardization

df["city"] = (

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
    df["city"]
    .str.strip()
    .str.lower()
    .str.replace(r"\s+", " ", regex=True)
```

)

8.5 Outlier Detection Using IQR

I

Q

R

=

Q

3

−

Q

1

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
IQR=Q
```

3

−Q

1

L

o

w

e

r

=

Q

1

−

1.5

(
I
Q
R
)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
Lower=Q
```

1

−1.5(IQR)

U
p
p
e
r
=
Q
3
+
1.5
(
I
Q
R
)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
Upper=Q
```

3

+1.5(IQR)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
Q1 = df["salary"].quantile(0.25)
Q3 = df["salary"].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = df[
    (df["salary"] < lower) |
    (df["salary"] > upper)
```

]

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
print(outliers)
```

Do not remove every outlier automatically. An outlier can be a real and important observation.

9. Feature Encoding

ML algorithms generally require numerical inputs.

9.1 Label Encoding

CONCEPT MAP

```
Low    → 0
Medium → 1
High   → 2
```

Use it for ordered categories.

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
mapping = {
    "low": 0,
    "medium": 1,
    "high": 2
```

}

df["risk_encoded"] = df["risk"].map(mapping)

9.2 One-Hot Encoding

city_Delhi

city_Mumbai

city_Pune

Use it for nominal categories with no natural order.

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(
    handle_unknown="ignore"
```

)

10. Feature Scaling

10.1 Standardization

z

=

x

–

μ

σ

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
z=
```

σ

x−μ

Parameters:

Symbol	Meaning

x

x	Original value

μ

μ	Mean

σ

σ	Standard deviation

z

z	Standardized value

The output usually has mean near 0 and standard deviation near 1.

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Important for:

KNN

SVM

Logistic Regression

Linear Regression with regularization

PCA

Neural Networks

10.2 Min-Max Scaling

x

′

=

x

−

x

m

i

n

x

m

a

x

−

x

m

i

n

x

,

=

x

max

−x

min

x−x

min

Usually transforms values into the range 0 to 1.

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)
```

10.3 Robust Scaling

Uses median and IQR, so it is more resistant to outliers.

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()
```

Tree-based algorithms usually do not require scaling.

11. Train-Test Split

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.model_selection import train_test_split
```

X_train, X_test, y_train, y_test = train_test_split(

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
```

)

Parameters:

Parameter	Meaning
test_size=0.2	20% testing data
random_state=42	Reproducible split
stratify=y	Maintains class ratio

For time-series data, do not randomly shuffle past and future observations.

12. Data Leakage

Data leakage happens when training uses information that would not be available at prediction time.

Examples:

Scaling before train-test split

Using future sales to predict past sales

Including target-derived columns

Using the test set during feature selection

Wrong:

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
X_scaled = scaler.fit_transform(X)
```

X_train, X_test = train_test_split(X_scaled)

Better:

X_train, X_test = train_test_split(X)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

> Best practice: use a pipeline.

13. Scikit-Learn Pipeline

A pipeline keeps preprocessing and the model together.

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

model = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression())
```

])

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
model.fit(X_train, y_train)
```

For mixed numerical and categorical data:

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline

numeric_features = ["age", "salary"]
categorical_features = ["city", "gender"]

numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
```

])

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
```

])


```
preprocessor = ColumnTransformer([
    ("num", numeric_pipeline, numeric_features),
    ("cat", categorical_pipeline, categorical_features)
```

])

14. Bias-Variance Tradeoff

Bias

Bias is error caused by an overly simple model.

CONCEPT MAP

```
High bias → Underfitting
```

Variance

Variance is error caused by excessive sensitivity to training data.

CONCEPT MAP

```
High variance → Overfitting
```

E

x

p

e

c

t

e

d

E

r

r

o

r

=

B

i

a

s

2

+

V

a

r
i
a
n
c
e
+
I
r
r
e
d
u
c
i
b
l
e
E
r
r
o
r
Expected Error=Bias
2
+Variance+Irreducible Error

Problem	Training Score	Test Score
Underfitting	Low	Low
Good fit	High	High
Overfitting	Very high	Low

15. Overfitting and Underfitting

Underfitting

The model is too simple.

Solutions:

Use a more powerful model

Create better features

Reduce regularization

Train longer

Add non-linear features

Overfitting

The model memorizes training data.

Solutions:

More training data

Cross-validation

Regularization

Reduce model complexity

Feature selection

Pruning

Early stopping

Dropout

Data augmentation

16. Linear Regression

Linear regression predicts a continuous value.

> Formula

y
^
=
β
0
+
β
1
x
1
+
β
2
x
2
+
□
+
β
p
x
p
y
^
=β
0
+β

1

x

1

+β

2

x

2

+□+β

p

x

p

Parameters:

Symbol	Name	Meaning
y ^ y ^		

	Prediction	Predicted target
β 0 β 0		

	Intercept	Base prediction
β j β j		

	Coefficient	Effect of feature
j j x j x j		

	Feature	Input value
p		

p	Feature count	Number of predictors

Example:

S
a
l
a
r
y
=
20
,
000
+
8
,
000
(
E
x
p
e
r
i
e
n
c
e
)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
Salary=20,000+8,000(Experience)
```

For 5 years:

S
a
l
a
r
y
=
20
,
000
+

8

,

000

(

5

)

=

60

,

000

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
Salary=20,000+8,000(5)=60,000
```

Cost Function

M

S

E

=

1

n

∑

i

=

1

n

(

y

i

–

y

^

i

)

2

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
MSE=
```

n

1

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
i=1
```

∑

n

(y

i

–

y

^

i

)

2

The model selects coefficients that minimize MSE.

Code

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
import pandas as pd
import numpy as np

from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
df = pd.DataFrame({
    "experience": [1, 2, 3, 4, 5, 6, 7],
    "salary": [
        30000, 38000, 47000,
        56000, 64000, 73000, 81000
    ]
```

})

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
X = df[["experience"]]
y = df["salary"]
```

X_train, X_test, y_train, y_test = train_test_split(

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
    X,
    y,
    test_size=0.25,
    random_state=42
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Intercept:", model.intercept_)
print("Coefficient:", model.coef_[0])
print("MAE:", mean_absolute_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R2:", r2_score(y_test, y_pred))
```

Main assumptions

Linear relationship

Independent observations

Constant error variance

Residuals approximately normal for statistical inference

Limited multicollinearity

17. Polynomial Regression

Polynomial regression captures curved relationships.

y

^

=

β

0

+

β

1

x

+

β

2

x

2

+

□

+

β

d

x

d

y

^

=β

0

+β

1

x+β

2

x

2

+□+β

d

x

d

Here

d

d is the polynomial degree.

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression

model = Pipeline([
    ("poly", PolynomialFeatures(
        degree=2,
        include_bias=False
    )),
    ("regressor", LinearRegression())
```

])

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
model.fit(X_train, y_train)
```

A very high degree can overfit.

18. Ridge Regression

Ridge adds L2 regularization.

J

=

M

S

E

+

λ

∑

j

=

1

p

β

j

2

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
J=MSE+λ
j=1
```

∑

p

β

j

2

Parameters:

Symbol	Meaning

J

J	Total objective

λ

λ	Regularization strength

β

j

β

j

	Model coefficient

p

p	Number of features

Ridge reduces large coefficients but generally does not make them exactly zero.

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.linear_model import Ridge

model = Ridge(alpha=1.0)
model.fit(X_train, y_train)
```

In scikit-learn, alpha represents the regularization strength.

19. Lasso Regression

Lasso uses L1 regularization.

J

=

M

S

E

+

λ

∑

j

=

1

p

□

β

j

□

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
J=MSE+λ
j=1
```

∑

p

□β

j

□

Lasso can make unimportant coefficients exactly zero.

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.linear_model import Lasso

model = Lasso(alpha=0.1)
model.fit(X_train, y_train)
```

Use Lasso when feature selection is useful.

20. Elastic Net

Elastic Net combines L1 and L2 penalties.

J

=

M

S

E

+

λ

[

α

∑

□

β

j

□

+

(

1
−
α
)
∑
β
j
2
]

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
J=MSE+λ[α∑ β
```

j
□+(1−α)∑β
j
2
]

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.linear_model import ElasticNet

model = ElasticNet(
    alpha=0.1,
    l1_ratio=0.5
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
model.fit(X_train, y_train)
```

l1_ratio controls the balance between L1 and L2.

21. Logistic Regression

Logistic regression predicts class probability.

Linear score

z
=
β
0
+
β
1
x
1
+
□
+

β

p

x

p

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
z=β
```

0

+β

1

x

1

+□+β

p

x

p

Sigmoid

p

=

1

1

+

e

−

z

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
p=
```

1+e

−z

1

Parameters:

Symbol	Meaning

p

p	Probability of class 1

z

z	Linear score

e

e	Euler's number

β_j

β_j

	Model coefficient

x_j

x_j

	Input feature

Decision:

$\hat{y} = \begin{cases} 1, & p \geq 0.5 \\ 0, & p < 0.5 \end{cases}$

$\hat{y} = \begin{cases} 1, & p \geq 0.5 \\ 0, & p < 0.5 \end{cases}$

Binary Cross-Entropy

$L = -\frac{1}{n} \sum_i$

=

1

n

[

y

i

log

□

(

p

i

)

+

(

1

−

y

i

)

log

□

(

1

−

p

i

)

]

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
L=-
```

n

1

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
i=1
```

∑

n

[y

i

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
 log(p
```

i

)+(1−y

i

)log(1−p

i

)]

Code

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report

model = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(
        max_iter=1000
    ))
```

])

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
y_probability = model.predict_proba(X_test)[:, 1]

print(classification_report(y_test, y_pred))
```

Odds and log-odds

O

d

d

s

=

p

1

−

p

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
Odds=
```

1−p

p

L

o

g

O

d

d
s
=
ln
□
(
p
1
−
p
)


```
LogOdds=ln(
```

1−p
p
)

Logistic regression models log-odds as a linear function of features.

22. K-Nearest Neighbors

KNN predicts based on nearby observations.

Euclidean Distance

d
(
x
,
y
)
=
∑
j
=
1
p
(
x
j
−
y
j
)
2

```
d(x,y)=
j=1
```

Σ

p

(x

j

−y

j

)

2

Parameters:

Symbol	Meaning

d

d	Distance

p

p	Number of features

x

j

,

y

j

x

j

,y

j

	Feature values

Classification:

y

^

=

M

o

d

e

(

k

n

e
a
r
e
s
t
l
a
b
e
l
s
)
y
^

=Mode(k nearest labels)

Regression:

y
^
=
1
k
∑
i
=
1
k
y
i
y
^
=
k
1


```
i=1
```

∑
k
y
i

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

model = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier(
        n_neighbors=5
    ))
```

])

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
model.fit(X_train, y_train)
```

> Important:

CONCEPT MAP

```
Small K → sensitive to noise
Large K → smoother but may underfit
```

Scaling is important

Prediction can be slow on large datasets

23. Decision Tree

A decision tree creates if-else rules.

Entropy

E

n

t

r

o

p

y

=

−

∑

c

=

1

C

p

c

log

□

2

(

p

c

)


```
Entropy=-
c=1
```

∑

C

p

c

log

2

(p

c

)

Parameters:

Symbol	Meaning

C

C	Number of classes

p

c

p

c

	Proportion of class

c

c

Gini Impurity

G

i

n

i

=

1

−

∑

c

=

1

C

p

c

2


```
Gini=1−
c=1
```

∑

C

p

c

2

Information Gain

I

G

=

I

m

p

u

r

i

t

y

(

p

a

r

e

n

t

)

−

∑

j

N

j

N

I

m

p

u

r

i

t

y

(

c

h

i

l

d

j

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
IG=Impurity(parent)−
```

j

∑

N

N

j

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
 Impurity(child
```

j

)

The tree selects splits that reduce impurity.

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=5,
    min_samples_split=10,
    random_state=42
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
model.fit(X_train, y_train)
```

Important hyperparameters:

Parameter	Meaning
max_depth	Maximum tree depth
min_samples_split	Minimum observations to split
min_samples_leaf	Minimum observations in a leaf
criterion	Split-quality measure

Advantages:

Easy to understand

Works with non-linear relationships

No scaling required

Handles interactions automatically

Disadvantage: easily overfits.

24. Random Forest

Random Forest trains many decision trees.

Classification:

y
^
=
M
o
d
e
{
h
1
(
x
)
,
h
2
(
x
)
,
,
h
B
(
x
)
}
y
^
=Mode{h
1
(x),h

2

(x),…,h

B

(x)}

Regression:

y

^

=

1

B

∑

b

=

1

B

h

b

(

x

)

y

^

=

B

1


```
b=1
```

∑

B

h

b

(x)

Parameters:

Symbol	Meaning

B

B	Number of trees

h

b

(

x

)

h

b

(x)	Prediction from tree

b

b

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=200,
    max_depth=10,
    min_samples_leaf=2,
    class_weight="balanced",
    random_state=42
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
model.fit(X_train, y_train)
```

Why it works:

Bootstrap samples create different trees

Random feature subsets reduce correlation

Voting or averaging reduces variance

25. Support Vector Machine

SVM finds a decision boundary with maximum margin.

Hyperplane

w

T

x

+

b

=

0

w

T

x+b=0

Prediction

y

^

=

s

i

g
n
(
w
T
x
+
b
)
y
^
=sign(w
T
x+b)
Margin
M
a
r
g
i
n
=
2
□
w
□

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
Margin=
```

□w□
2
Soft-margin objective
min
□
1
2
□
w
□
2
+
C

∑
i
=
1
n
ξ
i
min
2
1
□w□
2
+C

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE
i=1

∑
n
ξ
i

Parameters:

Symbol	Meaning
w	Hyperplane weights
b	Bias
C	Error penalty
ξ i ξ i	Slack variable

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

model = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC(
        kernel="rbf",
        C=1.0,
        gamma="scale",
        probability=True
    ))
```

])

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
model.fit(X_train, y_train)
```

RBF Kernel

K

(

x

,

z

)

=

exp

□

(

−

γ

□

x

−

z

□

2

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
K(x,z)=exp(-γ x-z
```

2

)

CONCEPT MAP

```
Large C → fewer training errors, higher overfitting risk
Small C → wider margin, more regularization
Large gamma → narrow influence
Small gamma → smoother boundary
```

26. Naive Bayes

Naive Bayes uses Bayes' theorem.

P
(
C
□
X
)
=
P
(
X
□
C
)
P
(
C
)
P
(
X
)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE
P(C X)= P(X) P(X C)P(C)

Parameters:

Symbol	Name	Meaning
(P(C	X))	Posterior
(P(X	C))	Likelihood

P
(
C
)

P(C)	Prior	Initial class probability

P
(
X
)

P(X)	Evidence	Total feature probability

Naive Bayes assumes features are conditionally independent.

Gaussian Naive Bayes

Used for continuous numerical features.

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.naive_bayes import GaussianNB

model = GaussianNB()
model.fit(X_train, y_train)
```

Multinomial Naive Bayes

Used for word counts and text data.

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline

model = Pipeline([
    ("tfidf", TfidfVectorizer()),
    ("classifier", MultinomialNB())
```

])

27. Bagging vs Boosting

Bagging	Boosting
Models train independently	Models train sequentially
Reduces variance	Mainly reduces bias
Random Forest	AdaBoost, Gradient Boosting
Parallelizable	Less parallel by nature

28. AdaBoost

AdaBoost gives more weight to previously misclassified observations.

Final prediction:

F

(

x

)

=

∑

m

=

1

M

α

m

h

m

(

x

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
F(x)=
m=1
```

∑

M

α

m

h

m

(x)

Learner weight:

α

m

=

1

2

ln

□

(

1

−

ϵ

m

ϵ

m

)

α

m

=

2

1

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
  ln(
```

ϵ

m

1−ϵ

m

)

Parameters:

Symbol	Meaning
M	

M	Number of weak learners
h	
m	
h	
m	

	Weak learner
ϵ	
m	
ϵ	
m	

	Weighted error
α	
m	
α	
m	

	Learner importance

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.ensemble import AdaBoostClassifier

model = AdaBoostClassifier(
    n_estimators=100,
    learning_rate=0.1,
    random_state=42
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
model.fit(X_train, y_train)
```

29. Gradient Boosting

Gradient Boosting trains each new model to correct earlier errors.

F

m

(

x

)

=

F
m
−
1
(
x
)
+
η
h
m
(
x
)
F
m
(x)=F
m−1
(x)+ηh
m
(x)

Parameters:

Symbol	Meaning
F m F m	
	Updated ensemble
F m − 1 F m−1	
	Previous ensemble
h m h m	

	New weak learner

η

η	**Learning rate**

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.ensemble import GradientBoostingClassifier

model = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.05,
    max_depth=3,
    random_state=42
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
model.fit(X_train, y_train)
```

30. XGBoost, LightGBM and CatBoost

XGBoost

Strengths:

Strong tabular performance

Regularization

Missing-value handling

Parallel computation

LightGBM

Strengths:

Fast on large datasets

Memory efficient

Leaf-wise growth

CatBoost

Strengths:

Strong categorical feature handling

Less manual encoding

Good default performance

Example using XGBoost:

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from xgboost import XGBClassifier

model = XGBClassifier(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=5,
    subsample=0.8,
    colsample_bytree=0.8,
    eval_metric="logloss",
    random_state=42
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
model.fit(X_train, y_train)
```

31. K-Means Clustering

K-Means groups similar observations.

Objective

J

=

∑

k

=

1

K

∑

x

i

□

C

k

□

x

i

−

μ

k

□

2


```
J=
k=1
```

∑

K

x

i

□C

k

∑

□x

i

−μ

k

□

2

Parameters:

Symbol	Meaning
K	

K	Number of clusters
C	
k	
C	
k	

	Cluster
k	
k	
μ	
k	
μ	
k	

	Cluster centroid
x	
i	
x	
i	

	Observation

Steps

1. Select K centroids
2. Assign each point to the nearest centroid
3. Recalculate centroids

4. Repeat until assignments stabilize

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

model = KMeans(
    n_clusters=3,
    n_init=10,
    random_state=42
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
clusters = model.fit_predict(X_scaled)
```

Elbow Method

Plot inertia for different values of

K

K.


```
inertias = []

for k in range(1, 10):
    model = KMeans(
        n_clusters=k,
        n_init=10,
        random_state=42
    )
    model.fit(X_scaled)
    inertias.append(model.inertia_)
```

32. Silhouette Score

s

(

i

)

=

b

(

i

)

−

a

(

i

)

max

□

(

a

(

i

)

,

b

(

i

)

)

```
s(i)=
max(a(i),b(i))
b(i)−a(i)
```

Parameters:

Symbol	Meaning
a	
(	
i	
)	
a(i)	Average distance within own cluster
b	
(	
i	
)	
b(i)	Nearest other-cluster distance

Range:

CONCEPT MAP

```
Near 1 → good clustering
Near 0 → overlapping clusters
Below 0 → likely wrong assignment
from sklearn.metrics import silhouette_score

score = silhouette_score(
    X_scaled,
    clusters
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
print(score)
```

33. Hierarchical Clustering

Hierarchical clustering builds a tree of clusters called a dendrogram.

Types:

Agglomerative: starts with individual points and merges

Divisive: starts with one cluster and splits

Common linkage methods:

Single

Complete

Average

Ward

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.cluster import AgglomerativeClustering

model = AgglomerativeClustering(
    n_clusters=3,
    linkage="ward"
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
clusters = model.fit_predict(X_scaled)
```

34. DBSCAN

DBSCAN forms clusters based on dense regions.

Important parameters:

Parameter	Meaning
eps	Maximum neighborhood distance
min_samples	Minimum points required for a dense region

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.cluster import DBSCAN

model = DBSCAN(
    eps=0.5,
    min_samples=5
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
clusters = model.fit_predict(X_scaled)
```

Advantages:

Finds irregularly shaped clusters

Does not require number of clusters

Detects noise

Disadvantages:

Sensitive to eps

Difficult with different cluster densities

Scaling is important

35. Gaussian Mixture Model

GMM assumes data is generated from multiple Gaussian distributions.

P

(

x

)

=

∑

k

=

1

K

π

k

N

(

x

□

μ

k

,

Σ

k

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE
P(x)= k=1

∑

K

π

k

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE
N(x μ

k

,Σ

k

)

Parameters:

Symbol	Meaning

K

K	Number of Gaussian components

π

k

π

k

	Component weight

μ

k

μ

k

	Mean

Σ

k

Σ

k

	Covariance matrix

Unlike K-Means, GMM provides probability-based soft assignments.

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.mixture import GaussianMixture

model = GaussianMixture(
    n_components=3,
    random_state=42
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
clusters = model.fit_predict(X_scaled)
probabilities = model.predict_proba(X_scaled)
```

36. PCA

PCA reduces feature dimensions while preserving maximum variance.

Standardization

z

i

j

=

x

i

j

−

μ

j

σ

j

z

ij

=

σ

j

x

ij

−μ

j

Covariance Matrix

Σ

=

1

n

−

1

X

T

X

Σ=

n−1

1

X

T

X

Eigen decomposition

Σ

v

j

=

λ

j

v

j

Σv

j

=λ

j

v

j

Parameters:

Symbol	Meaning

Σ

Σ	Covariance matrix

v

j

v

j

	Principal direction

λ

j

λ

j

	Variance captured

X

X	Centered data

Explained Variance Ratio

E

V

R

j

=

λ

j

∑

k

λ

k

EVR

j

=

∑

k

λ

k

λ

j

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X_scaled = StandardScaler().fit_transform(X)

pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X_scaled)

print(pca.explained_variance_ratio_)
```

PCA components are combinations of original features and may be difficult to interpret.

37. Feature Engineering

Feature engineering creates useful inputs from raw data.

Examples:

df["order_date"] = pd.to_datetime(df["order_date"])

df["order_month"] = df["order_date"].dt.month

df["order_day"] = df["order_date"].dt.day

df["order_weekday"] = df["order_date"].dt.dayofweek

df["avg_order_value"] = (


```
    df["total_spent"] /
    df["total_orders"].replace(0, 1)
```

)

Other examples:

Polynomial features

Interaction features

Log transformation

Binning

Lag features

Rolling averages

Text vectors

Image embeddings

Log Transformation

x

′

=

log

□

(

1

+

x

)

x

′

=log(1+x)

Useful for strongly right-skewed positive features.


```
import numpy as np
```

df["log_income"] = np.log1p(df["income"])

38. Feature Selection

Filter Methods

Independent of a model.

Correlation

Chi-square

ANOVA

Mutual information

Variance threshold

Wrapper Methods

Evaluate feature subsets using model performance.

Recursive Feature Elimination

Forward Selection

Backward Elimination

Embedded Methods

Feature selection happens during model training.

Lasso

Decision Tree

Random Forest

Boosting

> Example:

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import mutual_info_classif

selector = SelectKBest(
    score_func=mutual_info_classif,
    k=5
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
X_selected = selector.fit_transform(X, y)
```

39. Cross-Validation

Cross-validation provides a more reliable estimate of performance.

K-Fold CV

Split data into K folds.

Train on K−1 folds.

Validate on the remaining fold.

Repeat K times.

Mean score:

s

-

=

1

K

∑

i

=

1

K

s

i

s

-

=

K

1

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
i=1
```

∑

K

s

i

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(
    model,
    X,
    y,
    cv=5,
    scoring="accuracy"
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
print("Scores:", scores)
print("Mean:", scores.mean())
print("Std:", scores.std())
```

Use StratifiedKFold for classification.

Use TimeSeriesSplit for time-series problems.

40. Hyperparameter Tuning

Grid Search

Tries every specified combination.

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    random_state=42
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
parameters = {
    "n_estimators": [100, 200],
    "max_depth": [5, 10, None],
    "min_samples_leaf": [1, 2, 5]
```

}

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
grid = GridSearchCV(
    estimator=model,
    param_grid=parameters,
    cv=5,
    scoring="f1",
    n_jobs=-1
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
grid.fit(X_train, y_train)

print(grid.best_params_)
print(grid.best_score_)
```

Randomized Search

Tries random combinations and is faster for large search spaces.

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.model_selection import RandomizedSearchCV

search = RandomizedSearchCV(
    estimator=model,
    param_distributions=parameters,
    n_iter=10,
    cv=5,
    scoring="f1",
    random_state=42,
    n_jobs=-1
```

)

Always perform tuning using training data, not the final test set.

41. Regression Metrics

Mean Absolute Error

M

A

E

=

1

n

∑
i
=
1
n
□
y
i
−
y
^
i
□

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
MAE=
```

n
1

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
i=1
```

∑
n
□y
i
−
y
^
i
□

Easy to interpret and less sensitive to extreme errors.

Mean Squared Error

M
S
E
=
1
n
∑
i
=
1

n
(
y
i
–
y
^
i
)
2

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
MSE=
```

n
1

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
i=1
```

∑
n
(y
i
–
y
^
i
)
2

Penalizes large errors strongly.

Root Mean Squared Error

R
M
S
E
=
1
n
∑
i
=
1
n

(
y
i
–
y
^
i
)
2

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
RMSE=
```

n
1

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
i=1
```

∑
n
(y
i
–
y
^
i
)
2

Same unit as the target.

R-Squared

R
2
=
1
–
∑
(
y
i
–
y
^

i
)
2
∑
(
y
i
−
y
-
)
2
R
2
=1−
∑(y
i
−
y
-
)
2
∑(y
i
−
y
^
i
)
2
An
R
2
R
2
of 0.80 means the model explains approximately 80% of target variance.

Adjusted R-Squared

A
d
j
u

s
t
e
d
R
2
=
1
−
(
1
−
R
2
)
n
−
1
n
−
p
−
1
Adjusted R
2
=1−(1−R
2
)
n−p−1
n−1

It penalizes unnecessary features.

42. Classification Metrics

Confusion Matrix

Predicted Positive	Predicted Negative	
Actual Positive	TP	FN
Actual Negative	FP	TN

Accuracy

A
c
c

u
r
a
c
y
=
T
P
+
T
N
T
P
+
T
N
+
F
P
+
F
N


```
Accuracy=
```

TP+TN+FP+FN

TP+TN

Best when classes are balanced and errors have similar costs.

Precision

P
r
e
c
i
s
i
o
n
=
T
P
T

P

+

F

P

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
Precision=
```

TP+FP

TP

Out of predicted positives, how many were correct?

Useful when false positives are expensive.

Recall

R

e

c

a

l

l

=

T

P

T

P

+

F

N

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
Recall=
```

TP+FN

TP

Out of actual positives, how many were found?

Useful for medical diagnosis and fraud detection.

Specificity

S

p

e

c

i

f

i

c

i

t

y

=

T

N

T

N

+

F

P

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
Specificity=
```

TN+FP

TN

F1 Score

F

1

=

2

P

r

e

c

i

s

i

o

n

×

R

e

c

a

l

l

P

r

e

c

i

s

i

o

n

+

R

e

c

a

l

l

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
F1=2
```

Precision+Recall

Precision×Recall

Useful for imbalanced classification.

Code

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_auc_score
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1:", f1_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

43. ROC-AUC and Precision-Recall

True Positive Rate

T

P

R

=

T

P

T

P

+

F

N

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
TPR=
```

TP+FN

TP

False Positive Rate

F

P

R

=

F

P

F

P

+

T

N

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
FPR=
```

FP+TN

FP

ROC-AUC measures how well the model ranks positives above negatives.

For highly imbalanced data, the Precision-Recall curve is often more informative.

44. Class Imbalance

> Example:

99% genuine transactions

1% fraudulent transactions

A model predicting every transaction as genuine gets 99% accuracy but detects no fraud.

Solutions:

Use precision, recall, F1, PR-AUC

Class weights

Undersampling

Oversampling

SMOTE

Threshold tuning

Anomaly detection

Class weights

```
model = LogisticRegression(
    class_weight="balanced",
    max_iter=1000
```

)

SMOTE

Apply SMOTE only to training data.

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=42)
```

X_train_resampled, y_train_resampled = (

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
    smote.fit_resample(X_train, y_train)
```

)

45. Threshold Tuning

The default classification threshold is often 0.5, but it can be changed.

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
probabilities = model.predict_proba(X_test)[:, 1]

threshold = 0.3
predictions = (
    probabilities >= threshold
```

).astype(int)

Lower threshold:

Higher recall

More false positives

Higher threshold:

Higher precision

More false negatives

Select the threshold based on business cost.

46. Time-Series Basics

Time-series data is ordered by time.

Components:

Trend

Seasonality

Cyclic variation

Noise

Lag Feature

L

a

g

1
(
t
)
=
y
t
−
1
Lag
1
(t)=y
t−1
df["lag_1"] = df["sales"].shift(1)
df["lag_7"] = df["sales"].shift(7)
Moving Average
M
A
t
=
1
k
∑
i
=
0
k
−
1
y
t
−
i
MA
t
=
k
1


```
i=0
```

∑

k−1

y

t−i

df["moving_avg_7"] = (


```
    df["sales"]
    .shift(1)
    .rolling(7)
    .mean()
```

)

Differencing

Δ

y

t

=

y

t

−

y

t

−

1

Δy

t

=y

t

−y

t−1

df["difference"] = df["sales"].diff()

For time series:

Train on past

Test on future

Never use random split blindly

47. ARIMA

A

R

I

M

A

(

p

,

d

,

q

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE
`ARIMA(p,d,q)`

Parameters:

Parameter	Meaning

p

p	Number of autoregressive lags

d

d	Differencing order

q

q	Number of past error terms

AR model:

y

t

=

c

+

ϕ

1

y

t

−

1

+

□

+

ϕ

p

y

t

−

p

+

ϵ

t

y

t

=c+ϕ

1

y

t−1

+□+ϕ

p

y

t−p

+ϵ

t

ARIMA works best for suitable univariate time-series patterns and often expects stationarity after differencing.

48. Deep Learning Basics

Deep learning uses neural networks with multiple layers.

CONCEPT MAP

```
Input layer → Hidden layers → Output layer
```

Artificial Neuron

z

=

∑

j

=

1

p

w

j

x

j

+

b

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
z=
j=1
```

∑

p

w

j

x

j

+b

a

=

f

(

z

)


```
a=f(z)
```

Parameters:

Symbol	Meaning
x j	
x j	
	Input
w j	
w j	
	Weight
b	
b	Bias
f	
f	Activation
a	
a	Output

49. Activation Functions

ReLU

R

e

L

U

(

z

)
=
max
□
(
0
,
z
)


```
ReLU(z)=max(0,z)
```

Used in hidden layers.

Sigmoid

σ
(
z
)
=
1
1
+
e
−
z

σ(z)=
1+e
−z
1

Used for binary classification output.

Softmax

P
(
y
=
j
)
=
e
z
j
∑

k
=
1
K
e
z
k

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
P(y=j)=
```

∑

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
k=1
```

K
e
z
k
e
z
j

Used for multiclass output.

50. Forward Propagation

For layer
l
l:
z
(
l
)
=
W
(
l
)
a
(
l
−
1
)

+
b
(
l
)
z
(l)
=W
(l)
a
(l−1)
+b
(l)
a
(
l
)
=
f
(
z
(
l
)
)
a
(l)
=f(z
(l)
)
The network transforms inputs layer by layer.

51. Backpropagation

Backpropagation calculates gradients using the chain rule.

∂
L
∂
W
(
l
)
∂W

(l)

∂L

The optimizer then updates weights:

W

n

e

w

=

W

o

l

d

−

η

∂

L

∂

W

W

new

=W

old

−η

∂W

∂L

Here

η

η is the learning rate.

52. Gradient Descent

θ

t

+

1

=

θ

t

−

η

□

J

(

θ

t

)

θ

t+1

=θ

t

−η□J(θ

t

)

Parameters:

Symbol	Meaning
θ t θ t	Current parameter
η	Learning rate
□J	Gradient
J	Loss function

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
w = 10.0
learning_rate = 0.1

for _ in range(20):
    gradient = 2 * w
    w = w - learning_rate * gradient

print(w)
```

53. ANN Code

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
import tensorflow as tf

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

X, y = load_iris(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(4,)),
    tf.keras.layers.Dense(
        32,
        activation="relu"
    ),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(
        16,
        activation="relu"
    ),
    tf.keras.layers.Dense(
        3,
        activation="softmax"
    )
```

])

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=16,
    validation_split=0.2,
    verbose=0
```

)

loss, accuracy = model.evaluate(

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
    X_test,
    y_test,
    verbose=0
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
print("Accuracy:", accuracy)
```

54. CNN Basics

CNNs are used for images.

Main layers:

Convolution

Activation

Pooling

Flattening

Dense layer

Convolution

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n)$$


```
Y(i,j)=
```

$$m$$

∑

n

∑

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
X(i+m,j+n)K(m,n)
```

Output size:

O

u

t

p

u

t

=

□

N

+

2

P

−

F

S

□

+

1

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
Output=
```

S

N+2P−F

□+1

Parameters:

Symbol	Meaning

N

N	Input size

P

P	Padding

F

F	Filter size

S

S	Stride

55. RNN and LSTM

RNN is used for sequential data.

h

t

=

tanh

□

(

W

x

h

x

t

+

W

h

h

h

t

−

1

+

b

)

h

t

=tanh(W

xh

x

t

+W

hh

h

t−1

+b)

Problem: vanishing gradients.

LSTM solves this using gates:

Forget gate

Input gate

Cell state

Output gate

Applications:

Text

Speech

Time series

Sequential sensor data

Transformers are now commonly preferred for many NLP tasks because they process tokens using attention and allow better parallelization.

56. NLP Basics

Typical NLP pipeline:

Text cleaning

Tokenization

Stop-word handling

Stemming/Lemmatization

Vectorization

Model training

Evaluation

Bag of Words

Represents text using word counts.

TF-IDF

T

F

I

D

F

(

t

,

d

)

=

T

F

(

t

,

d

)

×

I

D

F

(

t

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE
TFIDF(t,d)=TF(t,d)×IDF(t)

I

D

F

(

t

)

=

log

□

(

N

D

F

(

t

)

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE
IDF(t)=log(DF(t)

N

)

Parameters:

Symbol	Meaning

t

t	Term

d

d	Document

N

N	Number of documents

D

F

(

t

)

DF(t)	Documents containing term

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

text_model = Pipeline([
    ("tfidf", TfidfVectorizer(
        stop_words="english",
        ngram_range=(1, 2)
    )),
    ("classifier", LogisticRegression(
        max_iter=1000
    ))
```

])

57. Embeddings

Embeddings convert items into dense vectors.

Examples:

Word embeddings

Sentence embeddings

Product embeddings

User embeddings

Image embeddings

Similar items usually have vectors close to each other.

Cosine similarity:

S

i

m

i

l

a

r

i

t

y

=

a

□

b

□

a

□

□

b

□

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
Similarity=
```

□a□□b□

a□b

Used in:

Search

Recommendations

RAG

Semantic similarity

Clustering

58. Recommendation Systems

Types:

Popularity-based

Content-based

Collaborative filtering

Hybrid

Content-based filtering

Recommends items similar to items the user liked.

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

vectorizer = TfidfVectorizer(
    stop_words="english"
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
item_vectors = vectorizer.fit_transform(
    item_descriptions
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
similarity_matrix = cosine_similarity(
    item_vectors
```

)

Collaborative filtering

Uses user-item interactions.

Matrix factorization approximates:

R

≈

U

V

T

R≈UV

T

where:

Symbol	Meaning

R

R	User-item rating matrix

U

U	User latent factors

V

V	Item latent factors

59. Explainable AI

Explainable AI helps answer:

Why did the model make this prediction?

Which features were important?

Can the decision be trusted?

Feature Importance

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
importance = model.feature_importances_
```

SHAP

SHAP assigns each feature a contribution value based on Shapley values.

Interpretation:

CONCEPT MAP

```
Positive SHAP value → pushes prediction upward
Negative SHAP value → pushes prediction downward
```

LIME

LIME builds a simple local model near one prediction.

Difference:

SHAP	LIME
Strong theoretical foundation	Local surrogate explanation
Local and global use	Mainly local
Can be slower	Often simple and fast

60. Error Analysis

Do not stop after viewing one metric.

Create an error table:

```
results = X_test.copy()
```

results["actual"] = y_test.values

results["predicted"] = y_pred

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
errors = results[
    results["actual"] !=
    results["predicted"]
```

]

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
print(errors.head())
```

Check errors by:

Class

Age group

Region

Product type

Time period

Missing-value pattern

Confidence score

Questions:

Is one group performing poorly?

Are labels incorrect?

Are important features missing?

Is the decision threshold unsuitable?

Is class imbalance causing errors?

61. Model Selection

Model selection is not only choosing the highest score.

Consider:

Validation performance

Business metric

Interpretability

Inference speed

Training cost

Data size

Feature type

Robustness

Fairness

Maintenance complexity

Recommended approach:

1. Create a simple baseline

2. Try multiple algorithms
3. Use cross-validation
4. Tune promising models
5. Evaluate once on the final test set

6. Select based on technical and business requirements

62. Model Deployment

Deployment makes a trained model available to users.

Train model

Save model

Create API or app

Containerize

Deploy

Monitor

Retrain

Save Pipeline

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
import joblib

joblib.dump(
    model,
    "customer_churn_model.joblib"
```

)

Load it:

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
model = joblib.load(
    "customer_churn_model.joblib"
```

)

63. FastAPI Deployment

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from fastapi import FastAPI
from pydantic import BaseModel
import pandas as pd
import joblib

app = FastAPI()

model = joblib.load(
    "customer_churn_model.joblib"
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
class CustomerInput(BaseModel):
    tenure: float
    monthly_charges: float
    contract: str
```

@app.get("/")

```
def home():
    return {
        "message": "ML API is running"
    }
```

@app.post("/predict")

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
def predict(data: CustomerInput):
    input_df = pd.DataFrame([{
        "tenure": data.tenure,
        "monthly_charges": data.monthly_charges,
        "contract": data.contract
    }])

    prediction = int(
        model.predict(input_df)[0]
    )

    probability = float(
        model.predict_proba(input_df)[0][1]
    )

    return {
        "prediction": prediction,
        "probability": probability
    }
```

Run:

uvicorn main:app --reload

64. Streamlit Deployment

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
import streamlit as st
import pandas as pd
import joblib

model = joblib.load(
    "customer_churn_model.joblib"
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
st.title("Customer Churn Prediction")

tenure = st.number_input(
    "Tenure",
    min_value=0.0
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
monthly_charges = st.number_input(
    "Monthly Charges",
    min_value=0.0
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
contract = st.selectbox(
    "Contract",
    [
        "Month-to-month",
        "One year",
        "Two year"
    ]
```

)


```
if st.button("Predict"):
    input_df = pd.DataFrame([{
        "tenure": tenure,
        "monthly_charges": monthly_charges,
        "contract": contract
    }])

    prediction = model.predict(input_df)[0]

    if prediction == 1:
        st.error("Customer may churn")
    else:
        st.success("Customer may stay")
```

Run:

streamlit run app.py

65. Model Monitoring

After deployment, monitor:

API latency

Error rate

Input distributions

Missing-value rate

Prediction distribution

Data drift

Concept drift

Model performance

Business impact

Data Drift

Input data distribution changes.

> Example:

Training customers were mostly 20–35 years old.

Production customers are mostly 50–70 years old.

Concept Drift

The relationship between features and target changes.

> Example:

Old fraud patterns no longer represent current fraud methods.

66. Major ML Project Structure

ml_project/

CONCEPT MAP

```
│
├── data/
│   ├── raw/
│   └── processed/
│
├── notebooks/
│   └── exploration.ipynb
│
├── src/
│   ├── preprocess.py
│   ├── train.py
│   ├── evaluate.py
│   └── predict.py
│
├── models/
│   └── model.joblib
│
├── app/
│   └── main.py
│
├── tests/
│   └── test_model.py
│
├── requirements.txt
├── Dockerfile
├── README.md
└── .gitignore
```

67. Complete End-to-End Classification Example

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
import os
import joblib
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import (
    StandardScaler,
    OneHotEncoder
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_auc_score
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
os.makedirs("models", exist_ok=True)
```

1. Load data

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
df = pd.read_csv("customer_churn.csv")
```

2. Remove exact duplicates

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
df = df.drop_duplicates()
```

3. Features and target

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
target = "churn"

X = df.drop(columns=[target])
y = df[target]
```

4. Column groups

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
numeric_features = [
    "tenure",
    "monthly_charges",
    "total_charges"
```

]

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
categorical_features = [
    "contract",
    "internet_service",
    "payment_method"
```

]

5. Pipelines

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
numeric_pipeline = Pipeline([
    (
        "imputer",
        SimpleImputer(strategy="median")
    ),
    (
        "scaler",
        StandardScaler()
    )
```

])

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
categorical_pipeline = Pipeline([
    (
        "imputer",
        SimpleImputer(
            strategy="most_frequent"
        )
    ),
    (
        "encoder",
        OneHotEncoder(
            handle_unknown="ignore"
        )
    )
```

])

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
preprocessor = ColumnTransformer([
    (
        "numeric",
        numeric_pipeline,
        numeric_features
    ),
    (
        "categorical",
        categorical_pipeline,
        categorical_features
    )
```

])

6. Final model pipeline

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
model = Pipeline([
    (
        "preprocessor",
        preprocessor
    ),
    (
        "classifier",
        RandomForestClassifier(
            n_estimators=200,
            max_depth=10,
            min_samples_leaf=2,
            class_weight="balanced",
            random_state=42
        )
    )
```

])

7. Split

X_train, X_test, y_train, y_test = (

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
    train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42,
        stratify=y
    )
```

)

8. Train

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
model.fit(X_train, y_train)
```

9. Predict

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]
```

10. Evaluate

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
print(
    "Accuracy:",
    accuracy_score(y_test, y_pred)
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
print(
    "Precision:",
    precision_score(
        y_test,
        y_pred,
        zero_division=0
    )
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
print(
    "Recall:",
    recall_score(
        y_test,
        y_pred,
        zero_division=0
    )
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
print(
    "F1:",
    f1_score(
        y_test,
        y_pred,
        zero_division=0
    )
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
print(
    "ROC-AUC:",
    roc_auc_score(y_test, y_prob)
```

)

MACHINE LEARNING: COMPLETE INTERVIEW GUIDE

```
print(
    "Confusion Matrix:\n",
    confusion_matrix(y_test, y_pred)
```

)


```
print(
    "Classification Report:\n",
    classification_report(
        y_test,
        y_pred,
        zero_division=0
    )
```

)

11. Save complete pipeline


```
joblib.dump(
    model,
    "models/churn_pipeline.joblib"
```

)

68. Most Asked ML Interview Questions

What is Machine Learning?

Machine Learning is a branch of AI where algorithms learn patterns from data and use them to make predictions or decisions on new data.

Supervised vs Unsupervised Learning?

Supervised learning uses labeled data with a target. Unsupervised learning has no target and discovers hidden patterns.

Regression vs Classification?

Regression predicts continuous numbers. Classification predicts classes or categories.

Parameter vs Hyperparameter?

Parameters are learned during training, such as coefficients. Hyperparameters are chosen before training, such as tree depth or learning rate.

What is overfitting?

Overfitting happens when a model learns training data too closely and performs poorly on unseen data.

What is underfitting?

Underfitting happens when a model is too simple and performs poorly on training and testing data.

What is regularization?

Regularization adds a penalty to control model complexity and reduce overfitting.

What is cross-validation?

Cross-validation evaluates a model on multiple train-validation splits to obtain a more reliable performance estimate.

Why is feature scaling required?

Distance-based and gradient-based algorithms can be dominated by features with larger numerical ranges. Scaling brings features to comparable scales.

Why do trees not require scaling?

Trees split values using thresholds. They do not depend on Euclidean distance or gradient magnitude.

What is data leakage?

Data leakage happens when the model receives information during training that would not be available at real prediction time.

Precision vs Recall?

Precision measures correctness among predicted positives. Recall measures how many actual positives were found.

When is recall important?

Recall is important when missing a positive case is costly, such as disease or fraud detection.

When is precision important?

Precision is important when false alarms are costly, such as marking important emails as spam.

What is F1 score?

F1 is the harmonic mean of precision and recall. It is useful when both metrics matter.

Bagging vs Boosting?

Bagging trains models independently and reduces variance. Boosting trains models sequentially and corrects earlier errors.

Random Forest vs XGBoost?

Random Forest uses independent trees and averaging. XGBoost builds trees sequentially to correct errors and often provides stronger tabular performance but requires more tuning.

What is PCA?

PCA is an unsupervised dimensionality-reduction method that creates components capturing maximum variance.

What is a baseline model?

A baseline is a simple reference model used to determine whether more complex models provide meaningful improvement.

Why use a pipeline?

A pipeline applies preprocessing consistently, prevents leakage and simplifies training, validation and deployment.

69. How to Explain an ML Project in an Interview

Use this order:

1. Problem statement
2. Business value
3. Dataset and source
4. Features and target
5. Data cleaning
6. EDA findings
7. Feature engineering
8. Algorithms tried
9. Evaluation metric
10. Final result
11. Deployment
12. Limitations

13. Future improvements

> Example:

I developed a customer churn prediction system to identify customers likely to leave a telecom service. I cleaned missing values, encoded categorical variables and created customer-behavior features such as tenure, monthly charges and contract type. I compared Logistic Regression, Random Forest and Gradient Boosting using stratified cross-validation. Because churn was imbalanced, I focused on recall and F1 score rather than accuracy alone. Random Forest gave the best validation performance. I saved the full preprocessing-model pipeline with Joblib and created a FastAPI endpoint for predictions.

70. Final ML Revision Table

Area	Must-Know Topics
Python	Lists, dictionaries, functions, OOP
NumPy	Arrays, broadcasting, matrix operations
Pandas	Cleaning, filtering, grouping, merging
Statistics	Mean, variance, probability, correlation
Preprocessing	Missing values, encoding, scaling
Regression	Linear, Ridge, Lasso, trees, boosting
Classification	Logistic, KNN, SVM, NB, forests
Clustering	K-Means, Hierarchical, DBSCAN, GMM
Dimensionality	PCA
Evaluation	MAE, RMSE, Accuracy, Precision, Recall, F1
Validation	Cross-validation, leakage prevention
Tuning	Grid Search, Random Search
Deep Learning	ANN, CNN, RNN, LSTM
NLP	TF-IDF, embeddings, classifiers
Deployment	Joblib, FastAPI, Streamlit
MLOps	Versioning, monitoring, retraining

Final Interview Answer

I understand the complete machine-learning lifecycle, starting from problem definition, data collection, EDA, cleaning, feature engineering and preprocessing. I can build regression, classification, clustering and dimensionality-reduction models using algorithms such as Linear Regression, Logistic Regression, KNN, Decision Tree, Random Forest, SVM, Naive Bayes, K-Means, PCA and boosting methods. I evaluate models using suitable metrics, prevent leakage using pipelines, perform cross-validation and hyperparameter tuning, handle imbalanced data and conduct error analysis. I also understand deep-learning basics, NLP, time series, model explainability, deployment using FastAPI or Streamlit, and production monitoring.

PART 6 | Complete Python: Beginner to Advanced

Every Important Concept with Easy Explanation and Code

Python is a high-level programming language used in:

Web development

Data analysis

Machine learning

Artificial intelligence

Automation

APIs

Testing

Desktop applications

Scripting

Basic Python program:

PYTHON: BEGINNER TO ADVANCED

```
print("Hello, Python!")
```

Output:

Hello, Python!

1. Python Installation and Execution

Check Python version:

python --version

or:

python3 --version

Run a Python file:

python app.py

Python files use the .py extension:

app.py

main.py

student.py

2. Python Syntax

Python uses indentation instead of curly braces.

Correct:

PYTHON: BEGINNER TO ADVANCED

```
age = 20

if age >= 18:
    print("Adult")
```

Incorrect:

PYTHON: BEGINNER TO ADVANCED

```
age = 20

if age >= 18:
print("Adult")
```

Python is case-sensitive:

PYTHON: BEGINNER TO ADVANCED

```
name = "Aamir"
Name = "Rahul"

print(name)
print(Name)
```

These are two different variables.

3. Comments

Comments explain code and are ignored by Python.

Single-line comment

This is a comment

PYTHON: BEGINNER TO ADVANCED

```
name = "Aamir"
```

Multi-line documentation

PYTHON: BEGINNER TO ADVANCED

```
def add(a, b):
    """
    This function adds two numbers.
    """
    return a + b
```

4. Variables

A variable stores a value.

PYTHON: BEGINNER TO ADVANCED

```
name = "Aamir"
age = 24
salary = 50000.50
is_employed = True
```

Python does not require explicit type declaration.

PYTHON: BEGINNER TO ADVANCED

```
x = 10
x = "Python"

print(x)
```

Variable naming rules

Valid:

PYTHON: BEGINNER TO ADVANCED

```
student_name = "Aamir"
age2 = 24
_total = 100
```

Invalid:

```
2age = 24
student-name = "Aamir"
class = "Python"
```

Recommended style:

PYTHON: BEGINNER TO ADVANCED

```
student_name = "Aamir"   # snake_case
```

5. Constants

Python does not enforce constants, but uppercase names are used by convention.

PYTHON: BEGINNER TO ADVANCED

```
PI = 3.14159
MAX_USERS = 100
```

6. Data Types

Important built-in types:

Type	Example	Meaning
int	10	Whole number
float	10.5	Decimal number
complex	2+3j	Complex number
str	"Python"	Text
bool	True	True or False
list	[1, 2]	Ordered mutable collection
tuple	(1, 2)	Ordered immutable collection
set	{1, 2}	Unique values
dict	{"name": "Aamir"}	Key-value pairs
NoneType	None	No value

Check a data type:

PYTHON: BEGINNER TO ADVANCED

```
age = 24

print(type(age))
```

Output:

PYTHON: BEGINNER TO ADVANCED

```
<class 'int'>
```

7. Integer, Float and Complex Numbers

PYTHON: BEGINNER TO ADVANCED

```
age = 24
price = 99.99
number = 2 + 3j

print(type(age))
print(type(price))
print(type(number))
```

Numeric operations

PYTHON: BEGINNER TO ADVANCED

```
a = 10
b = 3

print(a + b)   # Addition
print(a - b)   # Subtraction
print(a * b)   # Multiplication
print(a / b)   # Division
print(a // b)  # Floor division
print(a % b)   # Remainder
print(a ** b)  # Power
```

8. Type Conversion

Convert one type to another.

PYTHON: BEGINNER TO ADVANCED

```
age_text = "24"

age = int(age_text)
price = float("99.5")
number = str(100)
status = bool(1)

print(age)
print(price)
print(number)
print(status)
```

> Important:

int("hello") # ValueError

Safe conversion:

PYTHON: BEGINNER TO ADVANCED

```
value = "25"

try:
    number = int(value)
    print(number)
except ValueError:
    print("Invalid number")
```

9. Input and Output

Output using print()

PYTHON: BEGINNER TO ADVANCED

```
name = "Aamir"
age = 24

print(name)
print("Age:", age)
```

User input

PYTHON: BEGINNER TO ADVANCED

```
input() always returns a string.

name = input("Enter your name: ")
print("Hello", name)
```

Numerical input:

PYTHON: BEGINNER TO ADVANCED

```
age = int(input("Enter your age: "))
print("Next year age:", age + 1)
```

10. Formatted Strings

f-string

PYTHON: BEGINNER TO ADVANCED

```
name = "Aamir"
age = 24

print(f"My name is {name} and I am {age} years old.")
```

Number formatting

PYTHON: BEGINNER TO ADVANCED

```
price = 1234.56789

print(f"{price:.2f}")
print(f"{price:,.2f}")
```

Output:

1234.57

1,234.57

11. Operators

Arithmetic operators

PYTHON: BEGINNER TO ADVANCED

```
a = 10
b = 3

print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a % b)
print(a ** b)
```

Comparison operators

PYTHON: BEGINNER TO ADVANCED

```
a = 10
b = 20

print(a == b)
print(a != b)
print(a > b)
print(a < b)
print(a >= b)
print(a <= b)
```

Logical operators

PYTHON: BEGINNER TO ADVANCED

```
age = 25
has_id = True

print(age >= 18 and has_id)
print(age < 18 or has_id)
print(not has_id)
```

Assignment operators

PYTHON: BEGINNER TO ADVANCED

```
x = 10
```

x += 5

x -= 2

x *= 3

x /= 2

PYTHON: BEGINNER TO ADVANCED

```
print(x)
```

Membership operators

PYTHON: BEGINNER TO ADVANCED

```
fruits = ["apple", "banana", "mango"]

print("apple" in fruits)
print("orange" not in fruits)
```

Identity operators

PYTHON: BEGINNER TO ADVANCED

```
a = [1, 2]
b = a
c = [1, 2]

print(a is b)
print(a is c)
print(a == c)
```

Difference:

== checks values

is checks whether both references point to the same object

12. Conditional Statements

PYTHON: BEGINNER TO ADVANCED

```
if
age = 20

if age >= 18:
    print("Adult")
if-else
number = 7

if number % 2 == 0:
    print("Even")
else:
    print("Odd")
if-elif-else
marks = 82

if marks >= 90:
    print("Grade A+")
elif marks >= 75:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
else:
    print("Grade C")
```

Nested condition

PYTHON: BEGINNER TO ADVANCED

```
age = 25
has_license = True

if age >= 18:
    if has_license:
        print("Can drive")
    else:
        print("License required")
```

Ternary operator

PYTHON: BEGINNER TO ADVANCED

```
age = 20

result = "Adult" if age >= 18 else "Minor"

print(result)
```

13. Match-Case

match is similar to switch-case.

PYTHON: BEGINNER TO ADVANCED

```
day = 2
```

match day:

PYTHON: BEGINNER TO ADVANCED

```
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case _:
        print("Invalid day")
```

14. Loops

Loops repeat code.

PYTHON: BEGINNER TO ADVANCED

```
for loop
for number in range(1, 6):
    print(number)
```

Output:

1

2

3

4

5

PYTHON: BEGINNER TO ADVANCED

```
range()
print(list(range(5)))
print(list(range(1, 6)))
print(list(range(1, 10, 2)))
```

Loop through a list

PYTHON: BEGINNER TO ADVANCED

```
names = ["Aamir", "Rahul", "Priya"]

for name in names:
    print(name)
while loop
count = 1

while count <= 5:
    print(count)
    count += 1
```

15. Loop Control Statements

break

Stops the loop.

PYTHON: BEGINNER TO ADVANCED

```
for number in range(1, 10):
    if number == 5:
        break

    print(number)
```

continue

Skips the current iteration.

PYTHON: BEGINNER TO ADVANCED

```
for number in range(1, 6):
    if number == 3:
        continue

    print(number)
pass
```

Does nothing. It is used as a placeholder.

PYTHON: BEGINNER TO ADVANCED

```
for number in range(5):
    if number == 2:
        pass

    print(number)
```

Loop else

Runs when the loop completes normally.

PYTHON: BEGINNER TO ADVANCED

```
for number in range(1, 4):
    print(number)
else:
    print("Loop completed")
```

The else does not run when the loop stops using break.

16. Strings

A string is a sequence of characters.

PYTHON: BEGINNER TO ADVANCED

```
name = "Aamir"
message = 'Hello'
description = """This is
```

a multiline string."""

Indexing

PYTHON: BEGINNER TO ADVANCED

```
text = "Python"

print(text[0])
print(text[-1])
```

Output:

P

n

Slicing

PYTHON: BEGINNER TO ADVANCED

```
text = "Python Programming"

print(text[0:6])
print(text[:6])
print(text[7:])
print(text[::2])
print(text[::-1])
```

String immutability

Strings cannot be changed directly.

PYTHON: BEGINNER TO ADVANCED

```
text = "Python"
```

text[0] = "J" # TypeError

PYTHON: BEGINNER TO ADVANCED

```
text = "J" + text[1:]

print(text)
```

17. Important String Methods

PYTHON: BEGINNER TO ADVANCED

```
text = "  Python Programming  "

print(text.lower())
print(text.upper())
print(text.title())
print(text.capitalize())
print(text.strip())
print(text.replace("Python", "Java"))
print(text.startswith("  Python"))
print(text.endswith("  "))
```

Split and join

PYTHON: BEGINNER TO ADVANCED

```
sentence = "Python is easy"

words = sentence.split()

print(words)
print("-".join(words))
```

Find and count

PYTHON: BEGINNER TO ADVANCED

```
text = "banana"

print(text.find("na"))
print(text.count("a"))
```

Validation methods

PYTHON: BEGINNER TO ADVANCED

```
print("123".isdigit())
print("Python".isalpha())
print("Python123".isalnum())
print(" ".isspace())
```

18. Lists

A list is an ordered and mutable collection.

PYTHON: BEGINNER TO ADVANCED

```
numbers = [10, 20, 30, 40]
names = ["Aamir", "Rahul", "Priya"]
mixed = [1, "Python", True, 10.5]
```

Access list items

PYTHON: BEGINNER TO ADVANCED

```
numbers = [10, 20, 30, 40]

print(numbers[0])
print(numbers[-1])
print(numbers[1:3])
```

Modify list

numbers[0] = 100

PYTHON: BEGINNER TO ADVANCED

```
print(numbers)
```

19. Important List Methods

PYTHON: BEGINNER TO ADVANCED

```
numbers = [10, 20, 30]

numbers.append(40)
numbers.insert(1, 15)
numbers.extend([50, 60])

print(numbers)
```

Remove values:

PYTHON: BEGINNER TO ADVANCED

```
numbers.remove(20)
last_value = numbers.pop()
numbers.pop(0)

print(numbers)
print(last_value)
```

Other methods:

PYTHON: BEGINNER TO ADVANCED

```
numbers = [40, 10, 30, 20, 20]

numbers.sort()
print(numbers)

numbers.sort(reverse=True)
print(numbers)

numbers.reverse()
print(numbers)

print(numbers.count(20))
print(numbers.index(30))
```

Clear list:

PYTHON: BEGINNER TO ADVANCED

```
numbers.clear()
```

20. List Copy

Incorrect:

PYTHON: BEGINNER TO ADVANCED

```
a = [1, 2, 3]
b = a

b.append(4)

print(a)
```

Both variables point to the same list.

Correct shallow copy:

PYTHON: BEGINNER TO ADVANCED

```
a = [1, 2, 3]
b = a.copy()

b.append(4)

print(a)
print(b)
```

21. List Comprehension

List comprehension creates a list in a shorter way.

Traditional:

PYTHON: BEGINNER TO ADVANCED

```
squares = []

for number in range(1, 6):
    squares.append(number ** 2)

print(squares)
```

List comprehension:

PYTHON: BEGINNER TO ADVANCED

```
squares = [number ** 2 for number in range(1, 6)]

print(squares)
```

With condition:

PYTHON: BEGINNER TO ADVANCED

```
even_numbers = [
    number
    for number in range(1, 11)
    if number % 2 == 0
```

]

PYTHON: BEGINNER TO ADVANCED

```
print(even_numbers)
```

With if-else:

PYTHON: BEGINNER TO ADVANCED

```
labels = [
    "Even" if number % 2 == 0 else "Odd"
    for number in range(1, 6)
```

]

PYTHON: BEGINNER TO ADVANCED

```
print(labels)
```

22. Tuples

A tuple is ordered and immutable.

PYTHON: BEGINNER TO ADVANCED

```
coordinates = (10, 20)
colors = ("red", "green", "blue")
```

Access:

PYTHON: BEGINNER TO ADVANCED

```
print(coordinates[0])
```

Tuple cannot be modified:

coordinates[0] = 100 # TypeError

One-item tuple

PYTHON: BEGINNER TO ADVANCED

```
value = (10,)

print(type(value))
```

Tuple unpacking

PYTHON: BEGINNER TO ADVANCED

```
person = ("Aamir", 24, "Bihar")
```

name, age, state = person

PYTHON: BEGINNER TO ADVANCED

```
print(name)
print(age)
print(state)
```

Swapping values

PYTHON: BEGINNER TO ADVANCED

```
a = 10
b = 20
```

a, b = b, a

PYTHON: BEGINNER TO ADVANCED

```
print(a, b)
```

23. Sets

A set stores unique unordered values.

PYTHON: BEGINNER TO ADVANCED

```
numbers = {10, 20, 30, 20}

print(numbers)
```

Duplicate 20 is removed.

Set methods

PYTHON: BEGINNER TO ADVANCED

```
numbers = {10, 20, 30}

numbers.add(40)
numbers.update([50, 60])
numbers.remove(20)
numbers.discard(100)

print(numbers)
```

Difference:

PYTHON: BEGINNER TO ADVANCED

```
remove() gives an error if the value is missing
discard() does not give an error
```

24. Set Operations

PYTHON: BEGINNER TO ADVANCED

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a | b)   # Union
print(a & b)   # Intersection
print(a - b)   # Difference
print(a ^ b)   # Symmetric difference
```

Methods:

PYTHON: BEGINNER TO ADVANCED

```
print(a.union(b))
print(a.intersection(b))
print(a.difference(b))
```

25. Dictionaries

A dictionary stores key-value pairs.

PYTHON: BEGINNER TO ADVANCED

```
student = {
    "name": "Aamir",
    "age": 24,
    "course": "AI/ML"
```

}

Access:

PYTHON: BEGINNER TO ADVANCED

```
print(student["name"])
print(student.get("age"))
get() is safer:

print(student.get("salary", "Not available"))
```

Modify:

student["age"] = 25

student["city"] = "Jaipur"

Remove:

PYTHON: BEGINNER TO ADVANCED

```
student.pop("course")
student.popitem()
```

26. Dictionary Methods

PYTHON: BEGINNER TO ADVANCED

```
student = {
    "name": "Aamir",
    "age": 24,
    "city": "Jaipur"
```

}

PYTHON: BEGINNER TO ADVANCED

```
print(student.keys())
print(student.values())
print(student.items())
```

Loop:

PYTHON: BEGINNER TO ADVANCED

```
for key, value in student.items():
    print(key, value)
```

Dictionary comprehension:

PYTHON: BEGINNER TO ADVANCED

```
squares = {
    number: number ** 2
    for number in range(1, 6)
```

}

PYTHON: BEGINNER TO ADVANCED

```
print(squares)
```

27. Mutable vs Immutable

Mutable objects

Can be changed after creation:

list

dictionary

set

bytearray

Immutable objects

Cannot be changed after creation:

int

float

bool

string

tuple

frozenset

bytes

> Example:

PYTHON: BEGINNER TO ADVANCED

```
numbers = [1, 2, 3]
numbers.append(4)
```

The list changes.

PYTHON: BEGINNER TO ADVANCED

```
text = "Python"
text = text + " Programming"
```

A new string object is created.

28. Functions

A function is reusable code.

PYTHON: BEGINNER TO ADVANCED

```
def greet():
    print("Hello")

greet()
```

Function with parameters

PYTHON: BEGINNER TO ADVANCED

```
def greet(name):
    print(f"Hello, {name}")

greet("Aamir")
```

Return value

PYTHON: BEGINNER TO ADVANCED

```
def add(a, b):
    return a + b

result = add(10, 20)

print(result)
```

29. Types of Function Arguments

Positional arguments

PYTHON: BEGINNER TO ADVANCED

```
def introduce(name, age):
    print(name, age)

introduce("Aamir", 24)
```

Keyword arguments

PYTHON: BEGINNER TO ADVANCED

```
introduce(age=24, name="Aamir")
```

Default arguments

PYTHON: BEGINNER TO ADVANCED

```
def greet(name, message="Welcome"):
    print(f"{message}, {name}")

greet("Aamir")
greet("Aamir", "Good morning")
```

Do not use a mutable object as a default value.

Incorrect:

PYTHON: BEGINNER TO ADVANCED

```
def add_item(item, items=[]):
    items.append(item)
    return items
```

Correct:

PYTHON: BEGINNER TO ADVANCED

```
def add_item(item, items=None):
    if items is None:
        items = []

    items.append(item)
    return items
```

30. *args

- args accepts multiple positional arguments.

PYTHON: BEGINNER TO ADVANCED

```
def total(*numbers):
    return sum(numbers)

print(total(10, 20, 30, 40))
```

Inside the function, numbers is a tuple.

31. **kwargs

- *kwargs accepts multiple keyword arguments.

PYTHON: BEGINNER TO ADVANCED

```
def show_profile(**details):
    for key, value in details.items():
        print(key, value)

show_profile(
    name="Aamir",
    age=24,
    city="Jaipur"
```

)

Inside the function, details is a dictionary.

32. Parameter Order

Correct parameter order:

PYTHON: BEGINNER TO ADVANCED

```
def example(
    normal,
    default=10,
    *args,
    keyword_only=True,
    **kwargs
```

):

PYTHON: BEGINNER TO ADVANCED

```
    pass
```

33. Scope

Scope defines where a variable can be accessed.

Local scope

PYTHON: BEGINNER TO ADVANCED

```
def show():
    message = "Local variable"
    print(message)

show()
```

message cannot normally be accessed outside the function.

Global scope

PYTHON: BEGINNER TO ADVANCED

```
name = "Aamir"

def show():
    print(name)

show()
```

global

PYTHON: BEGINNER TO ADVANCED

```
count = 0

def increment():
    global count
    count += 1

increment()

print(count)
```

nonlocal

PYTHON: BEGINNER TO ADVANCED

```
def outer():
    count = 0

    def inner():
        nonlocal count
        count += 1
        return count

    return inner

counter = outer()

print(counter())
print(counter())
```

34. LEGB Rule

Python searches variables in this order:

CONCEPT MAP

```
L → Local
E → Enclosing
G → Global
B → Built-in
```

> Example:

PYTHON: BEGINNER TO ADVANCED

```
name = "Global"

def outer():
    name = "Enclosing"

    def inner():
        name = "Local"
        print(name)

    inner()

outer()
```

Output:

Local

35. Lambda Functions

A lambda is a small anonymous function.

PYTHON: BEGINNER TO ADVANCED

```
square = lambda number: number ** 2

print(square(5))
```

Normal function:

PYTHON: BEGINNER TO ADVANCED

```
def square(number):
    return number ** 2
```

Lambda with two arguments:

PYTHON: BEGINNER TO ADVANCED

```
add = lambda a, b: a + b

print(add(10, 20))
```

Use lambda for short operations, not complex logic.

36. map()

PYTHON: BEGINNER TO ADVANCED

```
map() applies a function to every item.

numbers = [1, 2, 3, 4]

squares = list(
    map(lambda number: number ** 2, numbers)
```

)

PYTHON: BEGINNER TO ADVANCED

```
print(squares)
```

List comprehension is often easier:

PYTHON: BEGINNER TO ADVANCED

```
squares = [number ** 2 for number in numbers]
```

37. filter()

PYTHON: BEGINNER TO ADVANCED

```
filter() keeps items that satisfy a condition.

numbers = [1, 2, 3, 4, 5, 6]

even_numbers = list(
    filter(lambda number: number % 2 == 0, numbers)
```

)

PYTHON: BEGINNER TO ADVANCED

```
print(even_numbers)
```

38. reduce()

PYTHON: BEGINNER TO ADVANCED

```
reduce() combines all values into one result.

from functools import reduce

numbers = [1, 2, 3, 4]

product = reduce(
    lambda a, b: a * b,
    numbers
```

)

PYTHON: BEGINNER TO ADVANCED

```
print(product)
```

39. Recursion

Recursion means a function calls itself.

Factorial

n

!

=

n

×

(

n

−

1

)

!

n!=n×(n−1)!

PYTHON: BEGINNER TO ADVANCED

```
def factorial(number):
    if number < 0:
        raise ValueError("Number must be non-negative")

    if number in (0, 1):
        return 1

    return number * factorial(number - 1)

print(factorial(5))
```

Flow:

PYTHON: BEGINNER TO ADVANCED

```
factorial(5)
```

5 × factorial(4)

5 × 4 × factorial(3)

5 × 4 × 3 × factorial(2)

5 × 4 × 3 × 2 × factorial(1)

120

Every recursion needs a base condition.

40. Built-in Functions

Important built-ins:

PYTHON: BEGINNER TO ADVANCED

```
numbers = [10, 20, 30, 40]

print(len(numbers))
print(sum(numbers))
print(min(numbers))
print(max(numbers))
print(sorted(numbers, reverse=True))
print(any([False, True, False]))
print(all([True, True, False]))
```

Other useful functions:

PYTHON: BEGINNER TO ADVANCED

```
print(abs(-10))
print(round(10.567, 2))
print(pow(2, 3))
print(divmod(10, 3))
```

41. enumerate()

PYTHON: BEGINNER TO ADVANCED

```
enumerate() gives index and value.

names = ["Aamir", "Rahul", "Priya"]

for index, name in enumerate(names, start=1):
    print(index, name)
```

42. zip()

PYTHON: BEGINNER TO ADVANCED

```
zip() combines multiple iterables.

names = ["Aamir", "Rahul", "Priya"]
marks = [90, 85, 88]

for name, mark in zip(names, marks):
    print(name, mark)
```

Create dictionary:

PYTHON: BEGINNER TO ADVANCED

```
students = dict(zip(names, marks))

print(students)
zip() stops when the shortest iterable ends.
```

43. Sorting

PYTHON: BEGINNER TO ADVANCED

```
sorted()
```

Returns a new sorted collection.

PYTHON: BEGINNER TO ADVANCED

```
numbers = [30, 10, 20]

result = sorted(numbers)

print(numbers)
print(result)
list.sort()
```

Changes the original list.

PYTHON: BEGINNER TO ADVANCED

```
numbers.sort()
```

Sort by key

PYTHON: BEGINNER TO ADVANCED

```
students = [
    {"name": "Aamir", "marks": 90},
    {"name": "Rahul", "marks": 80},
    {"name": "Priya", "marks": 95}
```

]

PYTHON: BEGINNER TO ADVANCED

```
students.sort(
    key=lambda student: student["marks"],
    reverse=True
```

)

PYTHON: BEGINNER TO ADVANCED

```
print(students)
```

44. Object-Oriented Programming

OOP organizes code using classes and objects.

Main concepts:

Class

Object

Attributes

Methods

Constructor

Encapsulation

Inheritance

Polymorphism

Abstraction

45. Class and Object

A class is a blueprint.

An object is an instance of a class.

PYTHON: BEGINNER TO ADVANCED

```
class Student:
    pass

student1 = Student()
student2 = Student()

print(type(student1))
```

46. Constructor and self

PYTHON: BEGINNER TO ADVANCED

```
__init__() initializes an object.
```

self represents the current object.

PYTHON: BEGINNER TO ADVANCED

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        return f"My name is {self.name}."

student = Student("Aamir", 24)

print(student.name)
print(student.introduce())
```

47. Instance and Class Variables

Instance variable

Different for every object.

PYTHON: BEGINNER TO ADVANCED

```
class Student:
    def __init__(self, name):
        self.name = name
```

Class variable

Shared by all objects.

PYTHON: BEGINNER TO ADVANCED

```
class Student:
    school = "UEM Jaipur"

    def __init__(self, name):
        self.name = name

student1 = Student("Aamir")
student2 = Student("Rahul")

print(student1.school)
print(student2.school)
```

48. Instance, Class and Static Methods

Instance method

Uses self.

PYTHON: BEGINNER TO ADVANCED

```
class Employee:
    company = "ABC"

    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def show(self):
        return f"{self.name}: {self.salary}"
```

Class method

Uses cls.

PYTHON: BEGINNER TO ADVANCED

```
class Employee:
    company = "ABC"

    @classmethod
    def change_company(cls, company):
        cls.company = company
```

Static method

Does not use self or cls.

PYTHON: BEGINNER TO ADVANCED

```
class Calculator:
    @staticmethod
    def add(a, b):
        return a + b

print(Calculator.add(10, 20))
```

49. Encapsulation

Encapsulation protects object data.

Public attribute

PYTHON: BEGINNER TO ADVANCED

```
class Account:
    def __init__(self, balance):
        self.balance = balance
```

Protected convention

PYTHON: BEGINNER TO ADVANCED

```
self._balance = balance
```

A single underscore means: “internal use.”

Private name mangling

PYTHON: BEGINNER TO ADVANCED

```
class Account:
    def __init__(self, balance):
        self.__balance = balance

    def get_balance(self):
        return self.__balance

account = Account(5000)

print(account.get_balance())
```

Python private attributes are not completely inaccessible; Python uses name mangling.

50. Property

property controls attribute access.

PYTHON: BEGINNER TO ADVANCED

```
class Employee:
    def __init__(self, salary):
        self._salary = salary

    @property
    def salary(self):
        return self._salary

    @salary.setter
    def salary(self, value):
        if value < 0:
            raise ValueError("Salary cannot be negative")

        self._salary = value

employee = Employee(50000)

employee.salary = 60000

print(employee.salary)
```

51. Inheritance

Inheritance allows a child class to reuse parent code.

PYTHON: BEGINNER TO ADVANCED

```
class Person:
    def __init__(self, name):
        self.name = name

    def introduce(self):
        return f"I am {self.name}"

class Student(Person):
    def study(self):
        return "I am studying"

student = Student("Aamir")

print(student.introduce())
print(student.study())
```

52. Types of Inheritance

Single inheritance

CONCEPT MAP

```
Parent → Child
```

Multilevel inheritance

CONCEPT MAP

```
Grandparent → Parent → Child
```

Multiple inheritance

CONCEPT MAP

```
Parent1 + Parent2 → Child
```

Hierarchical inheritance

CONCEPT MAP

```
One parent → Multiple children
```

Hybrid inheritance

Combination of inheritance types.

53. super()

PYTHON: BEGINNER TO ADVANCED

```
super() accesses parent class functionality.

class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, name, course):
        super().__init__(name)
        self.course = course

student = Student("Aamir", "AI/ML")

print(student.name)
print(student.course)
```

54. Method Overriding

A child class changes a parent method.

PYTHON: BEGINNER TO ADVANCED

```
class Animal:
    def sound(self):
        return "Animal sound"

class Dog(Animal):
    def sound(self):
        return "Bark"

dog = Dog()

print(dog.sound())
```

55. Polymorphism

Polymorphism means one interface can have different behaviors.

PYTHON: BEGINNER TO ADVANCED

```
class Dog:
    def sound(self):
        return "Bark"

class Cat:
    def sound(self):
        return "Meow"

animals = [Dog(), Cat()]

for animal in animals:
    print(animal.sound())
```

This is also an example of duck typing.

56. Operator Overloading

Customize operators for objects.

PYTHON: BEGINNER TO ADVANCED

```
class Number:
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return Number(self.value + other.value)

    def __repr__(self):
        return f"Number({self.value})"

a = Number(10)
b = Number(20)

print(a + b)
```

57. Abstraction

Abstraction hides implementation details and defines required behavior.

PYTHON: BEGINNER TO ADVANCED

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

rectangle = Rectangle(10, 5)

print(rectangle.area())
```

A class with an unimplemented abstract method cannot normally be instantiated.

58. Composition

Composition means a class strongly contains another object.

PYTHON: BEGINNER TO ADVANCED

```
class Engine:
    def start(self):
        return "Engine started"

class Car:
    def __init__(self):
        self.engine = Engine()

    def start(self):
        return self.engine.start()

car = Car()

print(car.start())
```

A car “has an” engine.

59. Aggregation

Aggregation is a weaker “has-a” relationship.

PYTHON: BEGINNER TO ADVANCED

```
class Teacher:
    def __init__(self, name):
        self.name = name

class Department:
    def __init__(self, teacher):
        self.teacher = teacher

teacher = Teacher("Rahul")
department = Department(teacher)

print(department.teacher.name)
```

The teacher can exist independently of the department.

60. Magic or Dunder Methods

Dunder means double underscore.

Important methods:

Method	Use
__init__	Constructor
__str__	User-friendly text
__repr__	Developer representation
__len__	Length
__eq__	Equality
__add__	Addition
__iter__	Iteration
__enter__	Context manager entry
__exit__	Context manager exit

> Example:

PYTHON: BEGINNER TO ADVANCED

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def __str__(self):
        return f"{self.name}: {self.marks}"

    def __len__(self):
        return len(self.name)

student = Student("Aamir", 90)

print(student)
print(len(student))
```

61. MRO

MRO means Method Resolution Order.

It determines where Python searches for a method in multiple inheritance.

PYTHON: BEGINNER TO ADVANCED

```
class A:
    pass

class B(A):
    pass

class C(A):
    pass

class D(B, C):
    pass

print(D.mro())
```

62. Exception Handling

Exceptions are runtime errors.

PYTHON: BEGINNER TO ADVANCED

```
try:
    number = int(input("Enter a number: "))
    result = 10 / number
    print(result)

except ValueError:
    print("Please enter a valid integer")

except ZeroDivisionError:
    print("Number cannot be zero")

else:
    print("No exception occurred")
```

finally:

PYTHON: BEGINNER TO ADVANCED

```
    print("Program finished")
```

63. raise

Use raise to create an error intentionally.

PYTHON: BEGINNER TO ADVANCED

```
def withdraw(balance, amount):
    if amount <= 0:
        raise ValueError("Amount must be positive")

    if amount > balance:
        raise ValueError("Insufficient balance")

    return balance - amount

print(withdraw(5000, 1000))
```

64. Custom Exception

PYTHON: BEGINNER TO ADVANCED

```
class InsufficientBalanceError(Exception):
    pass

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientBalanceError(
            "Not enough balance"
        )

    return balance - amount
```

65. File Handling

Open and read a file

PYTHON: BEGINNER TO ADVANCED

```
file = open("notes.txt", "r", encoding="utf-8")

content = file.read()

print(content)

file.close()
```

Better approach:

PYTHON: BEGINNER TO ADVANCED

```
with open(
    "notes.txt",
    "r",
    encoding="utf-8"
```

) as file:

PYTHON: BEGINNER TO ADVANCED

```
    content = file.read()

print(content)
```

The file is closed automatically.

66. File Modes

Mode	Meaning
r	Read
w	Write and replace
a	Append
x	Create new file
b	Binary mode
t	Text mode
+	Read and write

67. Write to File

PYTHON: BEGINNER TO ADVANCED

```
with open(
    "output.txt",
    "w",
    encoding="utf-8"
```

) as file:

PYTHON: BEGINNER TO ADVANCED

```
    file.write("Hello Python\n")
    file.write("File handling")
```

Append:

PYTHON: BEGINNER TO ADVANCED

```
with open(
    "output.txt",
    "a",
    encoding="utf-8"
```

) as file:

PYTHON: BEGINNER TO ADVANCED

```
    file.write("\nNew line")
```

68. Read File Line by Line

PYTHON: BEGINNER TO ADVANCED

```
with open(
    "output.txt",
    "r",
    encoding="utf-8"
```

) as file:

PYTHON: BEGINNER TO ADVANCED

```
    for line in file:
        print(line.strip())
```

Other methods:

PYTHON: BEGINNER TO ADVANCED

```
file.read()
file.readline()
file.readlines()
```

69. CSV Files

PYTHON: BEGINNER TO ADVANCED

```
import csv

with open(
    "students.csv",
    "w",
    newline="",
    encoding="utf-8"
```

) as file:

PYTHON: BEGINNER TO ADVANCED

```
    writer = csv.writer(file)

    writer.writerow(["Name", "Marks"])
    writer.writerow(["Aamir", 90])
    writer.writerow(["Rahul", 85])
```

Read CSV:

PYTHON: BEGINNER TO ADVANCED

```
with open(
    "students.csv",
    "r",
    encoding="utf-8"
```

) as file:

PYTHON: BEGINNER TO ADVANCED

```
    reader = csv.DictReader(file)

    for row in reader:
        print(row)
```

70. JSON Handling

PYTHON: BEGINNER TO ADVANCED

```
import json

student = {
    "name": "Aamir",
    "age": 24,
    "skills": ["Python", "SQL", "ML"]
```

}

PYTHON: BEGINNER TO ADVANCED

```
json_text = json.dumps(
    student,
    indent=4
```

)

PYTHON: BEGINNER TO ADVANCED

```
print(json_text)
```

Save JSON:

PYTHON: BEGINNER TO ADVANCED

```
with open(
    "student.json",
    "w",
    encoding="utf-8"
```

) as file:

PYTHON: BEGINNER TO ADVANCED

```
    json.dump(student, file, indent=4)
```

Read JSON:

PYTHON: BEGINNER TO ADVANCED

```
with open(
    "student.json",
    "r",
    encoding="utf-8"
```

) as file:

PYTHON: BEGINNER TO ADVANCED

```
    data = json.load(file)

print(data)
```

71. Modules

A module is a Python file containing reusable code.

calculator.py:

PYTHON: BEGINNER TO ADVANCED

```
def add(a, b):
    return a + b

PI = 3.14159
```

main.py:

PYTHON: BEGINNER TO ADVANCED

```
import calculator

print(calculator.add(10, 20))
print(calculator.PI)
```

Other import styles:

PYTHON: BEGINNER TO ADVANCED

```
from calculator import add
from calculator import add as addition
```

Avoid:

PYTHON: BEGINNER TO ADVANCED

```
from calculator import *
```

It can create name conflicts.

72. __name__ == "__main__"

PYTHON: BEGINNER TO ADVANCED

```
def main():
    print("Application running")

if __name__ == "__main__":
    main()
```

When the file runs directly:

__name__ becomes "__main__"

When imported:

__name__ becomes the module name

73. Packages

A package is a folder containing Python modules.

my_package/

CONCEPT MAP

```
├── __init__.py
├── calculator.py
└── formatter.py
```

Import:

PYTHON: BEGINNER TO ADVANCED

```
from my_package.calculator import add
```

74. pip

Install package:

pip install pandas

Uninstall:

pip uninstall pandas

Show installed packages:

pip list

Show package information:

pip show pandas

Create requirements file:

pip freeze > requirements.txt

Install from it:

pip install -r requirements.txt

75. Virtual Environment

A virtual environment isolates project dependencies.

Create:

python -m venv venv

Activate on Windows:

venv\Scripts\activate

Activate on macOS/Linux:

source venv/bin/activate

Deactivate:

deactivate

76. Iterables and Iterators

An iterable can be looped over.

Examples:

list

tuple

string

set

dictionary

An iterator returns one item at a time.

PYTHON: BEGINNER TO ADVANCED

```
numbers = [10, 20, 30]

iterator = iter(numbers)

print(next(iterator))
print(next(iterator))
print(next(iterator))
```

After all items, StopIteration is raised.

77. Custom Iterator

PYTHON: BEGINNER TO ADVANCED

```
class Counter:
    def __init__(self, start, end):
        self.current = start
        self.end = end

    def __iter__(self):
        return self

    def __next__(self):
        if self.current > self.end:
            raise StopIteration

        value = self.current
        self.current += 1

        return value

for number in Counter(1, 5):
    print(number)
```

78. Generators

A generator produces values one at a time using yield.

PYTHON: BEGINNER TO ADVANCED

```
def count_up_to(limit):
    number = 1

    while number <= limit:
        yield number
        number += 1

for number in count_up_to(5):
    print(number)
```

Generators save memory because they do not store every result at once.

79. Generator Expression

PYTHON: BEGINNER TO ADVANCED

```
squares = (
    number ** 2
    for number in range(1, 6)
```

)

PYTHON: BEGINNER TO ADVANCED

```
print(next(squares))
print(next(squares))
```

List comprehension creates all values immediately:

PYTHON: BEGINNER TO ADVANCED

```
squares = [
    number ** 2
    for number in range(1, 6)
```

]

Generator expression creates values lazily.

80. Closures

A closure remembers values from its enclosing function.

PYTHON: BEGINNER TO ADVANCED

```
def multiplier(factor):
    def multiply(number):
        return number * factor

    return multiply

double = multiplier(2)
triple = multiplier(3)

print(double(10))
print(triple(10))
```

factor remains available even after multiplier() finishes.

81. Decorators

A decorator adds behavior to a function without modifying the function itself.

PYTHON: BEGINNER TO ADVANCED

```
def decorator(function):
    def wrapper():
        print("Before function")
        function()
        print("After function")

    return wrapper
```

@decorator

PYTHON: BEGINNER TO ADVANCED

```
def greet():
    print("Hello")

greet()
```

82. Decorator with Arguments

PYTHON: BEGINNER TO ADVANCED

```
from functools import wraps

def logger(function):
    @wraps(function)
    def wrapper(*args, **kwargs):
        print(
            f"Calling {function.__name__}"
        )

        result = function(*args, **kwargs)

        print("Function completed")

        return result

    return wrapper
```

@logger

PYTHON: BEGINNER TO ADVANCED

```
def add(a, b):
    return a + b

print(add(10, 20))
```

@wraps preserves the original function name and metadata.

83. Decorator with Parameters

PYTHON: BEGINNER TO ADVANCED

```
from functools import wraps

def repeat(times):
    def decorator(function):
        @wraps(function)
        def wrapper(*args, **kwargs):
            result = None

            for _ in range(times):
                result = function(*args, **kwargs)

            return result

        return wrapper

    return decorator
```

@repeat(3)

PYTHON: BEGINNER TO ADVANCED

```
def greet():
    print("Hello")

greet()
```

84. Context Managers

A context manager manages setup and cleanup.

The with statement is commonly used for files.

Custom context manager:

PYTHON: BEGINNER TO ADVANCED

```
class DatabaseConnection:
    def __enter__(self):
        print("Connection opened")
        return self

    def __exit__(
        self,
        exception_type,
        exception_value,
        traceback
    ):
        print("Connection closed")

with DatabaseConnection():
    print("Using database")
```

85. contextlib

PYTHON: BEGINNER TO ADVANCED

```
from contextlib import contextmanager
```

@contextmanager

PYTHON: BEGINNER TO ADVANCED

```
def managed_resource():
    print("Resource started")

    try:
        yield "Resource"
    finally:
        print("Resource cleaned")

with managed_resource() as resource:
    print(resource)
```

86. Regular Expressions

Regular expressions search and validate text.

PYTHON: BEGINNER TO ADVANCED

```
import re

text = "My number is 9931879953"

match = re.search(r"\d{10}", text)

if match:
    print(match.group())
```

Important patterns

Pattern	Meaning
\d	Digit
\w	Word character
\s	Whitespace
.	Any character
+	One or more
*	Zero or more
?	Zero or one
^	Start
$	End
{n}	Exactly n
[abc]	a, b or c
[^abc]	Not a, b or c

87. Regex Functions

PYTHON: BEGINNER TO ADVANCED

```
import re

text = "Python 3.12 and Java 21"

print(re.findall(r"\d+", text))
print(re.sub(r"\d+", "NUMBER", text))
print(re.split(r"\s+", text))
```

Email validation:

PYTHON: BEGINNER TO ADVANCED

```
pattern = r"^[\w.-]+@[\w.-]+\.[A-Za-z]{2,}$"

email = "aamir@example.com"

if re.fullmatch(pattern, email):
    print("Valid email")
else:
    print("Invalid email")
```

88. Collections Module

Counter

PYTHON: BEGINNER TO ADVANCED

```
from collections import Counter

words = ["python", "java", "python", "sql"]

frequency = Counter(words)

print(frequency)
print(frequency.most_common(2))
```

defaultdict

PYTHON: BEGINNER TO ADVANCED

```
from collections import defaultdict

groups = defaultdict(list)
```

groups["IT"].append("Aamir")

groups["IT"].append("Rahul")

PYTHON: BEGINNER TO ADVANCED

```
print(groups)
```

deque

PYTHON: BEGINNER TO ADVANCED

```
from collections import deque

queue = deque([1, 2, 3])

queue.append(4)
queue.appendleft(0)
queue.popleft()

print(queue)
```

namedtuple

PYTHON: BEGINNER TO ADVANCED

```
from collections import namedtuple

Student = namedtuple(
    "Student",
    ["name", "marks"]
```

)

PYTHON: BEGINNER TO ADVANCED

```
student = Student("Aamir", 90)

print(student.name)
```

89. Dataclasses

Dataclasses reduce repeated class code.

PYTHON: BEGINNER TO ADVANCED

```
from dataclasses import dataclass
```

@dataclass

PYTHON: BEGINNER TO ADVANCED

```
class Student:
    name: str
    age: int
    marks: float = 0.0

student = Student(
    name="Aamir",
    age=24,
    marks=90
```

)

PYTHON: BEGINNER TO ADVANCED

```
print(student)
```

Frozen dataclass:

@dataclass(frozen=True)

PYTHON: BEGINNER TO ADVANCED

```
class Point:
    x: int
    y: int
```

A frozen dataclass is immutable.

90. Type Hints

Type hints improve readability and tooling.

PYTHON: BEGINNER TO ADVANCED

```
def add(a: int, b: int) -> int:
    return a + b
```

name: str = "Aamir"

marks: list[int] = [80, 90, 95]

Optional value:

PYTHON: BEGINNER TO ADVANCED

```
def find_user(
    user_id: int
```

) -> str | None:

PYTHON: BEGINNER TO ADVANCED

```
    if user_id == 1:
        return "Aamir"

    return None
```

Type hints are not automatically enforced at runtime.

91. typing

PYTHON: BEGINNER TO ADVANCED

```
from typing import Any, Callable

def process(
    values: list[int],
    operation: Callable[[int], int]
```

) -> list[int]:

PYTHON: BEGINNER TO ADVANCED

```
    return [operation(value) for value in values]

def square(value: int) -> int:
    return value ** 2

print(process([1, 2, 3], square))
```

92. Shallow Copy and Deep Copy

Shallow copy

Copies the outer object but nested objects are shared.

PYTHON: BEGINNER TO ADVANCED

```
import copy

original = [[1, 2], [3, 4]]
shallow = copy.copy(original)
```

shallow[0].append(100)

PYTHON: BEGINNER TO ADVANCED

```
print(original)
print(shallow)
```

Deep copy

Copies nested objects too.

PYTHON: BEGINNER TO ADVANCED

```
original = [[1, 2], [3, 4]]
deep = copy.deepcopy(original)
```

deep[0].append(100)

PYTHON: BEGINNER TO ADVANCED

```
print(original)
print(deep)
```

93. Memory Management

Python mainly uses:

Reference counting

Garbage collection

Private heap memory

Check object identity:

PYTHON: BEGINNER TO ADVANCED

```
a = [1, 2, 3]
b = a

print(id(a))
print(id(b))
```

Reference example:

PYTHON: BEGINNER TO ADVANCED

```
import sys

data = []

print(sys.getrefcount(data))
```

Garbage collector:

PYTHON: BEGINNER TO ADVANCED

```
import gc

gc.collect()
```

94. __slots__

__slots__ can reduce memory for many objects.

PYTHON: BEGINNER TO ADVANCED

```
class Employee:
    __slots__ = ("name", "salary")

    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
```

It also prevents adding undeclared attributes.

95. Descriptors

Descriptors control attribute access using:

__get__

__set__

__delete__

PYTHON: BEGINNER TO ADVANCED

```
class PositiveNumber:
    def __set_name__(self, owner, name):
        self.name = name

    def __get__(self, instance, owner):
        if instance is None:
            return self

        return instance.__dict__[self.name]

    def __set__(self, instance, value):
        if value < 0:
            raise ValueError(
                f"{self.name} must be positive"
            )

        instance.__dict__[self.name] = value

class Product:
    price = PositiveNumber()

    def __init__(self, price):
        self.price = price

product = Product(500)

print(product.price)
```

Properties, class methods and functions use the descriptor protocol internally.

96. Metaclasses

A metaclass creates classes.

Normal relationship:

Class creates objects

Metaclass creates classes

PYTHON: BEGINNER TO ADVANCED

```
class Meta(type):
    def __new__(
        cls,
        name,
        bases,
        namespace
    ):
        namespace["category"] = "Created by Meta"

        return super().__new__(
            cls,
            name,
            bases,
            namespace
        )

class Product(metaclass=Meta):
    pass

print(Product.category)
```

Metaclasses are advanced and should be used only when class creation needs to be customized.

97. Logging

Logging is better than print() for production applications.

PYTHON: BEGINNER TO ADVANCED

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format=(
        "%(asctime)s - "
        "%(levelname)s - "
        "%(message)s"
    )
```

)

PYTHON: BEGINNER TO ADVANCED

```
logging.info("Application started")
logging.warning("Low memory")
logging.error("Operation failed")
```

Log to file:

PYTHON: BEGINNER TO ADVANCED

```
logging.basicConfig(
    filename="app.log",
    level=logging.INFO
```

)

Levels:

DEBUG

INFO

WARNING

ERROR

CRITICAL

98. Assertions

Assertions check assumptions during development.

PYTHON: BEGINNER TO ADVANCED

```
def calculate_average(numbers):
    assert len(numbers) > 0, "List cannot be empty"

    return sum(numbers) / len(numbers)

print(calculate_average([10, 20, 30]))
```

Do not use assertions for user input validation because assertions can be disabled.

99. Unit Testing

Using unittest:

PYTHON: BEGINNER TO ADVANCED

```
import unittest

def add(a, b):
    return a + b

class TestCalculator(unittest.TestCase):
    def test_add(self):
        self.assertEqual(
            add(10, 20),
            30
        )

    def test_negative(self):
        self.assertEqual(
            add(-1, -2),
            -3
        )

if __name__ == "__main__":
    unittest.main()
```

Run:

python -m unittest

100. Pytest

Install:

pip install pytest

Test:

PYTHON: BEGINNER TO ADVANCED

```
def add(a, b):
    return a + b

def test_add():
    assert add(10, 20) == 30
```

Run:

pytest

Test exception:

PYTHON: BEGINNER TO ADVANCED

```
import pytest

def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")

    return a / b

def test_zero_division():
    with pytest.raises(ValueError):
        divide(10, 0)
```

101. SQLite Database

SQLite is included with Python.

PYTHON: BEGINNER TO ADVANCED

```
import sqlite3

connection = sqlite3.connect("students.db")
cursor = connection.cursor()

cursor.execute("""
```

CREATE TABLE IF NOT EXISTS students (

PYTHON: BEGINNER TO ADVANCED

```
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    marks REAL
```

)

""")

PYTHON: BEGINNER TO ADVANCED

```
cursor.execute(
    """
    INSERT INTO students (name, marks)
    VALUES (?, ?)
    """,
    ("Aamir", 90)
```

)

PYTHON: BEGINNER TO ADVANCED

```
connection.commit()

cursor.execute(
    "SELECT * FROM students"
```

)

PYTHON: BEGINNER TO ADVANCED

```
rows = cursor.fetchall()

for row in rows:
    print(row)

connection.close()
```

Use parameterized queries to prevent SQL injection.

102. HTTP APIs

Install Requests:

pip install requests

GET request:

PYTHON: BEGINNER TO ADVANCED

```
import requests

response = requests.get(
    "https://jsonplaceholder.typicode.com/users",
    timeout=10
```

)

PYTHON: BEGINNER TO ADVANCED

```
response.raise_for_status()

users = response.json()

for user in users[:3]:
    print(user["name"])
```

POST request:

PYTHON: BEGINNER TO ADVANCED

```
payload = {
    "title": "Python",
    "body": "Learning Python",
    "userId": 1
```

}

PYTHON: BEGINNER TO ADVANCED

```
response = requests.post(
    "https://jsonplaceholder.typicode.com/posts",
    json=payload,
    timeout=10
```

)

PYTHON: BEGINNER TO ADVANCED

```
print(response.json())
```

103. Environment Variables

Do not hard-code secrets.

PYTHON: BEGINNER TO ADVANCED

```
import os

database_url = os.getenv(
    "DATABASE_URL",
    "sqlite:///default.db"
```

)

PYTHON: BEGINNER TO ADVANCED

```
print(database_url)
```

Using .env:

pip install python-dotenv

.env:

PYTHON: BEGINNER TO ADVANCED

```
API_KEY=secret-value
```

Python:

PYTHON: BEGINNER TO ADVANCED

```
import os
from dotenv import load_dotenv

load_dotenv()

api_key = os.getenv("API_KEY")
```

Do not commit .env to Git.

104. Command-Line Arguments

Using argparse:

PYTHON: BEGINNER TO ADVANCED

```
import argparse

parser = argparse.ArgumentParser()

parser.add_argument(
    "--name",
    required=True
```

)

PYTHON: BEGINNER TO ADVANCED

```
parser.add_argument(
    "--age",
    type=int,
    default=18
```

)

PYTHON: BEGINNER TO ADVANCED

```
arguments = parser.parse_args()

print(arguments.name)
print(arguments.age)
```

Run:

python app.py --name Aamir --age 24

105. Date and Time

PYTHON: BEGINNER TO ADVANCED

```
from datetime import datetime, date, timedelta

now = datetime.now()
today = date.today()

print(now)
print(today)
print(now.strftime("%d-%m-%Y %H:%M:%S"))
```

Parse text:

PYTHON: BEGINNER TO ADVANCED

```
date_text = "2026-07-22"

parsed_date = datetime.strptime(
    date_text,
    "%Y-%m-%d"
```

)

PYTHON: BEGINNER TO ADVANCED

```
print(parsed_date)
```

Add days:

PYTHON: BEGINNER TO ADVANCED

```
future_date = today + timedelta(days=7)

print(future_date)
```

106. Random Numbers

PYTHON: BEGINNER TO ADVANCED

```
import random

print(random.randint(1, 10))
print(random.random())

items = ["Python", "Java", "SQL"]

print(random.choice(items))

random.shuffle(items)

print(items)
```

For reproducible results:

PYTHON: BEGINNER TO ADVANCED

```
random.seed(42)
```

For numerical work, NumPy random generators are often preferred.

107. Threading

Threading is useful for I/O-bound work.

Examples:

Downloading files

Calling APIs

Reading network data

PYTHON: BEGINNER TO ADVANCED

```
import threading
import time

def task(name):
    print(f"{name} started")
    time.sleep(2)
    print(f"{name} completed")

threads = [
    threading.Thread(
        target=task,
        args=(f"Task {number}",)
    )
    for number in range(3)
```

]

PYTHON: BEGINNER TO ADVANCED

```
for thread in threads:
    thread.start()

for thread in threads:
    thread.join()

print("All tasks completed")
```

108. Race Condition and Lock

A race condition occurs when threads modify shared data unsafely.

PYTHON: BEGINNER TO ADVANCED

```
import threading

counter = 0
lock = threading.Lock()

def increment():
    global counter

    for _ in range(100000):
        with lock:
            counter += 1

threads = [
    threading.Thread(target=increment)
    for _ in range(2)
```

]

PYTHON: BEGINNER TO ADVANCED

```
for thread in threads:
    thread.start()

for thread in threads:
    thread.join()

print(counter)
```

109. Multiprocessing

Multiprocessing is useful for CPU-bound work.

Examples:

Large calculations

Image processing

Data transformation

PYTHON: BEGINNER TO ADVANCED

```
from multiprocessing import Process

def task(number):
    print(
        f"Square of {number} is {number ** 2}"
    )

if __name__ == "__main__":
    processes = [
        Process(
            target=task,
            args=(number,)
        )
        for number in range(1, 4)
    ]

    for process in processes:
        process.start()

    for process in processes:
        process.join()
```

110. Process Pool

PYTHON: BEGINNER TO ADVANCED

```
from concurrent.futures import ProcessPoolExecutor

def square(number):
    return number ** 2

if __name__ == "__main__":
    with ProcessPoolExecutor() as executor:
        results = executor.map(
            square,
            [1, 2, 3, 4]
        )

    print(list(results))
```

111. Thread Pool

PYTHON: BEGINNER TO ADVANCED

```
from concurrent.futures import ThreadPoolExecutor

def fetch_data(number):
    return f"Data {number}"

with ThreadPoolExecutor(
    max_workers=3
```

) as executor:

PYTHON: BEGINNER TO ADVANCED

```
    results = executor.map(
        fetch_data,
        [1, 2, 3]
    )

print(list(results))
```

112. Async Programming

Async programming handles many I/O tasks efficiently.

PYTHON: BEGINNER TO ADVANCED

```
import asyncio

async def task(name, delay):
    print(f"{name} started")

    await asyncio.sleep(delay)

    print(f"{name} completed")

async def main():
    await asyncio.gather(
        task("Task 1", 2),
        task("Task 2", 1),
        task("Task 3", 3)
    )

asyncio.run(main())
```

Important keywords:

PYTHON: BEGINNER TO ADVANCED

```
async
await
asyncio.run()
asyncio.gather()
```

113. Threading vs Multiprocessing vs Async

Technique	Best for
Threading	I/O-bound tasks
Multiprocessing	CPU-bound tasks
Asyncio	Many concurrent I/O tasks

Python's GIL limits CPU-bound Python threads, but it does not prevent useful concurrency for I/O operations.

114. Functional Programming Concepts

Important ideas:

Pure functions

Immutability

Higher-order functions

First-class functions

map, filter, reduce

Functions are first-class objects:

PYTHON: BEGINNER TO ADVANCED

```
def greet(name):
    return f"Hello, {name}"

function = greet

print(function("Aamir"))
```

Function as argument:

PYTHON: BEGINNER TO ADVANCED

```
def execute(function, value):
    return function(value)

print(execute(str.upper, "python"))
```

115. Walrus Operator

The walrus operator := assigns and returns a value.

PYTHON: BEGINNER TO ADVANCED

```
numbers = [10, 20, 30, 40]

if (count := len(numbers)) > 3:
    print(f"List has {count} items")
```

Use it only when it makes code clearer.

116. Unpacking

Sequence unpacking

a, b, c = [10, 20, 30]

Star unpacking

first, *middle, last = [1, 2, 3, 4, 5]

PYTHON: BEGINNER TO ADVANCED

```
print(first)
print(middle)
print(last)
```

Function call unpacking

PYTHON: BEGINNER TO ADVANCED

```
def add(a, b):
    return a + b

numbers = [10, 20]

print(add(*numbers))
```

Dictionary unpacking:

PYTHON: BEGINNER TO ADVANCED

```
details = {
    "name": "Aamir",
    "age": 24
```

}

PYTHON: BEGINNER TO ADVANCED

```
def show(name, age):
    print(name, age)

show(**details)
```

117. Dictionary Merge

PYTHON: BEGINNER TO ADVANCED

```
a = {"name": "Aamir"}
b = {"age": 24}

combined = a | b

print(combined)
```

Alternative:

PYTHON: BEGINNER TO ADVANCED

```
combined = {**a, **b}
```

118. Structural Pattern Matching

PYTHON: BEGINNER TO ADVANCED

```
def process(command):
    match command:
        case ["add", first, second]:
            return int(first) + int(second)

        case ["show", name]:
            return f"Showing {name}"

        case _:
            return "Unknown command"

print(process(["add", "10", "20"]))
```

119. Packaging a Python Project

Typical structure:

my_project/

CONCEPT MAP

```
├── pyproject.toml
├── README.md
├── src/
│   └── my_package/
│       ├── __init__.py
│       └── calculator.py
└── tests/
    └── test_calculator.py
```

Basic pyproject.toml:

[build-system]

PYTHON: BEGINNER TO ADVANCED

```
requires = ["setuptools>=68"]
```

build-backend = "setuptools.build_meta"

[project]

PYTHON: BEGINNER TO ADVANCED

```
name = "my-python-package"
version = "0.1.0"
description = "Example Python package"
```

requires-python = ">=3.11"

Install project locally:

pip install -e .

120. Clean Code Practices

Use meaningful names

Bad:

PYTHON: BEGINNER TO ADVANCED

```
x = 50000
```

Better:

PYTHON: BEGINNER TO ADVANCED

```
monthly_salary = 50000
```

Keep functions small

PYTHON: BEGINNER TO ADVANCED

```
def calculate_tax(salary, tax_rate):
    return salary * tax_rate
```

Avoid repeated code

Create reusable functions.

Handle errors

PYTHON: BEGINNER TO ADVANCED

```
def divide(a, b):
    if b == 0:
        raise ValueError("b cannot be zero")

    return a / b
```

Follow PEP 8

Common conventions:

CONCEPT MAP

```
Variables and functions → snake_case
Classes → PascalCase
Constants → UPPER_CASE
```

Four spaces for indentation

121. Common Python Errors

SyntaxError

if age > 18

print("Adult")

NameError

print(undefined_variable)

TypeError

"10" + 5

ValueError

int("Python")

IndexError

PYTHON: BEGINNER TO ADVANCED

```
numbers = [1, 2, 3]
```

print(numbers[10])

KeyError

PYTHON: BEGINNER TO ADVANCED

```
student = {"name": "Aamir"}
```

print(student["age"])

AttributeError

PYTHON: BEGINNER TO ADVANCED

```
number = 10
```

number.append(20)

ModuleNotFoundError

The required package is not installed or the import path is wrong.

122. Common Interview Differences

List vs Tuple

List	Tuple
Mutable	Immutable
Uses []	Uses ()
More methods	Fewer methods
Good for changing data	Good for fixed data

Set vs List

Set	List
Unique values	Duplicates allowed
Unordered	Ordered
Fast membership check	Slower membership check
No indexing	Supports indexing

== vs is

==	is
Compares values	Compares object identity

Shallow vs Deep Copy

Shallow Copy	Deep Copy
Nested objects shared	Nested objects copied
copy.copy()	copy.deepcopy()

Iterator vs Generator

Iterator	Generator
Implements __iter__ and __next__	Uses yield
More code	Less code
Manual state management	Automatic state management

Thread vs Process

Thread	Process
Shared memory	Separate memory
Lightweight	Heavier
Best for I/O	Best for CPU work

123. Most Asked Python Coding Questions

Reverse a string

PYTHON: BEGINNER TO ADVANCED

```
text = "Python"

print(text[::-1])
```

Check palindrome

PYTHON: BEGINNER TO ADVANCED

```
def is_palindrome(text):
    cleaned = text.lower().replace(" ", "")
    return cleaned == cleaned[::-1]

print(is_palindrome("Madam"))
```

Count character frequency

PYTHON: BEGINNER TO ADVANCED

```
from collections import Counter

text = "programming"

print(Counter(text))
```

Find second largest number

PYTHON: BEGINNER TO ADVANCED

```
numbers = [10, 20, 30, 30, 40]

unique_numbers = sorted(
    set(numbers),
    reverse=True
```

)

PYTHON: BEGINNER TO ADVANCED

```
second_largest = (
    unique_numbers[1]
    if len(unique_numbers) >= 2
    else None
```

)

PYTHON: BEGINNER TO ADVANCED

```
print(second_largest)
```

Remove duplicates while preserving order

PYTHON: BEGINNER TO ADVANCED

```
numbers = [1, 2, 1, 3, 2, 4]

unique = list(dict.fromkeys(numbers))

print(unique)
```

Two Sum

PYTHON: BEGINNER TO ADVANCED

```
def two_sum(numbers, target):
    seen = {}

    for index, number in enumerate(numbers):
        required = target - number

        if required in seen:
            return [seen[required], index]

        seen[number] = index

    return []

print(two_sum([2, 7, 11, 15], 9))
```

Time complexity:

PYTHON: BEGINNER TO ADVANCED

```
O(n)
```

FizzBuzz

PYTHON: BEGINNER TO ADVANCED

```
for number in range(1, 16):
    if number % 15 == 0:
        print("FizzBuzz")
    elif number % 3 == 0:
        print("Fizz")
    elif number % 5 == 0:
        print("Buzz")
    else:
        print(number)
```

124. Recommended Python Learning Order

1. Installation and syntax
2. Variables and data types
3. Operators
4. Input and output
5. Conditional statements

6. Loops
7. Strings
8. Lists
9. Tuples
10. Sets
11. Dictionaries
12. Functions
13. Scope
14. Lambda and comprehensions
15. Recursion
16. OOP
17. Exception handling
18. File handling
19. Modules and packages
20. Virtual environments
21. Iterators and generators
22. Closures and decorators
23. Context managers
24. Regular expressions
25. Collections
26. Dataclasses and type hints
27. Testing and logging
28. Databases and APIs
29. Threading
30. Multiprocessing
31. Async programming
32. Memory management
33. Descriptors and metaclasses

34. Packaging and clean code

Final Interview Summary

Python is a high-level, interpreted and dynamically typed programming language. It supports procedural, object-oriented and functional programming. I understand Python data types, control statements, collections, functions, scope, lambda expressions, comprehensions, recursion, OOP, exception handling, file handling, modules, packages and virtual environments. I also understand advanced concepts such as iterators, generators, closures, decorators, context managers, regular expressions, dataclasses, type hints, testing, logging, APIs, databases, threading, multiprocessing, asynchronous programming, memory management, descriptors and packaging.

1. Installation and syntax
2. Variables and data types
3. Operators
4. Input and output
5. Conditional statements
6. Loops
7. Strings
8. Lists

9. Tuples
10. Sets
11. Dictionaries
12. Functions
13. Scope
14. Lambda and comprehensions
15. Recursion
16. OOP
17. Exception handling
18. File handling
19. Modules and packages
20. Virtual environments
21. Iterators and generators
22. Closures and decorators
23. Context managers
24. Regular expressions
25. Collections
26. Dataclasses and type hints
27. Testing and logging
28. Databases and APIs
29. Threading
30. Multiprocessing
31. Async programming
32. Memory management
33. Descriptors and metaclasses

34. Packaging and clean code

PART 7 | Complete Python: 34 Topics with Detailed Code

1. Installation and Syntax

Check Python installation

python --version

On some systems:

python3 --version

Run a Python file:

python app.py

Install a package:

pip install requests

Basic Python syntax

This is a single-line comment.

"""

This is a multiline string.

It is commonly used as a docstring.

"""

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print("Hello, Python!")

name = "Aamir"
age = 24
```

Python uses indentation to define blocks.

PYTHON: 34 TOPICS WITH DETAILED CODE

```
if age >= 18:
    print(f"{name} is an adult.")
else:
    print(f"{name} is a minor.")
```

Statements can continue inside parentheses.

PYTHON: 34 TOPICS WITH DETAILED CODE

```
total = (
    10
    + 20
    + 30
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print("Total:", total)
```

Important syntax rules

Python is case-sensitive.

PYTHON: 34 TOPICS WITH DETAILED CODE

```
name = "Aamir"
Name = "Rahul"

print(name)
print(Name)
```

Recommended naming styles:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
student_name = "Aamir"       # Variable/function: snake_case
MAX_USERS = 100              # Constant: UPPER_CASE
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class StudentProfile:        # Class: PascalCase
    pass
```

2. Variables and Data Types

A variable is a name that refers to an object.

PYTHON: 34 TOPICS WITH DETAILED CODE

```
name = "Aamir"
age = 24
height = 5.8
is_employed = False
nothing = None

print(type(name))
print(type(age))
print(type(height))
print(type(is_employed))
print(type(nothing))
```

Main built-in data types

Numeric types

PYTHON: 34 TOPICS WITH DETAILED CODE

```
integer_value = 100
float_value = 25.75
complex_value = 3 + 4j
```

Boolean

PYTHON: 34 TOPICS WITH DETAILED CODE

```
is_active = True
```

String

PYTHON: 34 TOPICS WITH DETAILED CODE

```
language = "Python"
```

List

PYTHON: 34 TOPICS WITH DETAILED CODE

```
skills = ["Python", "SQL", "ML"]
```

Tuple

PYTHON: 34 TOPICS WITH DETAILED CODE

```
coordinates = (10, 20)
```

Set

PYTHON: 34 TOPICS WITH DETAILED CODE

```
unique_numbers = {1, 2, 3}
```

Dictionary

PYTHON: 34 TOPICS WITH DETAILED CODE

```
student = {
    "name": "Aamir",
    "age": 24
```

}

None

PYTHON: 34 TOPICS WITH DETAILED CODE

```
result = None

values = [
    integer_value,
    float_value,
    complex_value,
    is_active,
    language,
    skills,
    coordinates,
    unique_numbers,
    student,
    result
```

]

PYTHON: 34 TOPICS WITH DETAILED CODE

```
for value in values:
    print(value, "->", type(value))
```

Dynamic typing

PYTHON: 34 TOPICS WITH DETAILED CODE

```
value = 10
print(value, type(value))

value = "Python"
print(value, type(value))

value = [1, 2, 3]
print(value, type(value))
```

Type conversion

PYTHON: 34 TOPICS WITH DETAILED CODE

```
age_text = "24"
price_text = "99.50"

age = int(age_text)
price = float(price_text)
number_text = str(500)
boolean_value = bool(1)

print(age, type(age))
print(price, type(price))
print(number_text, type(number_text))
print(boolean_value, type(boolean_value))
```

Mutable and immutable types

List is mutable.

PYTHON: 34 TOPICS WITH DETAILED CODE

```
numbers = [1, 2, 3]
numbers.append(4)

print(numbers)
```

String is immutable.

PYTHON: 34 TOPICS WITH DETAILED CODE

```
text = "Python"
```

text[0] = "J" # TypeError

PYTHON: 34 TOPICS WITH DETAILED CODE

```
new_text = "J" + text[1:]
print(new_text)
```

3. Operators

Arithmetic operators

PYTHON: 34 TOPICS WITH DETAILED CODE

```
a = 10
b = 3

print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Floor division:", a // b)
print("Remainder:", a % b)
print("Power:", a ** b)
```

Comparison operators

PYTHON: 34 TOPICS WITH DETAILED CODE

```
x = 10
y = 20

print(x == y)
print(x != y)
print(x > y)
print(x < y)
print(x >= y)
print(x <= y)
```

Logical operators

PYTHON: 34 TOPICS WITH DETAILED CODE

```
age = 24
has_id = True
has_permission = False

print(age >= 18 and has_id)
print(has_permission or has_id)
print(not has_permission)
```

Assignment operators

PYTHON: 34 TOPICS WITH DETAILED CODE

```
value = 10
```

value += 5

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(value)
```

value -= 2

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(value)
```

value *= 3

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(value)
```

value /= 2

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(value)
```

Membership operators

PYTHON: 34 TOPICS WITH DETAILED CODE

```
skills = ["Python", "SQL", "Machine Learning"]

print("Python" in skills)
print("Java" not in skills)
```

Identity operators

PYTHON: 34 TOPICS WITH DETAILED CODE

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]

print(a == c)   # Same values
print(a is c)   # Different objects

print(a is b)   # Same object
```

Bitwise operators

PYTHON: 34 TOPICS WITH DETAILED CODE

```
a = 5   # 0101
b = 3   # 0011

print("AND:", a & b)
print("OR:", a | b)
print("XOR:", a ^ b)
print("Left shift:", a << 1)
print("Right shift:", a >> 1)
```

4. Input and Output

Basic input

PYTHON: 34 TOPICS WITH DETAILED CODE

```
name = input("Enter your name: ")
print("Hello,", name)
input() always returns a string.

age_text = input("Enter your age: ")

print(age_text)
print(type(age_text))
```

Convert numerical input

PYTHON: 34 TOPICS WITH DETAILED CODE

```
try:
    first_number = float(input("Enter first number: "))
    second_number = float(input("Enter second number: "))

    total = first_number + second_number

    print("Total:", total)

except ValueError:
    print("Please enter valid numbers.")
```

Formatted output

PYTHON: 34 TOPICS WITH DETAILED CODE

```
name = "Aamir"
age = 24
salary = 52345.6789

print(f"Name: {name}")
print(f"Age: {age}")
print(f"Salary: ₹{salary:.2f}")
print(f"Formatted salary: ₹{salary:,.2f}")
```

Complete calculator

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def calculator():
    print("Simple Calculator")
    print("1. Add")
    print("2. Subtract")
    print("3. Multiply")
    print("4. Divide")

    try:
        choice = int(input("Choose an operation: "))
        number1 = float(input("Enter first number: "))
        number2 = float(input("Enter second number: "))

        if choice == 1:
            result = number1 + number2

        elif choice == 2:
            result = number1 - number2

        elif choice == 3:
            result = number1 * number2

        elif choice == 4:
            if number2 == 0:
                print("Cannot divide by zero.")
                return

            result = number1 / number2

        else:
            print("Invalid option.")
            return

        print(f"Result: {result:.2f}")
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    except ValueError:
        print("Invalid input.")
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
calculator()
```

5. Conditional Statements

PYTHON: 34 TOPICS WITH DETAILED CODE

```
if, elif, else
marks = 82

if marks >= 90:
    grade = "A+"

elif marks >= 80:
    grade = "A"

elif marks >= 70:
    grade = "B"

elif marks >= 60:
    grade = "C"

else:
    grade = "Fail"

print("Grade:", grade)
```

Nested conditions

PYTHON: 34 TOPICS WITH DETAILED CODE

```
age = 24
has_license = True

if age >= 18:
    if has_license:
        print("You can drive.")
    else:
        print("You need a driving license.")
else:
    print("You are too young to drive.")
```

Multiple conditions

PYTHON: 34 TOPICS WITH DETAILED CODE

```
username = "admin"
password = "python123"

if username == "admin" and password == "python123":
    print("Login successful.")
else:
    print("Invalid credentials.")
```

Ternary operator

PYTHON: 34 TOPICS WITH DETAILED CODE

```
number = 11

result = "Even" if number % 2 == 0 else "Odd"

print(result)
```

match-case

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def get_day_name(day_number):
    match day_number:
        case 1:
            return "Monday"
        case 2:
            return "Tuesday"
        case 3:
            return "Wednesday"
        case 4:
            return "Thursday"
        case 5:
            return "Friday"
        case 6:
            return "Saturday"
        case 7:
            return "Sunday"
        case _:
            return "Invalid day"
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(get_day_name(5))
```

6. Loops

PYTHON: 34 TOPICS WITH DETAILED CODE

```
for loop
for number in range(1, 6):
    print(number)
```

Range arguments

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(list(range(5)))
print(list(range(1, 6)))
print(list(range(2, 11, 2)))
print(list(range(10, 0, -1)))
```

Loop through collections

PYTHON: 34 TOPICS WITH DETAILED CODE

```
skills = ["Python", "SQL", "Machine Learning"]

for skill in skills:
    print(skill)
while loop
count = 1

while count <= 5:
    print(count)
    count += 1
```

break

PYTHON: 34 TOPICS WITH DETAILED CODE

```
for number in range(1, 11):
    if number == 6:
        break

    print(number)
```

continue

PYTHON: 34 TOPICS WITH DETAILED CODE

```
for number in range(1, 11):
    if number % 2 != 0:
        continue

    print(number)
pass
def future_function():
    pass
```

Loop with else

PYTHON: 34 TOPICS WITH DETAILED CODE

```
numbers = [10, 20, 30, 40]
search_value = 30

for number in numbers:
    if number == search_value:
        print("Value found.")
        break
else:
    print("Value not found.")
```

Multiplication table

PYTHON: 34 TOPICS WITH DETAILED CODE

```
number = 5

for multiplier in range(1, 11):
    result = number * multiplier

    print(f"{number} × {multiplier} = {result}")
```

Nested loops

PYTHON: 34 TOPICS WITH DETAILED CODE

```
for row in range(1, 4):
    for column in range(1, 4):
        print(f"Row {row}, Column {column}")
```

7. Strings

String creation

PYTHON: 34 TOPICS WITH DETAILED CODE

```
single_quote = 'Python'
double_quote = "Machine Learning"

multiline = """
```

Python is used for:

- Data Science
- Machine Learning
- Web Development

"""

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(multiline)
```

Indexing and slicing

PYTHON: 34 TOPICS WITH DETAILED CODE

```
text = "Python Programming"

print(text[0])
print(text[-1])
print(text[0:6])
print(text[:6])
print(text[7:])
print(text[::2])
print(text[::-1])
```

Important string methods

PYTHON: 34 TOPICS WITH DETAILED CODE

```
text = "  python programming language  "

print(text.strip())
print(text.upper())
print(text.lower())
print(text.title())
print(text.capitalize())
print(text.replace("python", "Java"))
print(text.count("g"))
print(text.find("programming"))
print(text.startswith("  python"))
print(text.endswith("  "))
```

Split and join

PYTHON: 34 TOPICS WITH DETAILED CODE

```
sentence = "Python is easy to learn"

words = sentence.split()

print(words)

joined_sentence = "-".join(words)

print(joined_sentence)
```

String validation

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print("12345".isdigit())
print("Python".isalpha())
print("Python123".isalnum())
print("hello world".islower())
print("HELLO".isupper())
print("   ".isspace())
```

Palindrome program

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def is_palindrome(text):
    cleaned_text = "".join(
        character.lower()
        for character in text
        if character.isalnum()
    )

    return cleaned_text == cleaned_text[::-1]
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
examples = [
    "Madam",
    "A man a plan a canal Panama",
    "Python"
```

]

PYTHON: 34 TOPICS WITH DETAILED CODE

```
for example in examples:
    print(example, "->", is_palindrome(example))
```

Character frequency

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def character_frequency(text):
    frequency = {}

    for character in text.lower():
        if character.isspace():
            continue

        frequency[character] = frequency.get(character, 0) + 1

    return frequency
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(character_frequency("Python Programming"))
```

8. Lists

A list is ordered, mutable and allows duplicates.

PYTHON: 34 TOPICS WITH DETAILED CODE

```
numbers = [10, 20, 30, 40]
mixed = [1, "Python", True, 5.5]

print(numbers)
print(mixed)
```

Indexing and slicing

PYTHON: 34 TOPICS WITH DETAILED CODE

```
numbers = [10, 20, 30, 40, 50]

print(numbers[0])
print(numbers[-1])
print(numbers[1:4])
print(numbers[::-1])
```

Add items

PYTHON: 34 TOPICS WITH DETAILED CODE

```
numbers = [10, 20, 30]

numbers.append(40)
numbers.insert(1, 15)
numbers.extend([50, 60])

print(numbers)
```

Remove items

PYTHON: 34 TOPICS WITH DETAILED CODE

```
numbers.remove(20)

last_item = numbers.pop()
first_item = numbers.pop(0)

print(numbers)
print("Removed last:", last_item)
print("Removed first:", first_item)
```

Sorting

PYTHON: 34 TOPICS WITH DETAILED CODE

```
numbers = [40, 10, 30, 20]

ascending = sorted(numbers)
descending = sorted(numbers, reverse=True)

print("Original:", numbers)
print("Ascending:", ascending)
print("Descending:", descending)

numbers.sort()
print("Changed original:", numbers)
```

List copy

PYTHON: 34 TOPICS WITH DETAILED CODE

```
original = [1, 2, 3]
copied = original.copy()

copied.append(4)

print("Original:", original)
print("Copied:", copied)
```

Student management example

PYTHON: 34 TOPICS WITH DETAILED CODE

```
students = []
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def add_student(name, marks):
    student = {
        "name": name,
        "marks": marks
    }

    students.append(student)
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def show_students():
    if not students:
        print("No students available.")
        return

    for index, student in enumerate(students, start=1):
        print(
            f"{index}. {student['name']} "
            f"- Marks: {student['marks']}"
        )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def calculate_average_marks():
    if not students:
        return 0

    total_marks = sum(
        student["marks"]
        for student in students
    )

    return total_marks / len(students)
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
add_student("Aamir", 90)
add_student("Rahul", 82)
add_student("Priya", 95)

show_students()

print(
    "Average Marks:",
    calculate_average_marks()
```

)

9. Tuples

A tuple is ordered and immutable.

PYTHON: 34 TOPICS WITH DETAILED CODE

```
coordinates = (10, 20)
student = ("Aamir", 24, "AI/ML")

print(coordinates)
print(student[0])
```

One-item tuple

PYTHON: 34 TOPICS WITH DETAILED CODE

```
single_value = (10,)

print(type(single_value))
```

Tuple unpacking

name, age, course = student

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(name)
print(age)
print(course)
```

Extended unpacking

PYTHON: 34 TOPICS WITH DETAILED CODE

```
numbers = (1, 2, 3, 4, 5)
```

first, *middle, last = numbers

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print("First:", first)
print("Middle:", middle)
print("Last:", last)
```

Return multiple values

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def calculate(number1, number2):
    total = number1 + number2
    difference = number1 - number2
    product = number1 * number2

    return total, difference, product
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
result = calculate(10, 5)

print(result)
```

total, difference, product = result

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(total)
print(difference)
print(product)
```

Named tuples

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from collections import namedtuple

Employee = namedtuple(
    "Employee",
    ["name", "department", "salary"]
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
employee = Employee(
    name="Aamir",
    department="AI/ML",
    salary=50000
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(employee.name)
print(employee.department)
print(employee.salary)
```

10. Sets

A set stores unique values.

PYTHON: 34 TOPICS WITH DETAILED CODE

```
numbers = {10, 20, 30, 20, 10}

print(numbers)
```

Add and remove

PYTHON: 34 TOPICS WITH DETAILED CODE

```
numbers.add(40)
numbers.update([50, 60, 70])

numbers.remove(20)
numbers.discard(100)

print(numbers)
```

Set operations

PYTHON: 34 TOPICS WITH DETAILED CODE

```
python_students = {
    "Aamir",
    "Rahul",
    "Priya"
```

}

PYTHON: 34 TOPICS WITH DETAILED CODE

```
sql_students = {
    "Aamir",
    "Neha",
    "Priya"
```

}

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(
    "Union:",
    python_students | sql_students
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(
    "Intersection:",
    python_students & sql_students
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(
    "Only Python:",
    python_students - sql_students
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(
    "Only one course:",
    python_students ^ sql_students
```

)

Subset and superset

PYTHON: 34 TOPICS WITH DETAILED CODE

```
a = {1, 2}
b = {1, 2, 3, 4}

print(a.issubset(b))
print(b.issuperset(a))
```

Remove duplicate values while preserving order

PYTHON: 34 TOPICS WITH DETAILED CODE

```
numbers = [1, 2, 1, 3, 2, 4]

unique_numbers = list(dict.fromkeys(numbers))

print(unique_numbers)
```

11. Dictionaries

A dictionary stores key-value pairs.

PYTHON: 34 TOPICS WITH DETAILED CODE

```
student = {
    "name": "Aamir",
    "age": 24,
    "course": "AI/ML"
```

}

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(student)
```

Access values

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(student["name"])
print(student.get("age"))
print(student.get("salary", "Not available"))
```

Add and update

student["city"] = "Jaipur"

student["age"] = 25

PYTHON: 34 TOPICS WITH DETAILED CODE

```
student.update({
    "email": "aamir@example.com",
    "skills": ["Python", "SQL", "ML"]
```

})

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(student)
```

Remove values

PYTHON: 34 TOPICS WITH DETAILED CODE

```
removed_course = student.pop("course")

print("Removed:", removed_course)

last_item = student.popitem()

print("Last item:", last_item)
```

Loop through dictionary

PYTHON: 34 TOPICS WITH DETAILED CODE

```
for key in student:
    print(key, student[key])

for key, value in student.items():
    print(key, "=", value)
```

Nested dictionary

PYTHON: 34 TOPICS WITH DETAILED CODE

```
company = {
    "employees": {
        101: {
            "name": "Aamir",
            "department": "AI/ML",
            "salary": 50000
        },
        102: {
            "name": "Rahul",
            "department": "Data Analytics",
            "salary": 45000
        }
    }
```

}

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(
    company["employees"][101]["name"]
```

)

Frequency-count program

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def word_frequency(sentence):
    frequency = {}

    words = sentence.lower().split()

    for word in words:
        clean_word = word.strip(".,!?")

        frequency[clean_word] = (
            frequency.get(clean_word, 0) + 1
        )

    return frequency
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
sentence = "Python is easy and Python is powerful"

print(word_frequency(sentence))
```

12. Functions

A function is reusable code.

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def greet():
    print("Hello")
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
greet()
```

Parameters and return values

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def add(number1, number2):
    result = number1 + number2

    return result
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
total = add(10, 20)

print(total)
```

Positional and keyword arguments

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def introduce(name, age, city):
    return (
        f"My name is {name}, "
        f"I am {age} years old, "
        f"and I live in {city}."
    )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(
    introduce("Aamir", 24, "Jaipur")
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(
    introduce(
        city="Jaipur",
        age=24,
        name="Aamir"
    )
```

)

Default arguments

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def calculate_salary(
    base_salary,
    bonus_percentage=10
```

):

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    bonus = (
        base_salary *
        bonus_percentage /
        100
    )

    return base_salary + bonus
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(calculate_salary(50000))
print(calculate_salary(50000, 20))
```

- args

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def calculate_total(*numbers):
    return sum(numbers)
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(calculate_total(10, 20))
print(calculate_total(10, 20, 30, 40))
```

- *kwargs

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def create_profile(**details):
    for key, value in details.items():
        print(f"{key}: {value}")
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
create_profile(
    name="Aamir",
    age=24,
    role="AI/ML Engineer"
```

)

Keyword-only arguments

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def register_user(
    username,
    *,
    is_admin=False,
    is_active=True
```

):

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    return {
        "username": username,
        "is_admin": is_admin,
        "is_active": is_active
    }
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(
    register_user(
        "aamir",
        is_admin=True
    )
```

)

Avoid mutable default arguments

Incorrect:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def add_skill_bad(skill, skills=[]):
    skills.append(skill)
    return skills
```

Correct:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def add_skill(skill, skills=None):
    if skills is None:
        skills = []

    skills.append(skill)

    return skills
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(add_skill("Python"))
print(add_skill("SQL"))
```

13. Scope

Python follows LEGB:

Local

Enclosing

Global

Built-in

Local and global scope

PYTHON: 34 TOPICS WITH DETAILED CODE

```
company_name = "OpenAI"
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def show_employee():
    employee_name = "Aamir"

    print("Company:", company_name)
    print("Employee:", employee_name)
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
show_employee()
```

global

PYTHON: 34 TOPICS WITH DETAILED CODE

```
count = 0
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def increment():
    global count

    count += 1
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
increment()
increment()

print(count)
```

nonlocal

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def create_counter():
    count = 0

    def increment():
        nonlocal count

        count += 1

        return count

    return increment
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
counter = create_counter()

print(counter())
print(counter())
print(counter())
```

LEGB demonstration

PYTHON: 34 TOPICS WITH DETAILED CODE

```
value = "Global"
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def outer():
    value = "Enclosing"

    def inner():
        value = "Local"

        print(value)

    inner()
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
outer()
```

14. Lambda and Comprehensions

Lambda

PYTHON: 34 TOPICS WITH DETAILED CODE

```
square = lambda number: number ** 2

print(square(5))
```

Lambda with sorting

PYTHON: 34 TOPICS WITH DETAILED CODE

```
students = [
    {"name": "Aamir", "marks": 90},
    {"name": "Rahul", "marks": 82},
    {"name": "Priya", "marks": 95}
```

]

PYTHON: 34 TOPICS WITH DETAILED CODE

```
sorted_students = sorted(
    students,
    key=lambda student: student["marks"],
    reverse=True
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(sorted_students)
```

List comprehension

PYTHON: 34 TOPICS WITH DETAILED CODE

```
squares = [
    number ** 2
    for number in range(1, 11)
```

]

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(squares)
```

Conditional list comprehension

PYTHON: 34 TOPICS WITH DETAILED CODE

```
even_squares = [
    number ** 2
    for number in range(1, 11)
    if number % 2 == 0
```

]

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(even_squares)
```

If-else comprehension

PYTHON: 34 TOPICS WITH DETAILED CODE

```
labels = [
    "Even" if number % 2 == 0 else "Odd"
    for number in range(1, 11)
```

]

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(labels)
```

Dictionary comprehension

PYTHON: 34 TOPICS WITH DETAILED CODE

```
square_map = {
    number: number ** 2
    for number in range(1, 6)
```

}

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(square_map)
```

Set comprehension

PYTHON: 34 TOPICS WITH DETAILED CODE

```
unique_lengths = {
    len(word)
    for word in [
        "Python",
        "Java",
        "SQL",
        "Python"
    ]
```

}

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(unique_lengths)
```

Generator expression

PYTHON: 34 TOPICS WITH DETAILED CODE

```
square_generator = (
    number ** 2
    for number in range(1, 6)
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
for value in square_generator:
    print(value)
```

15. Recursion

Recursion means a function calls itself.

Factorial

n

!

=

n

(

n

–

1

)

!

n!=n(n−1)!

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def factorial(number):
    if number < 0:
        raise ValueError(
            "Number must be non-negative"
        )

    if number in (0, 1):
        return 1

    return (
        number *
        factorial(number - 1)
    )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(factorial(5))
```

Fibonacci

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def fibonacci(number):
    if number < 0:
        raise ValueError(
            "Number must be non-negative"
        )

    if number in (0, 1):
        return number

    return (
        fibonacci(number - 1)
        + fibonacci(number - 2)
    )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
for number in range(10):
    print(fibonacci(number), end=" ")
```

The recursive Fibonacci version is slow because it repeats calculations.

Fibonacci with memoization

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from functools import lru_cache
```

@lru_cache(maxsize=None)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def fibonacci_fast(number):
    if number in (0, 1):
        return number

    return (
        fibonacci_fast(number - 1)
        + fibonacci_fast(number - 2)
    )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(fibonacci_fast(40))
```

Recursive sum of list

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def recursive_sum(numbers):
    if not numbers:
        return 0

    return (
        numbers[0]
        + recursive_sum(numbers[1:])
    )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(recursive_sum([10, 20, 30, 40]))
```

Binary search using recursion

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def binary_search(
    numbers,
    target,
    left,
    right
```

):

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    if left > right:
        return -1

    middle = (left + right) // 2

    if numbers[middle] == target:
        return middle

    if numbers[middle] > target:
        return binary_search(
            numbers,
            target,
            left,
            middle - 1
        )

    return binary_search(
        numbers,
        target,
        middle + 1,
        right
    )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
numbers = [10, 20, 30, 40, 50, 60]

index = binary_search(
    numbers,
    40,
    0,
    len(numbers) - 1
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print("Index:", index)
```

16. Object-Oriented Programming

Class, object, constructor and methods

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class BankAccount:
    bank_name = "Python Bank"

    def __init__(
        self,
        account_holder,
        opening_balance=0
    ):
        self.account_holder = account_holder
        self._balance = opening_balance
        self.transactions = []

    @property
    def balance(self):
        return self._balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError(
                "Deposit amount must be positive"
            )

        self._balance += amount

        self.transactions.append({
            "type": "deposit",
            "amount": amount,
            "balance": self._balance
        })

    def withdraw(self, amount):
        if amount <= 0:
            raise ValueError(
                "Withdrawal amount must be positive"
```



```
            )

        if amount > self._balance:
            raise ValueError(
                "Insufficient balance"
            )

        self._balance -= amount

        self.transactions.append({
            "type": "withdrawal",
            "amount": amount,
            "balance": self._balance
        })

    def show_statement(self):
        print(
            f"Account Holder: "
            f"{self.account_holder}"
        )

        for transaction in self.transactions:
            print(transaction)

    def __str__(self):
        return (
            f"BankAccount("
            f"holder={self.account_holder}, "
            f"balance={self._balance}"
            f")"
        )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
account = BankAccount(
    "Aamir",
    opening_balance=5000
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
account.deposit(2000)
account.withdraw(1500)

print(account)
print("Balance:", account.balance)

account.show_statement()
```

Inheritance and overriding

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class SavingsAccount(BankAccount):
    def __init__(
        self,
        account_holder,
        opening_balance=0,
        interest_rate=0.04
    ):
        super().__init__(
            account_holder,
            opening_balance
        )

        self.interest_rate = interest_rate

    def add_interest(self):
        interest = (
            self.balance *
            self.interest_rate
        )

        self.deposit(interest)

        return interest
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
savings = SavingsAccount(
    "Aamir",
    opening_balance=10000,
    interest_rate=0.05
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
interest = savings.add_interest()

print("Interest:", interest)
print(savings)
```

Abstract class

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from abc import ABC, abstractmethod
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class Employee(ABC):
    def __init__(self, name):
        self.name = name

    @abstractmethod
    def calculate_salary(self):
        pass
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class FullTimeEmployee(Employee):
    def __init__(
        self,
        name,
        monthly_salary
    ):
        super().__init__(name)

        self.monthly_salary = monthly_salary

    def calculate_salary(self):
        return self.monthly_salary
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class ContractEmployee(Employee):
    def __init__(
        self,
        name,
        hourly_rate,
        worked_hours
    ):
        super().__init__(name)

        self.hourly_rate = hourly_rate
        self.worked_hours = worked_hours

    def calculate_salary(self):
        return (
            self.hourly_rate *
            self.worked_hours
        )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
employees = [
    FullTimeEmployee("Aamir", 50000),
    ContractEmployee("Rahul", 500, 80)
```

]

PYTHON: 34 TOPICS WITH DETAILED CODE

```
for employee in employees:
    print(
        employee.name,
        employee.calculate_salary()
    )
```

Composition

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class Engine:
    def start(self):
        return "Engine started"
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class Car:
    def __init__(self, model):
        self.model = model
        self.engine = Engine()

    def start(self):
        return (
            f"{self.model}: "
            f"{self.engine.start()}"
        )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
car = Car("Tesla")

print(car.start())
```

17. Exception Handling

Basic exception handling

PYTHON: 34 TOPICS WITH DETAILED CODE

```
try:
    number = int(
        input("Enter a number: ")
    )

    result = 100 / number

except ValueError:
    print("Input must be an integer.")

except ZeroDivisionError:
    print("Cannot divide by zero.")

else:
    print("Result:", result)
```

finally:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    print("Execution finished.")
```

Raise exceptions

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def set_age(age):
    if not isinstance(age, int):
        raise TypeError(
            "Age must be an integer"
        )

    if age < 0:
        raise ValueError(
            "Age cannot be negative"
        )

    return age
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(set_age(24))
```

Custom exception

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class InsufficientBalanceError(Exception):
    """Raised when withdrawal exceeds balance."""
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class Account:
    def __init__(self, balance):
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            raise InsufficientBalanceError(
                "Not enough balance"
            )

        self.balance -= amount

        return self.balance
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
account = Account(1000)

try:
    account.withdraw(1500)

except InsufficientBalanceError as error:
    print("Transaction failed:", error)
```

Exception chaining

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def convert_to_integer(value):
    try:
        return int(value)

    except ValueError as error:
        raise ValueError(
            f"Cannot convert {value!r} "
            f"to integer"
        ) from error
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
try:
    convert_to_integer("Python")

except ValueError as error:
    print(error)
```

18. File Handling

Write and read text file

PYTHON: 34 TOPICS WITH DETAILED CODE

```
file_path = "students.txt"

students = [
    "Aamir,90\n",
    "Rahul,82\n",
    "Priya,95\n"
```

]

PYTHON: 34 TOPICS WITH DETAILED CODE

```
with open(
    file_path,
    "w",
    encoding="utf-8"
```

) as file:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    file.writelines(students)

with open(
    file_path,
    "r",
    encoding="utf-8"
```

) as file:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    content = file.read()

print(content)
```

Read line by line

PYTHON: 34 TOPICS WITH DETAILED CODE

```
with open(
    file_path,
    "r",
    encoding="utf-8"
```

) as file:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    for line_number, line in enumerate(
        file,
        start=1
    ):
        print(
            line_number,
            line.strip()
        )
```

Append

PYTHON: 34 TOPICS WITH DETAILED CODE

```
with open(
    file_path,
    "a",
    encoding="utf-8"
```

) as file:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    file.write("Neha,88\n")
```

CSV handling

PYTHON: 34 TOPICS WITH DETAILED CODE

```
import csv

csv_file = "employees.csv"

employees = [
    {
        "id": 1,
        "name": "Aamir",
        "department": "AI/ML"
    },
    {
        "id": 2,
        "name": "Rahul",
        "department": "Analytics"
    }
```

]

PYTHON: 34 TOPICS WITH DETAILED CODE

```
with open(
    csv_file,
    "w",
    newline="",
    encoding="utf-8"
```

) as file:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    fieldnames = [
        "id",
        "name",
        "department"
    ]

    writer = csv.DictWriter(
        file,
        fieldnames=fieldnames
    )

    writer.writeheader()
    writer.writerows(employees)

with open(
    csv_file,
    "r",
    encoding="utf-8"
```

) as file:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    reader = csv.DictReader(file)

    for row in reader:
        print(row)
```

JSON handling

PYTHON: 34 TOPICS WITH DETAILED CODE

```
import json

profile = {
    "name": "Aamir",
    "age": 24,
    "skills": [
        "Python",
        "SQL",
        "Machine Learning"
    ]
```

}

PYTHON: 34 TOPICS WITH DETAILED CODE

```
with open(
    "profile.json",
    "w",
    encoding="utf-8"
```

) as file:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    json.dump(
        profile,
        file,
        indent=4
    )

with open(
    "profile.json",
    "r",
    encoding="utf-8"
```

) as file:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    loaded_profile = json.load(file)

print(loaded_profile)
```

Path handling

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from pathlib import Path

folder = Path("data")
folder.mkdir(exist_ok=True)

file_path = folder / "notes.txt"

file_path.write_text(
    "Python File Handling",
    encoding="utf-8"
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(
    file_path.read_text(
        encoding="utf-8"
    )
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print("Exists:", file_path.exists())
print("Name:", file_path.name)
print("Suffix:", file_path.suffix)
```

19. Modules and Packages

Module example

Create calculator.py:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
PI = 3.14159
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def add(a, b):
    return a + b
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def subtract(a, b):
    return a - b
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def multiply(a, b):
    return a * b
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def divide(a, b):
    if b == 0:
        raise ValueError(
            "Cannot divide by zero"
        )

    return a / b
```

Use it in main.py:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
import calculator

print(calculator.add(10, 20))
print(calculator.PI)
```

Or:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from calculator import add, multiply

print(add(5, 6))
print(multiply(4, 7))
__name__ == "__main__"
def main():
    print("Program started")
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
if __name__ == "__main__":
    main()
```

Package structure

my_package/

CONCEPT MAP

```
├── __init__.py
├── math_tools.py
└── string_tools.py
```

math_tools.py:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def square(number):
    return number ** 2
```

string_tools.py:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def capitalize_words(text):
    return text.title()
```

Use package:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from my_package.math_tools import square
from my_package.string_tools import capitalize_words

print(square(5))
print(capitalize_words("python programming"))
```

20. Virtual Environments

Create virtual environment:

python -m venv venv

Activate on Windows:

venv\Scripts\activate

Activate on macOS/Linux:

source venv/bin/activate

Install packages:

pip install requests pandas

Create requirements file:

pip freeze > requirements.txt

Install dependencies:

pip install -r requirements.txt

Deactivate:

deactivate

Example requirements.txt:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
pandas==2.3.1
requests==2.32.4
```

Project structure:

python_project/

CONCEPT MAP

```
├── venv/
├── app.py
├── requirements.txt
└── README.md
```

21. Iterators and Generators

Iterator

PYTHON: 34 TOPICS WITH DETAILED CODE

```
numbers = [10, 20, 30]

iterator = iter(numbers)

print(next(iterator))
print(next(iterator))
print(next(iterator))
```

next(iterator) raises StopIteration.

Custom iterator

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class NumberRange:
    def __init__(self, start, end):
        self.current = start
        self.end = end

    def __iter__(self):
        return self

    def __next__(self):
        if self.current > self.end:
            raise StopIteration

        value = self.current
        self.current += 1

        return value
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
for number in NumberRange(1, 5):
    print(number)
```

Generator function

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def generate_numbers(start, end):
    current = start

    while current <= end:
        yield current

        current += 1
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
generator = generate_numbers(1, 5)

print(next(generator))
print(next(generator))

for number in generator:
    print(number)
```

Fibonacci generator

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def fibonacci_generator(limit):
    first = 0
    second = 1
    count = 0

    while count < limit:
        yield first

        first, second = (
            second,
            first + second
        )

        count += 1
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(
    list(
        fibonacci_generator(10)
    )
```

)

Generator pipeline

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def read_numbers():
    for number in range(1, 11):
        yield number
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def filter_even(numbers):
    for number in numbers:
        if number % 2 == 0:
            yield number
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def square_numbers(numbers):
    for number in numbers:
        yield number ** 2
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
pipeline = square_numbers(
    filter_even(
        read_numbers()
    )
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(list(pipeline))
```

22. Closures and Decorators

Closure

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def create_multiplier(factor):
    def multiply(number):
        return number * factor

    return multiply
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
double = create_multiplier(2)
triple = create_multiplier(3)

print(double(10))
print(triple(10))
```

Basic decorator

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def uppercase_decorator(function):
    def wrapper():
        result = function()

        return result.upper()

    return wrapper
```

@uppercase_decorator

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def greet():
    return "hello aamir"
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(greet())
```

Decorator with arguments

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from functools import wraps
import time
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def execution_time(function):
    @wraps(function)
    def wrapper(*args, **kwargs):
        start_time = time.perf_counter()

        result = function(*args, **kwargs)

        end_time = time.perf_counter()

        print(
            f"{function.__name__} took "
            f"{end_time - start_time:.6f} "
            f"seconds"
        )

        return result

    return wrapper
```

@execution_time

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def calculate_sum(limit):
    return sum(range(limit))
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(calculate_sum(1_000_000))
```

Decorator with configuration

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from functools import wraps
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def repeat(times):
    def decorator(function):
        @wraps(function)
        def wrapper(*args, **kwargs):
            result = None

            for _ in range(times):
                result = function(
                    *args,
                    **kwargs
                )

            return result

        return wrapper

    return decorator
```

@repeat(3)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def greet(name):
    print(f"Hello, {name}")
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
greet("Aamir")
```

Authentication decorator

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from functools import wraps
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def require_admin(function):
    @wraps(function)
    def wrapper(user, *args, **kwargs):
        if not user.get("is_admin"):
            raise PermissionError(
                "Admin access required"
            )

        return function(
            user,
            *args,
            **kwargs
        )

    return wrapper
```

@require_admin

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def delete_user(current_user, user_id):
    return (
        f"User {user_id} deleted by "
        f"{current_user['name']}"
    )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
admin = {
    "name": "Aamir",
    "is_admin": True
```

}

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(delete_user(admin, 101))
```

23. Context Managers

A context manager performs setup and cleanup.

File context manager

PYTHON: 34 TOPICS WITH DETAILED CODE

```
with open(
    "example.txt",
    "w",
    encoding="utf-8"
```

) as file:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    file.write("Context manager example")
```

Custom class context manager

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class DatabaseConnection:
    def __init__(self, database_name):
        self.database_name = database_name
        self.connected = False

    def __enter__(self):
        print(
            f"Connecting to "
            f"{self.database_name}"
        )

        self.connected = True

        return self

    def execute(self, query):
        if not self.connected:
            raise RuntimeError(
                "Database not connected"
            )

        print("Executing:", query)

    def __exit__(
        self,
        exception_type,
        exception_value,
        traceback
    ):
        self.connected = False

        print("Connection closed")

        # False means exceptions are not hidden.
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
        return False
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
with DatabaseConnection(
    "company.db"
```

) as connection:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    connection.execute(
        "SELECT * FROM employees"
    )
```

Context manager with contextlib

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from contextlib import contextmanager
```

@contextmanager

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def managed_resource(name):
    print(f"Opening resource: {name}")

    resource = {
        "name": name,
        "status": "open"
    }

    try:
        yield resource

    finally:
        resource["status"] = "closed"

        print(
            f"Closing resource: {name}"
        )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
with managed_resource(
    "ML Model"
```

) as resource:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    print(resource)
```

Suppressing an expected exception

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from contextlib import suppress

with suppress(FileNotFoundError):
    with open(
        "missing_file.txt",
        "r",
        encoding="utf-8"
    ) as file:
        print(file.read())

print("Program continued.")
```

24. Regular Expressions

PYTHON: 34 TOPICS WITH DETAILED CODE

```
import re

text = """
```

Contact Aamir at aamir@example.com.

Phone: 9931879953

Salary: 50000

"""

Search

PYTHON: 34 TOPICS WITH DETAILED CODE

```
phone_match = re.search(
    r"\b\d{10}\b",
    text
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
if phone_match:
    print(phone_match.group())
```

Find all

PYTHON: 34 TOPICS WITH DETAILED CODE

```
numbers = re.findall(
    r"\d+",
    text
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(numbers)
```

Email extraction

PYTHON: 34 TOPICS WITH DETAILED CODE

```
emails = re.findall(
    r"\b[\w.-]+@[\w.-]+\.[A-Za-z]{2,}\b",
    text
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(emails)
```

Replace

PYTHON: 34 TOPICS WITH DETAILED CODE

```
masked_text = re.sub(
    r"\b\d{10}\b",
    "XXXXXXXXXX",
    text
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(masked_text)
```

Split

PYTHON: 34 TOPICS WITH DETAILED CODE

```
sentence = "Python,SQL;Machine Learning|AI"

parts = re.split(
    r"[,;|]",
    sentence
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(parts)
```

Validation functions

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def validate_email(email):
    pattern = (
        r"^[\w.-]+@"
        r"[\w.-]+\."
        r"[A-Za-z]{2,}$"
    )

    return bool(
        re.fullmatch(
            pattern,
            email
        )
    )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def validate_phone(phone):
    return bool(
        re.fullmatch(
            r"[6-9]\d{9}",
            phone
        )
    )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(
    validate_email(
        "aamir@example.com"
    )
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(
    validate_phone(
        "9931879953"
    )
```

)

Named groups

PYTHON: 34 TOPICS WITH DETAILED CODE

```
log_line = (
    "2026-07-22 "
    "ERROR "
    "Database connection failed"
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
pattern = (
    r"(?P<date>\d{4}-\d{2}-\d{2}) "
    r"(?P<level>\w+) "
    r"(?P<message>.+)"
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
match = re.match(
    pattern,
    log_line
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
if match:
    print(match.groupdict())
```

25. Collections Module

Counter

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from collections import Counter

words = [
    "python",
    "sql",
    "python",
    "ml",
    "python",
    "sql"
```

]

PYTHON: 34 TOPICS WITH DETAILED CODE

```
frequency = Counter(words)

print(frequency)
print(frequency["python"])
print(frequency.most_common(2))
```

defaultdict

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from collections import defaultdict

employees_by_department = defaultdict(list)

employees = [
    ("Aamir", "AI/ML"),
    ("Rahul", "Analytics"),
    ("Priya", "AI/ML")
```

]

PYTHON: 34 TOPICS WITH DETAILED CODE

```
for name, department in employees:
    employees_by_department[
        department
    ].append(name)

print(
    dict(employees_by_department)
```

)

deque

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from collections import deque

queue = deque()

queue.append("Task 1")
queue.append("Task 2")
queue.append("Task 3")

while queue:
    current_task = queue.popleft()

    print("Processing:", current_task)
```

Bounded deque

PYTHON: 34 TOPICS WITH DETAILED CODE

```
recent_searches = deque(
    maxlen=3
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
recent_searches.append("Python")
recent_searches.append("SQL")
recent_searches.append("ML")
recent_searches.append("Pandas")

print(recent_searches)
```

ChainMap

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from collections import ChainMap

default_config = {
    "theme": "light",
    "language": "English"
```

}

PYTHON: 34 TOPICS WITH DETAILED CODE

```
user_config = {
    "theme": "dark"
```

}

PYTHON: 34 TOPICS WITH DETAILED CODE

```
config = ChainMap(
    user_config,
    default_config
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(config["theme"])
print(config["language"])
```

26. Dataclasses and Type Hints

Basic dataclass

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from dataclasses import dataclass, field
```

@dataclass

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class Student:
    name: str
    age: int
    marks: float
    skills: list[str] = field(
        default_factory=list
    )

    def add_skill(
        self,
        skill: str
    ) -> None:
        self.skills.append(skill)

    def is_passed(self) -> bool:
        return self.marks >= 40
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
student = Student(
    name="Aamir",
    age=24,
    marks=90
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
student.add_skill("Python")
student.add_skill("Machine Learning")

print(student)
print(student.is_passed())
```

Validation in __post_init__

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from dataclasses import dataclass
```

@dataclass

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class Product:
    name: str
    price: float
    quantity: int = 0

    def __post_init__(self):
        if self.price < 0:
            raise ValueError(
                "Price cannot be negative"
            )

        if self.quantity < 0:
            raise ValueError(
                "Quantity cannot be negative"
            )

    @property
    def total_value(self) -> float:
        return self.price * self.quantity
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
product = Product(
    "Laptop",
    50000,
    2
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(product.total_value)
```

Frozen dataclass

@dataclass(frozen=True)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class Point:
    x: int
    y: int
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
point = Point(10, 20)

print(point)
```

point.x = 100 # FrozenInstanceError

Type hints

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def calculate_average(
    values: list[float]
```

) -> float:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    if not values:
        raise ValueError(
            "Values cannot be empty"
        )

    return sum(values) / len(values)
```

marks: list[float] = [

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    90,
    82,
    95
```

]

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(calculate_average(marks))
```

Optional and union types

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def find_student(
    student_id: int
```

) -> str | None:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    students = {
        1: "Aamir",
        2: "Rahul"
    }

    return students.get(student_id)
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
result = find_student(3)

print(result)
```

Callable

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from collections.abc import Callable
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def process_values(
    values: list[int],
    operation: Callable[[int], int]
```

) -> list[int]:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    return [
        operation(value)
        for value in values
    ]
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def square(number: int) -> int:
    return number ** 2
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(
    process_values(
        [1, 2, 3],
        square
    )
```

)

27. Testing and Logging

unittest

PYTHON: 34 TOPICS WITH DETAILED CODE

```
import unittest
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def divide(a, b):
    if b == 0:
        raise ValueError(
            "Cannot divide by zero"
        )

    return a / b
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class TestCalculator(unittest.TestCase):
    def test_divide(self):
        self.assertEqual(
            divide(10, 2),
            5
        )

    def test_decimal_result(self):
        self.assertAlmostEqual(
            divide(10, 3),
            3.333333,
            places=5
        )

    def test_zero_division(self):
        with self.assertRaises(
            ValueError
        ):
            divide(10, 0)
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
if __name__ == "__main__":
    unittest.main()
```

Pytest style

PYTHON: 34 TOPICS WITH DETAILED CODE

```
import pytest
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def add(a, b):
    return a + b
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def test_add_positive_numbers():
    assert add(10, 20) == 30
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def test_add_negative_numbers():
    assert add(-5, -3) == -8
```

@pytest.mark.parametrize(

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    "a,b,expected",
    [
        (1, 2, 3),
        (10, 20, 30),
        (-1, 1, 0)
    ]
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def test_add_multiple_cases(
    a,
    b,
    expected
```

):

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    assert add(a, b) == expected
```

Run:

pytest -v

Logging

PYTHON: 34 TOPICS WITH DETAILED CODE

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format=(
        "%(asctime)s | "
        "%(levelname)s | "
        "%(name)s | "
        "%(message)s"
    )
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
logger = logging.getLogger(
    "employee_app"
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def calculate_salary(
    base_salary,
    bonus_percentage
```

):

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    logger.info(
        "Calculating salary"
    )

    if base_salary < 0:
        logger.error(
            "Negative base salary"
        )

        raise ValueError(
            "Salary cannot be negative"
        )

    bonus = (
        base_salary *
        bonus_percentage /
        100
    )

    final_salary = (
        base_salary + bonus
    )

    logger.info(
        "Salary calculated successfully"
    )

    return final_salary
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(
    calculate_salary(
        50000,
        10
    )
```

)

File logging

PYTHON: 34 TOPICS WITH DETAILED CODE

```
logging.basicConfig(
    filename="application.log",
    filemode="a",
    level=logging.INFO,
    format=(
        "%(asctime)s - "
        "%(levelname)s - "
        "%(message)s"
    )
```

)

28. Databases and APIs

SQLite database

PYTHON: 34 TOPICS WITH DETAILED CODE

```
import sqlite3

DATABASE_NAME = "company.db"
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def create_database():
    with sqlite3.connect(
        DATABASE_NAME
    ) as connection:
        cursor = connection.cursor()

        cursor.execute("""
            CREATE TABLE IF NOT EXISTS employees (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                name TEXT NOT NULL,
                department TEXT NOT NULL,
                salary REAL NOT NULL
            )
        """)

        connection.commit()
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def add_employee(
    name,
    department,
    salary
```

):

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    with sqlite3.connect(
        DATABASE_NAME
    ) as connection:
        cursor = connection.cursor()

        cursor.execute(
            """
            INSERT INTO employees
            (name, department, salary)
            VALUES (?, ?, ?)
            """,
            (
                name,
                department,
                salary
            )
        )

        connection.commit()
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def get_employees():
    with sqlite3.connect(
        DATABASE_NAME
    ) as connection:
        connection.row_factory = (
            sqlite3.Row
        )

        cursor = connection.cursor()

        cursor.execute(
            "SELECT * FROM employees"
        )

        rows = cursor.fetchall()

        return [
            dict(row)
            for row in rows
        ]
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
create_database()

add_employee(
    "Aamir",
    "AI/ML",
    50000
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
add_employee(
    "Rahul",
    "Analytics",
    45000
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(get_employees())
```

Update and delete

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def update_salary(
    employee_id,
    new_salary
```

):

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    with sqlite3.connect(
        DATABASE_NAME
    ) as connection:
        cursor = connection.cursor()

        cursor.execute(
            """
            UPDATE employees
            SET salary = ?
            WHERE id = ?
            """,
            (
                new_salary,
                employee_id
            )
        )

        connection.commit()
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def delete_employee(
    employee_id
```

):

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    with sqlite3.connect(
        DATABASE_NAME
    ) as connection:
        cursor = connection.cursor()

        cursor.execute(
            """
            DELETE FROM employees
            WHERE id = ?
            """,
            (employee_id,)
        )

        connection.commit()
```

REST API request

PYTHON: 34 TOPICS WITH DETAILED CODE

```
import requests
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def get_users():
    url = (
        "https://jsonplaceholder."
        "typicode.com/users"
    )

    try:
        response = requests.get(
            url,
            timeout=10
        )

        response.raise_for_status()

        return response.json()

    except requests.Timeout:
        print("Request timed out.")

    except requests.HTTPError as error:
        print(
            "HTTP error:",
            error
        )

    except requests.RequestException as error:
        print(
            "Request failed:",
            error
        )

    return []
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
users = get_users()

for user in users[:3]:
    print(
        user["name"],
        user["email"]
    )
```

POST request

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def create_post():
    url = (
        "https://jsonplaceholder."
        "typicode.com/posts"
    )

    payload = {
        "title": "Python",
        "body": "Learning APIs",
        "userId": 1
    }

    response = requests.post(
        url,
        json=payload,
        timeout=10
    )

    response.raise_for_status()

    return response.json()
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(create_post())
```

29. Threading

Threading is useful for I/O-bound tasks.

PYTHON: 34 TOPICS WITH DETAILED CODE

```
import threading
import time
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def download_file(
    file_name,
    delay
```

):

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    print(
        f"Starting {file_name}"
    )

    time.sleep(delay)

    print(
        f"Completed {file_name}"
    )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
threads = []

tasks = [
    ("file1.csv", 2),
    ("file2.csv", 1),
    ("file3.csv", 3)
```

]

PYTHON: 34 TOPICS WITH DETAILED CODE

```
for file_name, delay in tasks:
    thread = threading.Thread(
        target=download_file,
        args=(
            file_name,
            delay
        )
    )

    threads.append(thread)

    thread.start()

for thread in threads:
    thread.join()

print("All downloads completed.")
```

Race condition and lock

PYTHON: 34 TOPICS WITH DETAILED CODE

```
import threading

counter = 0
lock = threading.Lock()
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def increment_counter():
    global counter

    for _ in range(100000):
        with lock:
            counter += 1
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
threads = [
    threading.Thread(
        target=increment_counter
    )
    for _ in range(4)
```

]

PYTHON: 34 TOPICS WITH DETAILED CODE

```
for thread in threads:
    thread.start()

for thread in threads:
    thread.join()

print("Final counter:", counter)
```

Thread pool

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from concurrent.futures import (
    ThreadPoolExecutor,
    as_completed
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
import time
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def process_task(task_id):
    time.sleep(1)

    return (
        f"Task {task_id} completed"
    )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
with ThreadPoolExecutor(
    max_workers=3
```

) as executor:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    futures = [
        executor.submit(
            process_task,
            task_id
        )
        for task_id in range(1, 6)
    ]

    for future in as_completed(futures):
        print(future.result())
```

30. Multiprocessing

Multiprocessing is useful for CPU-bound work.

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from multiprocessing import Process
import os
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def calculate_square(number):
    print(
        f"Process {os.getpid()} "
        f"calculated {number ** 2}"
    )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
if __name__ == "__main__":
    processes = []

    for number in range(1, 6):
        process = Process(
            target=calculate_square,
            args=(number,)
        )

        processes.append(process)

        process.start()

    for process in processes:
        process.join()

    print("All processes completed.")
```

Process pool

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from concurrent.futures import (
    ProcessPoolExecutor
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def calculate_factorial(number):
    result = 1

    for value in range(
        1,
        number + 1
    ):
        result *= value

    return number, result
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
if __name__ == "__main__":
    numbers = [
        10,
        15,
        20,
        25
    ]

    with ProcessPoolExecutor() as executor:
        results = executor.map(
            calculate_factorial,
            numbers
        )

    for number, factorial in results:
        print(
            number,
            factorial
        )
```

Shared queue

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from multiprocessing import (
    Process,
    Queue
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def producer(queue):
    for number in range(1, 6):
        queue.put(number)

    queue.put(None)
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def consumer(queue):
    while True:
        item = queue.get()

        if item is None:
            break

        print(
            "Processed:",
            item ** 2
        )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
if __name__ == "__main__":
    queue = Queue()

    producer_process = Process(
        target=producer,
        args=(queue,)
    )

    consumer_process = Process(
        target=consumer,
        args=(queue,)
    )

    producer_process.start()
    consumer_process.start()

    producer_process.join()
    consumer_process.join()
```

31. Async Programming

Async programming is useful for many concurrent I/O operations.

PYTHON: 34 TOPICS WITH DETAILED CODE

```
import asyncio
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
async def fetch_data(
    task_name,
    delay
```

):

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    print(
        f"{task_name} started"
    )

    await asyncio.sleep(delay)

    print(
        f"{task_name} completed"
    )

    return {
        "task": task_name,
        "status": "success"
    }
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
async def main():
    results = await asyncio.gather(
        fetch_data("Task 1", 2),
        fetch_data("Task 2", 1),
        fetch_data("Task 3", 3)
    )

    print(results)
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
asyncio.run(main())
```

Create tasks manually

PYTHON: 34 TOPICS WITH DETAILED CODE

```
import asyncio
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
async def worker(name, delay):
    await asyncio.sleep(delay)

    return f"{name} finished"
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
async def main():
    task1 = asyncio.create_task(
        worker("Worker 1", 2)
    )

    task2 = asyncio.create_task(
        worker("Worker 2", 1)
    )

    print(await task1)
    print(await task2)
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
asyncio.run(main())
```

Timeout

PYTHON: 34 TOPICS WITH DETAILED CODE

```
import asyncio
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
async def slow_operation():
    await asyncio.sleep(5)

    return "Completed"
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
async def main():
    try:
        result = await asyncio.wait_for(
            slow_operation(),
            timeout=2
        )

        print(result)

    except asyncio.TimeoutError:
        print("Operation timed out.")
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
asyncio.run(main())
```

Async queue

PYTHON: 34 TOPICS WITH DETAILED CODE

```
import asyncio
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
async def producer(queue):
    for number in range(1, 6):
        await queue.put(number)

        print(
            "Produced:",
            number
        )

    await queue.put(None)
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
async def consumer(queue):
    while True:
        item = await queue.get()

        if item is None:
            break

        print(
            "Consumed:",
            item ** 2
        )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
async def main():
    queue = asyncio.Queue()

    await asyncio.gather(
        producer(queue),
        consumer(queue)
    )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
asyncio.run(main())
```

32. Memory Management

Python manages memory using:

Reference counting

Garbage collection

Private heap

Object interning

Object identity

PYTHON: 34 TOPICS WITH DETAILED CODE

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]

print(id(a))
print(id(b))
print(id(c))

print(a is b)
print(a is c)
```

Reference count

PYTHON: 34 TOPICS WITH DETAILED CODE

```
import sys

data = []

print(
    sys.getrefcount(data)
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
another_reference = data

print(
    sys.getrefcount(data)
```

)

Garbage collection

PYTHON: 34 TOPICS WITH DETAILED CODE

```
import gc
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class Node:
    def __init__(self, value):
        self.value = value
        self.reference = None
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
node1 = Node(1)
node2 = Node(2)

node1.reference = node2
node2.reference = node1
```

del node1

del node2

PYTHON: 34 TOPICS WITH DETAILED CODE

```
collected = gc.collect()

print(
    "Objects collected:",
    collected
```

)

Shallow and deep copy

PYTHON: 34 TOPICS WITH DETAILED CODE

```
import copy

original = [
    [1, 2],
    [3, 4]
```

]

PYTHON: 34 TOPICS WITH DETAILED CODE

```
shallow_copy = copy.copy(
    original
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
deep_copy = copy.deepcopy(
    original
```

)

shallow_copy[0].append(100)

deep_copy[1].append(200)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print("Original:", original)
print("Shallow:", shallow_copy)
print("Deep:", deep_copy)
```

Generator memory advantage

PYTHON: 34 TOPICS WITH DETAILED CODE

```
import sys

list_values = [
    number ** 2
    for number in range(100000)
```

]

PYTHON: 34 TOPICS WITH DETAILED CODE

```
generator_values = (
    number ** 2
    for number in range(100000)
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(
    "List size:",
    sys.getsizeof(list_values)
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(
    "Generator size:",
    sys.getsizeof(generator_values)
```

)

__slots__

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class NormalEmployee:
    def __init__(
        self,
        name,
        salary
    ):
        self.name = name
        self.salary = salary
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class SlottedEmployee:
    __slots__ = (
        "name",
        "salary"
    )

    def __init__(
        self,
        name,
        salary
    ):
        self.name = name
        self.salary = salary
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
normal = NormalEmployee(
    "Aamir",
    50000
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
slotted = SlottedEmployee(
    "Aamir",
    50000
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(
    "Normal object:",
    sys.getsizeof(normal)
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(
    "Slotted object:",
    sys.getsizeof(slotted)
```

)

33. Descriptors and Metaclasses

Descriptor

A descriptor controls attribute access.

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class PositiveNumber:
    def __set_name__(
        self,
        owner,
        name
    ):
        self.private_name = (
            f"_{name}"
        )

    def __get__(
        self,
        instance,
        owner
    ):
        if instance is None:
            return self

        return getattr(
            instance,
            self.private_name
        )

    def __set__(
        self,
        instance,
        value
    ):
        if not isinstance(
            value,
            (int, float)
        ):
            raise TypeError(
                "Value must be numeric"
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
            )

        if value < 0:
            raise ValueError(
                "Value cannot be negative"
            )

        setattr(
            instance,
            self.private_name,
            value
        )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class Product:
    price = PositiveNumber()
    quantity = PositiveNumber()

    def __init__(
        self,
        name,
        price,
        quantity
    ):
        self.name = name
        self.price = price
        self.quantity = quantity

    @property
    def total_value(self):
        return (
            self.price *
            self.quantity
        )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
product = Product(
    "Laptop",
    50000,
    2
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(product.total_value)

product.price = 45000

print(product.total_value)
```

Method descriptor example

Functions inside classes become bound methods.

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class Example:
    def show(self):
        return "Hello"
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
example = Example()

print(Example.show)
print(example.show)
```

Metaclass

A metaclass creates classes.

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class ModelMeta(type):
    def __new__(
        metaclass,
        class_name,
        bases,
        namespace
    ):
        if class_name != "BaseModel":
            if "predict" not in namespace:
                raise TypeError(
                    f"{class_name} must "
                    f"define predict()"
                )

        namespace[
            "model_type"
        ] = "Machine Learning Model"

        return super().__new__(
            metaclass,
            class_name,
            bases,
            namespace
        )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class BaseModel(
    metaclass=ModelMeta
```

):

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    pass
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class LinearModel(BaseModel):
    def predict(self, value):
        return 2 * value + 5
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
model = LinearModel()

print(model.predict(10))
print(model.model_type)
```

Class decorator as simpler alternative

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def add_description(cls):
    cls.description = (
        "Decorated class"
    )

    return cls
```

@add_description

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class Student:
    pass
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
print(Student.description)
```

34. Packaging and Clean Code

Recommended project structure

employee_management/

CONCEPT MAP

```
├── pyproject.toml
├── README.md
├── requirements.txt
├── .gitignore
├── src/
│   └── employee_management/
│       ├── __init__.py
│       ├── models.py
│       ├── services.py
│       ├── repository.py
│       └── main.py
└── tests/
    ├── test_services.py
    └── test_repository.py
```

models.py

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from dataclasses import dataclass
```

@dataclass

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class Employee:
    employee_id: int
    name: str
    department: str
    salary: float

    def __post_init__(self):
        if self.employee_id <= 0:
            raise ValueError(
                "Employee ID must be positive"
            )

        if not self.name.strip():
            raise ValueError(
                "Name cannot be empty"
            )

        if self.salary < 0:
            raise ValueError(
                "Salary cannot be negative"
            )
```

repository.py

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from .models import Employee
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class EmployeeRepository:
    def __init__(self):
        self._employees: dict[
            int,
            Employee
        ] = {}

    def add(
        self,
        employee: Employee
    ) -> None:
        if (
            employee.employee_id
            in self._employees
        ):
            raise ValueError(
                "Employee already exists"
            )

        self._employees[
            employee.employee_id
        ] = employee

    def get(
        self,
        employee_id: int
    ) -> Employee | None:
        return self._employees.get(
            employee_id
        )

    def get_all(
        self
    ) -> list[Employee]:
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
        return list(
            self._employees.values()
        )

    def delete(
        self,
        employee_id: int
    ) -> bool:
        if (
            employee_id
            not in self._employees
        ):
            return False

        del self._employees[
            employee_id
        ]

        return True
```

services.py

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from .models import Employee
from .repository import EmployeeRepository
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class EmployeeService:
    def __init__(
        self,
        repository: EmployeeRepository
    ):
        self.repository = repository

    def register_employee(
        self,
        employee_id: int,
        name: str,
        department: str,
        salary: float
    ) -> Employee:
        employee = Employee(
            employee_id=employee_id,
            name=name,
            department=department,
            salary=salary
        )

        self.repository.add(employee)

        return employee

    def calculate_department_salary(
        self,
        department: str
    ) -> float:
        employees = (
            self.repository.get_all()
        )

        return sum(
```

```
            employee.salary
            for employee in employees
            if employee.department
            == department
        )

    def increase_salary(
        self,
        employee_id: int,
        percentage: float
    ) -> Employee:
        if percentage < 0:
            raise ValueError(
                "Percentage cannot be negative"
            )

        employee = self.repository.get(
            employee_id
        )

        if employee is None:
            raise ValueError(
                "Employee not found"
            )

        increase = (
            employee.salary *
            percentage /
            100
        )

        employee.salary += increase

        return employee
```

main.py

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from .repository import EmployeeRepository
from .services import EmployeeService
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def main():
    repository = EmployeeRepository()
    service = EmployeeService(
        repository
    )

    service.register_employee(
        1,
        "Aamir",
        "AI/ML",
        50000
    )

    service.register_employee(
        2,
        "Rahul",
        "Analytics",
        45000
    )

    service.register_employee(
        3,
        "Priya",
        "AI/ML",
        60000
    )

    total_salary = (
        service.calculate_department_salary(
            "AI/ML"
        )
    )

    print(
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
        "AI/ML Total Salary:",
        total_salary
    )

    updated_employee = (
        service.increase_salary(
            1,
            10
        )
    )

    print(updated_employee)
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
if __name__ == "__main__":
    main()
```

pyproject.toml

[build-system]

PYTHON: 34 TOPICS WITH DETAILED CODE

```
requires = ["setuptools>=68"]
```

build-backend = "setuptools.build_meta"

[project]

PYTHON: 34 TOPICS WITH DETAILED CODE

```
name = "employee-management"
version = "0.1.0"
description = "Employee management example"
```

requires-python = ">=3.11"

[tool.pytest.ini_options]

PYTHON: 34 TOPICS WITH DETAILED CODE

```
pythonpath = ["src"]
testpaths = ["tests"]
```

.gitignore

venv/

.venv/

__pycache__/

- .pyc

.env

.idea/

.vscode/

.pytest_cache/

dist/

build/

- .egg-info/

Clean-code principles

Use meaningful names

Bad:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def c(a, b):
    return a * b
```

Better:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def calculate_total_price(
    unit_price,
    quantity
```

):

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    return unit_price * quantity
```

Single responsibility

Bad:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def process_employee(employee):
    # Validate employee
    # Save employee
    # Send email
    # Generate report
    pass
```

Better:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def validate_employee(employee):
    pass
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def save_employee(employee):
    pass
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def send_welcome_email(employee):
    pass
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def generate_employee_report(employee):
    pass
```

Avoid duplicated code

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def calculate_percentage(
    value,
    percentage
```

):

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    return (
        value *
        percentage /
        100
    )
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
salary_bonus = calculate_percentage(
    50000,
    10
```

)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
discount = calculate_percentage(
    2000,
    15
```

)

Use early returns

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def withdraw(balance, amount):
    if amount <= 0:
        raise ValueError(
            "Amount must be positive"
        )

    if amount > balance:
        raise ValueError(
            "Insufficient balance"
        )

    return balance - amount
```

Document public functions

PYTHON: 34 TOPICS WITH DETAILED CODE

```
def calculate_average(
    values: list[float]
```

) -> float:

PYTHON: 34 TOPICS WITH DETAILED CODE

```
    """
    Calculate the arithmetic mean.

    Args:
        values:
            Non-empty list of numbers.

    Returns:
        Arithmetic mean.

    Raises:
        ValueError:
            If values is empty.
    """

    if not values:
        raise ValueError(
            "Values cannot be empty"
        )

    return sum(values) / len(values)
```

Separate configuration

PYTHON: 34 TOPICS WITH DETAILED CODE

```
from dataclasses import dataclass
```

@dataclass(frozen=True)

PYTHON: 34 TOPICS WITH DETAILED CODE

```
class AppConfig:
    database_name: str
    debug: bool = False
    max_connections: int = 10
```

PYTHON: 34 TOPICS WITH DETAILED CODE

```
config = AppConfig(
    database_name="company.db",
    debug=True
```

)

Final Revision Table

Topic	Main Concept
Installation	Run Python and install packages
Variables	Store object references
Operators	Perform calculations and comparisons
Input/output	Read and display information
Conditions	Make decisions
Loops	Repeat operations
Strings	Work with text
Lists	Mutable ordered collection
Tuples	Immutable ordered collection
Sets	Unique values
Dictionaries	Key-value data
Functions	Reusable logic
Scope	Variable visibility
Lambda	Small anonymous functions
Recursion	Function calls itself
OOP	Classes and objects
Exceptions	Handle runtime errors
Files	Read and write data
Modules	Reusable Python files
Virtual environments	Isolate dependencies
Iterators	Produce one item at a time
Generators	Lazy iteration with yield
Closures	Remember enclosing values
Decorators	Add function behavior
Context managers	Setup and cleanup
Regex	Search and validate patterns
Collections	Specialized containers
Dataclasses	Data-focused classes
Testing	Verify code correctness
Logging	Record application events
Databases	Store structured data
APIs	Communicate between systems
Threading	Concurrent I/O work
Multiprocessing	Parallel CPU work
Async	Concurrent non-blocking I/O

Memory	References and garbage collection
Descriptors	Control attributes
Metaclasses	Control class creation
Packaging	Build reusable projects
Clean code	Maintainable, readable design

PART 8 | Complete Machine Learning Interview Cheat Sheet

Use this as a final revision sheet before AI/ML, Data Science, or Machine Learning interviews.

1. Machine Learning in One Line

Machine Learning: algorithms learn patterns from historical data and use those patterns to predict or make decisions on unseen data.

CONCEPT MAP

```
Data → Preprocessing → Model Training → Evaluation → Prediction → Deployment
```

2. Types of Machine Learning

Type	Target?	Purpose	Examples
Supervised	Yes	Predict output	Regression, Classification
Unsupervised	No	Discover patterns	Clustering, PCA
Semi-Supervised	Partially	Learn from few labels	Image classification
Self-Supervised	Generated labels	Representation learning	LLM pretraining
Reinforcement Learning	Reward	Learn actions	Robotics, games

Supervised Learning

Regression: continuous output.

Examples:

House Price

Salary

Sales

Temperature

Classification: categorical output.

Examples:

Spam / Not Spam

Fraud / Genuine

Churn / Stay

Disease / No Disease

3. Important ML Terms

Term	Meaning
Feature	Input variable
Target	Output variable
Observation	One row
Training	Model learns parameters
Inference	Model predicts new data
Parameter	Learned by model
Hyperparameter	Chosen before training
Loss	Error during training
Epoch	One pass through data
Batch	Small training subset
Baseline	Simple reference model

4. Complete ML Workflow

1. Understand business problem
2. Define target
3. Collect data
4. Perform EDA
5. Clean data
6. Engineer features
7. Split train/test
8. Build preprocessing pipeline
9. Train baseline
10. Train multiple models
11. Cross-validation
12. Hyperparameter tuning
13. Evaluate test performance
14. Error analysis
15. Explain predictions
16. Save pipeline
17. Deploy
18. Monitor drift/performance

19. Retrain

5. Train, Validation and Test Data

Train set

Used to learn model parameters.

Validation set

Used for:

Model selection

Hyperparameter tuning

Threshold selection

Test set

Used once for final unbiased evaluation.

Typical split:

70% Train

15% Validation

15% Test

or:

80% Train

20% Test + Cross Validation

6. Train-Test Split Code

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.model_selection import train_test_split
```

X_train, X_test, y_train, y_test = train_test_split(

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
```

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
stratify=y is useful for classification because it keeps approximately the same class ratio.
```

7. Data Leakage

Data leakage: information unavailable at prediction time enters model training.

Examples:

Scaling full dataset before splitting

Using future sales to predict past sales

Target-derived features

Imputing using test-set statistics

Selecting features using test data

Correct approach:

CONCEPT MAP

```
Split first → Fit preprocessing on training data → Transform validation/test
```

Best solution:

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.pipeline import Pipeline
```

8. Missing Values

Numerical

Common strategies:

Mean

Median

Model-based imputation

Median is safer with outliers.

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median")
```

Categorical

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
SimpleImputer(strategy="most_frequent")
```

or:

"Unknown"

9. Duplicate Data

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
df.duplicated().sum()

df = df.drop_duplicates()
```

Duplicates may artificially increase model confidence.

10. Outliers

Common methods:

IQR

Z-score

Isolation Forest

Domain rules

IQR

I

Q

R

=

Q

3

−

Q

1

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
IQR=Q
```

3

−Q

1

Lower bound:

Q

1

–

1.5

I

Q

R

Q

1

−1.5IQR

Upper bound:

Q

3

+

1.5

I

Q

R

Q

3

+1.5IQR

Do not automatically remove every outlier.

Fraud cases, high-value customers, or rare failures may be the most important observations.

11. Categorical Encoding

One-Hot Encoding

Use for nominal categories:

Delhi

Mumbai

Jaipur

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(
    handle_unknown="ignore"
```

)

Ordinal Encoding

Use when categories have order:

Low < Medium < High

> Example:

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
mapping = {
    "Low": 0,
    "Medium": 1,
    "High": 2
```

}

12. Feature Scaling

Standardization

z

=

x

−

μ

σ

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
z=
```

σ

x−μ

Where:

x

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
x = original value
```

μ

μ = mean

σ

σ = standard deviation

z

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
z = standardized value
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
```

Important for:

KNN

SVM

Logistic Regression

PCA

Neural Networks

Regularized Linear Models

Usually unnecessary for:

Decision Tree

Random Forest

XGBoost

LightGBM

CatBoost

13. Min-Max Scaling

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Usually converts features into:

0 to 1

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.preprocessing import MinMaxScaler
```

14. StandardScaler vs MinMaxScaler

StandardScaler	MinMaxScaler
Mean 0, SD 1	Range typically 0–1
Good general default	Useful for bounded inputs
Outlier sensitive	Very outlier sensitive
Good for SVM/PCA	Often used in neural networks

15. Feature Engineering

Feature engineering means creating useful model inputs.

Examples:

CONCEPT MAP

```
Date → Month, Day, Weekday
Total Spend / Orders → Average Order Value
DOB → Age
Purchase History → Days Since Last Purchase
Sales → Lag 1, Lag 7, Rolling Average
Text → TF-IDF
```

> Example:

df["avg_order_value"] = (

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
    df["total_spent"] /
    df["total_orders"].replace(0, 1)
```

)

16. Feature Selection

Three types:

Filter Methods

Correlation

Chi-square

ANOVA

Mutual Information

Variance Threshold

Wrapper Methods

Recursive Feature Elimination

Forward Selection

Backward Selection

Embedded Methods

Lasso

Decision Tree

Random Forest

Boosting

17. Bias-Variance Tradeoff

E
x
p
e
c
t
e
d
E
r
r
o
r
=
B
i
a
s
2
+
V
a
r
i
a
n
c
e
+
N
o
i
s
e

Expected Error=Bias
2
+Variance+Noise

High Bias

Model too simple

Underfitting

High Variance

Model too complex

Overfitting

Best model:

Low enough bias + controlled variance

18. Underfitting

Symptoms:

Low training score

Low validation/test score

Solutions:

Use stronger model

Add useful features

Reduce regularization

Increase training

Add non-linear terms

19. Overfitting

Symptoms:

Very high training score

Much lower validation/test score

Solutions:

Cross-validation

Regularization

Reduce model complexity

More training data

Feature selection

Tree pruning

Dropout

Early stopping

Data augmentation

20. Linear Regression

Used for continuous prediction.

y

^

=

β

0

+

β

1

x

1

+
β
2
x
2
+
□
+
β
p
x
p
y
^
=β
0
+β
1
x
1
+β
2
x
2
+□+β
p
x
p
Where:
y
^
y
^
= predicted value
β
0
β
0
= intercept
β

j

β

j

= coefficient

x

j

x

j

= feature

> Example:

S

a

l

a

r

y

=

20000

+

8000

(

E

x

p

e

r

i

e

n

c

e

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
Salary=20000+8000(Experience)
```

For 5 years:

S

a

l

a

r

y
=
60000

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
Salary=60000
```

Loss
M
S
E
=
1
n
∑
(
y
i
–
y
^
i
)
2

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
MSE=
```

n
1
∑(y
i
–
y
^
i
)
2
Code

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

predictions = model.predict(X_test)
```

21. Linear Regression Assumptions

Important assumptions:

Linear relationship

Independent observations

Constant residual variance

Residual normality for inference

No severe multicollinearity

Interview point:

Prediction does not require every assumption to be perfectly satisfied, but statistical interpretation of coefficients relies more strongly on them.

22. Ridge Regression

L2 regularization:

L

o

s

s

=

M

S

E

+

λ

∑

j

β

j

2


```
Loss=MSE+λ
```

j

∑

β

j

2

Effect:

Shrinks coefficients

Reduces overfitting

Does not normally make coefficients exactly zero

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.linear_model import Ridge

model = Ridge(alpha=1.0)
```

23. Lasso Regression

L1:

L

o

s

s

=

M

S

E

+

λ

∑

j

□

β

j

□

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
Loss=MSE+λ
```

j

∑

□β

j

□

Effect:

Can make coefficients exactly zero

Performs feature selection

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.linear_model import Lasso
```

24. Elastic Net

Combines L1 and L2.

L

o
s
s
=
M
S
E
+
λ
[
α
L
1
+
(
1
−
α
)
L
2
]


```
Loss=MSE+λ[αL1+(1−α)L2]
```

Useful when:

Many correlated features

Need both shrinking and feature selection

25. Logistic Regression

Used for classification.

Linear component:

z
=
β
0
+
β
1
x
1
+
□

+

β

p

x

p

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
z=β
```

0

+β

1

x

1

+□+β

p

x

p

Sigmoid:

P

(

y

=

1

)

=

1

1

+

e

−

z

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
P(y=1)=
```

1+e

−z

1

Threshold:

CONCEPT MAP

```
p ≥ 0.5 → Class 1
p < 0.5 → Class 0
```

Code:

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(
    max_iter=1000
)
```

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
model.fit(X_train, y_train)
```

26. Logistic Regression Loss

Binary cross entropy:

$$L = -\frac{1}{n}\sum [y_i \log(p_i) + (1-y_i)\log(1-p_i)$$

]

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
L=-
```

n

1

∑[y

i

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
 log(p
```

i

)+(1−y

i

)log(1−p

i

)]

Confident wrong predictions get heavily penalized.

27. Odds and Log-Odds

Odds:

O

d

d

s

=

p

1

−

p

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
Odds=
```

1−p

p

Log-odds:

log

□

(

p

1

−

p

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
log(
```

1−p

p

)

Logistic regression models:

L

o

g

O

d

d

s

=

β

0

+

β

1

x

1

+

□

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
LogOdds=β
```

0

+β

1

x

1

+□

28. K-Nearest Neighbors

KNN predicts based on closest examples.

Euclidean distance:

d

(

x

,
y
)
=
∑
i
(
x
i
–
y
i
)
2

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
d(x,y)=
```

i
∑
(x
i
−y
i
)
2

Classification:

Majority vote among K nearest neighbors

Regression:

Average target of K nearest neighbors

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(
    n_neighbors=5
```

)

> Important:

Scaling required

CONCEPT MAP

```
Small K → noisy/high variance
Large K → smoother/high bias
```

Prediction can be slow

29. Decision Tree

Decision Tree creates hierarchical if-else rules.

Gini

G

i

n

i

=

1

−

∑

p

i

2

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
Gini=1-∑p
```

i

2

Entropy

E

n

t

r

o

p

y

=

−

∑

p

i

log

□

2

p

i

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
Entropy=-∑p
```

i

log

2

p

i

Information Gain

I

G

=

E

n

t

r

o

p

y

(

p

a

r

e

n

t

)

–

W

e

i

g

h

t

e

d

E

n

t

r

o

p

y

(

c

h

i

l

d

r

e

n

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
IG=Entropy(parent)−WeightedEntropy(children)
```

Code:

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    max_depth=5,
    min_samples_leaf=5,
    random_state=42
```

)

Advantages:

Interpretable

Non-linear relationships

No scaling required

Feature interactions

Weakness:

Easily overfits

30. Random Forest

Random Forest combines multiple decision trees.

Classification:

P

r

e

d

i

c

t

i

o

n

=

M

a

j

o
r
i
t
y
V
o
t
e
(
T
r
e
e
s
)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
Prediction=MajorityVote(Trees)
```

Regression:

P
r
e
d
i
c
t
i
o
n
=
A
v
e
r
a
g
e
(
T
r
e

e

s

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
Prediction=Average(Trees)
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=200,
    max_depth=10,
    random_state=42
```

)

Why it reduces overfitting:

Bootstrap sampling

Random feature subsets

Averaging/voting

31. Bagging

Bagging means:

Bootstrap Aggregating

Process:

Create multiple bootstrap datasets

Train models independently

Combine predictions

Main purpose:

Reduce variance

Random Forest is the most common example.

32. Boosting

Boosting trains models sequentially.

CONCEPT MAP

```
Model 1 → errors
```

Model 2 focuses on errors

Model 3 corrects remaining errors

...

Common algorithms:

AdaBoost

Gradient Boosting

XGBoost

LightGBM

CatBoost

Main advantage:

Excellent tabular-data performance

33. AdaBoost

Final prediction:

F
(
x
)
=
∑
m
α
m
h
m
(
x
)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
F(x)=
```

m
∑
α
m
h
m
(x)

Weak learners with better performance get more weight.

Misclassified samples receive greater attention.

34. Gradient Boosting

F
m
(
x
)
=
F
m
−
1
(
x

)

+

η

h

m

(

x

)

F

m

(x)=F

m−1

(x)+ηh

m

(x)

Where:

F

m

F

m

= updated ensemble

h

m

h

m

= new learner

η

η = learning rate

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.ensemble import GradientBoostingClassifier

model = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.05,
    max_depth=3,
    random_state=42
```

)

35. XGBoost

Important parameters:

Parameter	Meaning
n_estimators	Number of trees
learning_rate	Contribution of each tree

Parameter	Meaning
max_depth	Tree complexity
subsample	Fraction of rows
colsample_bytree	Fraction of features
reg_alpha	L1 regularization
reg_lambda	L2 regularization

Strengths:

Regularization

Strong tabular performance

Missing value handling

Efficient implementation

36. XGBoost vs Random Forest

Random Forest	XGBoost
Trees independently trained	Sequential boosting
Mainly reduces variance	Reduces errors iteratively
Less tuning	More tuning
Strong baseline	Often stronger accuracy
Easy parallelization	More complex

37. LightGBM

> Key idea:

Leaf-wise tree growth

Advantages:

Fast

Memory efficient

Good for large datasets

Potential issue:

Can overfit small datasets

38. CatBoost

Best known for:

Handling categorical variables effectively

Advantages:

Less preprocessing

Strong defaults

Good tabular accuracy

39. Support Vector Machine

Decision boundary:

w

T

x

+

b

=

0

w

T

x+b=0

> Goal:

Find the maximum-margin separating hyperplane

Prediction:

s

i

g

n

(

w

T

x

+

b

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
sign(w
```

T

x+b)

Margin:

2

□

w

□

□w□

2

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.svm import SVC

model = SVC(
    kernel="rbf",
    C=1.0,
    gamma="scale"
```

)

40. SVM Hyperparameters

C

Large C:

Stronger penalty for mistakes

Narrower margin

Higher overfitting risk

Small C:

More regularization

Wider margin

Gamma

Large gamma:

Each point has narrow influence

Complex boundary

Small gamma:

Broader influence

Smoother boundary

41. SVM Kernels

Linear

$K(x, z) = x^T z$


```
K(x,z)=x
```

T

z

Polynomial

$K(x, z) =$

(
γ
x
T
z
+
r
)
d

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
K(x,z)=(γx
```

T
z+r)
d
RBF
K
(
x
,
z
)
=
e
−
γ
□
x
−
z
□
2

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
K(x,z)=e
```

−γ□x−z□
2

42. Naive Bayes

Based on Bayes theorem:
P
(
C

□
X
)
=
P
(
X
□
C
)
P
(
C
)
P
(
X
)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
P(C X)=
P(X)
P(X C)P(C)
```

Naive assumption:

Features are conditionally independent given the class.

Types:

Type	Data
Gaussian NB	Continuous
Multinomial NB	Word counts
Bernoulli NB	Binary features

Very useful for:

Spam detection

Document classification

Sentiment classification

43. K-Means

Unsupervised clustering algorithm.

Objective:

J
=
∑
k

=

1

K

∑

x

i

□

C

k

□

x

i

−

μ

k

□

2

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
J=
k=1
```

∑

K

x

i

□C

k

∑

□x

i

−μ

k

□

2

Steps:

Choose K centroids

Assign points

Recalculate centroids

Repeat

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.cluster import KMeans

model = KMeans(
    n_clusters=3,
    n_init=10,
    random_state=42
)
```

44. Elbow Method

Choose K by plotting:

K vs Inertia

Inertia:

Within-cluster sum of squared distances

Look for a point where improvement begins slowing sharply.

45. Silhouette Score

$$s(i) = b(i) - a(i)$$

$$\max \square (a(i), b(i))$$

```
s(i)=
max(a(i),b(i))
b(i)-a(i)
```

Where:

a

(

i

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
a(i) = within-cluster distance
```

b

(

i

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
b(i) = nearest other-cluster distance
```

Interpretation:

CONCEPT MAP

```
Near 1 → strong clustering
Near 0 → overlapping clusters
Negative → possibly wrong cluster
```

46. Hierarchical Clustering

Creates a dendrogram.

Types:

Agglomerative

Divisive

Linkage methods:

Single

Complete

Average

Ward

47. DBSCAN

Density-based clustering.

Important parameters:

eps

min_samples

Advantages:

No K required

Finds irregular clusters

Detects noise

Weakness:

Sensitive to parameter selection

Hard with varying densities

48. PCA

Principal Component Analysis reduces dimensions.

Steps:

Standardize

Compute covariance matrix

Find eigenvalues/eigenvectors

Sort components

Project data

Eigen equation:

A

v

=

λ

v

MACHINE LEARNING INTERVIEW CHEAT SHEET

$$Av=\lambda v$$

Explained variance ratio:

E

V

R

j

=

λ

j

∑

k

λ

k

EVR

j

=

∑

k

λ

k

λ

j

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

49. PCA vs Feature Selection

PCA	Feature Selection
Creates new components	Keeps original features
Lower interpretability	Better interpretability
Components are combinations	Real original columns
Dimensionality reduction	Remove unnecessary features

50. Cross Validation

K-Fold:

Split data into K folds

Train on K−1

Validate on remaining fold

Repeat K times

M

e

a

n

C

V

=

1

K

∑

i

=

1

K

S

c

o

r

e

i

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
MeanCV=
```

K

1

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
i=1
```

∑

K

Score

i

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(
    model,
    X,
    y,
    cv=5,
    scoring="accuracy"
```

)

51. Types of Cross Validation

Method	Best Use
K-Fold	General regression
Stratified K-Fold	Classification
Leave-One-Out	Very small datasets
TimeSeriesSplit	Time series
GroupKFold	Group-dependent data

52. Hyperparameters vs Parameters

Parameters

Learned:

Regression coefficients

Tree split thresholds

Neural-network weights

Hyperparameters

Chosen:

K in KNN

max_depth

learning_rate

C in SVM

n_estimators

53. Grid Search

Tests all specified combinations.

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.model_selection import GridSearchCV

grid = GridSearchCV(
    model,
    param_grid={
        "max_depth": [3, 5, 10],
        "n_estimators": [100, 200]
    },
    cv=5
```

)

Advantage:

Exhaustive

Disadvantage:

Expensive

54. Randomized Search

Samples combinations.


```
from sklearn.model_selection import RandomizedSearchCV
```

Advantage:

Faster for large search spaces

55. Bayesian Optimization

Uses previous experiments to decide which hyperparameters to test next.

Popular tools:

Optuna

Hyperopt

BayesianOptimization

56. Confusion Matrix

Predicted Positive	Predicted Negative	
Actual Positive	TP	FN
Actual Negative	FP	TN

57. Accuracy

A

c

c

u

r

a

c

y

=

T

P
+
T
N
T
P
+
T
N
+
F
P
+
F
N


```
Accuracy=
```

TP+TN+FP+FN
TP+TN

Use when:
Classes reasonably balanced
Error costs similar

58. Precision

P
r
e
c
i
s
i
o
n
=
T
P
T
P
+
F
P

```
Precision=
```

TP+FP

TP

Question:

Of everything predicted positive, how much was correct?

Use when false positives are costly.

> Example:

Spam filtering

59. Recall

R

e

c

a

l

l

=

T

P

T

P

+

F

N

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
Recall=
```

TP+FN

TP

Question:

Of all actual positives, how many did we detect?

Use when false negatives are costly.

Examples:

Disease detection

Fraud detection

60. Specificity

S

p

e

c

i
f
i
c
i
t
y
=
T
N
T
N
+
F
P


```
Specificity=
```

TN+FP
TN

Measures true negative detection.

61. F1 Score

F
1
=
2
P
r
e
c
i
s
i
o
n
×
R
e
c
a
l

l

P

r

e

c

i

s

i

o

n

+

R

e

c

a

l

l


```
F1=2
```

Precision+Recall

Precision×Recall

Good when:

Classes are imbalanced

Precision and recall both matter

62. ROC-AUC

ROC plots:

T

P

R

TPR

against:

F

P

R

=

F

P

F

P

+

T

N

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
FPR=
```

FP+TN

FP

AUC measures ranking performance.

CONCEPT MAP

```
1.0 → excellent
0.5 → random-like
```

63. Precision-Recall AUC

Often more informative than ROC-AUC when:

Positive class is very rare

> Example:

Fraud detection

64. Regression Metrics

MAE

M

A

E

=

1

n

∑

□

y

i

–

y

^

i

□

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
MAE=
```

n

1

∑□y

i

–
y
^
i
□
Easy to interpret.
MSE
M
S
E
=
1
n
∑
(
y
i
–
y
^
i
)
2

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
MSE=
```

n
1
∑(y
i
–
y
^
i
)
2
Strongly penalizes large errors.
RMSE
R
M
S

E
=
M
S
E

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
RMSE=
```

MSE

Same unit as target.

R²

R
2
=
1
−
∑
(
y
i
−
y
^
i
)
2
∑
(
y
i
−
y
-
)
2
R
2
=1−
∑(y
i
−
y

-

)
2
∑(y
i
–
y
^
i
)
2

65. MAE vs RMSE

MAE	RMSE
Linear penalty	Squared penalty
Robust to large errors	Punishes large errors
Easy interpretation	Same target unit
Median-optimal	Mean-sensitive

66. Adjusted R^2

A
d
j
u
s
t
e
d
R
2
=
1
–
(
1
–
R
2
)
n
–

1

n

−

p

−

1

AdjustedR

2

=1−(1−R

2

)

n−p−1

n−1

Where:

n

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
n = observations
```

p

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
p = predictor count
```

Useful when comparing models with different numbers of features.

67. Imbalanced Data

> Example:

99.5% Genuine

0.5% Fraud

Accuracy can be misleading.

Solutions:

Class weights

Random oversampling

Random undersampling

SMOTE

ADASYN

Threshold tuning

Better metrics

Anomaly detection

68. SMOTE

Creates synthetic minority samples.

Correct usage:

Only on training folds/data

Never before train-test split

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=42)
```

X_resampled, y_resampled = (

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
    smote.fit_resample(
        X_train,
        y_train
    )
```

)

69. Threshold Tuning

Default threshold:

0.5

Lower threshold:

CONCEPT MAP

```
Recall ↑
False positives ↑
```

Higher threshold:

CONCEPT MAP

```
Precision ↑
False negatives ↑
```

> Example:

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
probabilities = model.predict_proba(X_test)[:, 1]

predictions = (
    probabilities >= 0.30
```

).astype(int)

70. Multicollinearity

Occurs when predictors are strongly correlated.

Effects:

Unstable coefficients

Hard coefficient interpretation

Large standard errors

VIF:

V

I

F

j

=
1
1
−
R
j
2
VIF
j
=
1−R
j
2
1

High VIF may indicate multicollinearity.

71. Covariance

C
o
v
(
X
,
Y
)
=
∑
(
x
i
−
x
-
)
(
y
i
−
y
-
)
n

–

1

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
Cov(X,Y)=
```

n−1

∑(x

i

–

x

-

)(y

i

–

y

-

)

Positive:

Move together

Negative:

Move oppositely

72. Pearson Correlation

r

=

C

o

v

(

X

,

Y

)

σ

X

σ

Y

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
r=
```

σ

X

σ

Y

MACHINE LEARNING INTERVIEW CHEAT SHEET
`Cov(X,Y)`

Range:

- 1 to +1

Measures linear relationship.

73. Pearson vs Spearman

Pearson	Spearman
Linear relationship	Monotonic relationship
Raw values	Ranks
Sensitive to outliers	More robust
Continuous variables	Ordinal/non-normal data

74. Mean, Median and Mode

Mean

x

-

=

∑

x

i

n

x

-

=

n

∑x

i

Sensitive to outliers.

Median

Middle ordered value.

Robust to outliers.

Mode

Most frequent value.

Useful for categorical variables.

75. Variance

s

2

=
∑
(
x
i
−
x
-
)
2
n
−
1
s
2
=
n−1
∑(x
i
−
x
-
)
2

Measures spread.

76. Standard Deviation

s
=
s
2

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
s=
```

s
2

Same unit as data.

77. Z-Score

z
=
x

–

μ

σ

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
z=
```

σ

x−μ

Interpretation:

CONCEPT MAP

```
z = 2 → 2 SD above mean
z = -1 → 1 SD below mean
```

78. Normal Distribution

Properties:

Bell shaped

Symmetric

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
Mean = Median = Mode
```

Empirical rule:

68% within 1 SD

95% within 2 SD

99.7% within 3 SD

79. Central Limit Theorem

As sample size becomes sufficiently large, the distribution of sample means approaches normality under broad conditions.

Standard error:

S

E

=

σ

n

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
SE=
```

n

σ

Important for:

Confidence intervals

Hypothesis tests

A/B testing

80. Hypothesis Testing

Null hypothesis:

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
H0 = No effect/difference
```

Alternative:

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
H1 = Effect/difference exists
```

Decision:

CONCEPT MAP

```
p < α → Reject H0
p ≥ α → Fail to reject H0
```

Common:

α

=

0.05

α=0.05

81. Type I and Type II Errors

Type I

Reject true

H

0

H

0

.

False positive

Probability:

α

α

Type II

Fail to reject false

H

0

H

0

.

False negative

Probability:

β

β

Statistical power:

1

−

β

1−β

82. Common Statistical Tests

Test	Use
One-sample t-test	Sample mean vs known value
Independent t-test	Two independent means
Paired t-test	Before vs after
ANOVA	3+ group means
Chi-square	Categorical association
Mann-Whitney	Non-parametric two-group
Wilcoxon	Non-parametric paired
Kruskal-Wallis	Non-parametric 3+ group

83. A/B Testing

> Example:

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
A = Old checkout
B = New checkout
```

Steps:

Define hypothesis

Choose primary metric

Randomize users

Calculate sample size

Run experiment

Check data quality

Calculate effect

Test significance

Check confidence interval

Evaluate business importance

Decide

Do not use p-value alone.

Also evaluate:

Effect size

Confidence interval

Business impact

Guardrail metrics

84. Time Series

Components:

Trend

Seasonality

Cycles

Noise

Important rule:

Never randomly shuffle time order blindly.

Train:

Past

Test:

Future

85. Lag Features

L

a

g

1

(

t

)

=

y

t

−

1

Lag

1

(t)=y

t−1

```
df["lag_1"] = df["sales"].shift(1)
df["lag_7"] = df["sales"].shift(7)
```

86. Moving Average

M

A

t

=

1

k

∑

i

=

0

k

−

1

y

t

−

i

MA

t

=

k

1

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
i=0
```

∑

k−1

y

t−i

df["ma_7"] = (

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
    df["sales"]
    .shift(1)
    .rolling(7)
    .mean()
```

)

Use shift(1) to avoid target leakage when predicting the current/future value.

87. ARIMA

A

R

I

M

A

(

p

,

d

,

q

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
ARIMA(p,d,q)
```

Where:

p

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
p = autoregressive lags
```

d

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
d = differencing
```

q

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
q = moving-average error terms
```

Use for suitable univariate time series.

88. Deep Learning

Deep learning uses multi-layer neural networks.

Neuron:

z

=

∑

i

w

i

x

i

+

b

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
z=
```

i

∑

w

i

x

i

+b

Activation:

a

=

f

(

z

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
a=f(z)
```

89. Activation Functions

ReLU

R

e

L

U

(

z

)

=

max

□

(

0

,

z

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
ReLU(z)=max(0,z)
```

Common hidden-layer activation.

Sigmoid

σ

(

z

)

=

1

1

+

e

−

z

σ(z)=

1+e

−z

1

Binary output.

Softmax

P

(

y

=

j

)

=

e

z

j

∑

k

e

z

k

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
P(y=j)=
```

∑

k

e

z

k

e

z

j

Multiclass output.

90. Forward Propagation

z

(

l

)

=

W

(

l

)
a
(
l
−
1
)
+
b
(
l
)
z
(l)
=W
(l)
a
(l−1)
+b
(l)
a
(
l
)
=
f
(
z
(
l
)
)
a
(l)
=f(z
(l)
)
Data moves from input to output.

91. Backpropagation

Calculates gradients of loss using the chain rule.

Weight update:

W

n

e

w

=

W

o

l

d

−

η

∂

L

∂

W

W

new

=W

old

−η

∂W

∂L

Where:

L

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
L = loss
```

η

η = learning rate

92. Gradient Descent

θ

t

+

1

=

θ

t

−

η

□

J

(
θ
t
)
θ
t+1
=θ
t
−η□J(θ
t
)

Types:

Batch GD

Stochastic GD

Mini-Batch GD

93. Learning Rate

Too high:

Overshoot/diverge

Too low:

Very slow convergence

Common optimizers:

SGD

Momentum

RMSprop

Adam

AdamW

94. CNN

Convolutional Neural Networks are mainly used for images.

Main layers:

Convolution

Activation

Pooling

Flatten/Global Pooling

Dense

Applications:

Image classification

Object detection

Medical imaging

Face analysis

95. RNN

Used for sequential data.

h

t

=

tanh

□

(

W

x

x

t

+

W

h

h

t

−

1

+

b

)

h

t

=tanh(W

x

x

t

+W

h

h

t−1

+b)

Problem:

Vanishing/exploding gradients

96. LSTM

LSTM adds gates:

Forget gate

Input gate

Cell state

Output gate

Designed to preserve longer dependencies.

97. Transformers

Core idea:

Self-Attention

Scaled dot-product attention:

A
t
t
e
n
t
i
o
n
(
Q
,
K
,
V
)
=
s
o
f
t
m
a
x
(
Q
K
T
d
k
)
V


```
Attention(Q,K,V)=softmax(
```

d

k

QK

T

)V

Where:

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
Q = Query
K = Key
V = Value
```

Used in:

BERT

GPT-style models

LLMs

Vision Transformers

98. NLP

Typical pipeline:

Text cleaning

Tokenization

Vectorization

Model

Evaluation

Traditional representations:

Bag of Words

TF-IDF

Modern:

Embeddings

Transformers

LLMs

99. TF-IDF

T

F

I

D

F

(

t

,

d

)

=

T
F
(
t
,
d
)
×
I
D
F
(
t
)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
TFIDF(t,d)=TF(t,d)×IDF(t)
```

I
D
F
(
t
)
=
log
□
(
N
D
F
(
t
)
)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
IDF(t)=log(
DF(t)
```

N
)

Important for:

Spam classification

Document search

Text similarity

100. Embeddings

Embeddings convert objects into dense vectors.

Examples:

Words

Sentences

Images

Products

Users

Similar objects have similar vectors.

Cosine similarity:

S

i

m

i

l

a

r

i

t

y

=

A

□

B

□

A

□

□

B

□

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
Similarity=
```

□A□□B□

A□B

101. Recommendation Systems

Types:

Popularity Based

Content Based

Collaborative Filtering

Hybrid

Metrics:

Precision@K

Recall@K

MAP@K

NDCG

Hit Rate

102. RAG

Retrieval-Augmented Generation:

Documents

CONCEPT MAP

↓

Chunks

CONCEPT MAP

↓

Embeddings

CONCEPT MAP

↓

Vector Database

CONCEPT MAP

↓

Retrieve relevant chunks

CONCEPT MAP

↓

LLM

CONCEPT MAP

↓

Grounded answer

Important concepts:

Chunking

Embedding

Vector search

Top-K retrieval

Reranking

Context generation

Grounding

Evaluation

103. Explainable AI

Used to understand model behavior.

Methods:

Coefficients

Feature Importance

Permutation Importance

PDP

SHAP

LIME

104. SHAP

SHAP estimates contribution of each feature.

Interpretation:

CONCEPT MAP

```
Positive SHAP → pushes prediction upward
Negative SHAP → pushes downward
```

Useful for:

Global explanation

Local explanation

Feature contribution

105. LIME

LIME explains one prediction using a simple local surrogate model.

Difference:

CONCEPT MAP

```
SHAP → stronger theoretical foundation
LIME → local approximation
```

106. Feature Importance

Tree model:

model.feature_importances_

But impurity-based importance may be biased toward high-cardinality features.

Permutation importance is often a stronger model-agnostic check.

107. Error Analysis

After metrics, inspect incorrect predictions.

Ask:

Which segments fail?

Which classes fail?

Are labels wrong?

Are important features missing?

Is threshold wrong?

Is data distribution different?

Error types:

False Positive

False Negative

Large regression residual

Low-confidence prediction

108. Model Selection

Do not select a model only because it has the highest score.

Consider:

Validation performance

Business objective

Interpretability

Latency

Memory

Training time

Prediction cost

Fairness

Robustness

Deployment complexity

109. Baseline Model

Always create a baseline.

Classification examples:

Majority class

Logistic Regression

Regression:

Predict mean

Linear Regression

Your complex model should beat the baseline meaningfully.

110. ML Pipeline Code Template

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import (
    StandardScaler,
    OneHotEncoder
```

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.linear_model import LogisticRegression

numeric_features = [
    "age",
    "income"
```

]

```
categorical_features = [
    "city",
    "contract"
```

]


```
numeric_pipeline = Pipeline([
    (
        "imputer",
        SimpleImputer(
            strategy="median"
        )
    ),
    (
        "scaler",
        StandardScaler()
    )
```

])


```
categorical_pipeline = Pipeline([
    (
        "imputer",
        SimpleImputer(
            strategy="most_frequent"
        )
    ),
    (
        "encoder",
        OneHotEncoder(
            handle_unknown="ignore"
        )
    )
```

])


```
preprocessor = ColumnTransformer([
    (
        "numeric",
        numeric_pipeline,
        numeric_features
    ),
    (
        "categorical",
        categorical_pipeline,
        categorical_features
    )
```

])

```
model = Pipeline([
    (
        "preprocessor",
        preprocessor
    ),
    (
        "classifier",
        LogisticRegression(
            max_iter=1000
        )
    )
```

])

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
model.fit(
    X_train,
    y_train
```

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
predictions = model.predict(
    X_test
```

)

111. Complete Classification Evaluation

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_auc_score,
    average_precision_score
```

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
predictions = model.predict(
    X_test
```

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
probabilities = model.predict_proba(
    X_test
```

)[:, 1]

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
print(
    "Accuracy:",
    accuracy_score(
        y_test,
        predictions
    )
```

)

```
print(
    "Precision:",
    precision_score(
        y_test,
        predictions,
        zero_division=0
    )
```

)


```
print(
    "Recall:",
    recall_score(
        y_test,
        predictions,
        zero_division=0
    )
```

)


```
print(
    "F1:",
    f1_score(
        y_test,
        predictions,
        zero_division=0
    )
```

)


```
print(
    "ROC-AUC:",
    roc_auc_score(
        y_test,
        probabilities
    )
```

)


```
print(
    "PR-AUC:",
    average_precision_score(
        y_test,
        probabilities
    )
```

)


```
print(
    confusion_matrix(
        y_test,
        predictions
    )
```

)

```
print(
    classification_report(
        y_test,
        predictions,
        zero_division=0
    )
```

)

112. Regression Evaluation Code

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
import numpy as np

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
```

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
predictions = model.predict(
    X_test
```

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
mae = mean_absolute_error(
    y_test,
    predictions
```

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
rmse = np.sqrt(
    mean_squared_error(
        y_test,
        predictions
    )
```

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
r2 = r2_score(
    y_test,
    predictions
```

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
print("MAE:", mae)
print("RMSE:", rmse)
print("R2:", r2)
```

113. Save and Load Model

Save complete pipeline:

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
import joblib

joblib.dump(
    model,
    "model.joblib"
```

)

Load:

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
model = joblib.load(
    "model.joblib"
```

)

Saving the whole pipeline prevents preprocessing mismatch.

114. Deployment Workflow

Train model

CONCEPT MAP

↓

Save model

CONCEPT MAP

↓

Create API

CONCEPT MAP

↓

Validate request

CONCEPT MAP

↓

Load pipeline

CONCEPT MAP

↓

Predict

CONCEPT MAP

↓

Return result

CONCEPT MAP

↓

Monitor

Common tools:

FastAPI

Flask

Streamlit

Docker

AWS

Azure

GCP

115. FastAPI Example

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from fastapi import FastAPI
from pydantic import BaseModel
import pandas as pd
import joblib

app = FastAPI()

model = joblib.load(
    "model.joblib"
```

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
class InputData(BaseModel):
    age: float
    income: float
    city: str
    contract: str
```

@app.post("/predict")

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
def predict(data: InputData):
    input_df = pd.DataFrame([
        data.model_dump()
    ])

    prediction = int(
        model.predict(
            input_df
        )[0]
    )

    probability = float(
        model.predict_proba(
            input_df
        )[0][1]
    )

    return {
        "prediction": prediction,
        "probability": probability
    }
```

116. Model Monitoring

Monitor:

Model accuracy

Precision/Recall

Business KPI

Prediction distribution

Input distribution

Missing values

API latency

Failure rate

Data drift

Concept drift

117. Data Drift vs Concept Drift

Data Drift

Input distribution changes.

$$P_{train}(X) \neq P_{production}(X)$$

Concept Drift

Relationship between features and target changes.

$$P_{train}(Y \mid X) \neq P_{production}(Y \mid X)$$

118. MLOps Basics

Important tools/concepts:

Git

GitHub

Docker

MLflow

DVC

CI/CD

Cloud

Monitoring

Model Registry

Feature Store

Kubernetes

119. MLflow

Used for:

Experiment tracking

Parameters

Metrics

Artifacts

Models

Model registry

120. Docker

Purpose:

Package application + dependencies into reproducible container

Benefits:

Same environment locally and production

Easy deployment

Dependency isolation

121. Fairness in ML

Check whether model performance differs unfairly across groups.

Monitor:

Selection rate

False positive rate

False negative rate

Recall by group

Precision by group

Calibration

High-risk decisions need human oversight.

122. Privacy and Ethics

Important concepts:

Data minimization

Consent

Secure storage

Access control

Bias testing

Transparency

Human review

Explainability

Monitoring

123. Algorithm Selection Cheat Sheet

Problem	Good Starting Model
Continuous prediction	Linear Regression
Binary classification	Logistic Regression
Strong tabular baseline	Random Forest
High-performance tabular	Gradient Boosting / XGBoost
Small distance-based data	KNN
High-dimensional classification	SVM
Text classification	Logistic / Naive Bayes
Customer segmentation	K-Means
Irregular clusters	DBSCAN
Dimensionality reduction	PCA
Image classification	CNN / Transfer Learning
Sequence/time series	ARIMA / GBM / LSTM
Document QA	RAG
Recommendations	Collaborative/Hybrid

124. Algorithm Scaling Cheat Sheet

Algorithm	Scaling?
Linear Regression	Often useful
Logistic Regression	Yes
KNN	Yes
SVM	Yes
PCA	Yes
Neural Network	Yes
Naive Bayes	Depends
Decision Tree	No
Random Forest	No
Gradient Boosting	Usually no
XGBoost	Usually no
K-Means	Yes
DBSCAN	Usually yes

125. Algorithm Interpretability

Algorithm	Interpretability
Linear Regression	High
Logistic Regression	High
Decision Tree	High
KNN	Medium-low
Random Forest	Medium
Boosting	Medium-low
SVM	Low-medium
Neural Network	Low
K-Means	Medium
PCA	Low

126. Metric Selection Cheat Sheet

Problem	Preferred Metrics
Balanced classification	Accuracy, F1
Imbalanced classification	Precision, Recall, F1, PR-AUC
Fraud detection	Recall, Precision, PR-AUC
Medical screening	Recall/Sensitivity
Spam filtering	Precision + Recall
Churn	Recall, F1, ROC-AUC
Regression	MAE, RMSE, R^2
Forecasting	MAE, RMSE, MAPE
Clustering	Silhouette, inertia
Recommendations	Precision@K, Recall@K, NDCG

127. Common Interview Traps

Accuracy is always best?

No.

Imbalanced data can make accuracy misleading.

Higher R^2 always means better model?

No.

Check:

Test performance

Residuals

Overfitting

Adjusted R^2

Business usefulness

Correlation means causation?

No.

Trees need scaling?

Generally no.

PCA is feature selection?

No. PCA creates new components.

SMOTE before split?

No. This causes leakage.

Test set used for hyperparameter tuning?

No.

Can you remove all outliers?

No.

Random split for time series?

Usually no.

128. Top Interview Questions: One-Line Answers

What is ML?

Learning patterns from data to make predictions on unseen data.

What is supervised learning?

Learning from labeled input-target pairs.

What is unsupervised learning?

Finding patterns without a target variable.

Overfitting?

Model memorizes training data and generalizes poorly.

Underfitting?

Model is too simple to learn the underlying pattern.

Bias?

Error from overly simple assumptions.

Variance?

Sensitivity to training-data changes.

Regularization?

Penalty that controls model complexity.

Cross-validation?

Repeated train-validation evaluation across multiple folds.

Pipeline?

A chain of preprocessing and modeling steps executed consistently.

Data leakage?

Training uses information unavailable at real inference time.

Feature engineering?

Creating useful predictors from raw data.

Feature selection?

Choosing the most useful original features.

PCA?

Creating lower-dimensional components that retain maximum variance.

Bagging?

Independent models trained on resampled data and combined.

Boosting?

Sequential models that correct earlier errors.

Random Forest?

Bagged decision trees with random feature selection.

XGBoost?

Regularized gradient-boosted decision trees.

Precision?

Correct positives among predicted positives.

Recall?

Detected positives among actual positives.

F1?

Harmonic mean of precision and recall.

ROC-AUC?

Ability to rank positives above negatives across thresholds.

Gradient descent?

Iterative parameter updates in the negative gradient direction.

Learning rate?

Step size used during parameter updates.

129. How to Explain Any Algorithm

Use this six-part structure:

1. Definition
2. Working process
3. Formula / main mathematics
4. Important parameters
5. Advantages and disadvantages

6. Best use case

Example for Random Forest:

Random Forest is an ensemble algorithm made of many decision trees. Each tree is trained using a bootstrap sample and random subset of features. For classification, trees vote; for regression, predictions are averaged. Important parameters include number of trees, maximum depth, minimum leaf size, and number of features considered at each split. It reduces variance compared with a single tree but is less interpretable.

130. How to Explain Any ML Project

Use:

Problem

CONCEPT MAP

↓

Business value

CONCEPT MAP

↓

Dataset

CONCEPT MAP

↓

Features + Target

CONCEPT MAP

↓

EDA

CONCEPT MAP

↓

Cleaning

CONCEPT MAP

↓

Feature Engineering

CONCEPT MAP

↓

Models Compared

CONCEPT MAP

↓

Metric Selection

CONCEPT MAP

↓

Cross Validation

CONCEPT MAP

↓

Tuning

CONCEPT MAP

↓

Final Result

CONCEPT MAP

↓

Error Analysis

CONCEPT MAP

↓

Deployment

CONCEPT MAP

↓

Limitations

CONCEPT MAP

↓

Future Improvement

131. Best Project Interview Answer Pattern

The goal of my project was to solve [business problem]. I used [dataset], where the target was [target] and important features included [features]. I handled missing values, duplicates, categorical encoding and feature engineering through a preprocessing pipeline. I compared [models] using [cross-validation]. Because the problem was [balanced/imbalanced/regression], I evaluated the models using [metrics] instead of relying only on accuracy. The final model was [model] because it provided the best balance of validation performance, interpretability and business requirements. I saved the complete pipeline and deployed it using FastAPI/Streamlit.

132. Final 30-Minute Revision Order

Revise in this exact order:

Supervised vs unsupervised

Regression vs classification

Train/validation/test

Data leakage

Missing values + encoding + scaling

Bias vs variance

Overfitting vs underfitting

Linear Regression

Logistic Regression

KNN

Decision Tree

Random Forest

SVM

Naive Bayes

K-Means

PCA

Bagging vs boosting

XGBoost

Cross-validation

Hyperparameter tuning

Confusion matrix

Precision vs recall

F1 and ROC-AUC

Regression metrics

Imbalanced data

Feature engineering

Deep-learning basics

Deployment

Project explanation

Error analysis and limitations

Final One-Minute ML Interview Answer

I understand the complete machine-learning lifecycle from business problem definition, data collection, EDA and preprocessing to feature engineering, model training, evaluation, tuning, deployment and monitoring. I am comfortable with regression, classification, clustering and dimensionality-reduction algorithms including Linear Regression, Logistic Regression, KNN, Decision Tree, Random Forest, SVM, Naive Bayes, K-Means, PCA and boosting algorithms such as Gradient Boosting and XGBoost. I use pipelines and cross-validation to prevent leakage, select metrics according to business cost, handle class imbalance with techniques such as class weighting or threshold tuning, and perform error analysis before deployment. I also understand deep-learning basics, NLP, RAG, explainable AI, model monitoring and deployment using FastAPI, Streamlit and Docker.

Final 30-Minute Revision Order

Revise in this exact order:

Supervised vs unsupervised

Regression vs classification

Train/validation/test

Data leakage

Missing values + encoding + scaling

Bias vs variance

Overfitting vs underfitting

Linear Regression

Logistic Regression

KNN

Decision Tree

Random Forest

SVM

Naive Bayes

K-Means

PCA

Bagging vs boosting

XGBoost

Cross-validation

Hyperparameter tuning

Confusion matrix

Precision vs recall

F1 and ROC-AUC

Regression metrics

Imbalanced data

Feature engineering

Deep-learning basics

Deployment

Project explanation

Error analysis and limitations

30-Minute Final ML Interview Revision

Exact Order + Definition + Key Formula + Interview Answer

Use this immediately before an ML interview.

1. Supervised vs Unsupervised Learning

Supervised Learning

Data contains features + target.

CONCEPT MAP

```
Input X → Model → Target y
```

Examples:

House Price Prediction

Spam Detection

Customer Churn

Medical Diagnosis

Algorithms:

Linear Regression

Logistic Regression

KNN

Decision Tree

Random Forest

SVM

Naive Bayes

XGBoost

Unsupervised Learning

There is no target variable.

The model discovers hidden patterns.

Examples:

Customer Segmentation

Clustering products

Dimensionality reduction

Algorithms:

K-Means

Hierarchical Clustering

DBSCAN

PCA

> Interview Answer

Supervised learning uses labeled data containing input features and a target variable. Unsupervised learning does not have a target and is mainly used to discover patterns, clusters, or lower-dimensional representations.

2. Regression vs Classification

Regression

Predicts continuous values.

Examples:

```
MACHINE LEARNING INTERVIEW CHEAT SHEET
```

```
Salary = ₹50,000
```

House Price = ₹80 lakh

```
MACHINE LEARNING INTERVIEW CHEAT SHEET
```

```
Sales = 25,000 units
```

Algorithms:

Linear Regression

Decision Tree Regressor

Random Forest Regressor

SVR

XGBoost Regressor

Metrics:

MAE

MSE

RMSE

R^2

Classification

Predicts classes.

Examples:

Fraud / Genuine

Spam / Not Spam

Churn / Stay

Algorithms:

Logistic Regression

KNN

Decision Tree

Random Forest

SVM

Naive Bayes

XGBoost

Metrics:

Accuracy

Precision

Recall

F1

ROC-AUC

3. Train / Validation / Test

Training Set

Used to learn model parameters.

Validation Set

Used for:

Model selection

Hyperparameter tuning

Threshold selection

Test Set

Used only for final evaluation.

Typical split:

70% Train

15% Validation

15% Test

or:

80% Train

20% Test

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
with cross-validation on training data.
```

Code

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.model_selection import train_test_split
```

X_train, X_test, y_train, y_test = train_test_split(

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
```

)

Interview Rule

Never tune your model using the final test set.

4. Data Leakage

Data leakage means the model receives information during training that would not be available during real prediction.

Examples:

Scaling full data before split

Using future information

Using target-derived features

Using test data during feature selection

SMOTE before train-test split

Wrong:

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
X_scaled = scaler.fit_transform(X)
```

X_train, X_test = train_test_split(X_scaled)

Correct:

X_train, X_test = train_test_split(X)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
scaler.fit(X_train)

X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
```

Best:

Use a Pipeline.

5. Missing Values + Encoding + Scaling

Missing Numerical Values

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median")
```

Common strategies:

Mean

Median

Model-based imputation

Median is safer with outliers.

Missing Categorical Values

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
SimpleImputer(
    strategy="most_frequent"
```

)

One-Hot Encoding

For categories with no order.

Delhi

Mumbai

Jaipur

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(
    handle_unknown="ignore"
```

)

Ordinal Encoding

For ordered categories.

Low < Medium < High

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
mapping = {
    "Low": 0,
    "Medium": 1,
    "High": 2
```

}

Standardization

z

=

x

–

μ

σ

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
z=
```

σ

x−μ

Where:

x

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
x = original value
```

μ

μ = mean

σ

σ = standard deviation

Used for:

KNN

SVM

PCA

Logistic Regression

Regularized Regression

Neural Networks

6. Bias vs Variance

Bias

Error caused by an overly simple model.

CONCEPT MAP

```
High Bias → Underfitting
```

Variance

Error caused by excessive sensitivity to training data.

CONCEPT MAP

```
High Variance → Overfitting
```

Conceptual formula:

E

r

r

o

r

=

B

i

a

s

2

+

V

a

r

i

a

n

c

e

+

I

r

r

e

d

u

c

i

b

l

e

N

o

i

s

e

MACHINE LEARNING INTERVIEW CHEAT SHEET
`Error=Bias`

2

+Variance+Irreducible Noise

> Interview Answer

A good model balances bias and variance so that it learns useful patterns without memorizing the training data.

7. Overfitting vs Underfitting

Underfitting	Overfitting	
Training Score	Low	Very high
Test Score	Low	Much lower
Main Problem	High bias	High variance
Model	Too simple	Too complex

Reduce Underfitting

More complex model

Better features

Train longer

Reduce regularization

Reduce Overfitting

Cross-validation

Regularization

More data

Reduce model complexity

Pruning

Dropout

Early stopping

8. Linear Regression

Used for continuous prediction.

> Formula

y

^

=

β

0

+

β

1

x

1

+

β

2

x

2

+

□

+

β

p

x

p

y

^

=β

0

+β

1

x

1

+β

2

x

2

+□+β

p

x

p

Parameters:

y

^

y

^

= prediction

β

0

β

0

= intercept

β
i
β
i
= coefficient
x
i
x
i
= feature

> Example:

S
a
l
a
r
y
=
20
,
000
+
8
,
000
(
E
x
p
e
r
i
e
n
c
e
)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
Salary=20,000+8,000(Experience)
```

For 5 years:

S
a
l
a
r
y
=
20
,
000
+
8
,
000
(
5
)
=
60
,
000


```
Salary=20,000+8,000(5)=60,000
```

Loss
M
S
E
=
1
n
∑
(
y
i
−
y
^
i
)
2

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
MSE=
```

n

1

∑(y

i

−

y

^

i

)

2

Code

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()

model.fit(
    X_train,
    y_train
```

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
prediction = model.predict(
    X_test
```

)

Assumptions

Linear relationship

Independent observations

Constant residual variance

No severe multicollinearity

Residual normality for statistical inference

9. Logistic Regression

Used mainly for binary classification.

Linear Score

z

=

β

0

+

β

1

x

1

+

□

+

β

p

x

p

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
z=β
```

0

+β

1

x

1

+□+β

p

x

p

Sigmoid

p

=

1

1

+

e

−

z

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
p=
```

1+e

−z

1

Decision:

CONCEPT MAP

```
p >= 0.5 → Class 1
p < 0.5 → Class 0
```

Loss

L

=

−

1

n

∑

[

y

log

□

(

p

)

+

(

1

−

y

)

log

□

(

1

−

p

)

]

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
L=-
```

n

1

∑[ylog(p)+(1−y)log(1−p)]

Code

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(
    max_iter=1000
```

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
model.fit(
    X_train,
    y_train
```

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
predictions = model.predict(
    X_test
```

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
probabilities = model.predict_proba(
    X_test
```

)[:, 1]

10. KNN

KNN predicts using nearby observations.

Euclidean Distance

$$d(x, y) = \sum_i (x_i - y_i)^2$$

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
d(x,y)=
```

$$i \sum (x_i - y_i)^2$$

Classification:

Majority vote among K neighbors

Regression:

Average target among K neighbors

Code

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(
    n_neighbors=5
```

)

> Important:

Scaling required

CONCEPT MAP

```
Small K → High variance
Large K → High bias
```

Prediction can be slow

11. Decision Tree

Creates if-else rules.

Gini Impurity

$$Gini = 1 - \sum p_i^2$$

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
Gini=1−∑p
```

i^2

Entropy

$$Entropy$$

o
p
y
=
−
∑
p
i
log
□
2
(
p
i
)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
Entropy=-∑p
```

i
log
2
(p
i
)
Information Gain
I
G
=
I
m
p
u
r
i
t
y
(
p
a
r
e
n

t
)
–
W
e
i
g
h
t
e
d
I
m
p
u
r
i
t
y
(
c
h
i
l
d
r
e
n
)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
IG=Impurity(parent)-WeightedImpurity(children)
```

Code

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    max_depth=5,
    min_samples_leaf=5,
    random_state=42
```

)

Advantages:

Interpretable

Non-linear

No scaling required

Captures feature interactions

Problem:

Can overfit easily

12. Random Forest

Random Forest combines many decision trees.

Classification:

P

r

e

d

i

c

t

i

o

n

=

M

a

j

o

r

i

t

y

V

o

t

e


```
Prediction=MajorityVote
```

Regression:

P

r

e

d

i

c

t

i

o
n
=
A
v
e
r
a
g
e
(
T
r
e
e
P
r
e
d
i
c
t
i
o
n
s
)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
Prediction=Average(TreePredictions)
```

Code

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=200,
    max_depth=10,
    random_state=42
```

)

Why it works:

Bootstrap sampling

Random feature subsets

Many diverse trees

Voting/Averaging

Main effect:

Reduces variance compared with one tree

13. SVM

SVM finds a maximum-margin decision boundary.

Hyperplane

w

T

x

+

b

=

0

w

T

x+b=0

Prediction

s

i

g

n

(

w

T

x

+

b

)


```
sign(w
```

T

x+b)

Margin

M

a

r

g

i

n

=

2

□

w

□

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
Margin=
```

□w□

2

Code

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.svm import SVC

model = SVC(
    kernel="rbf",
    C=1.0,
    gamma="scale"
)
```

C

Large C:

Less tolerance for training errors

Complex boundary

Higher overfitting risk

Small C:

More regularization

Wider margin

Gamma

Large gamma:

Narrow influence

Complex boundary

Small gamma:

Smooth boundary

14. Naive Bayes

Based on Bayes' theorem.

P

(

C

□

X

)

=

P

(

X

□

C

)

P

(

C

)

P

(

X

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
P(C X)=
P(X)
P(X C)P(C)
```

Assumption:

Features are conditionally independent given the class.

Types:

CONCEPT MAP

```
Gaussian NB → Continuous data
Multinomial NB → Text/counts
Bernoulli NB → Binary features
```

Code

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.naive_bayes import GaussianNB

model = GaussianNB()

model.fit(
    X_train,
    y_train
```

)

Excellent interview example:

Spam Detection

15. K-Means

Unsupervised clustering algorithm.

Objective:

J

=

∑

k

=

1

K

∑
x
i
□
C
k
□
x
i
–
μ
k
□
2

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
J=
k=1
```

∑
K
x
i
□C
k
∑
□x
i
−μ
k
□
2

Where:

K

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
K = clusters
```

C
k
C
k
= cluster
μ

k

μ

k

= centroid

Steps:

Select K centroids

Assign nearest points

Recalculate centroids

Repeat

Code

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.cluster import KMeans

model = KMeans(
    n_clusters=3,
    n_init=10,
    random_state=42
```

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
clusters = model.fit_predict(
    X_scaled
```

)

Use:

Customer Segmentation

16. PCA

Principal Component Analysis reduces dimensions.

Core concept:

Keep directions with maximum variance.

Eigenvalue equation

A

v

=

λ

v

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
Av=λv
```

Explained variance:

E

V

R

i

=

λ

i

∑

j

λ

j

EVR

i

=

∑

j

λ

j

λ

i

Code

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.decomposition import PCA

pca = PCA(
    n_components=2
```

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
X_pca = pca.fit_transform(
    X_scaled
```

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
print(
    pca.explained_variance_ratio_
```

)

> Remember:

PCA creates new components.

It is not feature selection.

17. Bagging vs Boosting

Bagging	Boosting
Independent learners	Sequential learners
Parallel training possible	Corrects previous errors
Reduces variance	Mainly reduces bias
Random Forest	AdaBoost/XGBoost

Bagging:

CONCEPT MAP

```
Train many models independently → combine.
```

Boosting:

CONCEPT MAP

```
Model 1 → errors → Model 2 fixes them → ...
```

18. XGBoost

XGBoost is an optimized gradient-boosting algorithm.

Main idea:

$$F_m(x) = F_{m-1}(x) + \eta h_m(x)$$

Parameters:

Parameter	Meaning
n_estimators	Number of trees

Parameter	Meaning
learning_rate	Tree contribution
max_depth	Tree complexity
subsample	Fraction of rows
colsample_bytree	Fraction of columns
reg_alpha	L1
reg_lambda	L2

> Interview Answer

XGBoost builds trees sequentially, where each new tree tries to correct the errors of the existing ensemble. It includes regularization and efficient optimizations, making it very effective for tabular datasets.

19. Cross-Validation

K-Fold:

Divide into K folds

Train on K−1

Validate on remaining fold

Repeat K times

C

V

M

e

a

n

=

1

K

∑

S

c

o

r

e

i

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
CVMean=
```

K

1

∑Score

i

Code

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(
    model,
    X,
    y,
    cv=5,
    scoring="accuracy"
```

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
print(scores.mean())
```

For classification:

Stratified K-Fold

For time series:

TimeSeriesSplit

20. Hyperparameter Tuning

Hyperparameters are values chosen before training.

Examples:

max_depth

n_estimators

learning_rate

C

gamma

K

Grid Search

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
from sklearn.model_selection import GridSearchCV

grid = GridSearchCV(
    model,
    {
        "max_depth": [3, 5, 10],
        "n_estimators": [100, 200]
    },
    cv=5
```

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
grid.fit(
    X_train,
    y_train
```

)

Random Search

Better for large search spaces.

CONCEPT MAP

```
Grid Search → tests all combinations
Random Search → tests sampled combinations
```

21. Confusion Matrix

Predicted +	Predicted -	
Actual +	TP	FN
Actual -	FP	TN

Know all four:

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
TP = Correct positive
TN = Correct negative
FP = False alarm
FN = Missed positive
```

22. Precision vs Recall

Precision

P

r

e

c

i

s

i

o

n

=

T

P

T

P

+

F

P

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
Precision=
```

TP+FP

TP

Meaning:

Of predicted positives, how many were correct?

Use when false positives are costly.

> Example:

Spam detection

Recall

R

e

c

a

l

l

=

T

P

T

P

+

F

N

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
Recall=
```

TP+FN

TP

Meaning:

Of actual positives, how many were found?

Use when false negatives are costly.

Examples:

Cancer detection

Fraud detection

23. F1 and ROC-AUC

F1

F

1

=

2

P

r

e

c

i

s

i

o

n
×
R
e
c
a
l
l
P
r
e
c
i
s
i
o
n
+
R
e
c
a
l
l


```
F1=2
```

Precision+Recall

Precision×Recall

Use for imbalanced datasets when precision and recall both matter.

ROC Curve

Plots:

T
P
R
TPR

against:

F
P
R
=
F

P

F

P

+

T

N

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
FPR=
```

FP+TN

FP

ROC-AUC

Measures ranking quality across thresholds.

1.0 = Perfect

0.5 = Random-like

For very rare classes, also check:

Precision-Recall AUC

24. Regression Metrics

MAE

M

A

E

=

1

n

∑

□

y

i

−

y

^

i

□

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
MAE=
```

n

1

∑□y

i

–
y
^
i
□
Easy interpretation.
MSE
M
S
E
=
1
n
∑
(
y
i
–
y
^
i
)
2

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
MSE=
```

n
1
∑(y
i
–
y
^
i
)
2
Penalizes large errors.
RMSE
R
M
S

E
=
M
S
E

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
RMSE=
```

MSE

Same unit as target.

R^2

R
2
=
1
−
∑
(
y
i
−
y
^
i
)
2
∑
(
y
i
−
y
-
)
2
R
2
=1−
∑(y
i
−
y

-

)

2

∑(y

i

−

y

^

i

)

2

Higher is generally better, but check generalization and residuals.

25. Imbalanced Data

> Example:

99% Genuine

1% Fraud

A model predicting everything genuine gets 99% accuracy but is useless.

Better metrics:

Precision

Recall

F1

PR-AUC

ROC-AUC

Solutions:

Class weights

Oversampling

Undersampling

SMOTE

ADASYN

Threshold tuning

> Important:

SMOTE only on training data/folds.

26. Feature Engineering

Feature engineering means converting raw information into useful model features.

Examples:

CONCEPT MAP

```
Date → Month
DOB → Age
Sales → Lag Features
Total Spend / Orders → Average Order Value
Text → TF-IDF
```

Code:

df["date"] = pd.to_datetime(

```
MACHINE LEARNING INTERVIEW CHEAT SHEET
```

```
    df["date"]
```

)

df["month"] = (

```
MACHINE LEARNING INTERVIEW CHEAT SHEET
```

```
    df["date"].dt.month
```

)

df["weekday"] = (

```
MACHINE LEARNING INTERVIEW CHEAT SHEET
```

```
    df["date"].dt.dayofweek
```

)

df["avg_order_value"] = (

```
MACHINE LEARNING INTERVIEW CHEAT SHEET
```

```
    df["total_spent"] /
    df["total_orders"].replace(0, 1)
```

)

Interview point:

Good features can improve a simple model more than unnecessarily complicated algorithms.

27. Deep-Learning Basics

Neuron:

z

=

∑

i

w

i

x

i

+

b

```
MACHINE LEARNING INTERVIEW CHEAT SHEET
```

```
z=
```

i

∑

w

i

x

i

+b

Output:

a

=

f

(

z

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
a=f(z)
```

Architecture:

CONCEPT MAP

```
Input → Hidden Layers → Output
```

ReLU

R

e

L

U

(

z

)

=

max

□

(

0

,

z

)

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
ReLU(z)=max(0,z)
```

Sigmoid

σ

(

z
)
=
1
1
+
e
−
z
σ(z)=
1+e
−z
1

Softmax
P
(
y
=
j
)
=
e
z
j
∑
k
e
z
k

MACHINE LEARNING INTERVIEW CHEAT SHEET

```
P(y=j)=
```

∑
k
e
z
k
e
z
j

Training:

Forward propagation

Calculate loss

Backpropagation

Gradient descent

Repeat

Weight update:

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W}$$

28. Deployment

Deployment means making a trained model available to users.

Flow:

Train

CONCEPT MAP

↓

Save model

CONCEPT MAP

↓

API/Web App

CONCEPT MAP

```
↓
```

User input

```
CONCEPT MAP
↓
```

Prediction

```
CONCEPT MAP
↓
```

Monitoring

Save model:

```
MACHINE LEARNING INTERVIEW CHEAT SHEET
import joblib

joblib.dump(
    model,
    "model.joblib"
)
```

Load:

```
MACHINE LEARNING INTERVIEW CHEAT SHEET
model = joblib.load(
    "model.joblib"
)
```

Common tools:

FastAPI

Flask

Streamlit

Docker

AWS

Azure

GCP

> Best practice:

Save the complete preprocessing + model pipeline.

29. Project Explanation

Always explain an ML project in this order:

1. Problem
2. Business value
3. Dataset
4. Features
5. Target
6. EDA
7. Data cleaning

8. Feature engineering
9. Algorithms compared
10. Metrics
11. Cross-validation
12. Hyperparameter tuning
13. Final model
14. Result
15. Deployment
16. Limitations

17. Future improvements

Strong Interview Pattern

The goal of my project was to predict X. I used Y dataset and identified target Z. I cleaned missing and duplicate values, encoded categorical features, scaled required numerical features, and performed feature engineering. I compared several models using cross-validation and selected the final model based on the most appropriate metric. After tuning, I performed error analysis, saved the complete pipeline, and deployed it through an API or web application.

30. Error Analysis and Limitations

Model evaluation should not end with one score.

Create:

Actual

Predicted

Probability

Error Type

Segment

Check:

False positives

False negatives

Low-confidence cases

Largest regression errors

Specific customer segments

Regions

Age groups

Products

Time periods

Questions:

Are labels wrong?

Is a feature missing?

Is one group performing badly?

Is there class imbalance?

Is threshold incorrect?

Is there data drift?

Common Project Limitations

Small dataset

Imbalanced classes

Synthetic data

Limited features

Outdated data

No external validation

No real-time data

Potential bias

Model drift

Limited explainability

> Interview Answer

After selecting the model, I perform error analysis instead of only reporting accuracy. I inspect false positives, false negatives, low-confidence cases and performance across important segments. This helps identify data-quality problems, missing features, threshold issues and potential bias. I also clearly communicate limitations such as dataset size, imbalance, drift and lack of external validation.

Ultra-Short Last 5-Minute Revision

Memorize these:

Topic	Remember
Supervised	Target available
Unsupervised	No target
Regression	Continuous output
Classification	Class output
Leakage	Future/test information enters training
Bias	Underfitting
Variance	Overfitting
Linear Regression	Continuous + MSE
Logistic Regression	Classification + sigmoid
KNN	Distance + K neighbors
Decision Tree	Gini/Entropy
Random Forest	Bagging many trees
SVM	Maximum margin
Naive Bayes	Bayes + independence
K-Means	Centroid clustering
PCA	Maximum variance components
Bagging	Parallel + variance reduction
Boosting	Sequential error correction
Cross-validation	Repeated validation
Grid Search	All parameter combinations
Precision	TP/(TP+FP)

Topic	Remember
Recall	TP/(TP+FN)
F1	Precision + Recall balance
MAE	Absolute error
RMSE	Penalizes large error
Imbalance	Accuracy can mislead
Feature Engineering	Create useful inputs
DL	Forward + loss + backprop
Deployment	Model → API → production
Error Analysis	Understand model mistakes